

Documentação

Módulo Estratégias

TODOS OS TÓPICOS

VERSÃO 4.0

21/11/2025



Sumário

1	Introdução	1
2	Estrutura de uma Estratégia	1
2.1	Área de declaração de parâmetros de entrada	2
2.2	Área de declaração de variáveis e funções	2
2.3	Área de código	2
3	Fluxo de Execução de uma Estratégia	3
4	Variáveis, tipos de dados e constantes	5
4.1	Tipos de dados	5
4.2	Séries de dados	5
4.3	Arrays	5
4.4	Correlação de múltiplos ativos	6
4.5	Acessando dados anteriores	9
4.6	Sombreamento de funções por variáveis	10
4.7	Constantes	10
5	Controle de Fluxo	14
5.1	If then else (Se então senão)	14
5.2	For (To / DownTo) Do (Para (Até / DownTo) Faça)	14
5.3	While (Enquanto)	15
5.4	Repeat (Repita)	16
6	Operadores	17
6.1	Operadores matemáticos	17
6.2	Operadores lógicos	17
7	Function (Função)	19
7.1	Criando Funções (Sintaxe)	19
7.2	Funções de biblioteca	19
8	Procedure (Procedimento)	20
8.1	Criando Procedimentos (Sintaxe)	20
9	Back-Testing	20
9.1	Modo de Execução do Backtest: OHLC e Tick a Tick	21
10	Automação de Estratégias	22
10.1	Criando uma estratégia de execução	22
10.2	Criando uma automação	24
10.3	Opções de entrada	26
10.4	Modo de Execução	26
10.5	Horários de Operação	28
10.6	Opções de saída	28
10.7	Risco	29
10.8	Segurança	29
10.9	Acompanhamento	30
10.10	Raio-X	33
10.11	Regras de Execução	35
10.12	Utilização da Função ConsoleLog na Automação	36
11	Estratégias	38
11.1	Abrir Estratégias	38
11.2	Central de Estratégias	38
11.3	Exportar / Importar Estratégias	39
11.4	Screening	40



11.5	Inserir Regra de Coloração	41
11.6	Editor de Estratégias	41
11.7	Propriedades do Editor de Estratégias	43
11.8	Nova Estratégia	43
11.9	Termômetro de Estratégia	44
12	Block Builder	47
12.1	Tipos de Estratégias Suportadas	47
12.2	Criando uma Estratégia	47
12.3	Aplicando a Estratégia no Gráfico	48
12.4	Exemplo Prático	48
13	Funções de Alarme	50
13.1	Função Alert	50
14	Funções de Back-Testing	51
14.1	Função BuyAtMarket	51
14.2	Função BuyLimit	52
14.3	Função BuyPosition	53
14.4	Função BuyPositionQty	54
14.5	Função BuyPrice	55
14.6	Função BuyStop	56
14.7	Função BuyToCoverAtMarket	58
14.8	Função BuyToCoverLimit	59
14.9	Função BuyToCoverStop	60
14.10	Função CancelPendingOrders	62
14.11	Função ClosePosition	63
14.12	Função DailyResult	64
14.13	Função HasPendingOrders	65
14.14	Função HasPosition	66
14.15	Função IsBought	67
14.16	Função IsSold	68
14.17	Função MyPrice	69
14.18	Função Position	70
14.19	Função PositionQty	71
14.20	Função Price	72
14.21	Função ReversePosition	73
14.22	Função SellPosition	74
14.23	Função SellPositionQty	75
14.24	Função SellPrice	76
14.25	Função SellShortAtMarket	77
14.26	Função SellShortLimit	78
14.27	Função SellShortStop	79
14.28	Função SellToCoverAtMarket	80
14.29	Função SellToCoverLimit	81
14.30	Função SellToCoverStop	82
14.31	Função SendOrder	84
14.32	Função OpenResult	85
14.33	Função XRay	86
14.34	Função NumberOfPendingOrders	87
15	Funções de Calendário	89
15.1	Função BarAnnualization	89
15.2	Função BarDuration	90
15.3	Função BarDurationF	91
15.4	Função Bartype	92
15.5	Função CalcDate	93
15.6	Funcao CalcTime	94
15.7	Função Closed	95



15.8	Função CloseM	96
15.9	Função CloseW	97
15.10	Função CloseY	98
15.11	Função CurrentAssetDate	99
15.12	Função CurrentDate	100
15.13	Função CurrentTime	101
15.14	Função Date	102
15.15	Função DayOfMonth	103
15.16	Função DayOfWeek	104
15.17	Função DaysToExpiration	105
15.18	Função ELDate	106
15.19	Função ELDate_Consol	107
15.20	Função FindBar	108
15.21	Função Friday	109
15.22	Função HighD	110
15.23	Função HighM	111
15.24	Função HighW	112
15.25	Função HighY	113
15.26	Função LastCalcDate	114
15.27	Função LastCalcTime	115
15.28	Função LastDayOfMonth	116
15.29	Função LowD	117
15.30	Função LowM	118
15.31	Função LowW	119
15.32	Função LowY	120
15.33	Função MinutesIntoWeek	121
15.34	Função Monday	123
15.35	Função Month	124
15.36	Função Next3rdFriday	125
15.37	Função OpenD	126
15.38	Função OpenM	127
15.39	Função OpenW	128
15.40	Função OpenY	129
15.41	Função RS_BarsPerDay	130
15.42	Função Saturday	131
15.43	Função Sunday	132
15.44	Função Thursday	133
15.45	Função Time	134
15.46	Função TimeExchange	135
15.47	Função TimeToMinutes	136
15.48	Função Today	137
15.49	Função Tuesday	138
15.50	Função VolumeD	139
15.51	Função VolumeM	140
15.52	Função VolumeW	141
15.53	Função VolumeY	142
15.54	Função Wednesday	143
15.55	Função Year	144
15.56	Função Yesterday	145
16	Funções de Candlestick	146
16.1	Função C_3WhSolds_3BlkCrows	146
16.2	Função C_BullEng_BearEng	148
16.3	Função C_Doji	149
16.4	Função C_Hammer_HangingMan	150
16.5	Função C_MornDoji_EveDoji	151
16.6	Função C_MornStar_EveStar	153
16.7	Função C_PierceLine_DkCloud	154



16.8	Função C_ShootingStar	155
17	Exemplos	156
17.1	Função DiMaisDiMenos	156
17.2	Função IFR	157
17.3	Função PaintVar	158
18	Funções Gráficas	159
18.1	Função AvgPrice	159
18.2	Função BarCount	160
18.3	Função ConsoleLog	161
18.4	Função CurrentBar	162
18.5	Função GetPlotColor	163
18.6	Função GetPlotWidth	164
18.7	Função GraphicInterval	165
18.8	Função GraphicOffset	166
18.9	Função HorizontalLine	167
18.10	Função HorizontalLineCustom	168
18.11	Função LastBarOnChart	170
18.12	Função Leader	171
18.13	Função LineSegment	172
18.14	Função MaxBarsBack	174
18.15	Função MaxBarsForward	175
18.16	Função MedianPrice	176
18.17	Função NoPlot	177
18.18	Função PaintBar	178
18.19	Funções Plot	179
18.20	Função PlotCandle	180
18.21	Função PlotN	182
18.22	Função PlotText	183
18.23	Função Range	185
18.24	Função RangeLeader	186
18.25	Função RGB	187
18.26	Função SetPlotColor	188
18.27	Função SetPlotStyle	189
18.28	Função SetPlotType	191
18.29	Função SetPlotWidth	192
18.30	Função TrueHigh	193
18.31	Função TrueLow	194
18.32	Função TrueRange	195
18.33	Função TrueRangeCustom	196
18.34	Função TypicalPrice	197
18.35	Função VerticalLine	198
18.36	Função VerticalLineCustom	199
18.37	Função WeightedClose	201
19	Indicadores	202
19.1	Função TWOMVVolatility	202
19.2	Função ADX	204
19.3	Função AccAgressSaldo	205
19.4	Função AccuDistr	206
19.5	Função AccuDistrW	207
19.6	Função AdaptiveMovingAverage	208
19.7	Função AgressionVolBalance	209
19.8	Função AgressionVolBuy	210
19.9	Função AgressionVolSell	211
19.10	Função AllTimeHigh	212
19.11	Função AllTimeLow	213
19.12	Função ArmsEaseOfMov	214



Documentação Módulo Estratégias

19.13	Função AroonLin	216
19.14	Função AroonOsc	217
19.15	Função AutoFibonacci	218
19.16	Função AutoFibonacciCustom	220
19.17	Função Averan	221
19.18	Função AvgAgent	222
19.19	Função AvgAgrBuySell	223
19.20	Função AvgAgrTotal	225
19.21	Função AvgSeparation	227
19.22	Função AvgTrueRange	229
19.23	Função BTCPrice	230
19.24	Função BTCSpread	231
19.25	Função BalanceAgent	232
19.26	Função BalanceOfPower	233
19.27	Função BearPower	235
19.28	Função BidAsk	237
19.29	Função BollingerBPerc	238
19.30	Função BollingerBandW	240
19.31	Função BollingerBands	242
19.32	Função BullPower	244
19.33	Função CCI	246
19.34	Função ChaikinMoneyFlow	247
19.35	Função ChaikinOsc	248
19.36	Função ChainSetup	249
19.37	Função COHENPRICEWAVE	250
19.38	Função COHENWEISWAVE	251
19.39	Função ContadorDeCandle	252
19.40	Função DTOscillator	253
19.41	Função DarvasBox	255
19.42	Função DecisionPoints	256
19.43	Função DiPDIM	258
19.44	Função DiDiIndex	259
19.45	Função DivergenceDetector	261
19.46	Função DonchianCH	264
19.47	Função EMA	265
19.48	Função Envelope	266
19.49	Função Euroinvest	268
19.50	Função FastStochastic	270
19.51	Função FastWMA	272
19.52	Função FgzRobot	273
19.53	Função FinancialVol	274
19.54	Função ForceIndex	275
19.55	Função FrassonATR	277
19.56	Função FrassonVH	279
19.57	Função FullStochastic	281
19.58	Função FuraChao	282
19.59	Função FuraTeto	283
19.60	Função HeikinAshi	284
19.61	Função HighestVolume	286
19.62	Função HighestVolumePrice	288
19.63	Função HiLoActivator	290
19.64	Função HistVolatility	292
19.65	Função HullMovingAverage	294
19.66	Função IchimokuCloud	295
19.67	Função ImpliedVolatility	297
19.68	Função KVO	298
19.69	Função KeltnerCH	299
19.70	Função LSVolatilityIndex	301



Documentação Módulo Estratégias

19.71	Função LinearRegressionChannel	302
19.72	Função MACD	304
19.73	Função MFI	305
19.74	Função MIMA	306
19.75	Função MIMAROC	307
19.76	Função MADRIDTRENDSQUEEZE	308
19.77	Função MAGOMACD	309
19.78	Função MAGOMEDIAS	310
19.79	Função Momentum	312
19.80	Função MomentumStochastic	314
19.81	Função MoneyFlow	315
19.82	Função MoneyFlowIndex	316
19.83	Função MADRIDMOVINGAVERAGE	317
19.84	Função NelogicaBottomFinder	318
19.85	Função NelogicaPullBackFinder	319
19.86	Função NelogicaWeisWave	320
19.87	Função OBV	321
19.88	Função OBVAvg	322
19.89	Função OnBalanceTR	323
19.90	Função OpenDaily	324
19.91	Função OpenInterest	325
19.92	Função PTAX	326
19.93	Função PTAXFuturo	327
19.94	Função ParabolicSAR	328
19.95	Função Phibo	329
19.96	Função Pivot	330
19.97	Função PowerMeter	332
19.98	Função PRICENERY	334
19.99	Função PriceOsc	335
19.100	Função PriceVolumeTrend	337
19.101	Função PriorCote	338
19.102	Função QuantityVol	339
19.103	Função RBG	340
19.104	Função ROC	341
19.105	Função RSI	343
19.106	Função Rafi	344
19.107	Função TENDENCYTRACKER	345
19.108	Função Ravi	346
19.109	Função RenkoVTwo	347
19.110	Função RsiStochastic	349
19.111	Função SMA	350
19.112	Função SafeZoneDownTrend	352
19.113	Função SafeZoneUpTrend	353
19.114	Função SampleAverage	354
19.115	Função SampleBollinger	355
19.116	Função SampleMACD	356
19.117	Função SampleRSI	357
19.118	Função SampleStochastic	358
19.119	Função SampleVWAP	359
19.120	Função Santo	360
19.121	Função SlowStochastic	361
19.122	Função SOMOSRENKOTRENDMPONTO	363
19.123	Função StdDevConstPeriod	364
19.124	Função StopATR	365
19.125	Função TOSCANABRUNELLO	367
19.126	Função TOSCANAPICCOLA	368
19.127	Função TOSCANAPICCOLO	369
19.128	Função TOSCANAPRODUTIVIDADE	370



```
EXPORTSYMBOL(groupsFree);  
// export the groupinfo to  
objects.active = modifier;  
// (modifier-ob)) to the modifier
```

19.129	Função TOSCANART	372
19.130	Função TRIX	373
19.131	Função TRIXM	374
19.132	Função TRPTAX	375
19.133	Função TRPTAXFuturo	376
19.134	Função Tilson	377
19.135	Função TimeAgrBuySell	378
19.136	Função TimeAgrSellBuy	379
19.137	Função TimeAgrTotal	380
19.138	Função TopBottomDetector	381
19.139	Função TREND AVERAGE	382
19.140	Função TREND CLOUD	384
19.141	Função TREND DIFUSOR	386
19.142	Função TREND FLOW	388
19.143	Função TREND SNIPER	390
19.144	Função TREND SOLDIER	392
19.145	Função TRADENCIATREND VARIATION	394
19.146	Função TREND VOLAG	396
19.147	Função Trades	397
19.148	Função TwoMVAgression	398
19.149	Função TwoMVPower	399
19.150	Função VSS	400
19.151	Função VWAP	401
19.152	Função VWAPDate	402
19.153	Função VWAPMonthly	403
19.154	Função VWAPWeekly	404
19.155	Função VWMA	405
19.156	Função Valerie	406
19.157	Função VolumeAgent	407
19.158	Função VolumeAtPrice	408
19.159	Função Williams	409
20	Funções de Livro	410
20.1	Função AskPrice	410
20.2	Função AskSize	411
20.3	Função BidPrice	412
20.4	Função BidSize	413
20.5	Função BookSpread	414
20.6	Função BuyOfferCount	415
20.7	Função GetAsset	418
20.8	Função GetFeed	421
20.9	Função IsBMF	424
20.10	Função Lote	425
20.11	Função MinPriceIncrement	426
20.12	Função MinOrderIncrement	427
20.13	Função MinOrderQty	428
20.14	Função SellOfferCount	429
20.15	Função TotalBuyQtd	432
20.16	Função TotalSellQtd	435
21	Funções Matemáticas	438
21.1	Função ABS	438
21.2	Função Arctangent	439
21.3	Função Ceiling	440
21.4	Função Combination	441
21.5	Função Cos	442
21.6	Função Cosine	443
21.7	Função Cotangent	444



21.8	Função Exp	445
21.9	Função ExpValue	446
21.10	Função ExtremePriceRatio	447
21.11	Função Factorial	448
21.12	Função FastD	449
21.13	Função FastK	450
21.14	Função Floor	451
21.15	Função FracPortion	452
21.16	Função GCD	453
21.17	Função IntPortion	454
21.18	Função Log	455
21.19	Função LogN	456
21.20	Função Max	457
21.21	Função Min	458
21.22	Função MinutesToTime	459
21.23	Função Mod	460
21.24	Função Neg	461
21.25	Função NumUnits	462
21.26	Função PercentR	463
21.27	Função Permutation	464
21.28	Função Pos	465
21.29	Função Power	466
21.30	Função PowerF	467
21.31	Função Random	468
21.32	Função Round	469
21.33	Função Round2Fraction	470
21.34	Função Sign	471
21.35	Função Sin	472
21.36	Função Sine	473
21.37	Função SlowK	474
21.38	Função Sqrt	475
21.39	Função Square	476
21.40	Função Tangent	477
21.41	Função UltimateOscillator	478
21.42	Função Volatility	479
21.43	Função VolumeOsc	480
21.44	Função VolumeROC	481
22	Funções de Opções	482
22.1	Função Delta	482
22.2	Função Gamma	484
22.3	Função Rho	486
22.4	Função Strike	488
22.5	Função Theta	489
22.6	Função Vega	491
23	Screening	493
23.1	Função Select	493
24	Funções de Usuário	494
24.1	Função Cum	494
24.2	Função FastKCustom	495
24.3	Função HarmonicMean	497
24.4	Função Highest	498
24.5	Função HighestBar	499
24.6	Função Lowest	500
24.7	Função LowestBar	501
24.8	Função Media	502
24.9	Função MediaExp	503



Documentação Módulo Estratégias

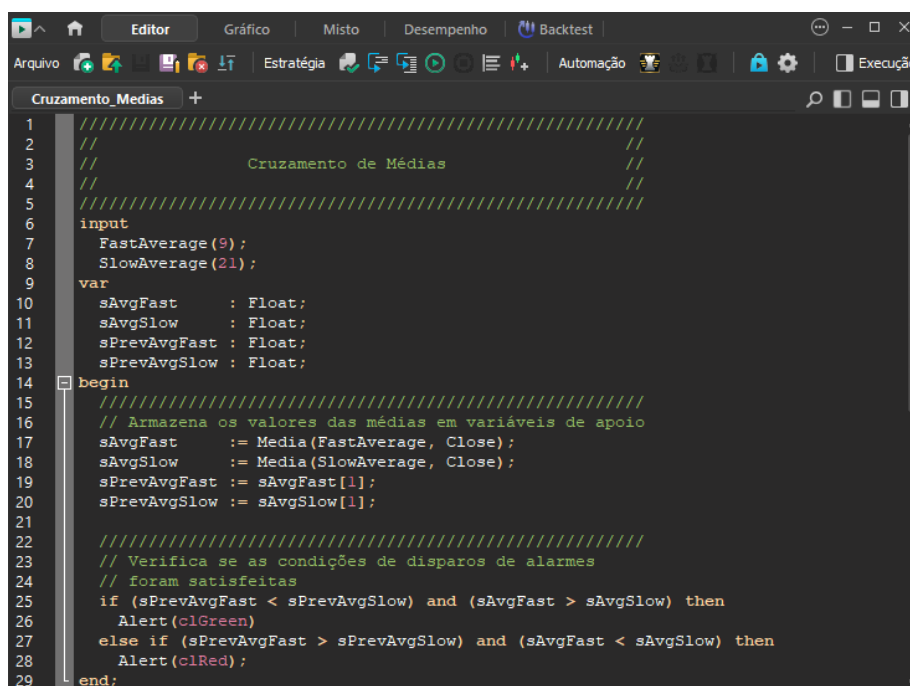
24.10	Função MidPoint	504
24.11	Função PercentChange	505
24.12	Função PriceOscillator	506
24.13	Função RateOfChange	507
24.14	Função SlowD	508
24.15	Função StdDevs	509
24.16	Função Summation	510
24.17	Função TriAverage	511
24.18	Função UlcerIndex	512
24.19	Função WAverage	513
24.20	Função WellesSum	514
24.21	Função xAverage	515
25	Funções Utils	516
25.1	Função AssetHasValue	516
25.2	Função BoolToString	517
25.3	Função CompareFloat	518
25.4	Função FloatToString	519
26	Anexos	520
26.1	Exemplos de funcionamento de estratégias	520

1 Introdução

A NTSL (Nelogica Trading System Language) é uma linguagem avançada desenvolvida para a criação de sistemas algorítmicos de operação com alto desempenho e eficiência. Projetada para atender às necessidades de traders e investidores, a NTSL permite a implementação de estratégias automatizadas de forma intuitiva e flexível. Seu grande diferencial é a facilidade de uso, possibilitando a escrita de códigos em português, tornando o desenvolvimento e a compreensão das estratégias mais acessíveis para todos os usuários.

2 Estrutura de uma Estratégia

Observe o trecho de código abaixo que representa o indicador média móvel. Esse código apresenta as três áreas que definem a estrutura de uma estratégia. São elas: área de declaração de parâmetros de entrada, área de declaração de variáveis e funções e área de código.



```
1  //////////////////////////////////////
2  //
3  //      Cruzamento de Médias      //
4  //
5  //////////////////////////////////////
6  input
7      FastAverage(9);
8      SlowAverage(21);
9  var
10     sAvgFast      : Float;
11     sAvgSlow      : Float;
12     sPrevAvgFast  : Float;
13     sPrevAvgSlow  : Float;
14 begin
15     //////////////////////////////////////
16     // Armazena os valores das médias em variáveis de apoio
17     sAvgFast      := Media(FastAverage, Close);
18     sAvgSlow      := Media(SlowAverage, Close);
19     sPrevAvgFast  := sAvgFast[1];
20     sPrevAvgSlow  := sAvgSlow[1];
21
22     //////////////////////////////////////
23     // Verifica se as condições de disparos de alarmes
24     // foram satisfeitas
25     if (sPrevAvgFast < sPrevAvgSlow) and (sAvgFast > sAvgSlow) then
26         Alert(clGreen)
27     else if (sPrevAvgFast > sPrevAvgSlow) and (sAvgFast < sAvgSlow) then
28         Alert(clRed);
29 end;
```

A área de parâmetros de entrada compreende toda a região entre a palavra reservada input (parâmetro) e a palavra reservada var. A área de variáveis e funções começa com a palavra var e estende-se até a palavra begin (início). Finalmente, a região de código inicia-se com a palavra reservada begin (início) e finaliza na palavra end (fim), conforme imagem abaixo:

Entrada	input (parametro)
Variáveis e Funções	var
Código	begin (inicio) end (fim)



```
add_data(the_data);\nEXPORTSYMBOL(groupsfree);\n/* export the groupinfo to\nobjects.active = modifier;\n(modifier-obj))*/#modifierexport(a
```

2.1 Área de declaração de parâmetros de entrada

Na área de declaração de parâmetros de entrada informamos todos os parâmetros externos que a estratégia usará. Esses parâmetros são fundamentais, pois:

- Definem a interface com o mundo externo, ou seja, é onde usuário poderá alterar e o que servirá como parâmetro de chamada caso a estratégia seja utilizada em outra interface/estratégia.
- Define os itens que serão analisados no processo de otimização.

Para definir um parâmetro: NOME_DO_PARÂMETRO (VALOR_DE_INICIALIZAÇÃO)

Exemplo:

Input	Definição
Preço (Close);	Define um parâmetro inicializado com o valor de fechamento de preços da série de dados.
Período (2);	Define um parâmetro chamado Período com o valor 2.

2.2 Área de declaração de variáveis e funções

Na área de declaração de variáveis informamos todas as variáveis que serão utilizadas na estratégia. Nesta região também descrevemos funções que desejamos usar no código. As funções devem ser sempre codificadas após a declaração das variáveis.

Para declarar uma variável: NOME_DA_VARIAVEL: TIPO

Exemplo:

Variável	Definição
sResult : Float;	Cria uma variável chamada sResult do tipo ponto flutuante.
nIndex : Integer;	Cria uma variável chamada nIndex para armazenar número inteiros.

2.3 Área de código

Nesta parte descreve-se o código propriamente dito, ou seja, as regras que utilizam parâmetros, variáveis e outros dados para calcular sinais e indicadores.



3 Fluxo de Execução de uma Estratégia

O código de uma estratégia é executado de maneira sequencial. Porém, ele é executado sequencialmente candle por candle, como se houvesse um laço que percorre toda série de dados de um ativo.

Sendo assim, a sequência de comandos existente na área de código será executada uma vez para cada candle existente. Para cada candle, será executado o código da estratégia em um loop implícito. A regra em pseudocódigo é, portanto:

- Tendo uma série de dados X de tamanho N;
- Para cada item X[i], com i variando de 0 até N-1, executa-se a área de código da estratégia;
- A cada novo dado do ativo, o código da estratégia continua sendo executado, avaliando o último e o penúltimo candle;

Importante:

- O mesmo candle pode ser processado diversas vezes. Isso ocorre muito em tempo real, onde cada trade pode gerar uma notificação de reprocessamento do fluxo de execução.
- Quando uma ordem é executada, o fluxo de execução reprocessa o mesmo candle para ativar as ordens Covers.
- Dependendo da notificação dos candles, podem ser reprocessados candles passados. Isso deve ser levado em consideração pelo programador.
- Sempre que um candle passado é reprocessado, serão reprocessados todos candles até o atual.
- Em tempo real, quando o código está executando ao mesmo tempo que recebe novos dados do mercado, o código da estratégia é executado diversas vezes para o último e o penúltimo candle durante a execução, podendo também executar em candles passados.

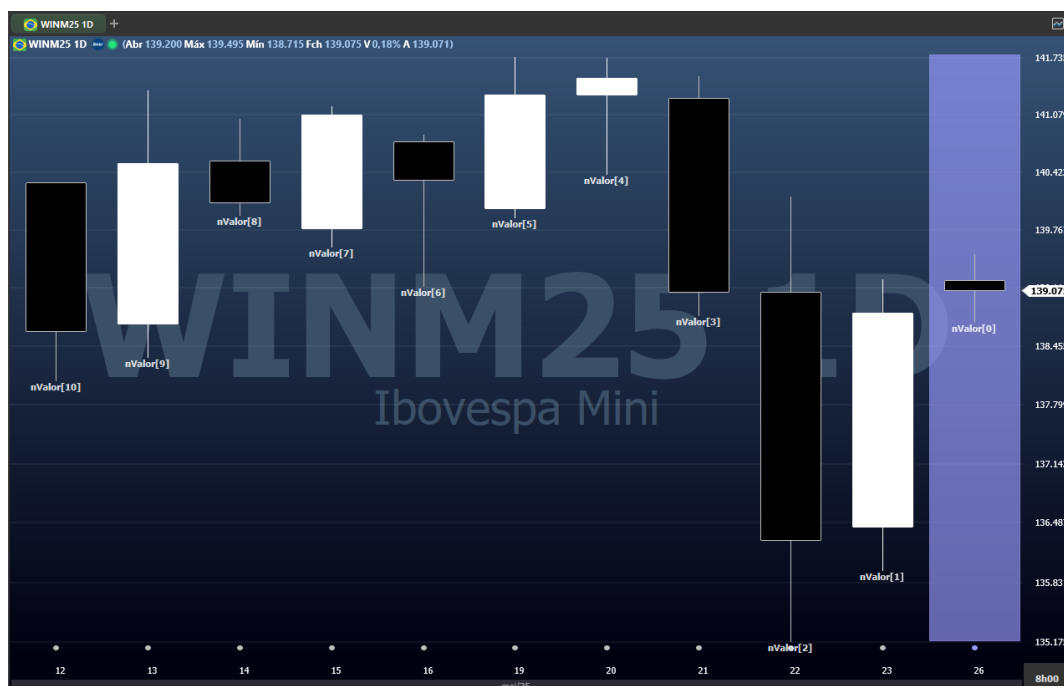
Esses processamentos podem alterar valores de variáveis globais ou de série, e podem causar mudanças na coloração para candles passados ou mudança de comportamento quando o código é pausado e executado novamente.

Quando estiver utilizando séries em sua estratégia, é importante lembrar que o primeiro tick do candle terá o valor [0] zerado. Como comentamos anteriormente, sua estratégia será executada não apenas no último candle e sim também ao passado, devido aos cálculos que precisam ser feitos para garantir que a NTSL funcione corretamente. É importante ressaltar que não se pode assumir que um candle no passado não será avaliado, porém, se desejar, é possível inibir o recálculo de algum valor utilizando a função **LastBarOnChart()** em uma condição no início do código para evitar isso. É fundamental compreender que o [0] sempre está se referindo ao candle que está sendo avaliado, e não necessariamente será do último candle.

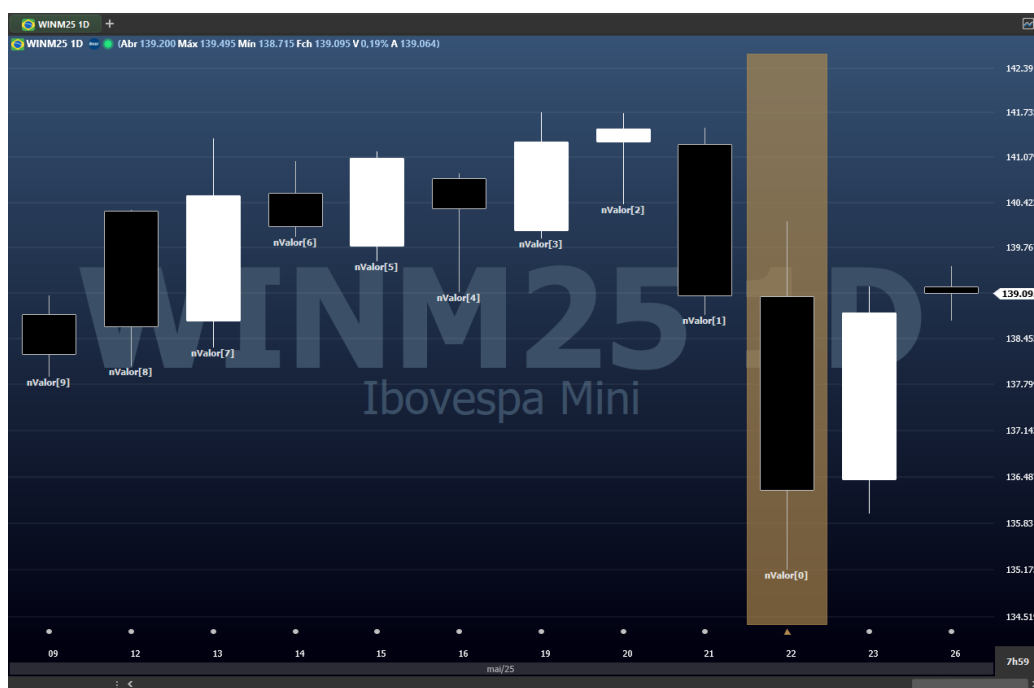
Abaixo, um exemplo de como uma variável arbitrária **"nValor"** funcionaria caso o último candle estivesse sendo avaliado. Caso eu quisesse acessar o valor dela há 2 candles passados, utilizaria **"nValor[2]"**.



Documentação Módulo Estratégias



Porém, caso o código esteja sendo executado em um candle no passado, o **nValor[0]** seria referente ao candle atual do processamento, e não ao último candle como no exemplo passado. Os outros candles também seriam acessados com base no candle que está sendo processado no momento, como no exemplo abaixo.





4 Variáveis, tipos de dados e constantes

As variáveis são utilizadas para armazenar dados, conforme seu tipo específico.

4.1 Tipos de dados

- **Float ou Real** - Representa números de ponto flutuante.
- **Integer ou Inteiro** - Representa números inteiros.
- **Boolean ou Booleano** - Representação lógica: True ou Verdadeiro e False ou Falso
- **Serie** - Representa uma série de dados.

A NTSL suporta a conversão implícita de valores Float a Inteiros, isto é, atribuir um valor Float diretamente a um valor Integer. O valor inteiro será o valor do ponto flutuante, descartando as casas decimais. Por exemplo, se eu atribuir **2.99** ou **2.35** a um inteiro, o valor do inteiro será **2**. Para outros comportamentos de arredondamento, a linguagem disponibiliza as funções Ceiling, Floor e Round. Para mais detalhes sobre essas funções, consulte esse mesmo manual na seção de funções.

As variáveis na linguagem oferecem uma flexibilidade muito maior do que na maioria dos sistemas de programação conhecidos.

Todas as variáveis são globais, e deve ser dada uma atenção especial quando utilizadas. Ao programar usando variáveis globais, deve ser levado em consideração o comportamento do processamento do fluxo de execução, a cada vez que o fluxo de execução é realizado as variáveis podem ter seu valor alterado com base no último valor que tinham.

Quando uma variável for acessada utilizando colchetes (exemplo: [x]), ela será considerada uma série de dados pelo compilador. Seu comportamento é diferente: séries iniciam cada candle com seu valor zerado. Também é possível navegar entre valores atuais e passados, mediante indexação.

4.2 Séries de dados

As séries de dados são utilizadas para obter dados específicos de cada Candle.

Séries:

- **Abertura** ou **Open** - Retorna o valor de abertura de cada candle.
- **Fechamento** ou **Close** - Retorna o dado de fechamento.
- **Minima** ou **Low** - Retorna o valor de mínima.
- **Maxima** ou **High** - Retorna o dado de máxima.
- **Quantidade** ou **Quantity** - Retorna o valor de contratos negociados de cada candle.
- **Volume** - Retorna o dado de volume financeiro de cada candle.

4.3 Arrays

Análogo ao tipo Serie, a estrutura de um Array permite o armazenamento de dados de um tipo específico, contudo, após sua criação, seu comprimento será fixo, o que permite uma alocação menor de memória. O acesso aos elementos de um Array é efetuado mediante indexação.

Estrutura da declaração:

```
1 var
2 Variavel : Array[Tamanho] Of Tipo;
```

A fim de exemplificação, segue um exemplo de declaração de um Array estático de 10 elementos, o qual é preenchido com valores de 1 a 10 (primeira estrutura de repetição), e efetuado o somatório dos valores:



```
1 var
2   i, sum : Integer;
3   lista : array[1..10] of Integer;
4 begin
5   for i := 1 to 10 do
6     lista[i] := i;
7   sum := 0;
8   for i := 1 to 10 do
9     sum := sum + i;
10    Plot(sum);
11 end;
```

4.4 Correlação de múltiplos ativos

A função **Asset** permite acessar dados de outros ativos dentro da estratégia, possibilitando correlações, comparações entre períodos e aplicação de indicadores em múltiplos instrumentos. Essa funcionalidade amplia consideravelmente as possibilidades de análise e automação, permitindo o uso de séries de diferentes ativos, intervalos e tipos de gráfico.

Estrutura básica:

```
Asset(Ticker : String, Bolsa : Const, Período : Período, Offset : Integer,
      Fator : Integer);
```

Parâmetros:

Ticker: **String**; Ticker do ativo.

Bolsa: **Const**; Bolsa na qual o ativo está listado.

Período: **Período** [Opcional]; Define o tipo de gráfico (ex: `itMinute`, `itDaily`).

Offset: **Integer** [Opcional]; Define o valor numérico do período.

Fator: **Integer** [Opcional]; Aplicável apenas para gráficos do tipo ponto figura.

Importante: O uso dos parâmetros opcionais de período, intervalo e fator está disponível apenas para usuários das plataformas **Ultra** ou **Automação 2.0**.

Exemplo de uso da função Asset

Permite declarar e acessar séries de dados de um ativo diferente do carregado no gráfico principal. Com isso, é possível comparar ativos distintos ou utilizar informações de múltiplos instrumentos dentro da mesma estratégia.

```
1 const
2   A1 = Asset("WINFUT", feedBMF, itSecond, 10);
3
4 begin
5   Plot(A1.close);
6   Plot2(A1.open);
7 end;
```

Período personalizado (mesmo ativo)

Permite acessar o mesmo ativo carregado no gráfico principal, mas em outro intervalo ou período. Ao utilizar apenas períodos customizados, a variável que utiliza a função **Asset** deve ser declarada como **constante**.

```
1 const
2   A1 = Asset(GetAsset, itSecond, 10);
3
4 begin
5   Plot(A1.close);
6 end;
```

Indicadores com ativos

Indicadores podem agora ser utilizados diretamente como propriedades do **Asset**, permitindo aplicar estudos técnicos sobre diferentes ativos e períodos.

```
1  const
2    A1 = Asset("WINFUT", feedBMF, itSecond, 10);
3
4  begin
5    Plot(A1.ADX(9, 9));
6  end;
```

Os seguintes indicadores podem ser utilizados como propriedades do **Asset**:

- | | | |
|-----------------------|----------------------|---------------------|
| • AccuDistr | • ForceIndex | • PriorCote |
| • AccuDistrW | • FrassonATR | • Ravi |
| • ADX | • FullStochastic | • RenkoVTwo |
| • AgressionVolBalance | • FuraChao | • ROC |
| • AgressionVolBuy | • FuraTeto | • RSI |
| • AgressionVolSell | • HILOActivator | • RSIStochastic |
| • AroonLin | • HistVolatility | • SafeZoneDownTrend |
| • AroonOsc | • IchimokuCloud | • SafeZoneUpTrend |
| • ArmsEaseOfMov | • KeltnerCh | • Santo |
| • AvgSeparation | • KVO | • SMA |
| • AvgTrueRange | • MACD | • SlowStochastic |
| • BalanceOfPower | • MFI | • StopATR |
| • BollingerBandW | • MIMA | • Tilson |
| • BollingerBands | • MIMAROC | • TopBottomDetector |
| • CCI | • Momentum | • TRIX |
| • ChaikinMoneyFlow | • MomentumStochastic | • TRIXM |
| • ContadorDeCandle | • MoneyFlow | • VWAP |
| • DiPDIM | • MoneyFlowIndex | • VWAPDate |
| • DidiIndex | • OBV | • VWAPMonthly |
| • DonchianCh | • OBVAvg | • VWAPWeekly |
| • EMA | • ParabolicSAR | • VWMA |
| • Envelope | • PhIbo | • Williams |
| • FastStochastic | • Pivot | |
| • FastWMA | • PriceVolumeTrend | |

Declaração via input

Também é possível declarar ativos como entradas da estratégia, permitindo alterar parâmetros sem recompilar o código.



```
1 input
2   A1("DOLFUT", feedBMF, itSecond, 10);
3
4 begin
5   Plot(A1.ADX(9, 9));
6 end;
```

Exemplo de correlação entre múltiplos ativos

Exemplo de estratégia que colore os candles conforme a relação entre WDO e WIN.

```
1 const
2   WDO = Asset("WDOFUT", feedBMF);
3   WIN = Asset("WINFUT", feedBMF);
4
5 input
6   Perodo(20);
7
8 var
9   wdoMedia : Float;
10  winMedia : Float;
11
12 begin
13   wdoMedia := MediaExp(Perodo, WDO.close);
14   winMedia := MediaExp(Perodo, WIN.close);
15
16   if (WDO.close > wdoMedia) and (WIN.close < winMedia) then
17     PaintBar(clLime)
18   else if (WDO.close < wdoMedia) and (WIN.close > winMedia) then
19     PaintBar(clAqua);
20 end;
```

Consultando o nome e feed do ativo

As funções **GetAsset** e **GetFeed** retornam respectivamente o ticker e a bolsa do ativo declarado.

```
1 const
2   WDO = Asset("WDOFUT", feedBMF);
3 begin
4   if (MaxBarsForward = 0) then
5     PlotText("Ticker: " + WDO.GetAsset +
6             " - Bolsa: " + WDO.GetFeed,
7             clRed, 0, 7);
8 end;
```

Propriedades acessíveis via função Asset

A função **Asset** permite criar um ativo diferente do gráfico principal e acessar suas informações e séries de preço e volume. As séries de preço e volume podem ser consultadas na **seção 4.2** deste manual.

- **GetFeed** – Retorna o feed de dados do ativo.
- **GetAsset** – Retorna o próprio ativo.
- **AskPrice** – Preço do ask (preço de venda).
- **AskSize** – Tamanho do ask (quantidade disponível para venda).
- **BidPrice** – Preço do bid (preço de compra).
- **BidSize** – Tamanho do bid (quantidade disponível para compra).



- **BookSpread** – Diferença entre o melhor bid e o melhor ask.
- **Lote** – Tamanho do lote do ativo.
- **MinPriceIncrement** – Incremento mínimo de preço.
- **Strike** – Preço de exercício (para derivativos como opções).

Bolsas disponíveis

As bolsas disponíveis dependem do acesso do usuário. Os feeds podem ser consultados na aba “Constantes” da plataforma.

- **feedBCB**
- **feedBMF**
- **feedBMFSynthetic**
- **feedBovespa**
- **feedCME**
- **feedDowJones**
- **feedEconomic**
- **feedMetrics**
- **feedNasdaq**
- **feedNyse**
- **feedOXR**
- **feedVector**
- **feedActivTrades**

Observações

- O parâmetro **Fator** é utilizado apenas para gráficos de ponto figura.
- Cada ativo pode ter seu próprio intervalo e período, independentes do ativo principal.
- O limite máximo é de **30 ativos** por estratégia.
- Em gráficos atemporais, o fechamento dos candles auxiliares pode não coincidir com o do ativo principal, o que pode causar divergências entre o backtest/replay e a automação em tempo real.

4.5 Acessando dados anteriores

O dado atual de uma variável pode ser descrito por Variável[0]. O número entre colchetes indica o dado de quantos períodos anteriores deseja-se acessar (sendo 0, portanto, da barra corrente). Para ilustrar melhor, vamos a um exemplo:

```
1 sResult := Preco[1];
```

A linha de código acima está atribuindo o valor da variável Preco[1] para a variável sResult. Imagine que Preco corresponde a todos os valores de fechamento da série de dados de um certo ativo, como na tabela abaixo:

Data	Posição	Valor
05/10/2010	Preço[0]	12,43
04/10/2010	Preço[1]	12,40
01/10/2010	Preço[2]	11,39
30/09/2010	Preço[3]	12,51
29/09/2010	Preço[4]	12,38
dd/mm/aaaa	Preço[5]	xx,xx

Portanto, Preço[1] refere-se ao valor de ontem do preço de fechamento (sResult vale então 12,40 em nosso exemplo). Dessa forma, o número inteiro que especificarmos entre colchetes indica ao sistema quantos períodos no passado deve-se acessar a informação.

4.6 Sombreamento de funções por variáveis

Na NTSL, variáveis e funções compartilham o mesmo espaço de nomes no escopo do script. Isso significa que, ao declarar uma variável com o mesmo identificador de uma função existente, a função será sobrescrita por essa variável no contexto atual. Esse comportamento é conhecido como **sombreamento**.

Quando isso ocorre, a função original deixa de estar acessível enquanto a variável estiver em escopo, o que pode resultar em erros de execução, especialmente ao utilizar nomes de funções nativas como **BuyToCover**, **SellToCover**, entre outras.

Exemplo: Sombreamento de função nativa

```
1 var
2   BuyToCover: integer;
3 begin
4   BuyToCover := 1; // A função BuyToCover é sobrescrita por esta variável
5   if Close > Media(Close, 20) then
6     BuyToCover(); // Erro: BuyToCover agora é um inteiro, não uma função
7 end;
```

No exemplo acima, a função **BuyToCover** é uma função nativa usada para envio de ordens de recompra a descoberto. No entanto, ao declarar uma variável com o mesmo nome (**BuyToCover: integer**), a função original foi sobrescrita. A tentativa de invocação da função resulta em erro, pois o nome agora se refere a um valor inteiro.

Boas práticas

- Evite declarar variáveis com nomes iguais a funções nativas, como **BuyToCover**, **SellShortStop**, **Media**, etc.
- Prefira nomes mais descritivos para variáveis, como **qtdeCompra**, **reversaoLong**, **ativarSinal**, etc.
- Sempre revise se o nome escolhido para a variável não corresponde a nenhuma função da linguagem.

4.7 Constantes

São utilizadas para a inserção de parâmetros de determinadas funções, estes valores não podem ser alterados pelo usuário.

Lado da Ordem:

- **osCompra** - Ordem de compra.
- **osVenda** - Ordem de venda.

Opções:



- **optCall** - Opção de compra.
- **optPut** - Opção de venda.

Tipo de Ordem:

- **otLimit** - Tipo limite.
- **otMarket** - A mercado.
- **otStopLimit** - Tipo stop.

Cores:

- **clAqua** ou **clAzulClaro** - Cor azul-claro
- **clBlack** ou **clPreto** - Cor preta
- **clAzul** - Cor azul
- **clCreme** - Cor creme
- **clCinzaEscuro** - Cor cinza-escuro
- **clFucsia** - Cor fúcsia
- **clCinza** - Cor cinza
- **clVerde** - Cor verde
- **clVerdeLima** - Cor verde-limão
- **clCinzaClaro** - Cor cinza-claro
- **clMarrom** - Cor marrom
- **clCinzaMedio** - Cor Cinza médio
- **clVerdeClaro** - Cor verde-claro
- **clAzulMarinho** - Cor azul-marinho
- **clVerdeOliva** - Cor verde-oliva
- **clPurpura** - Cor púrpura
- **clVermelho** - Cor vermelha
- **clPrata** - Cor prata
- **clAzulClaro** - Cor azul-claro
- **clTeal** - Cor Verde-azulado
- **clBranco** - Cor branca
- **clAmarelo** - Cor amarela

Constantes Matemáticas:

- **Math_PI** – Número Pi
- **Math_Sqrt2** – Raiz quadrada de 2
- **Math_Euler** – Número de Euler
- **Math_Log2E** – $\log_2(e)$
- **Math_Log10E** – $\log_{10}(e)$
- **Math_Ln2** – $\ln(2)$



- **Math_Ln10** – $\ln(10)$
- **Math_PI_Div2** – $\pi/2$
- **Math_PI_Div4** – $\pi/4$
- **Math_1DivPI** – $1/\pi$
- **Math_2DivPI** – $2/\pi$
- **Math_2DivSqrtPI** – $2/\sqrt{\pi}$
- **Math_1DivSqrt2** – $1/\sqrt{2}$
- **Math_SqrtPI** – Raiz quadrada de π
- **Math_EulerMascheroni** – Constante de Euler-Mascheroni
- **Math_Phi** – Número de Ouro
- **Math_EmbreeTrefethen** – Constante de Embree-Trefethen
- **Math_FeigenbaumDelta** – Constante de Feigenbaum (δ)
- **Math_FeigenbaumAlpha** – Constante de Feigenbaum (α)
- **Math_TwinPrime** – Constante dos primos gêmeos
- **Math_MeisselMertens** – Constante de Meissel-Mertens
- **Math_BrunTwinPrimes** – Constante de Brun para primos gêmeos
- **Math_BrunQuadruples** – Constante de Brun para primos quádruplos
- **Math_BruijnNewman** – Constante de Bruijn-Newman
- **Math_Catalan** – Constante de Catalan
- **Math_LandauRamanujan** – Constante de Landau-Ramanujan
- **Math_Viswanath** – Constante de Viswanath
- **Math_Legendre** – Constante de Legendre
- **Math_RamanujanSoldner** – Constante de Ramanujan-Soldner
- **Math_ErdosBorwein** – Constante de Erdős-Borwein
- **Math_Liouville** – Constante de Liouville
- **Math_MinInt** – Menor inteiro suportado
- **Math_MaxInt** – Maior inteiro suportado

Constantes de Período:

- **itAggressor** – Período baseado no agressor da operação (quem iniciou a negociação).
- **itDaily** – Período diário.
- **itInversion** – Período por inversão de direção do preço.
- **itKagiChart** – Período Kagi, muda de direção conforme variação definida.
- **itLote** – Período baseado em quantidade de lotes negociados.
- **itMinute** – Período por minutos fixos.
- **itMonthly** – Período mensal.
- **itPointFigure** – Período Ponto Figura, baseado em variação de preço.



- **itQtyd** – Período baseado em quantidade de negócios.
- **itRange** – Período baseado em variação de preço fixa.
- **itRenko** – Período Renko, muda conforme variação de preço definida.
- **itSecond** – Período por segundos fixos.
- **itTrade** – Período por microquantidade de trades.
- **itVarInv** – Período por variação inversa.
- **itVariation** – Período baseado em variação de preço ou percentual.
- **itWeekly** – Período semanal.
- **itYearly** – Período anual.

Tipo de Média:

- **kaADX** - Média baseada no indicador ADX.
- **kaArithmetic** - Média aritmética simples.
- **kaExponential** - Média móvel exponencial.
- **kaWeighted** - Média móvel ponderada.
- **kaWelles** - Média de Welles Wilder.

Tipo de Valor:

- **tvClose** - Valor de fechamento.
- **tvMax** - Valor máximo.
- **tvMin** - Valor mínimo.
- **tvOpen** - Valor de abertura.



5 Controle de Fluxo

As instruções de fluxo são utilizadas para administrar a sequência de execução das instruções dentro de um programa. A NTSL apresenta três tipos desse tipo:

5.1 If then else (Se então senão)

Se (condição) for verdadeiro executa-se a listagem (comandos 1), caso (condição) seja falso executa-se (comandos 2). Tanto (comandos 1) quanto (comandos 2) podem ser sequências de instruções.

A expressão (condição) pode ser qualquer tipo de teste que resulte em verdadeiro ou falso, alguns exemplos:

```
1 if (Close > Close[1])
2 if (nIndex = 10)
3 if (bCond = false) and (Volume > 1000)
```

A seguir um exemplo de coloração de candles de acordo com a condição presente no **If then else (Se então senão)**.

```
1 begin
2   if (close = close[1]) then
3     PaintBar(clYellow)
4   else if (close > close[1]) then
5     PaintBar(clGreen)
6   else
7     PaintBar(clRed);
8 end;
```

Se o valor de fechamento da barra atual (Close equivale a Close[0]) for igual ao fechamento da barra anterior (Representado por Close[1]) executa-se o código que segue a palavra reservada **THEN (ENTÃO)**. Caso contrário, o sistema executa o código subsequente até chegar na palavra reservada **ELSE (SENÃO)**.

Exemplo com múltiplas instruções begin e end (início e fim)

Quando a listagem (comandos 1 ou comandos 2) contiver mais de uma instrução, é necessário delimitar o bloco com **begin (início)** e **end (fim)**. Veja o exemplo:

```
1 begin
2   if (close = close[1]) then
3     begin
4       PaintBar(clYellow);
5       PlotText("Fechamento igual ao anterior", clYellow);
6     end
7   else if (close > close[1]) then
8     begin
9       PaintBar(clGreen);
10      PlotText("Fechamento superior ao anterior", clGreen);
11    end
12  else
13    begin
14      PaintBar(clRed);
15      PlotText("Fechamento inferior ao anterior", clRed);
16    end;
17 end;
```

5.2 For (To / DownTo) Do (Para (Até / DownTo) Faça)

O comando **FOR (PARA)** é utilizado para definir um loop controlado, ou seja, o bloco é executado repetidamente até que a variável de contagem saia do valor inicial e atinja o valor final.



Para a variável controladora, pode-se incrementá-la, a partir do comando **To (Até)**, ou decrementá-la, ao utilizar na estrutura o **DownTo**.

Exemplo (For To Do (Para Até Faça)):

Observe o código da média móvel, a variável de contagem nIndex começa com o valor 0 e deve chegar ao valor de Período - 1. No exemplo, Período foi definido como parâmetro de entrada, para definir o tamanho da média. Assim, caso seja parametrizado o valor 0 (Período), o comando FOR criará um loop de 9 iterações (de 0 até 8) para cada barra, calculando assim o valor médio para a posição atual.

```
1  input
2    Preco(close);
3    Período(9);
4  var
5    sResult : Float;
6    nIndex  : Integer;
7  begin
8    sResult := 0;
9    for nIndex := 0 to Período - 1 do
10     begin
11       sResult := sResult + Preco[nIndex];
12     end;
13   Plot(sResult / Período);
14 end;
```

Exemplo (For To Do x For DownTo Do (Para Até Faça x Para DownTo Faça)):

A fim didático, segue um exemplo de cálculo de fatorial, utilizando as duas estruturas.

For To Do (Para Até Faça):

```
1  input
2    n(6);
3  var
4    fatorial : Integer;
5    i        : Integer;
6  begin
7    fatorial := 1;
8    For i := 1 To n Do
9      fatorial := fatorial * i;
10   Plot(fatorial);
11 end;
```

For DownTo Do (Para DownTo Faça):

```
1  input
2    n(6);
3  var
4    fatorial : Integer;
5    i        : Integer;
6  begin
7    fatorial := 1;
8    For i := n DownTo 1 Do
9      fatorial := fatorial * i;
10   Plot(fatorial);
11 end;
```

5.3 While (Enquanto)

A execução da estratégia ao chegar no comando WHILE testa o resultado de (condição). Caso (condição) seja



verdadeiro (true) a listagem (comandos) é executada. Após a execução a (condição) volta a ser testada, assim, o loop apenas irá se encerrar quando (condição) deixar de ser verdadeira.

Exemplo:

No código a seguir, reescrevemos o indicador média móvel utilizando a instrução **WHILE (ENQUANTO)** ao invés de **FOR (PARA)**.

```
1  input
2    Preco(close);
3    Perodo(9);
4  var
5    sResult : Float;
6    nIndex  : Integer;
7  begin
8    sResult := 0;
9    nIndex  := 0;
10   while (nIndex <= Perodo - 1) do
11     begin
12       sResult := sResult + Preco[nIndex];
13       nIndex  := nIndex + 1;
14     end;
15   Plot(sResult / Perodo);
16 end;
```

5.4 Repeat (Repita)

Este comando difere-se do For e While, no sentido de que a condição será testada no final da estrutura, e não no início, portanto, as instruções do bloco serão executadas pelo menos uma vez.

Exemplo:

A fim de comparação com as outras estruturas já abordadas, segue o exemplo de cálculo de fatorial:

```
1  input
2    n(5);
3  var
4    fatorial : Integer;
5    i        : Integer;
6  begin
7    fatorial := 1;
8    i := 1;
9    repeat
10     begin
11       fatorial := fatorial * i;
12       i := i + 1;
13     end
14   until (i > n);
15   Plot(fatorial);
16 end;
```


6 Operadores

Os operadores constituem os símbolos matemáticos e lógicos usados em cálculos e comparações.

6.1 Operadores matemáticos

Os operadores matemáticos são:

Operador	Descrição	Exemplo	Resultado
+	Adição	5+4	9
-	Subtração	5-4	1
*	Multiplicação	5*4	20
/	Divisão	5/4	1,25

O operador de divisão possui a maior força de precedência, seguido por multiplicação. Assim, como consta na imagem abaixo:

Expressão	Resultado
10*10/5+2	22
10*10+2	102
50+100*10/2-2	549

6.2 Operadores lógicos

Os operadores lógicos são usados principalmente para comparações e para combinar várias condições em uma expressão lógica. Aqui está uma explicação de seu uso:

"E" lógico

Representado pela palavra reservada **and (e)**, retornará TRUE somente quando as duas condições de teste forem verdadeiras conforme tabela verdade abaixo:

Condição 1	AND	Condição 2	Resultado
FALSE	AND	FALSE	FALSE
FALSE	AND	TRUE	FALSE
TRUE	AND	FALSE	FALSE
TRUE	AND	TRUE	TRUE

"OU" lógico

Representado pela palavra reservada **or (ou)**, retornará TRUE (verdadeiro) sempre que pelo menos uma das condições de teste for verdadeira, conforme tabela verdade abaixo:

Condição 1	OR	Condição 2	Resultado
FALSE	OR	FALSE	FALSE
FALSE	OR	TRUE	TRUE
TRUE	OR	FALSE	TRUE
TRUE	OR	TRUE	TRUE



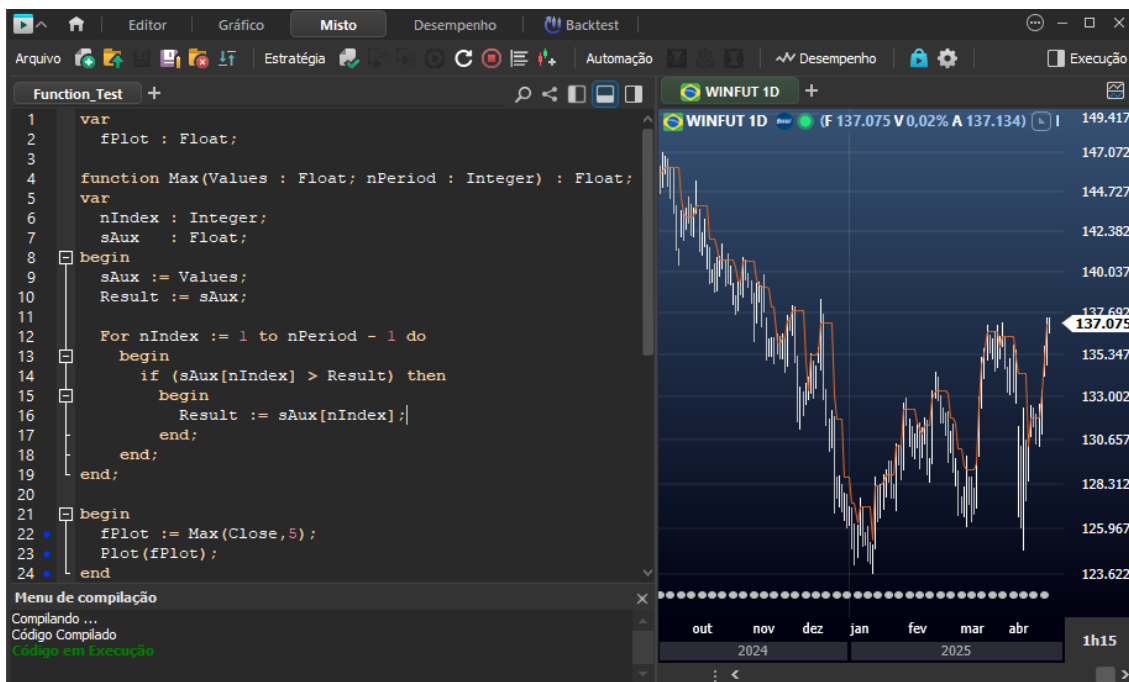
"NOT" lógico

Representado pela palavra reservada **not (não)**, retornará TRUE (verdadeiro) quando uma condição de teste for falsa, conforme tabela verdade abaixo:

Condição	NOT	Resultado
FALSE	NOT	TRUE
TRUE	NOT	FALSE

7 Function (Função)

Conforme visto, funções são declaradas e descritas na área de declaração de variáveis e funções, abaixo um exemplo de funções:



Observe que primeiro é declarada a variável usada na área principal. É sempre interessante manter o código o mais claro e organizado possível e as funções desempenham um papel fundamental nessa tarefa.

7.1 Criando Funções (Sintaxe)

```

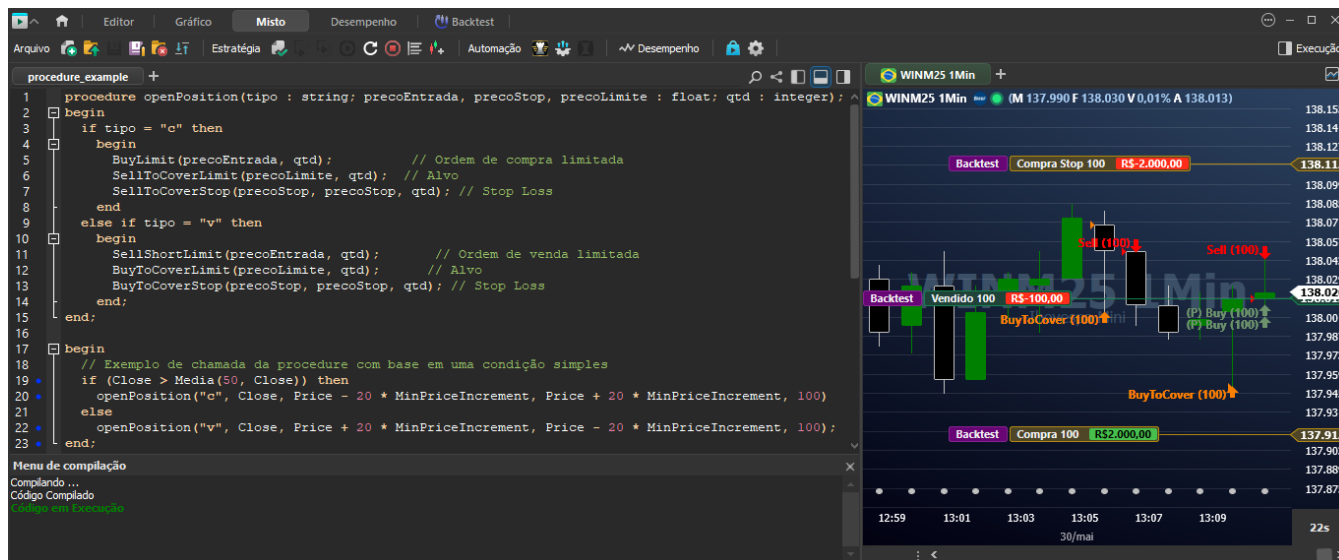
1  Função NomeDaFunção (parâmetro1 : Tipo;
2    parâmetro2 : Tipo; parâmetroN : Tipo): Tipo de Retorno;
3  begin
4    Comandos
5  end;
  
```

7.2 Funções de biblioteca

Além do usuário poder criar seus próprios indicadores, é possível utilizar a biblioteca do sistema, ou seja, o usuário pode utilizar estratégias já criadas em novas. Dentro das funcionalidades de bibliotecas, o usuário poderá colorir os gráficos de acordo com as condições determinadas pelo seu indicador.

8 Procedure (Procedimento)

Procedures são semelhantes às funções, mas não retornam um valor. Elas são usadas para agrupar comandos que podem ser chamados em diferentes partes do código, ajudando a manter a organização e legibilidade. A seguir, temos um exemplo prático de uma procedure para abrir posições no mercado com base em parâmetros.



8.1 Criando Procedimentos (Sintaxe)

```

1 procedimento NomeDaProcedure(parâmetro1 : Tipo;
2   parâmetro2 : Tipo; parâmetroN : Tipo);
3 begin
4   Comandos
5 end;

```

9 Back-Testing

A funcionalidade de Back-testing permite ao usuário avaliar uma determinada estratégia, teoria ou modelo através de uma análise de dados históricos.

Lista de funcionalidades utilizadas para Back-Testing:

- Lista de funções;
- Criar regra de execução;
- Execução da estratégia.

Após criada a estratégia de Back-Testing, para ser adicionado diretamente no gráfico, clique no botão direito sobre o mouse e selecione a opção "Inserir Estratégia de Execução".

O código de back-testing simula o mesmo comportamento da automação, as simulações de execução de ordens são processadas todas ao final do fluxo de execução do candle. Quando ocorre aumento de posição ao final do fluxo de execução, o código é reprocessado para ativar as ordens de cobertura da operação (ordens ToCover). O usuário pode observar esse comportamento quando utilizar o modo debug.



9.1 Modo de Execução do Backtest: OHLC e Tick a Tick

O Profit Ultra permite escolher entre dois modos de execução para o backtest: OHLC ou Tick a Tick. Abaixo, explicamos como cada um desses modos opera e suas características específicas.

O modo OHLC (Open, High, Low, Close) utiliza apenas os preços de abertura, máxima, mínima e fechamento de cada candle para realizar os cálculos e simular o movimento das operações. As saídas Limit e Stop utilizam os preços de máxima, mínima e fechamento para simular as execuções.

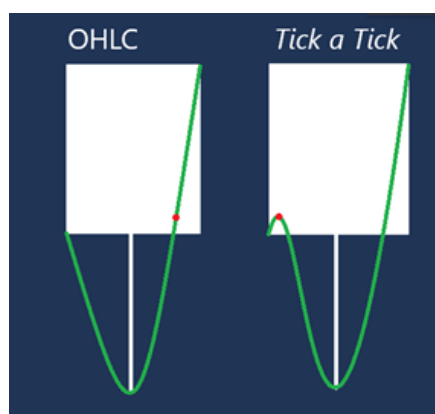
Como os movimentos dentro do candle não são considerados, as simulações são menos detalhadas em relação ao fluxo real de preços intraday. Em casos de ordens de stop ou gain, o movimento do preço é verificado com base nos níveis OHLC do candle.

O modo Tick a Tick utiliza exatamente o fluxo dos preços dentro de cada candle para simular as operações. Isso garante maior precisão ao refletir como as ordens seriam executadas em tempo real. Permitindo simulações de saídas Limit e Stop nos exatos pontos em que seriam executadas durante o pregão.

Em simulações OHLC, o backtest pode concluir que um stop ou um gain não foi atingido ao avaliar apenas os preços do OHLC, por achar que a ordem executou após a variação de preço. Já no modo Tick a Tick, o mesmo stop pode ser executado corretamente ao analisar os preços intraday, assegurando a aderência à lógica operacional real.

Por utilizar uma granularidade mais fina, o modo Tick a Tick tem menor disponibilidade de histórico quando comparado ao OHLC. Isso pode afetar simulações de períodos longos. Essa limitação pode ser encontrada ao executar uma estratégia no modo Tick a Tick.

Considere a imagem abaixo, que ilustra dois candles de alta, onde a linha verde representa o movimento real dos preços dentro do candle e o ponto vermelho, a ponto onde uma ordem foi enviada. No caso de um backtest enviando uma ordem alguns ticks acima da abertura, no modo OHLC, a ordem só seria executada após o preço atingir a mínima (LOW) e, posteriormente, alcançar o fechamento (CLOSE). Já no modo Tick a Tick, a ordem poderia ser executada assim que o preço atinge o nível programado, mesmo antes do candle ser concluído, permitindo, por exemplo, que uma ordem de LOSS seja disparada antes do fechamento do candle.



10 Automação de Estratégias

O módulo de Automação de Estratégias tem o objetivo de automatizar as estratégias de execução criadas no Editor de Estratégias. Ele está disponível nas versões mais avançadas das plataformas Nelógica, em modo simulação de forma gratuita e disponível para contas reais por meio de módulos opcionais.

O novo módulo de automação espelha o funcionamento do backtest das estratégias para execução de ordens reais e em simulador. Para fazer o gerenciamento dessas automações pode-se acessar o menu Estratégias > Automação de Estratégias.

A janela de Automações de Estratégias permite que possamos criar e gerenciar automações, acompanhar o resultado de todas as automações de maneira simplificada. Caso queira um detalhamento maior da automação pode-se clicar com o botão direito e ir em "Detalhes da Automação". Nos detalhes é possível acompanhar todos os eventos que a execução gerou, assim como diversos indicadores de performance da automação.

Automações

Comparador

+

Nova Automação

Todas as Contas

Resumo

|| Pausar Todas

Pausar + Zerar posições

xray

Última modificação: 23/05/2025

Status

Conta e Carteira

Ativo

Período

Posição

Res. Aberto

Res. Dia

Parâmetros

Executando

Sim

995555969 - Nicolas Dar

xray1

WINM25

1 Min

-

R\$ 0,00

0,00 pts

R\$ -20,00

-25,00 pts

-

...

Executando

Sim

995555969 - Nicolas Dar

xray2

WINM25

1 Min

-

R\$ 0,00

0,00 pts

R\$ -20,00

-25,00 pts

-

...

Executando

Sim

995555969 - Nicolas Dar

xray4

WINM25

1 Min

-

R\$ 0,00

0,00 pts

R\$ -20,00

-25,00 pts

-

...

Executando

Sim

995555969 - Nicolas Dar

xray5

WINM25

1 Min

-

R\$ 0,00

0,00 pts

R\$ -20,00

-25,00 pts

-

...

Executando

Sim

995555969 - Nicolas Dar

xray6

WINM25

5 Min

-

R\$ 0,00

0,00 pts

R\$ -50,00

-22,73 pts

-

...

Executando

Sim

995555969 - Nicolas Dar

xray7

WINM25

30 Seg

1 V

R\$ 0,00

0,00 pts

R\$ -51,00

-31,88 pts

-

...

Executando

Auto

xray7 - Nicolas Darvas

Resumo

Eventos

Raio-X

|| Pausar

Zerar

...

Resumo

Status

Ativo

WINM25

Período

30 Segundo(s)

Zerar às

-

Qtd

1/10

Est OCO

<None>

Tipo exec

Tempo real

Lado

Ambos

Limite ganho

R\$ 0,00

Limite perda

R\$ 0,00

Posição

1 V

Ord abertas

2

Qtd aberta

-

Op Totais

6

Op Consec.

1 Gain

Preço médio

R\$ 138.910,00

Res aberto

R\$ 1,00

Res aberto%

5,00 Pts

Res total

R\$ -50,00

Res total%

-31,25 Pts

Limite Financeiro(R\$):

0 / ∞

Limite Mini:

0 / ∞

Limite Cheio:

0 / ∞

Os próximos capítulos detalham desde a criação de uma estratégia de execução utilizando o novo módulo de Automação de Estratégias, como o usuário pode acompanhar essas automações, e por fim detalha regras gerais da execução das estratégias.

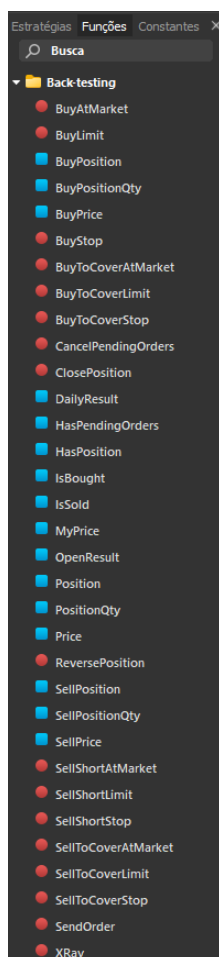
10.1 Criando uma estratégia de execução

Para criar uma automação você precisará criar uma estratégia de execução, que pode ser criada a partir do Editor de Estratégias. Se você já sabe o que é isso, pode seguir para o próximo passo.

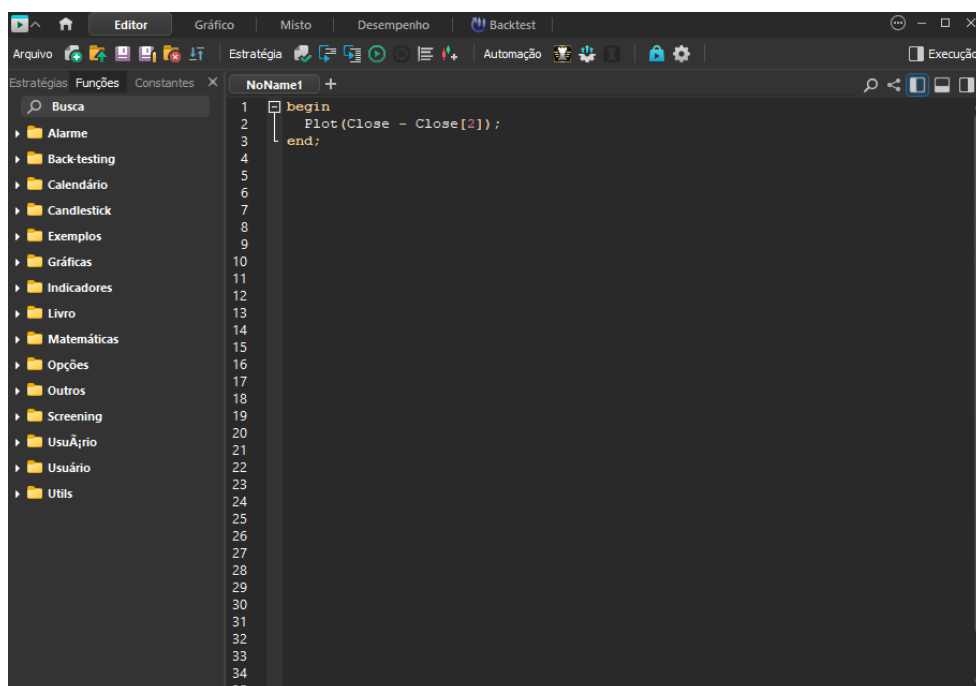
Ao abrir o Editor de Estratégias, você pode criar sua estratégia de execução utilizando funções de backtest (essas funções irão caracterizar a estratégia como sendo uma estratégia de execução e isso possibilitará a sua seleção no próximo passo para que a estratégia seja automatizada). Para criar uma estratégia de execução você deve usar funções do módulo backtest, que pode ser visto na imagem abaixo:

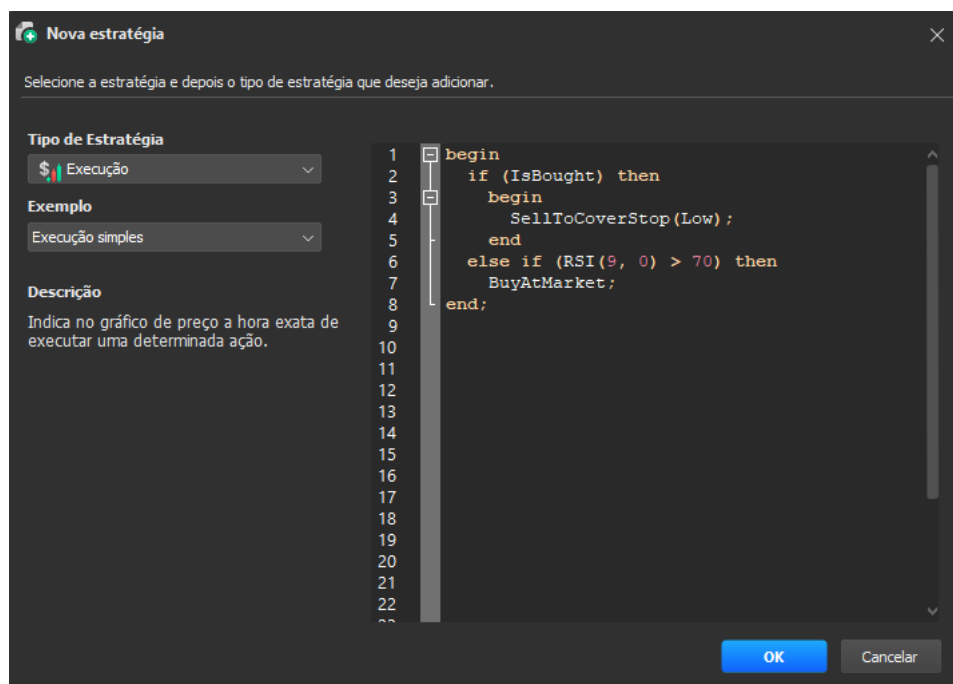


Documentação Módulo Estratégias



Sugerimos que você utilize a função Nova estratégia, e selecione exemplos de estratégias de execução se você não estiver familiarizado com programação ou com o Editor de Estratégias.



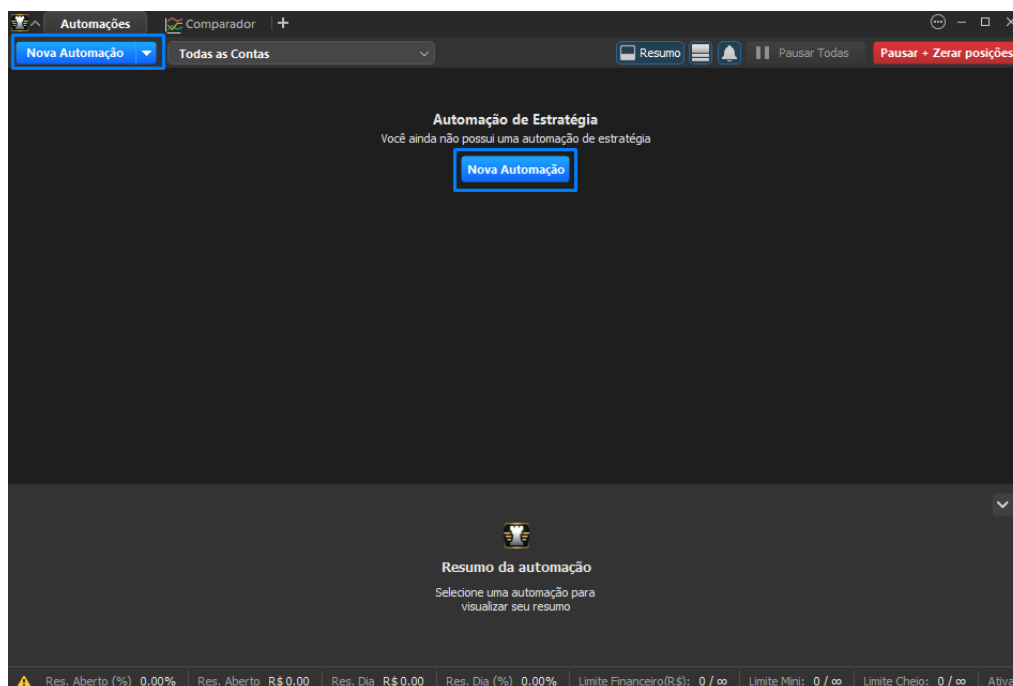


Abaixo é apresentado um exemplo de estratégia, utilizando o indicador IFR/RSI, que pode ser utilizado no módulo de estratégias automatizadas:

```
1 ///////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
2 //                                                                 IFR //
3 ///////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
4 begin
5     // Verifica se está comprado
6     if (IsBought) then
7         begin
8             // Fecha a posição com uma venda caso o IFR
9             // esteja superior a 70 (sobrecomprado)
10            if (RSI(9,0) > 70) then
11                SellToCoverAtMarket;
12            end
13            // Verifica se está vendido
14        else if (IsSold) then
15            begin
16                // Fecha a posição com uma compra caso o IFR
17                // esteja inferior a 30 (sobrevendido)
18                if (RSI(9,0) < 30) then
19                    BuyToCoverAtMarket;
20                end;
21            end;
22        end;
```

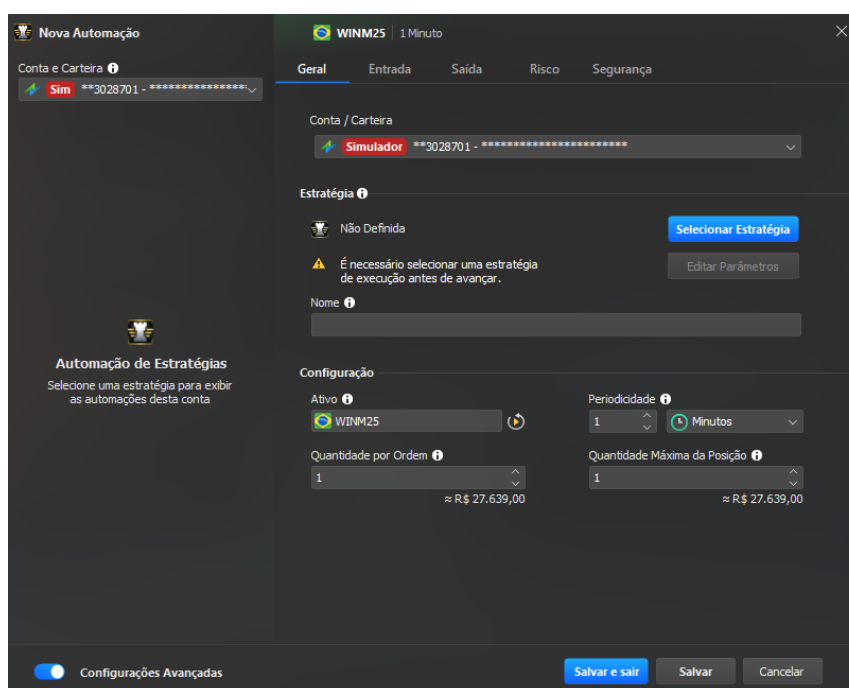
10.2 Criando uma automação

Para criar uma nova automação, basta abrir a interface de controle e localizar o botão "Nova Automação", disponível no menu superior da janela. Esse botão estará sempre visível nessa área da interface. Caso ainda não exista nenhuma automação criada, o mesmo botão "Nova Automação" será exibido em destaque no centro da tela.



Após clicar nesse botão será aberta a janela de criação da estratégia automatizada onde será configurada os parâmetros da automação em duas etapas. A primeira etapa contém configurações essenciais e obrigatórias para a criação da automação.

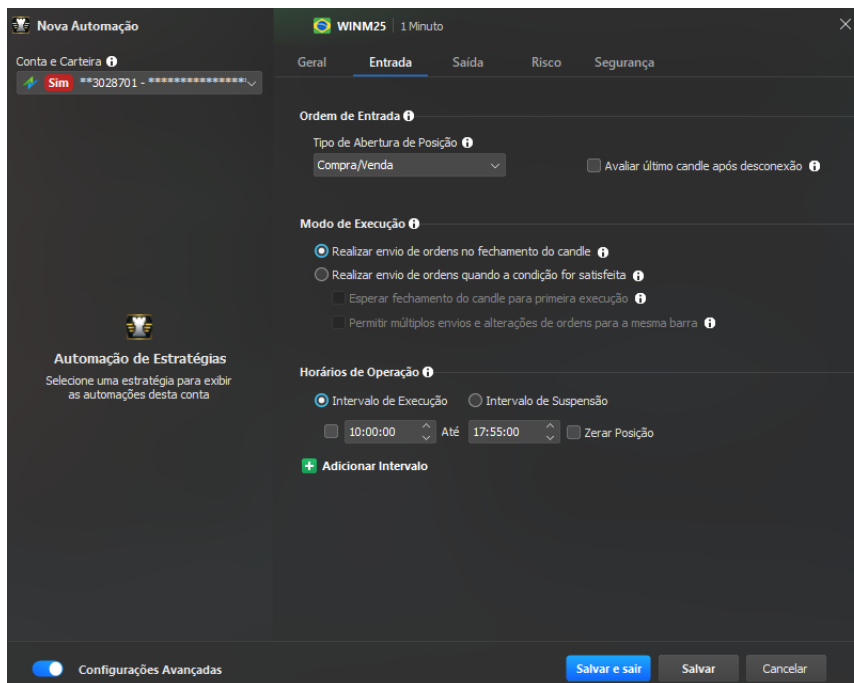
Na primeira etapa, nas configurações obrigatórias, é possível definir a conta, a estratégia de execução, ativo alvo, quantidade por ordem, quantidade máxima da posição e o período que a estratégia usará como base para as análises. A Estratégia de execução é o “cérebro” da automação. Aqui será selecionado aquela estratégia implementada no editor. Ao clicar no botão “Selecionar Estratégia” estarão listadas todas as estratégias de execução presentes na plataforma. Além disso, caso a estratégia selecionada possua parâmetros de entrada, os mesmos serão listados nessa janela para que possam ser editados da melhor maneira para a estratégia específica.



A próxima etapa contém configurações opcionais que podem ser exploradas nos menus acima quando as Configurações Avançadas estiverem ativas (canto inferior esquerdo). Aqui podemos configurar opções de Entrada e Saída, Risco e Segurança. Essas opções estão detalhadas a seguir:

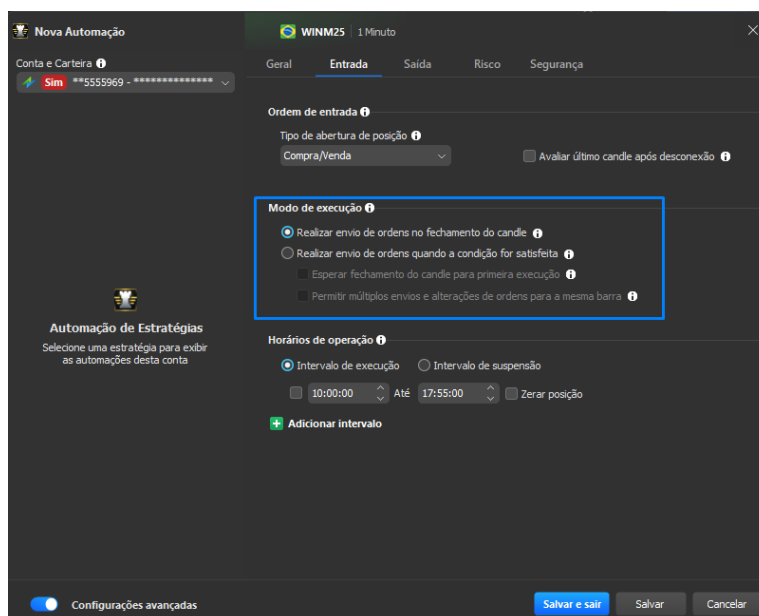
10.3 Opções de entrada

Nesta seção é possível especificar parâmetros que alterem a estratégia de abertura de ordens da automatização. Em ordem de entrada você pode especificar qual vai ser o tipo de abertura de posição, ou seja, se a estratégia irá abrir novas posições apenas de compra, ou apenas de venda ou ambos os lados. É possível também escolher se você deseja confirmar cada ordem de aumento de posição. Neste caso, a confirmação de ordens será mostrada para cada nova ordem da estratégia.



10.4 Modo de Execução

Há dois modos de execução: **"Realizar envio de ordens no fechamento do candle"** e **"Realizar envio de ordens quando a condição for satisfeita"**. A mudança de uma opção para a outra causa grande impacto na maneira como o código da estratégia é interpretado. Abaixo, é detalhado como cada modo se comporta.



Ordens no Fechamento do Candle

Esta é a configuração padrão do modo de execução da automação e é a que traz maior semelhança com as operações realizadas por meio do backtest no Editor de Estratégias. Para ativar este modo selecione a opção **"Realizar envio de ordens no fechamento do candle"**.

A cada fechamento de candle, o código para envio de ordens é reavaliado e se torna disponível para o envio de novas ordens de entrada, ou a alteração de ordens de entrada abertas ou de ordens Cover, caso haja posição. Deste modo, a escolha de tempo gráfico é crucial para sua automação.

Ordens que aumentem a exposição para o mercado e foram enviadas ao final de um candle e que não forem executadas até o final do candle seguinte serão canceladas ou editadas quando o próximo candle finalizar, de acordo com a estratégia do usuário. Uma vez que, apenas sabemos que um candle foi finalizado quando o próximo iniciar em gráficos atemporais, poderá ocorrer a execução de ordens em um candle subsequente caso o cancelamento chegue na bolsa após a execução da mesma.

É importante ressaltar que a automação poderá apresentar divergências em relação ao backtest ao ser comparado com a execução da automação em uma conta de simulação (ordens simuladas) ou conta real (ordens reais), operando ao vivo no mercado real. Isso se deve a diversos fatores, como: Spread, Margem, RLP, Delays na Bolsa, Alta volatilidade de preço de alguns ativos, entre outros.

Essas diferenças acontecem primordialmente em séries de dados atemporais, as quais podem causar a criação de muitos candles em pouco tempo. No backtest, por termos zero delay, as ordens são criadas sempre no candle subsequente, porém no mercado real, onde há delay, por menor que ele seja, ordens podem ser executadas em candles subsequentes já que não houve tempo na criação do cancelamento dessas ordens.

Além disso, há uma segunda opção caso esse modo seja selecionado. Ativando essa função, você terá uma execução mais próxima ao backtest, já que o código executará apenas no fechamento do candle ou com uma atualização de posição. Com a opção desativada, o código é executado diversas vezes no mesmo candle.

Ordens ao Satisfazer Condição

Este é o modo avançado de envio de ordens automatizadas. Ele não é compatível com o backtest da aplicação, uma vez que realiza operações a qualquer momento durante a formação dos candles, o que impede o backtest de simular adequadamente esse modo de execução. Para ativar este modo, selecione a opção **"Realizar envio de ordens quando a condição for satisfeita"**.

Neste modo, o código é constantemente avaliado durante a execução da automação, e as ordens podem ser enviadas a qualquer momento durante a formação do candle, desde que o código satisfaça a condição de envio. Como mostrado na imagem acima, existem duas opções que podem ser configuradas neste modo. Detalhamos cada uma delas a seguir:

A primeira opção, **"Esperar o fechamento do candle para primeira execução"** faz com que a automação aguarde o encerramento do candle atual para começar a executar ordens. Ou seja, somente quando um novo candle for iniciado é que as ordens serão enviadas.

A segunda opção, **"Permitir múltiplos envios e alterações de ordens para a mesma barra"**, quando habilitada, permite que a automação envie e modifique ordens de entrada e saída à medida que o código satisfaça as condições de envio. Isso ocorre respeitando um intervalo mínimo entre as alterações, como medida de proteção contra envio excessivo de ordens (flood). Caso essa opção não esteja selecionada, a automação limitará o envio e a modificação de ordens a apenas uma vez por barra, sempre que a condição definida no código for satisfeita.

A estratégia de atualização das ordens, quando a opção de múltiplos envios para a mesma barra estiver habilitada, seguirá uma equação linear que depende da proximidade das ordens em relação ao preço atual. Ou seja:

- Se as ordens estiverem muito distantes do preço atual, serão atualizadas com menor frequência;
- Se estiverem muito próximas do preço atual, serão atualizadas com maior frequência.

Vale ressaltar que todas as ordens são atualizadas simultaneamente. No entanto, a frequência de atualização é definida com base na ordem mais próxima do preço atual.

Existe um intervalo mínimo de atualização de 10 segundos. Assim, as ordens serão enviadas ou alteradas, no máximo, a cada 10 segundos.

O temporizador de envio de ordens só é atualizado quando ocorre alguma modificação. Ou seja, se o código enviar uma ordem diferente das que já estão apregoadas, o sistema irá verificar quando ocorreu a última modificação e determinar o momento da próxima atualização, liberando então novos envios ou alterações.

Assim como no modo de atualização ao fechamento do candle, as ordens de saída sempre serão atualizadas quando houver um aumento de posição, a fim de realizar o rebalanceamento das ordens de cobertura.

Neste modo de execução, as condições para envio de ordens são avaliadas de forma contínua e em tempo real, permitindo que a entrada ocorra imediatamente assim que a condição especificada for satisfeita como, por exemplo, no cruzamento de médias móveis ou no rompimento de um nível de preço pré-definido. Essa abordagem proporciona maior agilidade e precisão na execução, pois as ordens podem ser enviadas no exato momento em que as condições são atendidas.

Lembre-se de que este modo não é compatível com o backtest de estratégias, pois depende de dados em tempo real, o que inviabiliza a previsão precisa do comportamento da automação. Para testar o desempenho de sua estratégia neste modo, é necessário utilizar o recurso de replay de mercado ou realizar os testes diretamente no pregão, utilizando uma conta de simulação.

10.5 Horários de Operação

Nesta seção é possível configurar os períodos em que a estratégia poderá ou não enviar ordens, conforme as opções disponíveis:

- **Intervalo de Execução:** define o período em que a estratégia estará habilitada para enviar ordens. Após o horário final do intervalo, a estratégia não será pausada, mas deixará de enviar novas ordens automaticamente. Caso deseje, é possível marcar a opção Zerar Posição para que, ao final do intervalo, todas as posições abertas sejam encerradas automaticamente.
- **Intervalo de Suspensão:** ao invés de definir quando a automação irá operar, este modo permite configurar períodos em que a estratégia não deverá operar. Durante esse intervalo, nenhuma nova ordem será enviada. Assim como no modo anterior, é possível habilitar a opção Zerar Posição para forçar o encerramento de posições abertas no início do intervalo de suspensão.

Essas configurações permitem maior controle sobre a atuação da estratégia durante o pregão, possibilitando, por exemplo, evitar operações em horários de baixa liquidez ou eventos de maior volatilidade.

10.6 Opções de saída

Nesta outra etapa da configuração, é possível alterar configurações de saída de sua automação.

Aqui é possível configurar uma estratégia OCO que será aberta junto com suas ordens, mas atenção, caso você já tenha ordens de saída implementadas em sua estratégia de negociação (ordens de cobertura), estas não serão mais executadas já que as ordens OCO terão preferência.



Também é possível configurar nesta seção a zeragem automática por horário. A plataforma precisa estar aberta para que a zeragem aconteça, assim como para que a automação funcione.

A captura de tela mostra a aba 'Saída' de uma interface de configuração. No topo, há uma barra de navegação com as opções 'Geral', 'Entrada', 'Saída' (destacada), 'Risco' e 'Segurança'. Abaixo, o título 'Saída por Estratégia OCO' é seguido por um menu suspenso com o valor '<Nenhuma>' e um ícone de engrenagem. Um texto explicativo indica que a estratégia não possui regra de saída e sugere a seleção de uma estratégia OCO. Abaixo, há uma opção 'Zerar às' com um ícone de informação e um campo de tempo configurado para '17:55:00'.

10.7 Risco

Seguindo as configurações temos as configurações de risco da automação. Na seção Pausar e Zerar ao Alcançar é possível configurar parâmetros que irão pausar e encerrar as posições abertas de acordo com um objetivo de ganho, ou um limite de perda. Caso você não deseje encerrar a posição quando atingir um objetivo de ganho seleciona a opção "Não Zerar ao Atingir Objetivo de Ganho".

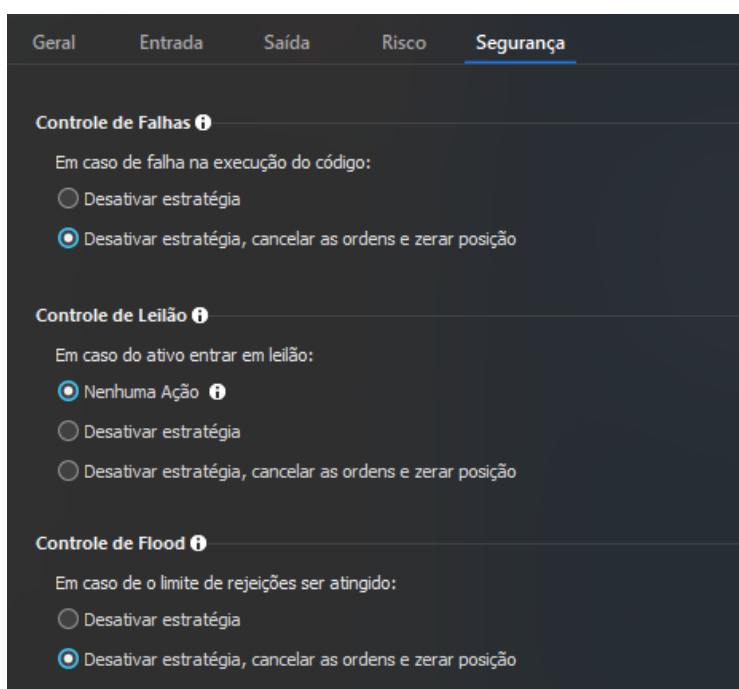
Já a seção Pausar ao Alcançar possibilita que a automação seja pausada quando alcançados os números de perdas consecutivas e máximo de operações configurados.

Aqui também disponibilizamos um atalho para que você possa configurar o risco global da carteira/conta no gerenciador de risco da plataforma.

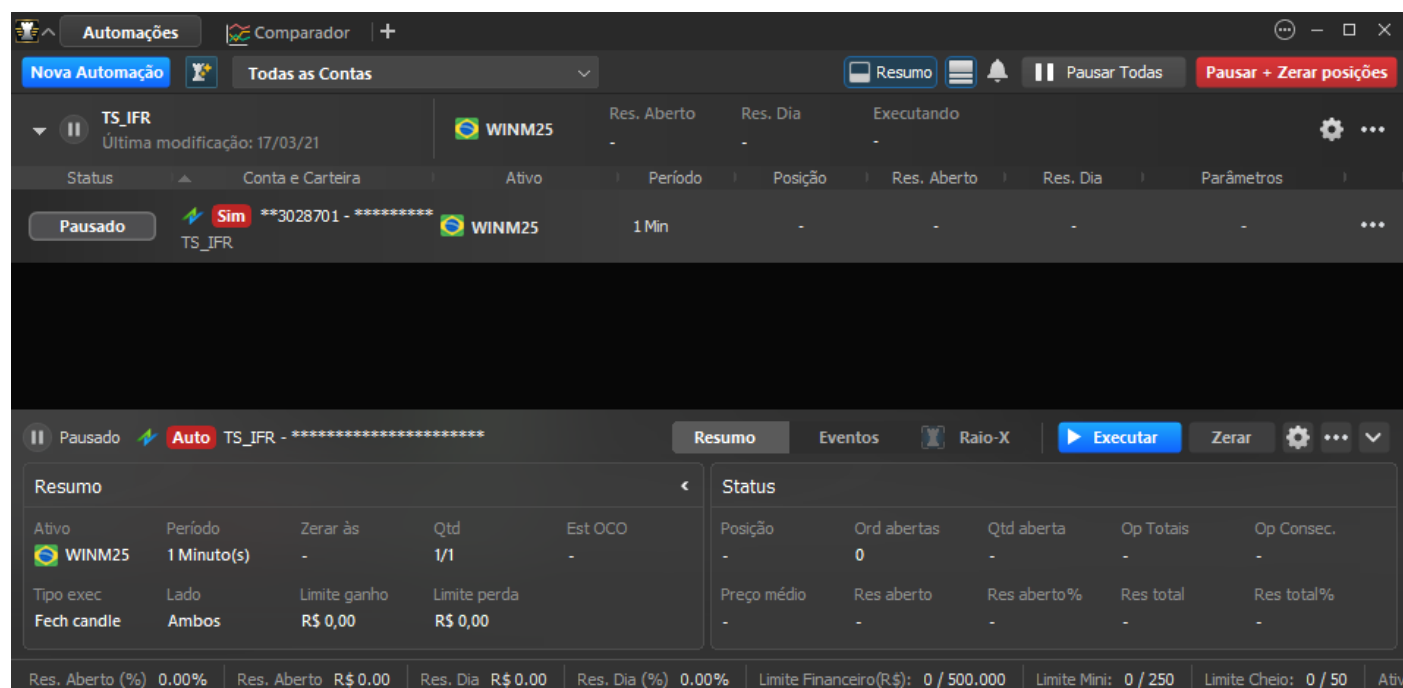
A captura de tela mostra a aba 'Risco' da interface de configuração. A barra de navegação no topo permanece a mesma. A seção 'Pausar e Zerar ao alcançar' contém duas opções: 'Objetivo de Ganho Diário' e 'Limite de Perda Diário', ambas com campos de entrada configurados para '0,00' e ícones de moeda (R\$). Há também uma opção 'Não Zerar ao Atingir Objetivo de Ganho'. A seção 'Pausar ao alcançar' contém 'Perdas Consecutivas' e 'Número de Operações', ambas com campos de entrada configurados para '1'. A seção 'Pausar ao atingir limites da conta' mostra campos para 'Perda da conta for maior que' e 'Ganho da conta for maior que', ambos com o valor '-'. Um botão 'Editar Limites' está disponível ao lado.

10.8 Segurança

Por fim, na seção Segurança, temos algumas configurações para definir o comportamento da estratégia em caso ocorra um erro de execução no código da mesma ou eventuais mudanças do mercado, como a entrada do ativo em leilão.

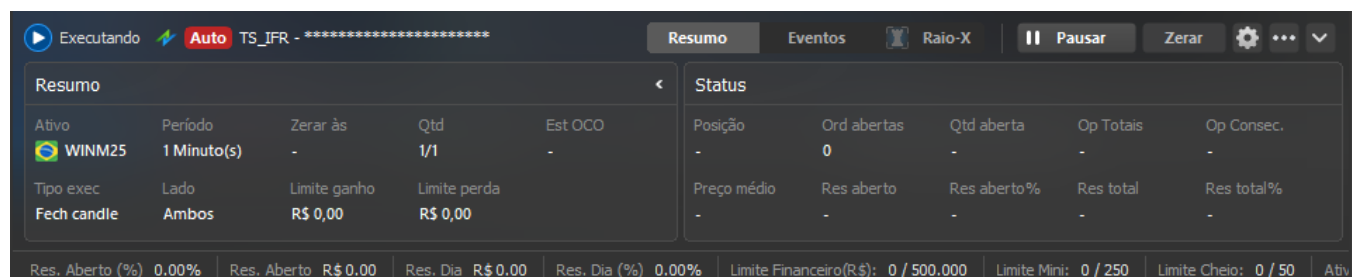


Uma vez criada a automação, você pode visualizá-la na janela Automação de Estratégias:



10.9 Acompanhamento

Para acompanhar a sua automação você pode visualizar um resumo rápido das principais informações na lista de automações da janela Automação de Estratégias. Nela você pode visualizar o status, conta e carteira em que a automação está vinculada, ativo, nome da estratégia, período gráfico e os resultados relacionados à execução. Você também pode visualizar nesta janela um resumo das estatísticas relacionadas a essa estratégia que contém mais detalhes da estratégia:



Executando **Auto** TS_IFR - *****

Resumo

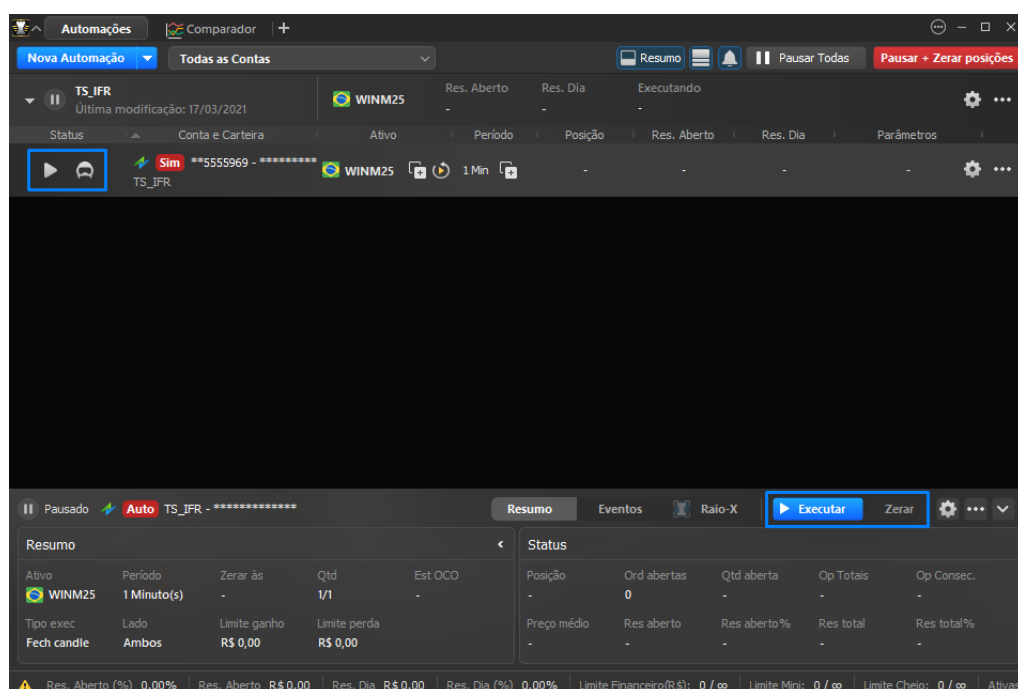
Ativo	Período	Zerar às	Qtd	Est OCO
WINM25	1 Minuto(s)	-	1/1	-
Tipo exec	Lado	Limite ganho	Limite perda	
Fech candle	Ambos	R\$ 0,00	R\$ 0,00	

Status

Posição	Ord abertas	Qtd aberta	Op Totais	Op Consec.
-	0	-	-	-
Preço médio	Res aberto	Res aberto%	Res total	Res total%
-	-	-	-	-

Res. Aberto (%) 0.00% | Res. Aberto R\$ 0.00 | Res. Dia R\$ 0.00 | Res. Dia (%) 0.00% | Limite Financeiro(R\$): 0 / 500.000 | Limite Mini: 0 / 250 | Limite Cheio: 0 / 50 | Ativ

Para ligar/pausar uma automação de maneira individual pode-se utilizar o painel, e clicar em executar:



Automações Comparador +

Nova Automação Todas as Contas

TS_IFR Última modificação: 17/03/2021

WINM25 Res. Aberto Res. Dia Executando

TS_IFR **5555969 - *****

WINM25 1 Min

Resumo **Executar** Zerar

Resumo

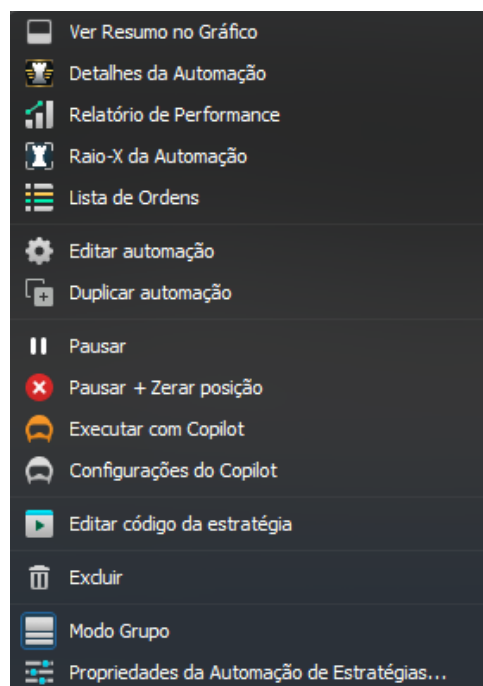
Ativo	Período	Zerar às	Qtd	Est OCO
WINM25	1 Minuto(s)	-	1/1	-
Tipo exec	Lado	Limite ganho	Limite perda	
Fech candle	Ambos	R\$ 0,00	R\$ 0,00	

Status

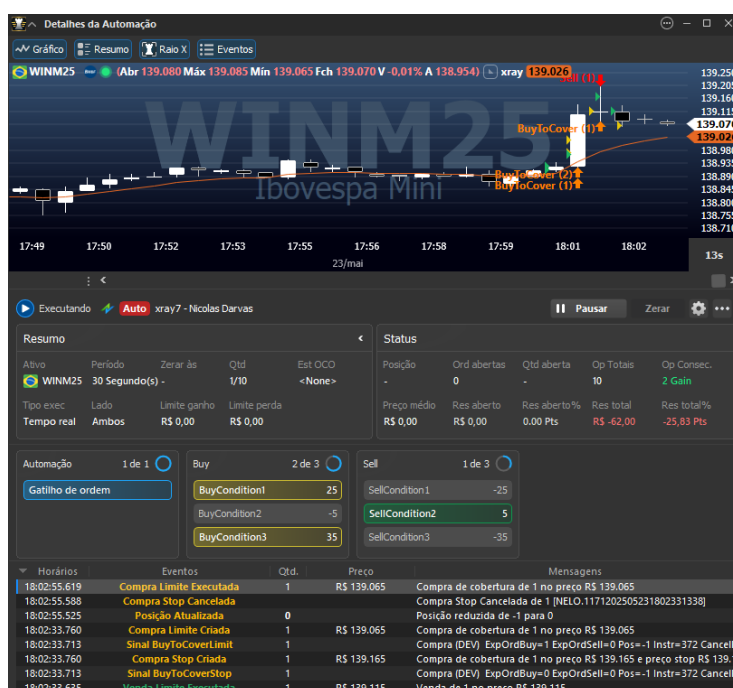
Posição	Ord abertas	Qtd aberta	Op Totais	Op Consec.
-	0	-	-	-
Preço médio	Res aberto	Res aberto%	Res total	Res total%
-	-	-	-	-

Res. Aberto (%) 0.00% | Res. Aberto R\$ 0.00 | Res. Dia R\$ 0.00 | Res. Dia (%) 0.00% | Limite Financeiro(R\$): 0 / ∞ | Limite Mini: 0 / ∞ | Limite Cheio: 0 / ∞ | Ativas:

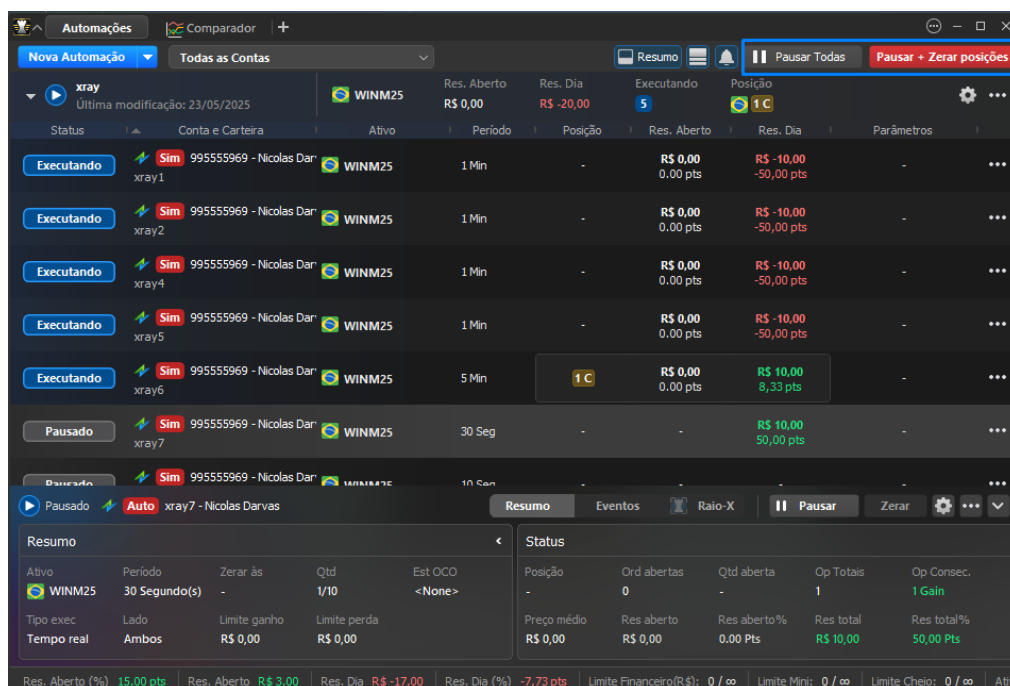
Através do menu de contexto também é possível zerar a posição de uma estratégia pausada ou Pausar + Zerar Posição de uma estratégia em execução. Podemos também editar o código da automação no Editor de Estratégias, Excluir a automação que irá excluir também a carteira atrelada a ela. Para realizar edições na sua automação, você pode, dentro do interface de controle, clicar com botão direito na estratégia desejada e ir em "Editar Automação" ou clicar diretamente na engrenagem. Lembrando que caso a sua automação esteja ligada, ao realizar alguma edição ela será pausada.



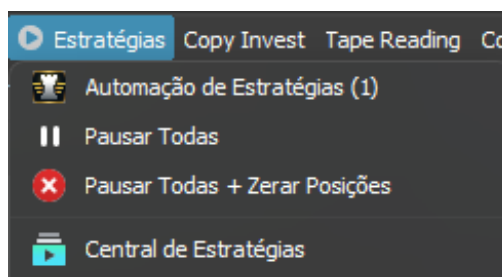
Para acompanhar os detalhes do que a execução está executando, vá em Detalhes da Automação. Nesta janela você poderá acompanhar tanto as ordens e sinais que a estratégia está gerando, como também acompanhar a performance e resultados da estratégia mais detalhadamente.



Na janela de Automações é possível utilizar os botões no canto superior direito da interface de controle para pausar todas as automações em execução ou para pausar e zerar todas as posições em aberto:



Caso você possua alguma automação ligada será possível identificar mesmo com a janela de estratégias automatizadas fechadas, pois no menu superior da sua Plataforma será destacado em azul o menu de Estratégias.



Neste menu você será informado quantas estratégias você possui em execução e permitirá que você tome ações rápidas através do menu como pausar tudo ou pausar e zerar tudo.

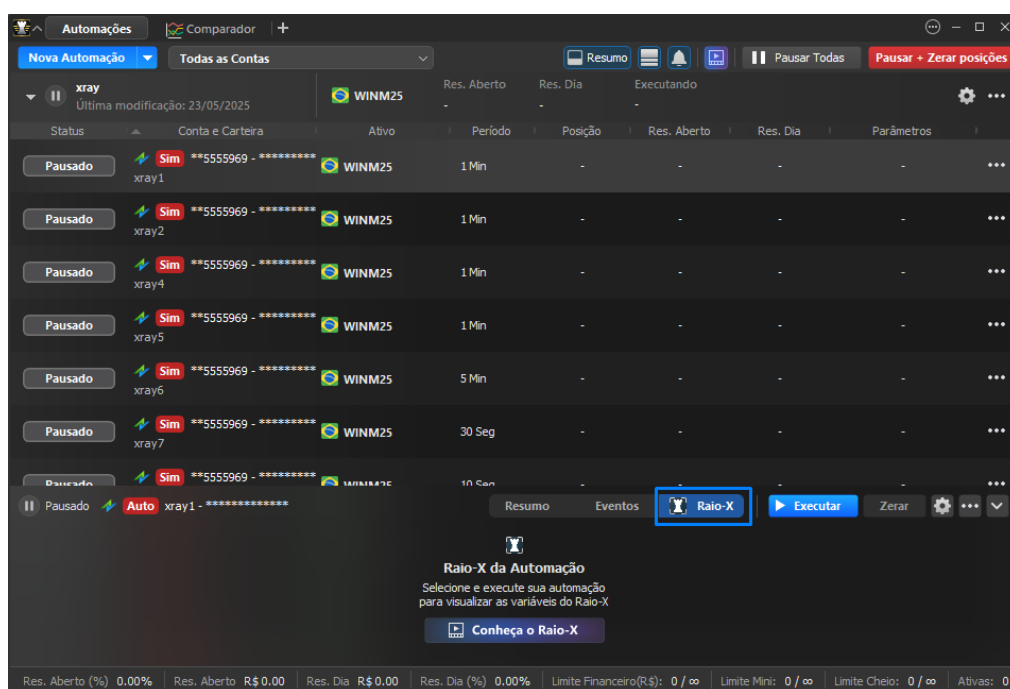
10.10 Raio-X

Disponível exclusivamente nos produtos com o módulo **Automações Premium** (ou superior), a função XRay é um dos principais recursos introduzidos na **Automação 2.0**. Ela foi desenvolvida para proporcionar maior visibilidade e controle sobre as condições e variáveis envolvidas na execução de estratégias automatizadas.

Objetivo

A função XRay permite:

- Acompanhar o estado das condições utilizadas na lógica da estratégia.
- Visualizar os valores atuais das variáveis envolvidas na tomada de decisão.
- Agrupar visualmente as condições no Gerenciador de Automações, utilizando cores e nomes distintos para facilitar a leitura (ex: amarelo para compra, verde para venda).

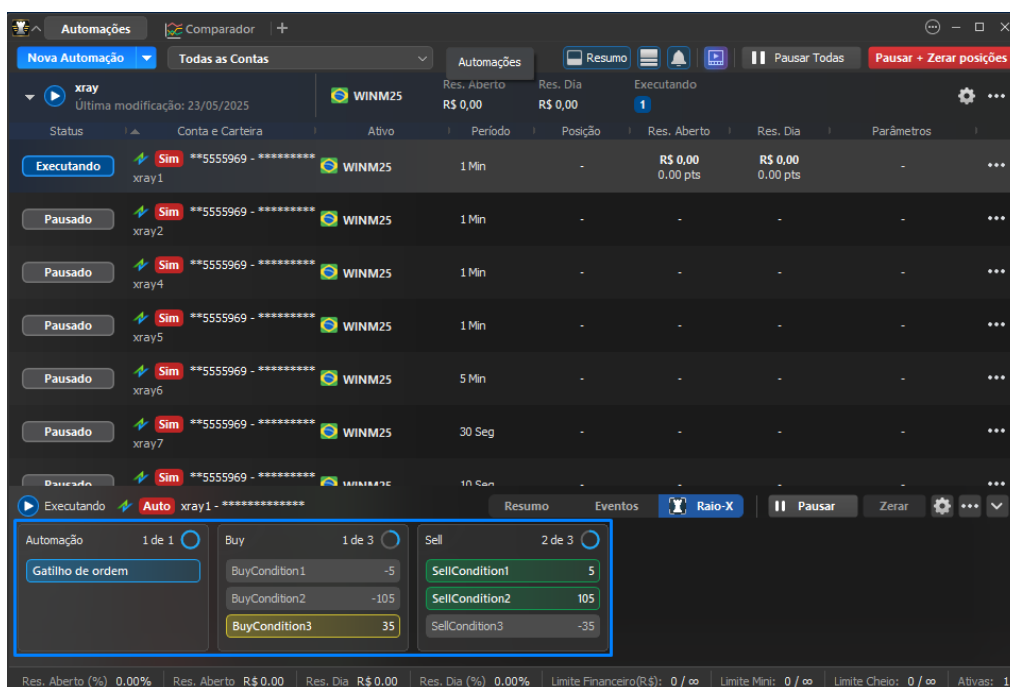


Para que as informações apareçam corretamente no Gerenciador de Automações, é necessário inserir a função XRay no código da estratégia. A chamada da função deve indicar o nome da condição, seu valor lógico, o grupo ao qual pertence, o valor associado e a cor correspondente.

Visualização no Gerenciador de Automações

Ao executar uma automação com a função XRay no código, o botão **Raio-X** ficará disponível no Gerenciador de Automações. Através dele, é possível:

- Verificar quais condições foram satisfeitas durante a execução.
- Observar os valores das variáveis associadas a cada condição.
- Distinguir entre diferentes grupos de condições (como compra e venda) por meio de cores e identificadores.



Diferenciais do recurso

- Facilita o entendimento do comportamento da automação durante a execução.
- Permite identificar rapidamente falhas lógicas ou inconsistências nas condições.
- Atua como uma ferramenta de depuração visual para aprimoramento das estratégias.

Para mais informações sobre a função XRay, consulte a seção **14.33 - Função XRay** no capítulo **Funções de Back-testing**.

10.11 Regras de Execução

As automações de estratégias funcionam com as mesmas regras que são utilizadas no backtest das estratégias de execução. Para entender melhor o que cada função faz relacionada à execução de ordens, recomendamos primeiramente a leitura da seção **14 - Funções de Back-Testing**. Aqui detalhamos as regras por trás dos mecanismos de automação para facilitar o entendimento do usuário na hora de implementar uma estratégia de execução já pensando na sua automação.

Reprocessamento do candle em aumentos de posição

Caso houver execução de alguma ordem (aumento de posição), o código é reprocessado para a criação de eventuais ordens de cobertura da operação (ordens ToCover). A reexecução da estratégia para posicionamento de ordens de cobertura acontece quando ordens forem executadas, ou seja, podem ocorrer no meio de um candle para não deixar o usuário exposto a riscos do mercado. Nesse reprocessamento, por mais que o código faça a chamada de ordens que poderiam aumentar a exposição, essas ordens não são enviadas. Mais detalhes sobre ordens de cobertura podem ser vistos a seguir.

Ordens para aumento de posição (Buy e Sell)

É possível enviar ordens Buy e Sell tanto quando você estiver comprado ou vendido, porém quando uma ordem Buy for enviada enquanto você estiver em uma posição vendida, ela será tratada como uma ordem de cobertura automaticamente. O mesmo acontece para uma ordem Sell, quando você estiver uma posição comprada, ela será tratada como uma ordem de cobertura. Caso contrário, elas apenas aumentaram a sua posição até o limite definido na configuração da estratégia.

Ordens conflitantes são automaticamente gerenciadas pelo automatizador, ou seja, quando for enviada uma ordem BuyToCover e você estiver em uma posição comprada ou zerada, ela será ignorada, da mesma maneira se for enviada uma ordem SellToCover e você estiver em uma posição vendida ou zerada, ela também será ignorada.

Ordens de cobertura de posição (BuyToCover e SellToCover)

Ordens de cobertura, ou ordens ToCover, nunca irão inverter a sua posição; elas garantem que a ordem contrária vai respeitar sempre a posição da operação. Recomendamos que o programador utilize sempre ordens de saída de posição como ordens ToCover explicitamente no código para garantir que o operacional que o usuário está programando está correto. Para isso, as ordens ToCover são sempre enviadas como ordens OCO, logo você não precisa se preocupar em gerenciar e cancelar eventuais ordens de cobertura que poderiam ficar abertas após a execução de apenas uma das pernas de saída.

Ordens de cobertura são enviadas ou atualizadas a toda mudança de candle, cabe ao usuário gerenciar a quantidade de cada ordem caso você esteja posicionado em mais de um lote para cobrir corretamente a exposição ao mercado. Na finalização do candle, caso o código indique um envio de um mesmo número de ordens de cobertura, será realizada uma edição da OCO para os novos valores de preço correspondentes. Por outro lado, caso o envio aumente ou diminua o número de ordens para aquele candle, a OCO aberta será cancelada, e será enviada uma nova OCO com as saídas definidas pelo código.

Ordens OCO

Caso o usuário configure uma ordem OCO pela configuração da automação, as ordens de cobertura definidas pela estratégia serão desconsideradas, já que todas ordens já vão possuir essa cobertura natural da ordem OCO.



Indicadores que atualizam valores anteriores

Se forem utilizados indicadores que alterem seus valores em candles antigos (candles já finalizados), o valor do indicador que será considerado será o do momento em que o candle finalizar, desse modo caso não houver a entrada da execução na virada do candle, não será feito envio de ordens para aquele candle no futuro, mesmo que algum indicador altere seu valor no passado e troque a decisão de envio de ordem. Alguns exemplos de indicadores que podem causar esse comportamento são indicadores que tem apenas um valor durante todo dia, por exemplo: **Preço Ask e Bid** e também indicadores que decidem apenas depois de alguns candles, como por exemplo o **Detector de topos e fundos** ou a linha Chikou Span (2) do indicador **IchimokuCloud**, entre outros.

Cálculo Automático do Preço Limite (a partir do Preço Stop)

A NTSL possui suporte para o cálculo automático do **preço limite** em ordens do tipo Stop, quando esse valor não é explicitamente definido no código. Esse mecanismo garante que as ordens sejam enviadas com parâmetros válidos e aceitos pela bolsa ou corretora, prevenindo rejeições por ultrapassarem as bandas de preço permitidas. Esse processo define a diferença máxima entre o preço stop e o preço limite, com base nas características do ativo e nas regras do ambiente de negociação. As regras aplicadas são as seguintes:

- **Ativos com valor mínimo de tick configurado** (variação mínima permitida entre ofertas):
 - **Criptomoedas:** diferença igual a **1 tick**.
 - **Contratos ISDI1** (ex.: swaps, compromissadas de índice): diferença igual a **7 ticks**.
 - **Opções:** diferença igual a **20 ticks**.
 - **Contratos futuros da BMF:**
 - * **Criptoativos:** diferença igual a **100 ticks**.
 - * **Demais contratos futuros da BMF:** diferença igual a **30 ticks**.
- **Ações negociadas na B3 (mercado Bovespa):** limite de **15% sobre o preço de referência**, respeitando a margem operacional dentro do limite de **20%** estabelecido pela bolsa.
- **Ativos operados por integração externa:** limite de **1% sobre o preço**.
- **Demais ativos não contemplados pelas regras anteriores:** limite de **5% sobre o preço**.

Caso o ativo **não possua tick mínimo configurado** (ou seja, o valor seja igual a zero), será aplicado automaticamente um **valor padrão de 1% sobre o preço de referência** como fallback de segurança.

Esse comportamento assegura que, mesmo na ausência de um valor de limite explicitamente definido na ordem, o sistema utilizará parâmetros compatíveis com os requisitos operacionais do mercado, garantindo o envio bem-sucedido da ordem.

10.12 Utilização da Função ConsoleLog na Automação

A função `ConsoleLog` pode ser utilizada dentro das automações em NTSL para registrar eventos em tempo real diretamente nos **eventos da automação**.

Essa funcionalidade é especialmente útil para depuração de estratégias, acompanhamento de variáveis internas e verificação de condições durante a execução.

Como funciona

- As mensagens registradas por `ConsoleLog` ficam visíveis na **janela de Detalhes da Automação**.
- O comportamento da emissão dos logs varia conforme o **modo de execução** da automação:
 - **Modo Candle a Candle:** os logs são enviados a cada **novo candle** e a cada **reavaliação de candle**.
 - **Modo Ao Satisfazer Condição:** os logs são enviados a cada **5 segundos** ou a cada **reavaliação de candle** por mudança de posição.

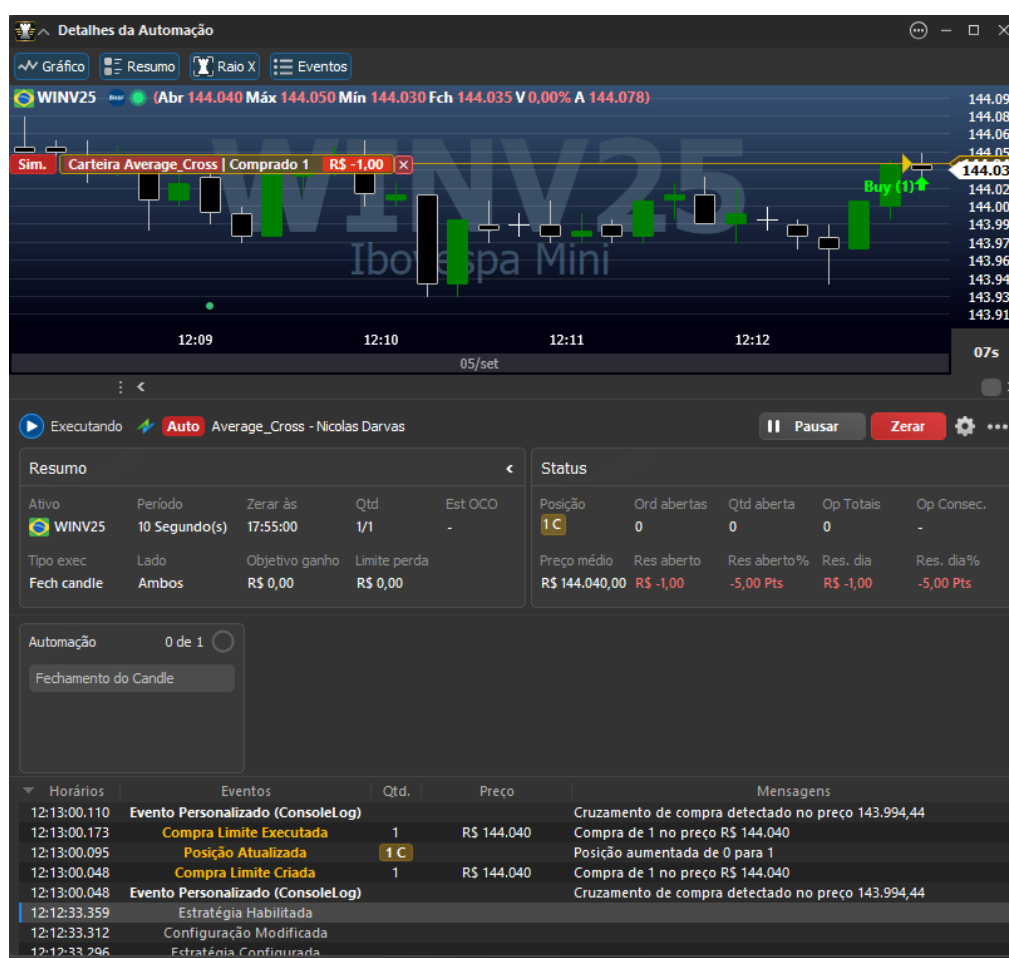
Exemplo de Uso

```

1  var
2      sAvgFast, sAvgSlow, sPrevAvgFast, sPrevAvgSlow : Float;
3  begin
4      sAvgFast := Media(9, Close);
5      sAvgSlow := Media(21, Close);
6      sPrevAvgFast := sAvgFast[1];
7      sPrevAvgSlow := sAvgSlow[1];
8      if (sPrevAvgFast < sPrevAvgSlow) and (sAvgFast > sAvgSlow) then
9          begin
10             ConsoleLog("Cruzamento de compra detectado no preço " + sPrevAvgFast);
11             buyLimit(Close, 1);
12          end
13      else if (sPrevAvgFast > sPrevAvgSlow) and (sAvgFast < sAvgSlow) then
14          begin
15             ConsoleLog("Cruzamento de venda detectado no preço " + sPrevAvgSlow);
16             SellShortLimit(Close, 1);
17          end;
18  end;

```

No exemplo acima, sempre que ocorrer um cruzamento entre as médias móveis, a automação registrará um **log informativo** na **janela de Detalhes da Automação**, permitindo acompanhar em tempo real os eventos da estratégia.



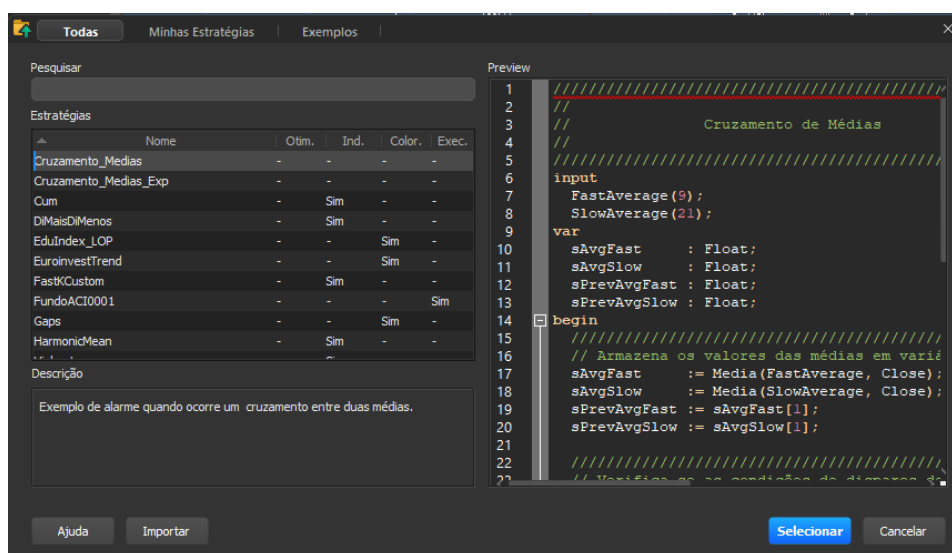
11 Estratégias

11.1 Abrir Estratégias

Na opção de "Abrir Estratégias", o usuário terá acesso a três abas, elas são:

- **Todas:** O usuário poderá ver todas as estratégias dentro do sua plataforma;
- **Minhas Estratégias:** O usuário irá filtrar para somente exibir todas as estratégias criadas por ele dentro da plataforma;
- **Exemplos:** O usuário irá filtrar para exibir exemplos de estratégias que já vem como padrão na plataforma.

Além das abas, o usuário também poderá pré-visualizar o seu código de estratégia para confirmar informações.



11.2 Central de Estratégias

Na aba Minhas Estratégias, localizada na Central de Estratégias, o usuário pode visualizar e gerenciar todas as estratégias criadas.

Para editar uma estratégia, basta clicar no botão "Editar" localizado ao lado direito da linha correspondente à estratégia. Ao clicar nesse botão, a estratégia será aberta automaticamente no Editor de Estratégias para edição.

Além disso, é possível acessar outras opções de gerenciamento através do menu de contexto. Para abrir esse menu, o usuário deve: clicar com o botão direito do mouse sobre a estratégia desejada, ou clicar no ícone de três pontos (...) localizado ao final da linha da estratégia.

As opções disponíveis no menu de contexto incluem:

- **Abrir no editor:** carrega a estratégia no Editor de Estratégias.
- **Compartilhar Estratégia:** permite compartilhar com outros usuários.
- **Anunciar Estratégia:** publica a estratégia na loja da plataforma.
- **Renomear Estratégia:** altera o nome da estratégia.
- **Excluir Estratégia:** remove a estratégia de forma permanente.

Essas funcionalidades tornam a gestão das estratégias mais acessível e organizada.

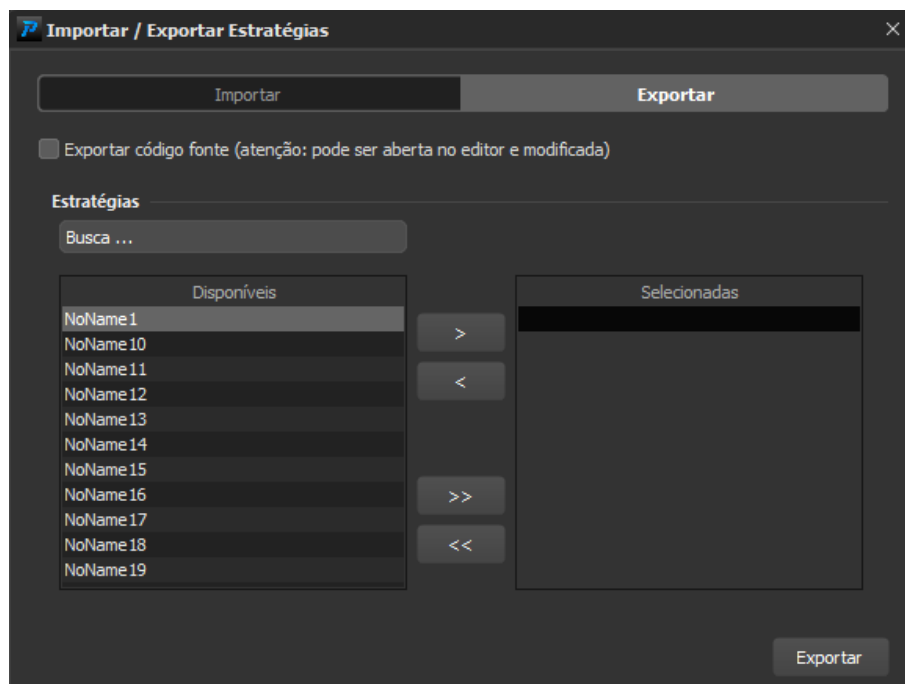


Estratégia	Código fonte	Otimização	Indicador	Coloração	Execução	Alarme	Screening	Texto	Última modificação
NoName1	✓	📄	✓	📄	📄	📄	📄	📄	02/05/2025
NoName10	✓	📄	✓	📄	📄	📄	📄	📄	02/05/2025
NoName11	✓	📄	✓	📄	📄	📄	📄	📄	02/05/2025
NoName12	✓	📄	✓	📄	📄	📄	📄	📄	02/05/2025
NoName13	✓	📄	✓	📄	📄	📄	📄	📄	02/05/2025
NoName14	✓	📄	✓	📄	📄	📄	📄	📄	02/05/2025
NoName15	✓	📄	✓	📄	📄	📄	📄	📄	02/05/2025
NoName16	✓	📄	✓	📄	📄	📄	📄	📄	02/05/2025
NoName17	✓	📄	✓	📄	📄	📄	📄	📄	02/05/2025
NoName18	✓	📄	✓	📄	📄	📄	📄	📄	02/05/2025
NoName19	✓	📄	✓	📄	📄	📄	📄	📄	02/05/2025
NoName2	✓	📄	✓	📄	📄	📄	📄	📄	02/05/2025
NoName20	✓	📄	✓	📄	📄	📄	📄	📄	02/05/2025
NoName3	✓	📄	✓	📄	📄	📄	📄	📄	02/05/2025

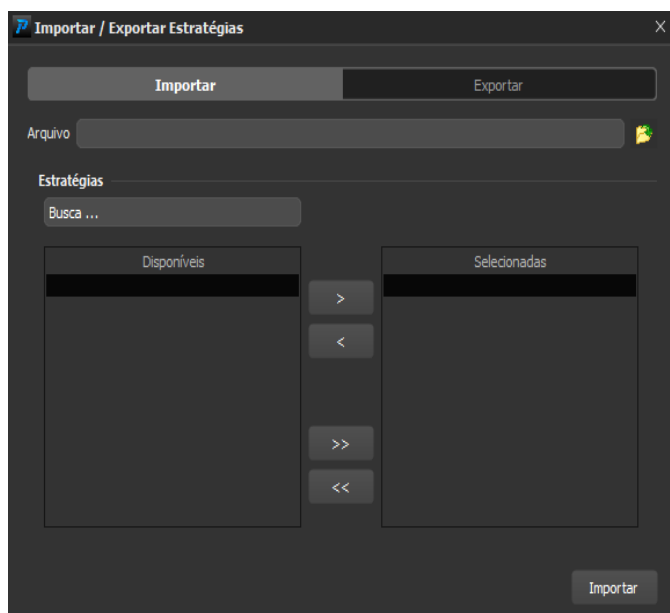
11.3 Exportar / Importar Estratégias

Nesta funcionalidade, permite ao usuário exportar as estratégias criadas por eles para que possam ser importadas novamente.

O usuário também tem a possibilidade de querer exportar o código fonte da estratégia ou apenas o arquivo executável.



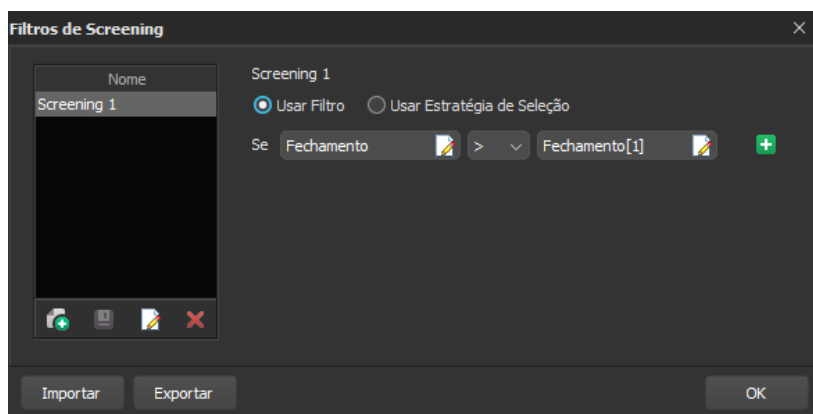
Na importação, o usuário tem a funcionalidade de escolher quais estratégias serão carregadas e adicionadas junto à sua plataforma Nelogica.



11.4 Screening

A funcionalidade de Screening mostra, de forma visual, os ativos que se encontram na base de dados e que satisfazem as condições da estratégia.

No filtro, ao clicar no botão "+", será criada uma condição para que, se Condição 1 e Condição 2 sejam verdadeiras, irá mostrar o ativo dentro da aba selecionada.



Caso o usuário deseje utilizar outras informações, ele irá poder clicar no botão "Mais" que se encontra ao lado da variável para selecionar outras condições.

Ao clicar no botão "Aplicar" a estratégia criada é aplicada à lista e irá mostrar os ativos que satisfazem a condição.

Screening - Screening 1

Filtro: Screening 1 Periodicidade: 1 Dia Lista de Ativos: IBRA

Aplicar

Ativo	Último	Hora	Variação	Máximo	Mínimo	Fechamento Ant	Abertura
OPCT3	5,61	17:07:00	3,51%	5,61	5,39	5,42	5,42
MLAS3	1,24	17:07:00	1,64%	1,26	1,22	1,22	1,23
CBAV3	4,03	17:07:50	2,54%	4,03	3,87	3,93	3,97
SRNA3	10,13	17:07:00	3,37%	10,16	9,62	9,80	9,74
SLCE3	20,03	18:39:48	1,21%	20,15	19,75	19,79	19,91
RCSL3	2,63	17:05:00	2,73%	2,70	2,58	2,56	2,58
CSAN3	8,01	18:39:53	3,09%	8,11	7,79	7,77	8,02
JALL3	4,09	17:07:00	2,25%	4,09	3,97	4,00	4,00
RAIZ4	1,85	18:37:29	2,21%	1,87	1,81	1,81	1,82
CASH3	6,99	18:39:23	8,88%	7,25	6,53	6,42	6,60
EVEN3	5,81	18:29:39	1,04%	5,81	5,72	5,75	5,74
RECV3	13,22	18:38:19	2,96%	13,31	12,87	12,84	12,97
NTCO3	9,54	18:35:47	0,32%	9,63	9,35	9,51	9,58
CSNA3	9,35	18:38:43	0,54%	9,48	9,17	9,30	9,40
EMBR3	66,12	18:39:19	1,85%	66,66	63,99	64,92	64,73
LOGG3	21,27	18:25:08	1,29%	21,37	20,80	21,00	21,00
SMT03	19,63	18:37:17	0,62%	19,71	19,15	19,51	19,44
MRFG3	21,59	18:39:44	0,51%	21,59	20,56	21,48	21,46
VAMO3	5,05	18:34:35	2,64%	5,07	4,86	4,92	4,95

Screening 1 + 42 Ativos

11.5 Inserir Regra de Coloração

A funcionalidade de inserir regra de coloração permite ao usuário colorir o gráfico de acordo como ele determinou os parâmetros nos gráficos dos ativos.

As regras de coloração serão feitas seguindo a ideologia de um indicador ou seja, respeitando o layout atual da janela.



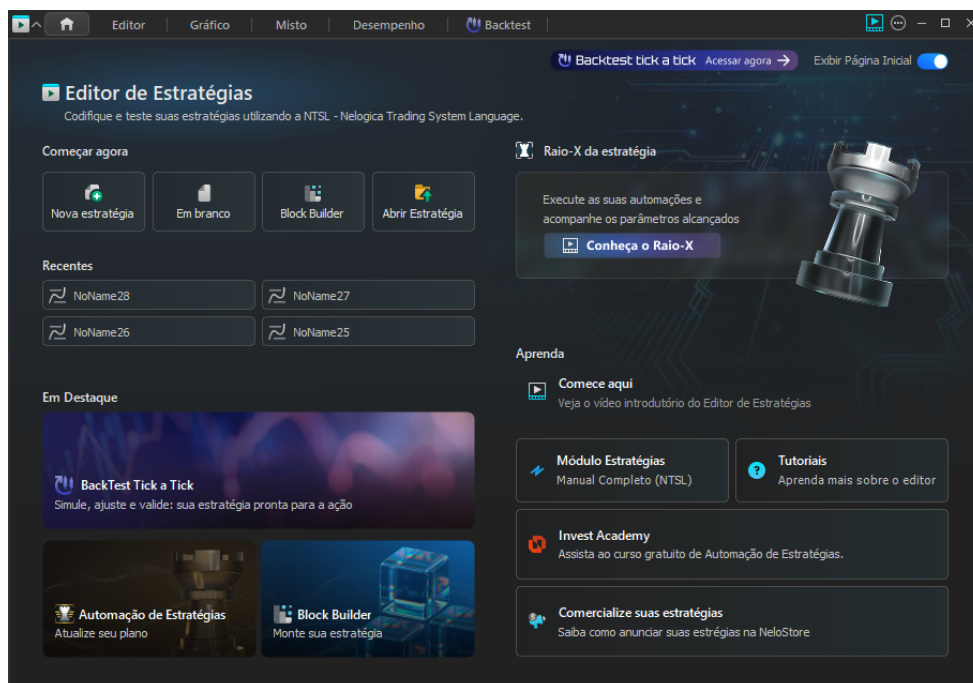
11.6 Editor de Estratégias

A janela de Editor de estratégias é onde o usuário poderá criar suas próprias estratégias, juntamente com a funcionalidade de visualizar diretamente no gráfico a estratégia criada.

O Editor de Estratégias possui três abas:

- **Editor:** Onde o usuário escreve a estratégia seguindo as instruções da linguagem NTSL.

- **Gráfico:** Onde o usuário visualiza a estratégia após executada diretamente no gráfico;
- **Misto:** É onde o usuário visualiza as informações da aba Editor e a aba Gráfico em uma só, onde a cada vez que ele execute o código, já irá aparecer diretamente no gráfico.
- **Desempenho:** Ao executar uma estratégia de execução pelo editor, o usuário poderá visualizar o desempenho do relatório de performance.
- **Backtest:** Onde o usuário pode escolher entre modo de execução OHLC ou Tick a Tick e visualizar a estratégia após executada no gráfico.



Dentro do Editor de estratégias o usuário irá possuir as seguintes opções:

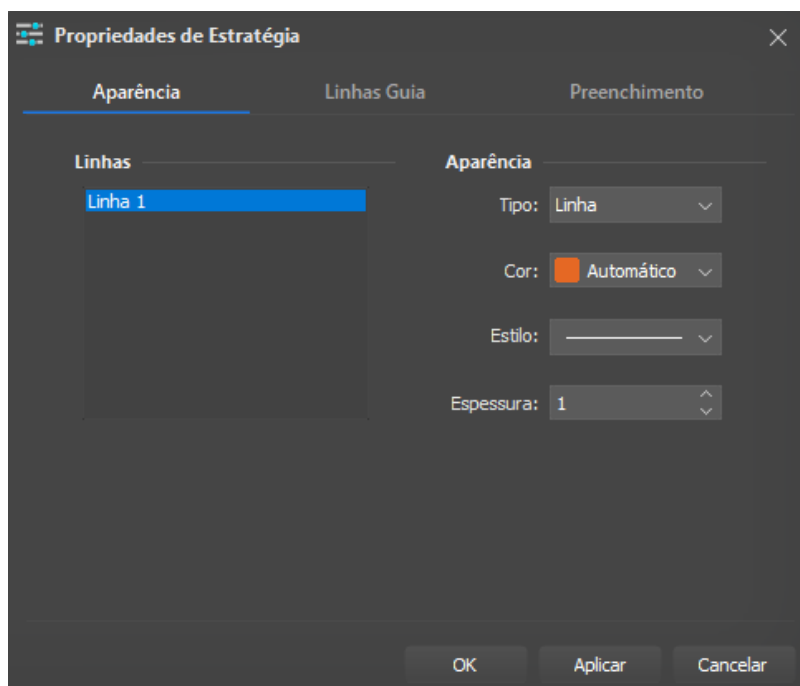
- **Nova Estratégia:** Onde o usuário irá poder criar uma nova estratégia;
- **Abrir Estratégia:** Permite ao usuário abrir estratégias já criadas e editá-las;
- **Fechar:** Fecha a aba da estratégia atual;
- **Salvar:** Salva a estratégia atual;
- **Salvar Como:** Salva a estratégia atual, podendo ser adicionado uma descrição da mesma;
- **Importar/Exportar:** Abre a janela de importar e exportar estratégia;
- **Verificar Sintaxe:** Realiza a leitura do código verificando se há erros e a transforma em uma estratégia executável;
- **Próximo:** Mostra passo-a-passo o que o código da estratégia está realizando e mostrando os valores naquele momento;
- **Entrar:** Semelhante ao Trace, mostra passo a passo o que a estratégia está realizando no momento de criação, porém, quando há funções no código ele irá abrir a função para mostrar ao usuário que a função está executando;
- **Executar:** Após apertar o botão "Compilar", o botão executar irá executar a estratégia criada e a mostra no gráfico;
- **Parar:** Tem como funcionalidade parar a estratégia para que o usuário pare a execução da estratégia criada;
- **Formatação automática:** Executa a formatação automática do código;

- **Aumentar base:** Expande a base do ativo para a execução da estratégia;
- **Automatizar estratégia:** Cria uma nova automação com a estratégia selecionada no editor;
- **Otimizar estratégia:** Abre a otimização de parâmetros com a estratégia selecionada no editor;
- **Propriedades:** Tem como funcionalidade mostrar as propriedades que irão constituir a estratégia, como desenho no gráfico, linhas guias e escala;

11.7 Propriedades do Editor de Estratégias

Nas propriedades do Editor de estratégias, o usuário irá poder utilizar valores e informações adicionais no momento de criação da estratégia:

- **Parâmetros:** Permite ao usuário utilizar estratégias já criadas como parâmetros para uma nova estratégia junto com o valor desejado para a mesma;
- **Aparência:** Permite ao usuário determinar se deseja que a estratégia seja mostrada em linha ou em histograma;
- **Linhas Guia:** Permite ao usuário criar linhas para se basear como exemplos de linha de suporte e resistência;
- **Preenchimento:** Configuração para visualizar um preenchimento entre as linhas.

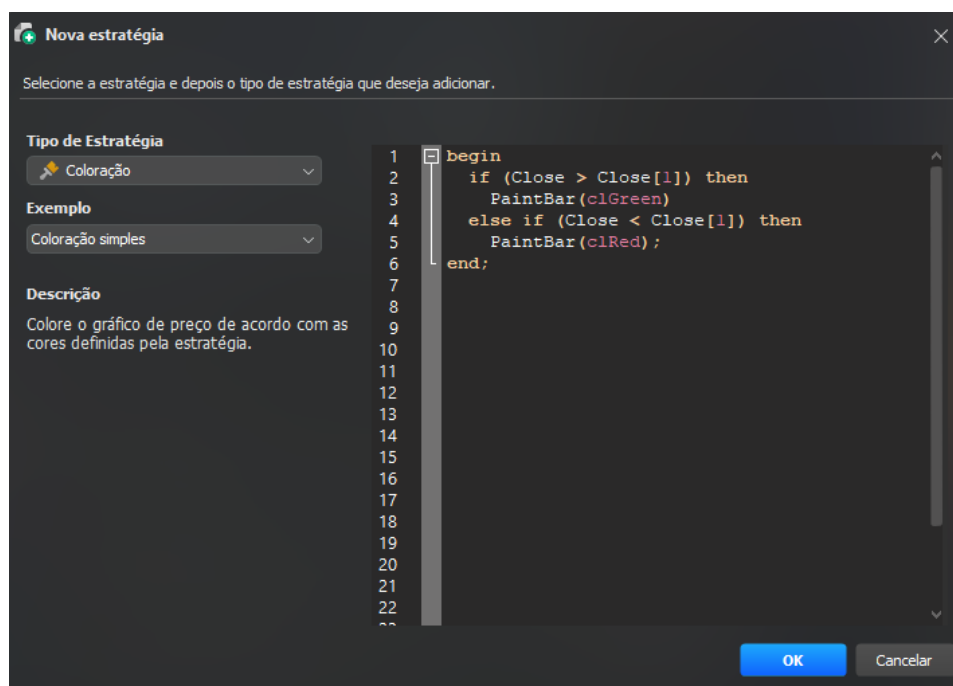


11.8 Nova Estratégia

Ao clicar no botão de Nova Estratégia, o usuário poderá escolher entre as opções abaixo:

- **Em branco (Texto):** Ao escolher desta maneira, o usuário irá criar uma estratégia em branco;
- **Indicador:** Ao abrir a estratégia, irá carregar um exemplo de estratégia de indicador para o usuário;
- **Coloração:** Ao abrir a estratégia, irá carregar um exemplo de regra de coloração para o usuário;
- **Execução:** Será criado um exemplo, com uma função para estratégia de execução;
- **Texto:** Ao abrir a estratégia, irá carregar um exemplo de estratégia de texto;
- **Screening:** Será inserido um exemplo para a criação de um filtro para o Screening;

- **Alarme:** Será criado um exemplo, com a função Alert.

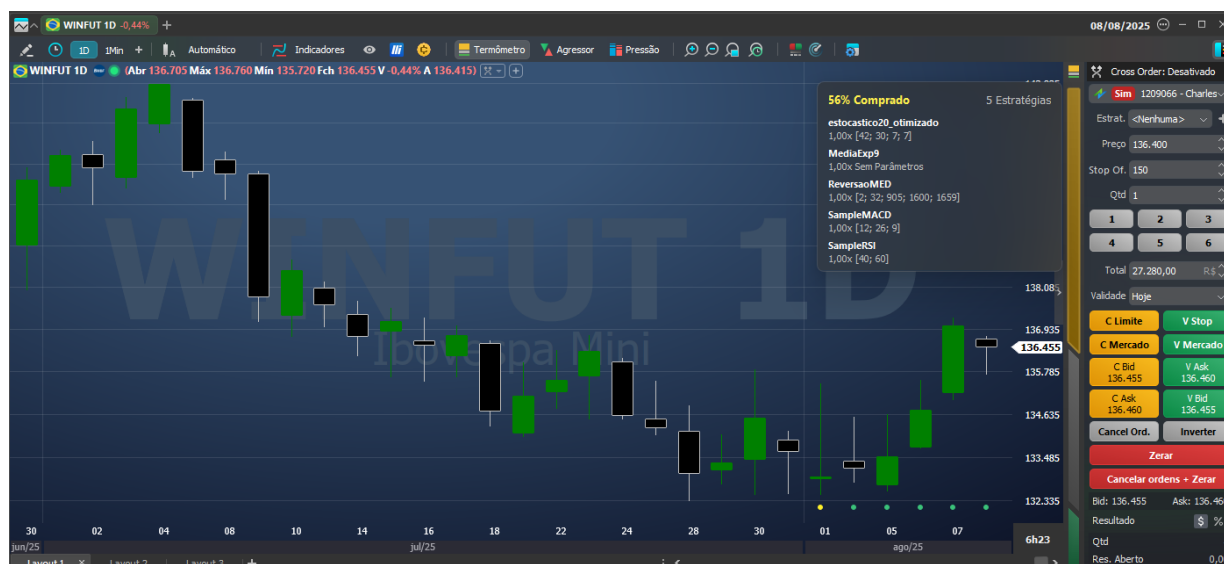


11.9 Termômetro de Estratégia

O **Termômetro de Estratégia** é uma ferramenta que permite monitorar, de forma consolidada, o comportamento de múltiplas estratégias de execução configuradas no gráfico. Ele apresenta, em tempo real, quantas dessas estratégias estão compradas, vendidas ou sem posição, proporcionando uma leitura rápida e clara da tendência predominante entre elas.

Essa visão consolidada é especialmente útil para acompanhar o comportamento coletivo das estratégias, seja durante o pregão ou em simulações.

Observação: Esta ferramenta está disponível apenas para usuários que possuem o módulo de Automação Premium ou superior.



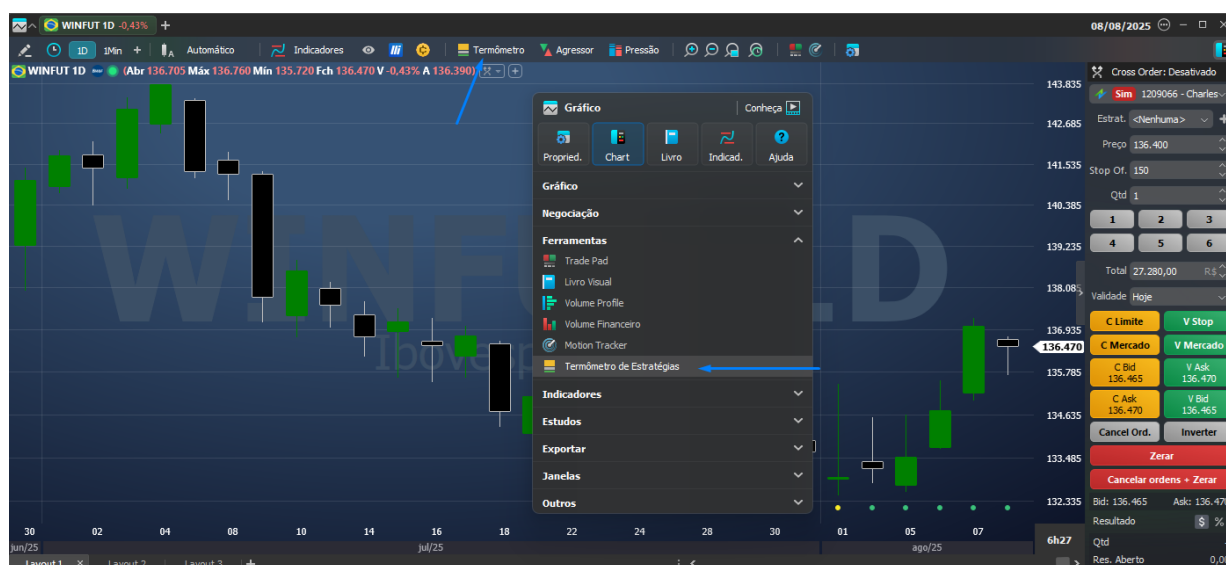
Principais Funcionalidades

- **Configuração de múltiplas estratégias:** Adicione várias estratégias de execução ao termômetro, cada uma com seus próprios parâmetros e multiplicadores.
- **Ajuste de parâmetros:** Modifique parâmetros de cada estratégia diretamente na janela de configuração.
- **Criação e salvamento de perfis:** Salve diferentes configurações de termômetros para uso posterior.
- **Leitura visual consolidada:** Visualize rapidamente o balanço entre estratégias compradas, vendidas e neutras.

Acessando o Termômetro

O Termômetro pode ser aberto de duas formas:

1. Pelo cabeçalho do gráfico, clicando no ícone do termômetro.
2. Pelo menu de contexto do gráfico, acessando o item Termômetro de Estratégias.

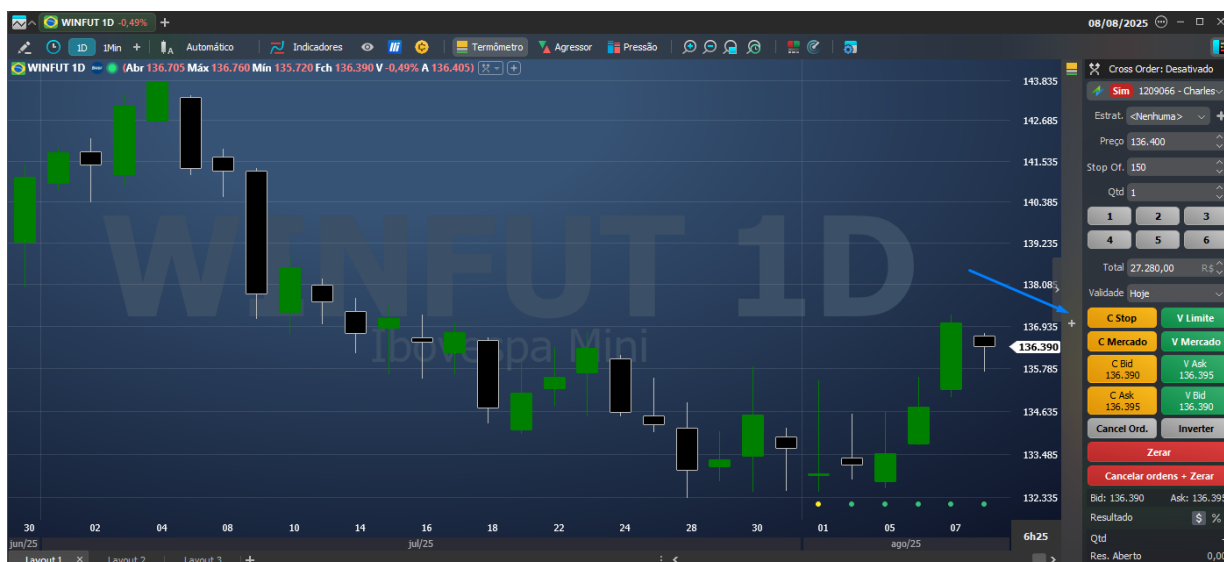


Adicionando Estratégias ao Termômetro

1. Com o Termômetro habilitado, clique sobre a **barra lateral direita do gráfico** onde ele é exibido.
2. Na janela de configuração que será aberta:
 - Escolha as estratégias de execução desejadas.
 - Ajuste seus parâmetros e multiplicadores.
 - Salve a configuração caso queira reutilizá-la no futuro.



Documentação Módulo Estratégias



Adicionar ao Termômetro

Pesquisar

Abrir/Salvar Termômetro ⓘ

Termômetro

- ☒ estocastico20_otimizado [42, 30, 7, 7]
- ☐ FundoACI0001 Sem Parâmetros
- ☒ IFR2 [14, 30]
- ☒ inversaoOrdem [10, 4]
- ☒ MaximaMinima [10, 2, 2]
- ☒ MediaExp9 Sem Parâmetros
- ☒ ReversaoMED [2, 32, 905, 1600, 1659]
- ☐ SampleAverage [18, 68]
- ☒ SampleBollinger [20, 2]
- ☒ SampleMACD [12, 26, 9]
- ☒ SampleRSI

estocastico20_otimizado 1,00x | [42, 30, 7, 7]

IFR2 1,00x | [14, 30]

inversaoOrdem 1,00x | [10, 4]

MaximaMinima 1,00x | [10, 2, 2]

MediaExp9 1,00x | Sem Parâmetros

ReversaoMED 1,00x | [2, 32, 905, 1600, 1659]

SampleBollinger 1,00x | [20, 2]

SampleMACD 1,00x | [12, 26, 9]

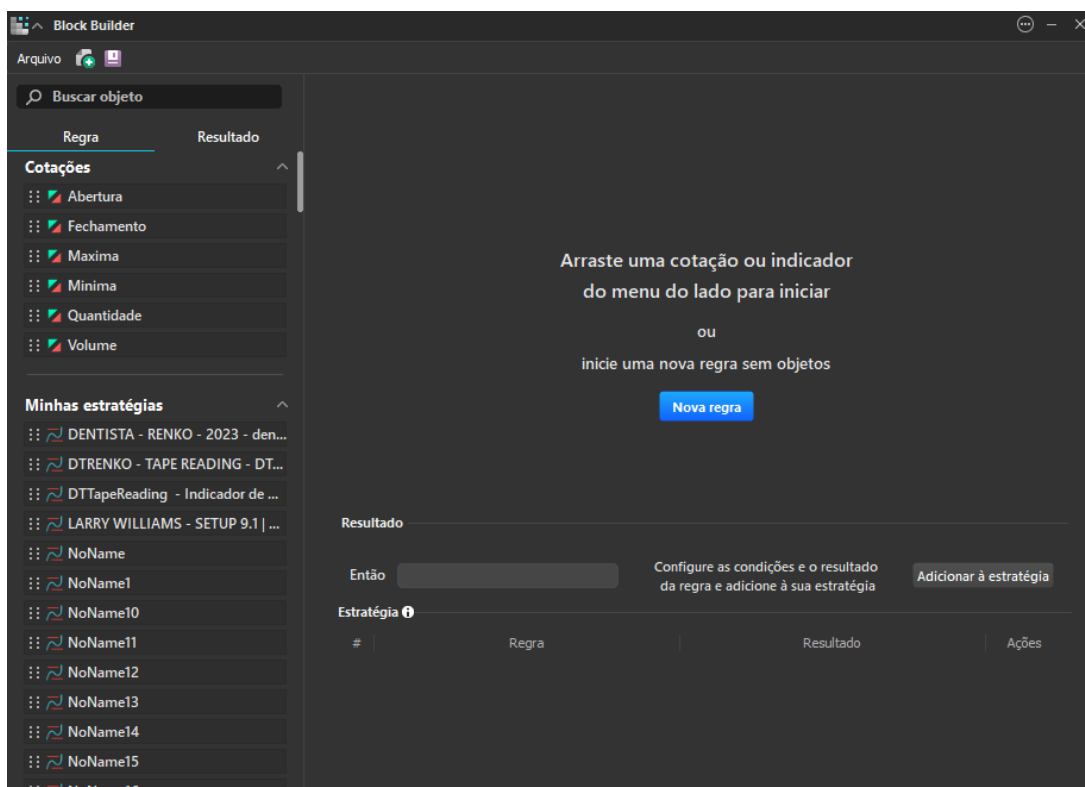
SampleRSI 1,00x | [40, 60]

Selecionadas: 9

Salvar Cancelar

12 Block Builder

O **Block Builder** é uma ferramenta visual que permite criar estratégias personalizadas de forma intuitiva, sem a necessidade de escrever código. Utilizando blocos pré-configurados, o usuário pode desenvolver regras para coloração, alarmes, execução automática e outras funcionalidades, com redução de erros e maior agilidade no processo.



12.1 Tipos de Estratégias Suportadas

O Block Builder permite a criação dos seguintes tipos de estratégias:

- **Estratégias de Indicadores:** para análise técnica personalizada.
- **Regras de Coloração:** para destacar condições específicas no gráfico.
- **Estratégias de Execução:** para automatizar operações (Backtest e Automação).
- **Estratégias de Alarmes:** para alertas personalizados.
- **Estratégias de Seleção (Screening):** para filtrar ativos com base em critérios definidos.

12.2 Criando uma Estratégia

A criação de uma nova estratégia no Block Builder envolve basicamente três etapas: definir condições, configurar os resultados esperados e salvar a estratégia.

1. **Definir Condições:** Na aba lateral esquerda, arraste os blocos desejados para o espaço de trabalho. Em seguida, clique no ícone de lápis para configurar os parâmetros da condição. Os tipos de valores que podem ser utilizados incluem:
 - **Numérico:** valores inteiros ou decimais definidos pelo usuário.
 - **Cotação:** valores de mercado como Abertura, Máxima, Mínima, Fechamento e Quantidade.
 - **Indicador:** valores derivados de indicadores criados pelo próprio usuário, com parâmetros customizáveis.



- **Cotações Anteriores:** permite usar dados de candles anteriores por meio da guia Variáveis e séries de dados.

A janela de diálogo 'Edição' apresenta opções para configurar a lógica da estratégia. No topo, há uma seção 'Valor' com três radio buttons: 'Personalizado' (valor 1), 'Cotação' (selecionado, com dropdown para 'Fechamento') e 'Indicador' (dropdown para 'AccAgressSaldo(0)'). Abaixo, a seção 'Cotações Anteriores' possui um checkbox 'Deslocar' (desativado) e um campo de valor '1' com uma seta de deslocamento e o texto 'período(s)'. Na base, há botões 'OK', 'Cancelar' e 'Ajuda'.

2. **Adicionar Condições:** Utilize o botão "+" para inserir novas condições. É possível combinar as condições utilizando operadores lógicos **E** e **OU**, permitindo maior flexibilidade na definição das regras.
3. **Definir Resultados:** Acesse a aba **Resultado** e arraste a função desejada para o campo correspondente. Por exemplo, em uma regra de coloração, você pode definir uma cor a ser aplicada ao candle quando as condições forem atendidas.
4. **Adicionar Novas Condições com Resultados Diferentes:** Para criar novas regras com ações distintas, clique em **Adicione à estratégia** e repita o processo conforme necessário.
5. **Salvar Estratégia:** Após definir todas as condições e resultados, clique em **Criar estratégia**, atribua um nome à mesma e salve.

12.3 Aplicando a Estratégia no Gráfico

Para aplicar uma regra de coloração criada no gráfico:

1. Clique com o botão direito do mouse no fundo da janela gráfica.
2. Selecione a opção **Inserir Regra de Coloração**.
3. Pesquise pelo nome da estratégia salva e selecione-a.

12.4 Exemplo Prático

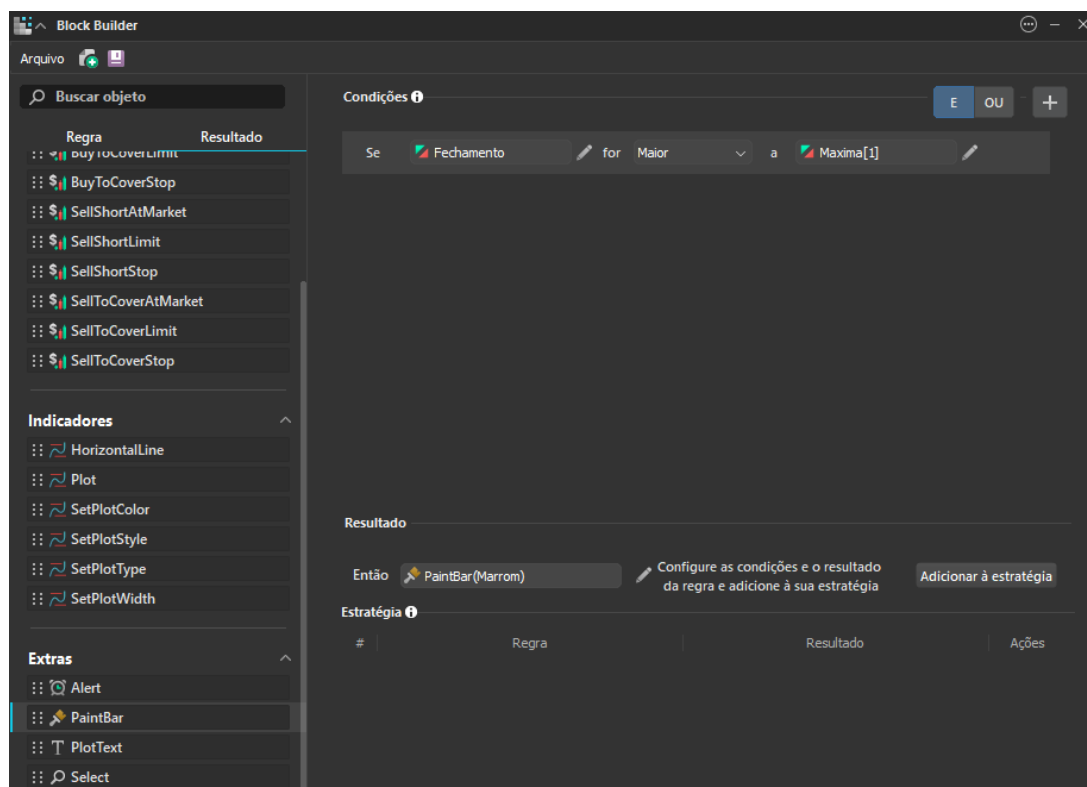
Suponha que você deseje pintar o candle atual de marrom quando o seu fechamento for maior que a máxima do candle anterior:

- **Condição:** Fechamento atual > Máxima do candle anterior (com deslocamento = 1).
- **Resultado:** Aplicar cor marrom ao candle atual.

Essa configuração permite destacar visualmente os candles que atendem ao critério especificado, facilitando a análise visual no gráfico.



Documentação Módulo Estratégias



13 Funções de Alarme

13.1 Função Alert

Disponível em todas versões.

A função **Alert()** tem como finalidade gerar um alarme ao usuário.

Sintaxe:

```
Função Alert(Color: Integer): Void;
```

Parâmetros:

Color: **Integer**; Determina a cor desejada para o popup de notificação, no momento de execução do alarme.

Exemplo de uso:

No exemplo abaixo, a função **Alert()** irá emitir um alarme (e um popup com coloração vermelha) caso a média móvel aritmética de **9** períodos for menor que a média de **21**.

```
1 begin
2   if (Media(9,Close) < Media(21,Close)) then
3     Alert(c1Red);
4   end;
```

Exemplo Gráfico:



14 Funções de Back-Testing

14.1 Função BuyAtMarket

Disponível em todas versões.

A função **BuyAtMarket()** tem como funcionalidade realizar uma ordem de compra a mercado.

Se o parâmetro de quantidade não for especificado, a ordem usará o valor definido no campo "Quantidade" da aba de Execução do editor de estratégias, com o mínimo sendo 1 lote.

Na automação, será considerada a quantidade configurada no campo "Quantidade por Ordem". Quando o parâmetro é especificado, a estratégia ignora essas configurações e utiliza a quantidade informada no código, que deve ser múltipla do lote do ativo.

Sintaxe:

```
Função BuyAtMarket(Quantity: Float): Void;
```

Parâmetros:

Quantity: **Float** [Opcional]; Determina a quantidade a ser executada na ordem de compra.

Exemplo de uso

No exemplo, caso a mínima do candle atual for igual a do candle anterior, será realizada uma simulação de ordem a mercado.

```
1 begin
2   if (Low = Low[1]) then
3     BuyAtMarket;
4   end;
```

Exemplo Gráfico:



14.2 Função BuyLimit

Disponível em todas versões.

A função `BuyLimit()` possui como finalidade enviar uma ordem de compra do tipo limite. Se a quantidade não for especificada, será usada a definida na aba de Execução do editor de estratégias, com mínimo de 1 lote. Em automações, utiliza-se a quantidade do campo "Quantidade por Ordem". Quando a quantidade é especificada no código, ela ignora as configurações padrão e deve ser múltipla do lote do ativo.

Sintaxe:

```
Função BuyLimit(Limit: Float; Quantity: Float): Void;
```

Parâmetros:

Limit: `Float`; Determina o preço limite para inserção de ordem.

Quantity: `Float` [Opcional]; Determina a quantidade a ser executada na ordem de compra.

Exemplo de uso:

No exemplo, caso não tenha nenhuma posição aberta, será enviada uma ordem de compra do tipo limite.

```
1 begin
2   if not HasPosition then
3     BuyLimit(close,lote);
4 end;
```

Exemplo Gráfico:





14.3 Função BuyPosition

Disponível em todas versões.

A função `BuyPosition()` retorna o tamanho da posição em lote da compra.

Sintaxe:

```
Função BuyPosition(): Integer;
```

Parâmetros:

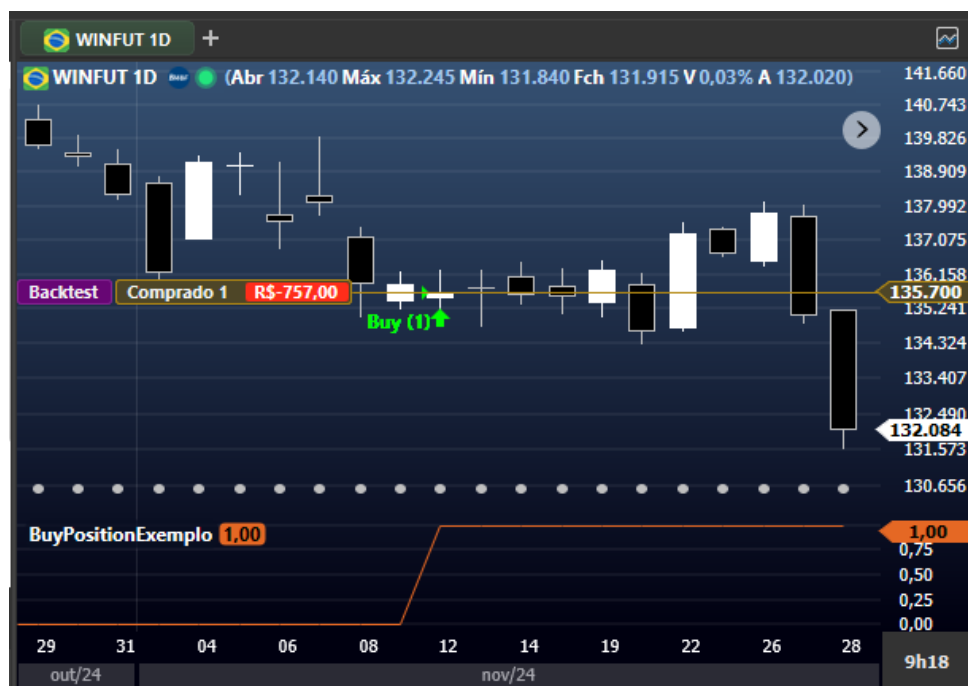
Sem parâmetros.

Exemplo de uso:

No exemplo, será realizada a simulação de uma ordem de compra caso a mínima do candle atual for igual a do candle anterior, então a variável `buyPos` irá receber o total da posição de compra.

```
1 var
2   buyPos : Integer;
3 begin
4   buyPos := BuyPosition;
5   if (Low = Low[1]) then
6     BuyAtMarket;
7     Plot(buyPos);
8 end;
```

Exemplo Gráfico:



14.4 Função BuyPositionQty

Disponível em todas versões.

A função `BuyPositionQty()` retorna o tamanho da posição em quantidade da compra.

Sintaxe:

```
Função BuyPositionQty(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, será realizada a simulação de uma ordem de compra caso o fechamento do candle atual for menor ao do candle anterior, então a variável `buyPos` irá receber o tamanho da posição em quantidade de compra.

```
1 var
2   buyPos : Float;
3 begin
4   buyPos := BuyPositionQty;
5   if (Close > Close[1]) then
6     BuyAtMarket;
7     Plot(BuyPositionQty);
8 end;
```

Exemplo Gráfico:



14.5 Função BuyPrice

Disponível em todas versões.

A função **BuyPrice()** retorna o preço de compra da posição.

Sintaxe:

```
Função BuyPrice(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, será realizada a simulação de uma ordem de compra caso a mínima do candle atual for igual a do candle anterior, então será atribuído à variável **pCompra** o valor de compra da posição.

```
1 var
2   pCompra : Float;
3 begin
4   pCompra := BuyPrice;
5   if (Low = Low[1]) then
6     BuyAtMarket();
7     Plot(BuyPrice);
8 end;
```

Exemplo Gráfico:





14.6 Função BuyStop

Disponível em todas versões.

A partir da função **BuyStop()** permite criar uma ordem do tipo Stop, onde o preço stop define o gatilho e o limite determina até que preço a ordem pode ser executada. Se a quantidade não for especificada, será usada a do campo Quantidade na aba de Execução (mínimo de 1 lote).

Em automações, é usada a quantidade do campo "Quantidade por Ordem". Quando a quantidade é informada no código, ela ignora as configurações padrão e deve ser múltipla do lote do ativo.

Sintaxe:

```
Função BuyStop(Stop : Float; Limit: Float; Quantity: Float): Void;
```

Parâmetros:

Stop: **Float**; Valor do tipo Float que será o gatilho da ordem.

Limit: **Float** [Opcional]; Valor do tipo Float que será o limite de preço aceito para execução.

Quantity: **Float** [Opcional]; Determina a quantidade a ser executada na ordem de compra.

Exemplo de uso:

No exemplo, caso a mínima do candle atual seja diferente a do candle anterior, será inserida uma ordem de compra tipo stop, no nível especificado na variável **pStop**.

```
1 input
2   pStop(133000);
3   lote(1);
4 begin
5   if (Low <> Low[1]) then
6     BuyStop(pStop,pStop,lote);
7 end;
```



Exemplo Gráfico:



14.7 Função BuyToCoverAtMarket

Disponível em todas versões.

A função **BuyToCoverAtMarket()** envia uma ordem de compra caso exista uma posição vendida em aberto. Se a quantidade não for especificada, será usada a definida na aba de Execução, com mínimo de 1 lote. Na automação, utiliza-se a quantidade do campo "Quantidade por Ordem".

Se a quantidade for especificada no código, ela prevalece sobre as configurações e deve ser múltipla do lote do ativo.

Sintaxe:

Função **BuyToCoverAtMarket**(Quantity: **Float**): **Void**;

Parâmetros:

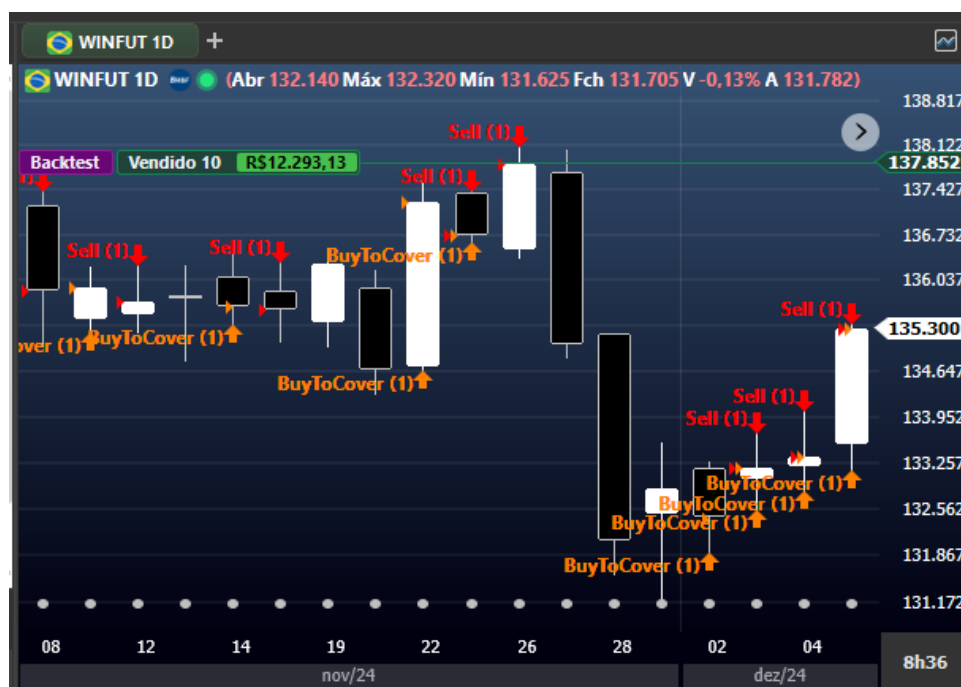
Quantity: **Float** [Opcional]; Determina a quantidade a ser executada na ordem de compra.

Exemplo de uso:

No exemplo abaixo, caso a mínima atual seja maior que a mínima anterior e exista uma posição de venda, será executada uma ordem a mercado, utilizando a função **BuyToCoverAtMarket()** para fechar a operação.

```
1 begin
2   SellShortAtMarket;
3   if ((Low > Low[1]) and (IsSold)) then
4     BuyToCoverAtMarket;
5   end;
```

Exemplo Gráfico:





14.8 Função BuyToCoverLimit

Disponível em todas versões.

A função `BuyToCoverLimit()` envia uma ordem de compra limite para finalizar a operação. Se a quantidade não for especificada, será usada a configurada na aba de Execução, com mínimo de 1 lote. Na automação, utiliza-se a quantidade do campo "Quantidade por Ordem".

Se especificada no código, a quantidade deve ser múltipla do lote do ativo e prevalece sobre as configurações.

Sintaxe:

```
Função BuyToCoverLimit(Limit: Float; Quantity: Float): Void;
```

Parâmetros:

Limit: **Float**; Valor do tipo Float que será o limite de preço aceito para execução.

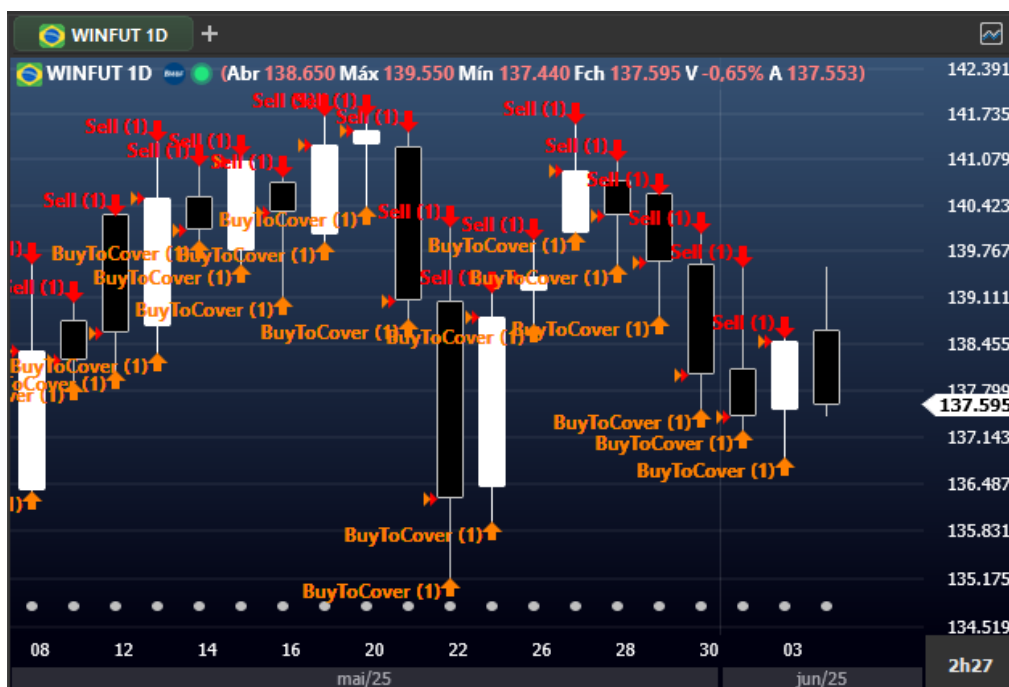
Quantity: **Float** [Opcional]; Determina a quantidade a ser executada na ordem de compra.

Exemplo de uso:

No exemplo, caso exista uma posição de venda, será inserida a ordem no preço de fechamento para zerar a posição.

```
1 begin
2   SellShortAtMarket;
3   if (IsSold) then
4     BuyToCoverLimit(Close, lote);
5   end;
```

Exemplo Gráfico:





14.9 Função BuyToCoverStop

Disponível em todas versões.

A função **BuyToCoverStop()** envia uma ordem de compra do tipo Stop se houver uma posição de venda no ativo. Se a quantidade não for especificada, será usada a configurada na aba de Execução, com mínimo de 1 lote. Na automação, utiliza-se a quantidade do campo "Quantidade por Ordem". Se especificada no código, a quantidade deve ser múltipla do lote do ativo e prevalece sobre as configurações.

Sintaxe:

```
Função BuyToCoverStop(Stop : Float; Limit: Float; Quantity: Float): Void;
```

Parâmetros:

Stop: **Float**; Valor do tipo Float que será o gatilho da ordem.

Limit: **Float** [Optional]; Valor do tipo Float que será o limite de preço aceito para execução.

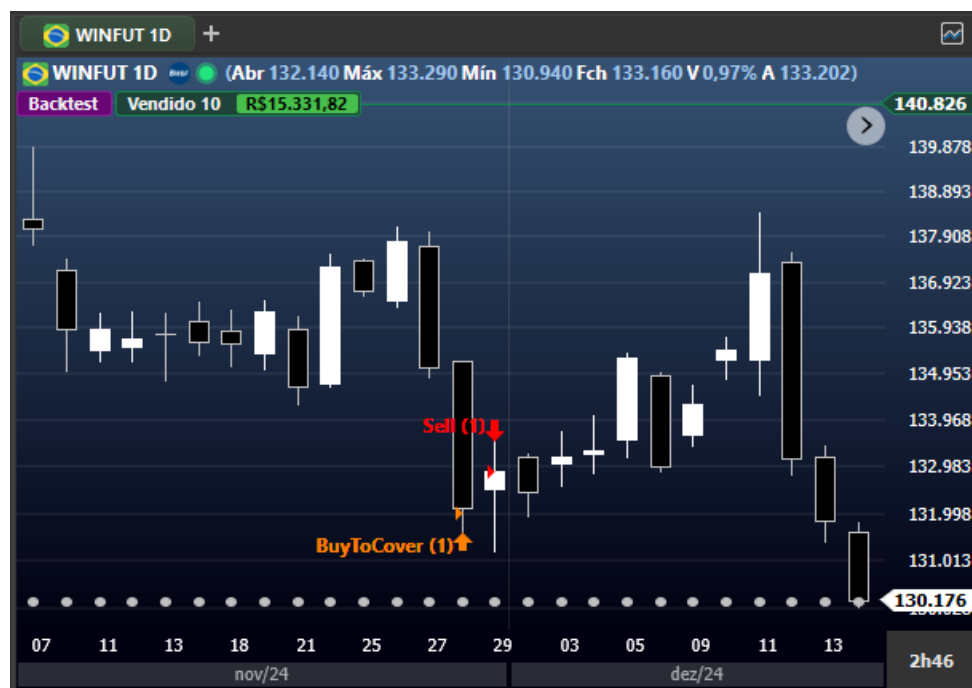
Quantity: **Float** [Opcional]; Determina a quantidade a ser executada na ordem de compra.

Exemplo de uso:

No exemplo, caso a mínima do candle atual for igual a do candle anterior e exista uma posição de venda, será inserida uma ordem de compra tipo stop, no preço armazenado na variável **pStop**.

```
1 input
2   pstop(132000);
3 begin
4   SellShortAtMarket;
5   if (Low = Low[1]) and (IsSold) then
6     BuyToCoverStop(pStop, pStop);
7 end;
```

Exemplo Gráfico:





14.10 Função CancelPendingOrders

Disponível em todas versões.

A função `CancelPendingOrders()` possui como recurso efetuar o cancelamento de todas ordens enviadas até o momento de sua chamada, no fluxo atual de execução, e as ordens abertas.

Sintaxe:

```
Função CancelPendingOrders(): Void;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, caso a mínima do candle atual for igual a do candle anterior e exista uma posição de venda, será inserida uma ordem de compra tipo stop, no preço armazenado na variável `pStop`, caso existam ordens pendentes, ocorrerá o cancelamento e não será plotado nada no gráfico.

```
1 input
2   pstop(132000);
3 begin
4   SellShortAtMarket;
5   if (Low = Low[1]) and (IsSold) then
6     BuyToCoverStop(pStop, pstop);
7   if (HasPendingOrders) then
8     CancelPendingOrders;
9 end;
```

Exemplo Gráfico:



14.11 Função ClosePosition

Disponível em todas versões.

A função `ClosePosition()` envia ordens para encerrar a posição. Esta função cancela todas ordens enviadas até o momento de sua chamada, no fluxo atual de execução, e as ordens abertas. Após confirmação do cancelamento das ordens, será enviada uma ordem para fechar a posição.

Sintaxe:

```
Função ClosePosition(): Void;
```

Parâmetros:

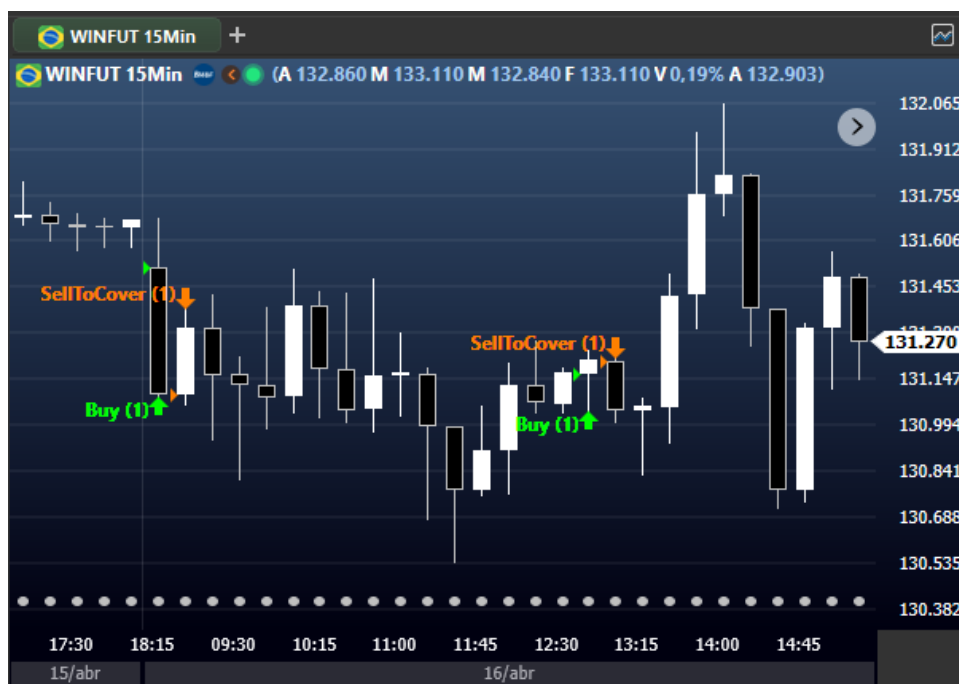
Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, caso a mínima do candle atual seja igual a do candle anterior será realizada a simulação de uma ordem de compra e então a posição será finalizada.

```
1 begin
2   if (Low = Low[1]) then
3     BuyAtMarket;
4   if HasPosition then
5     closeposition;
6 end;
```

Exemplo Gráfico:



14.12 Função DailyResult

Disponível em todas versões.

A partir da função `DailyResult()` pode-se verificar o resultado diário das operações. O parâmetro `OpenResult` é opcional, o valor default é verdadeiro.

Sintaxe:

```
Função DailyResult(OpenResult: Boolean): Float;
```

Parâmetros:

`OpenResult: Boolean` [Opcional]; Se deve retornar o resultado aberto:

- **Verdadeiro**: Retorna o resultado fechado + aberto;
- **Falso**: Retorna apenas o resultado fechado;

Exemplo de uso:

No exemplo, será o retornado resultado fechado + aberto das operações do dia.

```
1 var
2   dResult : Float;
3 begin
4   if not HasPosition then
5     BuyAtMarket;
6   dResult := DailyResult(True);
7   Plot(dResult);
8 end;
```

Exemplo Gráfico:



14.13 Função HasPendingOrders

Disponível em todas versões.

A função `HasPendingOrders()` retorna se há ordens pendentes.

Sintaxe:

Função `HasPendingOrders()`: **Boolean**;

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, caso a mínima do candle atual for igual a do candle anterior e exista uma posição de venda, será inserida uma ordem de compra tipo stop, no preço armazenado na variável `pStop`, e caso existam ordens pendentes, será plotado a `MedianPrice()` no gráfico.

```
1 input
2   pstop(132000);
3 begin
4   SellShortAtMarket;
5   if (Low = Low[1]) and (IsSold) then
6     BuyToCoverStop(pStop, pstop);
7   if (HasPendingOrders) then
8     Plot(MedianPrice);
9 end;
```

Exemplo Gráfico:



14.14 Função HasPosition

Disponível em todas versões.

A função `HasPosition()` retorna verdadeiro se a posição não é zero.

Sintaxe:

```
Função HasPosition(): Boolean;
```

Parâmetros:

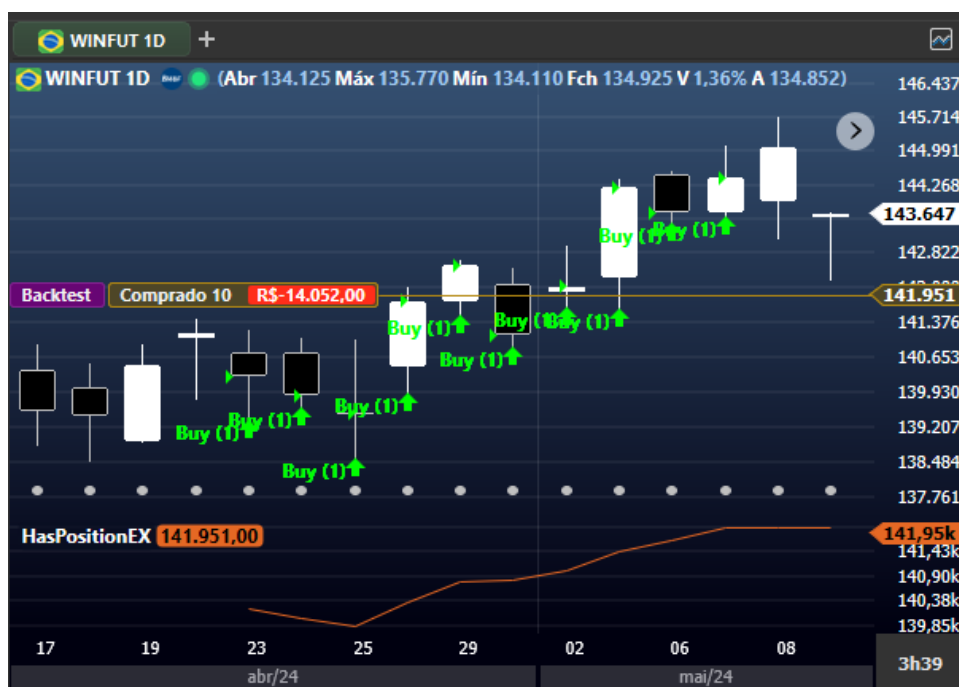
Sem parâmetros.

Exemplo de uso:

No exemplo, ocorrerá a simulação de uma ordem a mercado usando a função `HasPosition()` para validar se há posição ou não e, caso exista posição, ocorrerá o plot de `BuyPrice()` no gráfico.

```
1 begin
2   BuyAtMarket;
3   if HasPosition then
4     Plot(BuyPrice);
5 end;
```

Exemplo Gráfico:





14.15 Função IsBought

Disponível em todas versões.

A função `IsBought()` tem como funcionalidade determinar se há uma posição de compra em aberto.

Sintaxe:

```
Função IsBought(): Boolean;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, caso exista uma posição de compra, haverá a aplicação de uma coloração (verde).

```
1 begin
2   BuyAtMarket;
3   if (IsBought) then
4     PaintBar(c1Green);
5 end;
```

Exemplo Gráfico:





14.16 Função IsSold

Disponível em todas versões.

A função `IsSold()` tem como funcionalidade determinar se há uma posição de venda em aberto.

Sintaxe:

```
Função IsSold(): Boolean;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, caso exista uma posição de venda, haverá a aplicação de uma coloração (vermelho).

```
1 begin
2   SellShortAtMarket;
3   if (IsSold) then
4     PaintBar(c1Red);
5 end;
```

Exemplo Gráfico:



14.18 Função Position

Disponível em todas versões.

A função `Position()` retorna o tamanho da posição em lote, positivo para compra e negativo para venda.

Sintaxe:

Função `Position()`: `Integer`;

Parâmetros:

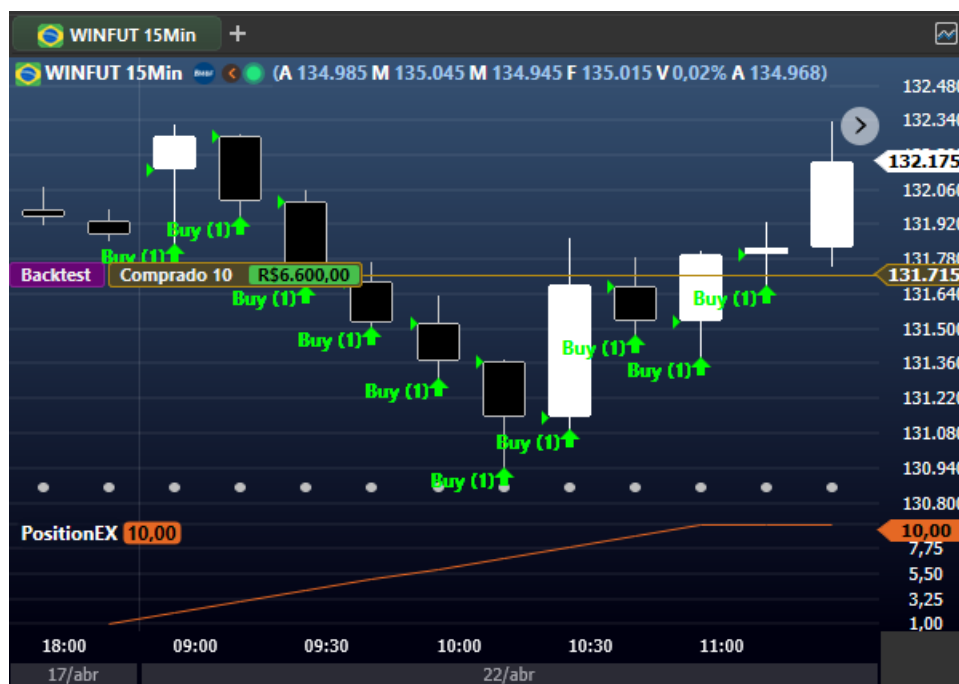
Sem parâmetros.

Exemplo de uso:

No exemplo, será atribuído à variável `vPosition`, a posição, e será simulada uma ordem de compra a mercado e então o plot da função no gráfico.

```
1 var
2   vPosition : Integer;
3 begin
4   vPosition := Position;
5   BuyAtMarket;
6   if (IsBought) then
7     Plot(vPosition);
8 end;
```

Exemplo Gráfico:



14.19 Função PositionQty

Disponível em todas versões.

A função `PositionQty()` retorna o tamanho da posição em quantidade, positivo para compra e negativo para venda.

Sintaxe:

Função `PositionQty(): Float;`

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, será atribuído à variável `vPosition`, a posição, e será simulada uma ordem de compra a mercado e então o plot da função no gráfico.

```
1 var
2   vPosition : Float;
3 begin
4   vPosition := PositionQty;
5   BuyAtMarket;
6   if (IsBought) then
7     Plot(vPosition);
8 end;
```

Exemplo Gráfico:



14.20 Função Price

Disponível em todas versões.

A função `Price()` retorna o preço de compra ou venda da posição, dependendo se estiver comprado ou vendido.

Sintaxe:

```
Função Price(): Float;
```

Parâmetros:

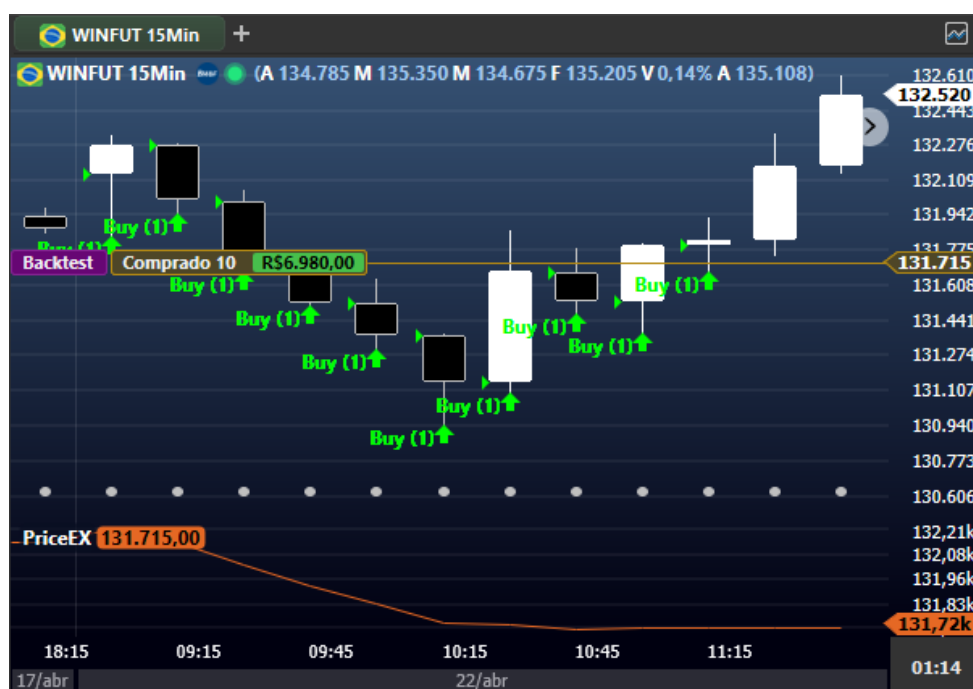
Sem parâmetros.

Exemplo de uso:

No exemplo, será atribuído à variável `vPrice`, o valor da função `Price()`, será simulada uma ordem de compra a mercado e então o plot da função no gráfico.

```
1 var
2   vPrice : Float;
3 begin
4   vPrice := Price;
5   BuyAtMarket;
6   if (IsBought) then
7     Plot(vPrice);
8 end;
```

Exemplo Gráfico:



14.21 Função ReversePosition

Disponível em todas versões.

A função `ReversePosition()` tem como funcionalidade realizar o envio de ordens, a fim de inversão da posição. Esta função cancela todas ordens enviadas até o momento de sua chamada, no fluxo atual de execução, e as ordens abertas. Após confirmação do cancelamento das ordens, será enviada uma ordem para reverter a posição.

Sintaxe:

```
Função ReversePosition(): Void;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, caso a mínima do candle atual for menor a do candle anterior, e exista uma posição de compra, será realizada a reversão.

```
1 begin
2   BuyAtMarket;
3   if (Low < Low[1]) and (IsBought = True) then
4     ReversePosition;
5 end
```

Exemplo Gráfico:



14.22 Função SellPosition

Disponível em todas as versões.

A função `SellPosition()` retorna o tamanho da posição em lote de venda.

Sintaxe:

```
Função SellPosition(): Void;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, a variável `sellPos`, irá receber a posição de venda.

```
1 var
2   sellPos : Integer;
3 begin
4   SellShortAtMarket;
5   sellPos := SellPosition;
6   Plot(sellPos);
7 end;
```

Exemplo Gráfico:



14.23 Função SellPositionQty

Disponível em todas versões.

A função `SellPositionQty()` retorna o tamanho da posição em quantidade de venda.

Sintaxe:

```
Função SellPositionQty(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, a variável `sellPos`, irá receber o total da posição de venda.

```
1 var
2   sellPos : Float;
3 begin
4   SellShortAtMarket;
5   sellPos := SellPositionQty;
6   Plot(sellPos);
7 end;
```

Exemplo Gráfico:



14.24 Função SellPrice

Disponível em todas versões.

A função `SellPrice()` retorna o preço de venda da posição.

Sintaxe:

```
Função SellPrice(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, será atribuído à variável `pVenda`, o valor de venda da posição, ocorrerá uma simulação de venda e o plot da função no gráfico.

```
1 var
2   pVenda : Float;
3 begin
4   SellShortAtMarket;
5   pVenda := SellPrice;
6   Plot(pVenda);
7 end;
```

Exemplo Gráfico:



14.25 Função SellShortAtMarket

Disponível em todas versões.

A função `SellShortAtMarket()` envia uma ordem de venda a mercado. Se a quantidade não for especificada, será usada a definida na aba de Execução, com mínimo de 1 lote. Na automação, utiliza-se a quantidade do campo "Quantidade por Ordem".

Se especificada no código, essa quantidade deve ser múltipla do lote do ativo e prevalece sobre as configurações.

Sintaxe:

```
Função SellShortAtMarket(Quantity: Float): Void;
```

Parâmetros:

Quantity: **Float** [Opcional]; Determina a quantidade a ser executada na ordem de compra.

Exemplo de uso:

No exemplo abaixo, será executada uma ordem de venda a mercado, caso a mínima do candle atual seja igual a do candle anterior.

```
1 begin
2   if (Low = Low[1]) then
3     SellShortAtMarket;
4 end;
```

Exemplo Gráfico:





14.26 Função SellShortLimit

Disponível em todas versões.

Descrição:

A função `SellShortLimit()` envia uma ordem de venda limite. Se a quantidade não for especificada, será usada a definida na aba de Execução, com mínimo de 1 lote. Na automação, utiliza-se a quantidade do campo "Quantidade por Ordem"

Se especificada no código, a quantidade deve ser múltipla do lote do ativo e prevalece sobre as configurações.

Sintaxe:

```
Função SellShortLimit(Limit: Float; Quantity: Float): Void;
```

Parâmetros:

Limit: **Float**; Preço para a inserção da ordem.

Quantity: **Float** [Opcional]; Determina a quantidade a ser executada na ordem de compra.

Exemplo de uso:

No exemplo abaixo, será executada uma ordem de venda a mercado, caso a mínima do candle atual seja igual a do candle anterior.

```
1 begin
2   if (Low = Low[1]) then
3     SellShortLimit(Close, lote);
4 end;
```

Exemplo Gráfico:



14.27 Função SellShortStop

Disponível em todas versões.

A função **SellShortStop()** envia uma ordem de venda do tipo stop. Se a quantidade não for especificada, será usada a definida na aba de Execução, com mínimo de 1 lote. Na automação, utiliza-se a quantidade do campo "Quantidade por Ordem".

Se especificada no código, a quantidade deve ser múltipla do lote do ativo e prevalece sobre as configurações.

Sintaxe:

```
Função SellShortStop(Stop: Float; Limit: Float; Quantity: Float): Void;
```

Parâmetros:

Stop: **Float**; Valor do tipo Float que será o gatilho da ordem;

Limit: **Float**[Opcional]; Valor do tipo Float que será o limite de preço aceito para execução.

Quantity: **Float** [Opcional]; Determina a quantidade a ser executada na ordem de compra.

Exemplo de uso:

No exemplo, caso a máxima atual for maior do que a anterior, será inserida uma ordem de venda stop, no preço armazenado na variável pAux.

```
1 input
2   pAux(136000);
3 begin
4   if (High > High[1]) then
5     SellShortStop(pAux, pAux);
6 end
```

Exemplo Gráfico:



14.28 Função SellToCoverAtMarket

Disponível em todas versões.

A função `SellToCoverAtMarket()` envia uma ordem de venda a mercado se houver uma posição de compra. Se a quantidade não for especificada, será usada a definida na aba de Execução, com mínimo de 1 lote. Na automação, utiliza-se a quantidade do campo "Quantidade por Ordem". Se especificada no código, a quantidade deve ser múltipla do lote do ativo e prevalece sobre as configurações.

Sintaxe:

Função `SellToCoverAtMarket(Quantity: Float): Void;`

Parâmetros:

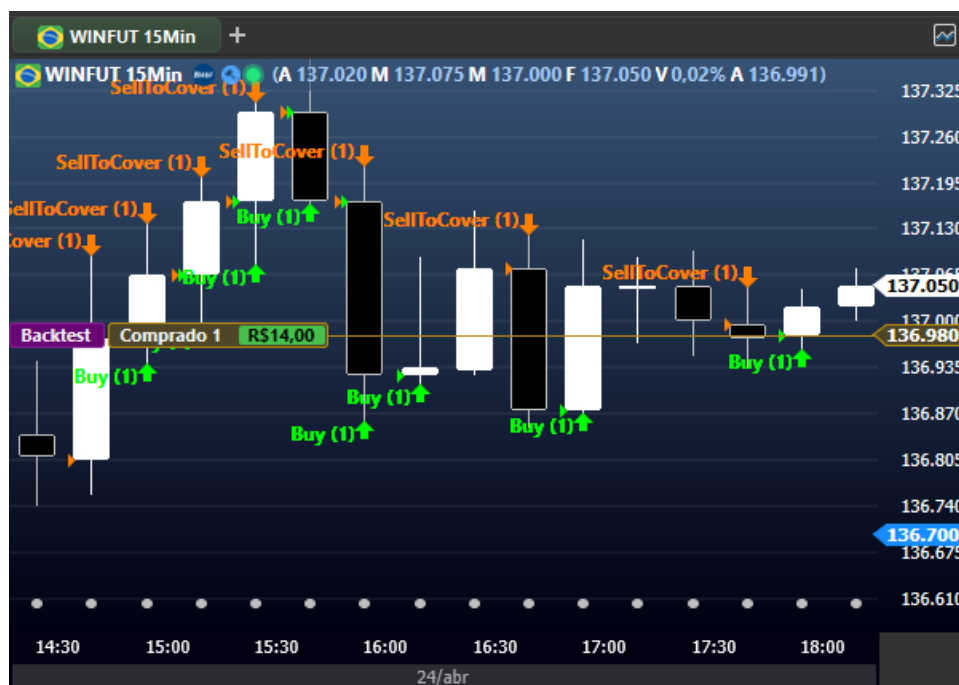
Quantity: Float [Opcional]; Determina a quantidade a ser executada na ordem de compra.

Exemplo de uso:

No exemplo abaixo, caso a máxima atual seja maior que a máxima anterior e exista uma posição de compra, será executada uma ordem a mercado, para fechar a operação.

```
1 begin
2   if not HasPosition then
3     BuyAtMarket;
4   if (High > High[1]) and (Isbought) then
5     SellToCoverAtMarket;
6 end;
```

Exemplo Gráfico:



14.29 Função SellToCoverLimit

Disponível em todas versões.

A função **SellToCoverLimit()** envia uma ordem de venda limite para finalizar a operação. Se a quantidade não for especificada, será usada a definida na aba de Execução, com mínimo de 1 lote. Na automação, utiliza-se a quantidade do campo "Quantidade por Ordem".

Se especificada no código, a quantidade deve ser múltipla do lote do ativo e prevalece sobre as configurações.

Sintaxe:

Função **SellToCoverAtLimit**(Limit: **Float**; Quantity: **Float**): **Void**;

Parâmetros:

Limit: **Float**; Preço para a inserção da ordem.

Quantity: **Float** [Opcional]; Determina a quantidade a ser executada na ordem de compra.

Exemplo de uso:

No exemplo, caso exista uma posição de compra, será inserida a ordem no preço de fechamento para zerar a posição.

```
1 begin
2   if not HasPosition then
3     BuyAtMarket;
4   if (High > High[1]) and (Isbought) then
5     SellToCoverLimit(Close);
6 end;
```

Exemplo Gráfico:





14.30 Função SellToCoverStop

Disponível em todas versões.

A função `SellToCoverStop()` envia uma ordem stop de venda caso exista uma posição de compra. Se a quantidade não for especificada, será usada a definida na aba de Execução, com mínimo de 1 lote. Na automação, utiliza-se a quantidade do campo "Quantidade por Ordem".

Se especificada no código, a quantidade deve ser múltipla do lote do ativo e prevalece sobre as configurações.

Sintaxe:

```
Função SellToCoverStop(Stop: Float; Limit: Float; Quantity: Float): Void;
```

Parâmetros:

Stop: **Float**; Valor do tipo Float que será o gatilho da ordem;

Limit: **Float** [Optional]; Valor que será o limite do preço aceito para execução.

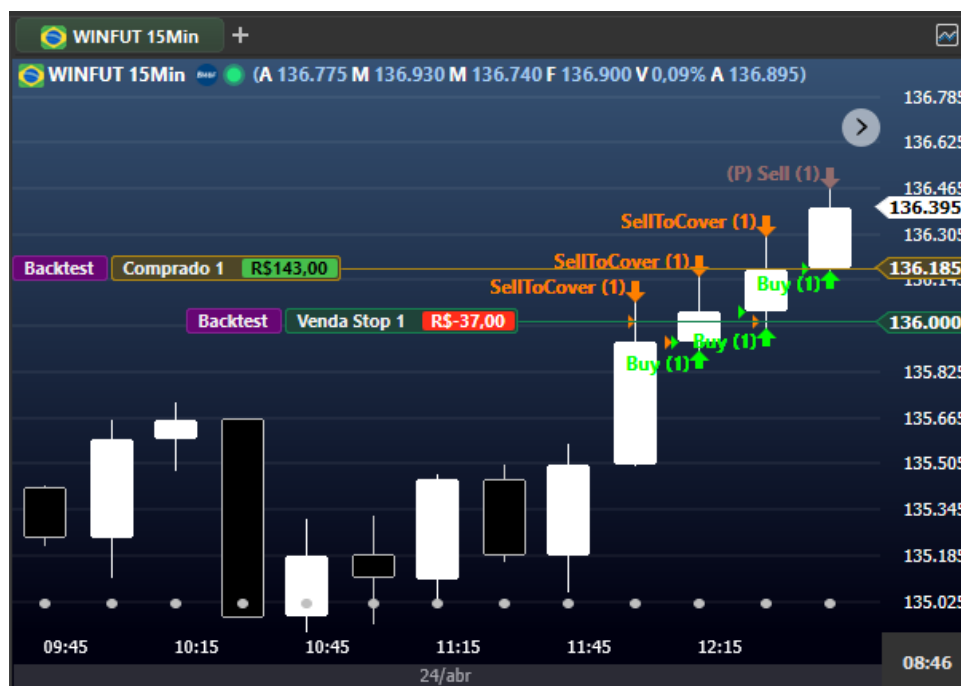
Quantity: **Float** [Opcional]; Determina a quantidade a ser executada na ordem.

Exemplo de uso:

No exemplo, caso a máxima do candle atual for maior que a do candle anterior e exista uma posição de compra, será inserida uma ordem de venda do tipo stop, no preço de pAux.

```
1 input
2   pAux(136000);
3 begin
4   if not HasPosition then
5     BuyAtMarket;
6   if (High > High[1]) then
7     SellToCoverStop(pAux,pAux,lote);
8 end
```

Exemplo Gráfico:



14.31 Função SendOrder

Disponível em todas versões.

A partir da função `SendOrder()` pode-se enviar ordens customizadas, determinando o lado, tipo e quantidade.

Sintaxe:

```
Função SendOrder(Side: Integer; Tipe: Integer; Quantity: Float; Limit: Float; Stop: Float): Void;
```

Parâmetros:

Side: **Integer**; Determina o lado da ordem;

Type: **Integer**; Tipo da ordem;

Quantity: **Float**; Determina a quantidade a ser executada na ordem.

Limit: **Float**; Quantidade de contratos / lotes.

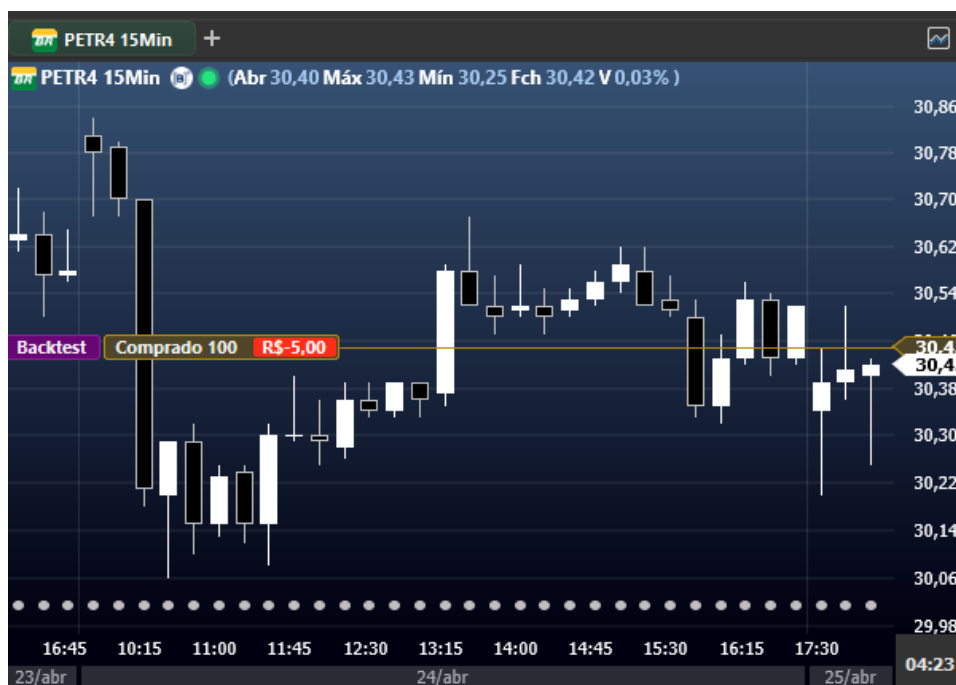
Stop: **Float**; Valor do tipo Float que será o gatilho da ordem;

Exemplo de uso:

No exemplo, será efetuada a inserção de uma ordem de compra, tipo Stop, no preço **30.45**, com limite de execução até o nível **30.55**.

```
1 begin
2   if not HasPosition then
3     SendOrder(osBuy, otStopLimit, 100, 30.55, 30.45);
4 end;
```

Exemplo Gráfico:



14.32 Função OpenResult

Disponível em todas versões.

A partir da função `OpenResult()` pode-se verificar o resultado diário das operações em aberto.

Sintaxe:

```
Função OpenResult(): Float;
```

Parâmetros:

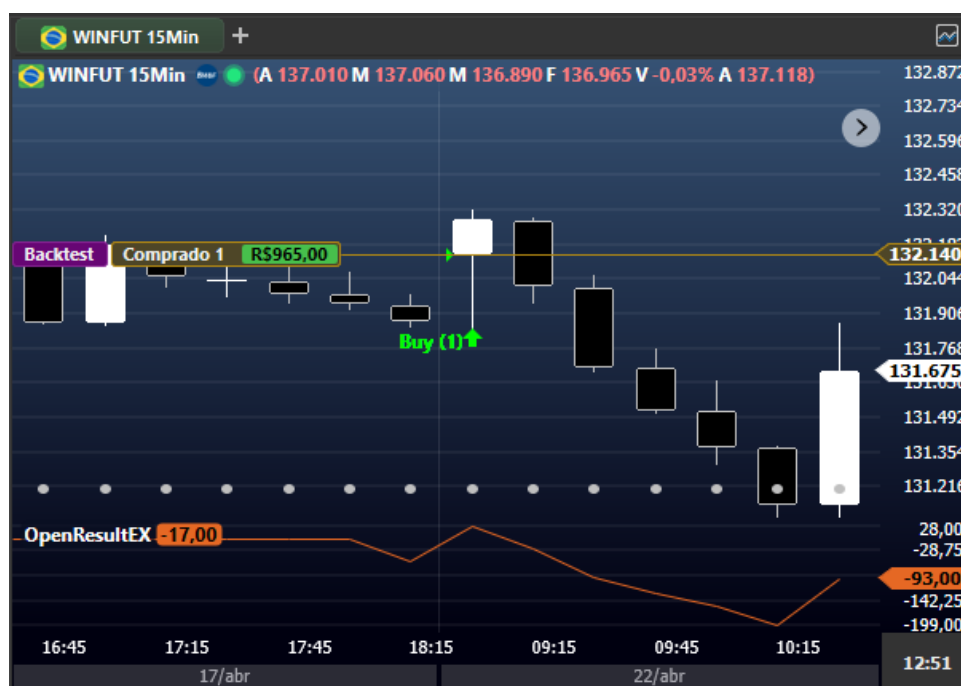
Sem parâmetros.

Exemplo de uso:

No exemplo, será o retornado resultado aberto das operações do dia.

```
1 var
2   oResult : Float;
3 begin
4   if not HasPosition then
5     BuyAtMarket;
6   oResult := OpenResult;
7   Plot(oResult);
8 end;
```

Exemplo Gráfico:





14.33 Função XRay

Disponível com o módulo Automações Premium ou superior.

A função **XRay()** permite o monitoramento em tempo real de variáveis e condições dentro de uma estratégia automatizada, exibindo essas informações no botão **Raio-X** do **Gerenciador de Automações**.

Sintaxe:

```
Função XRay(strValue: String; bValue: Boolean; strGroup: String; dValue: Float;  
ColorON: Integer; ColorOF: Integer): Void;
```

Parâmetros:

strValue: String; Nome identificador da condição ou variável que será exibido no painel Raio-X;

bValue: Boolean; Valor lógico que representa o estado atual da condição (verdadeiro ou falso);

strGroup: String; Define o nome para o grupo que será exibido no raio-x durante a execução ;

dValue: Float [Opcional]; Define o valor da variável;

colorON: Integer [Opcional]; Cor quando a condição for verdadeira;

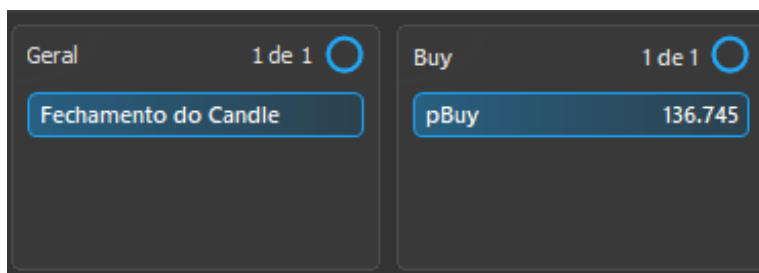
colorOF: Integer [Opcional]; Cor quando a condição for falsa;

Exemplo de uso:

No exemplo, será exibido o valor em tempo real da automação ao utilizar a função **XRay()**.

```
1 var  
2   pBuy : Float;  
3 begin  
4   pBuy := BuyPrice;  
5   if (Low <= Low[1]) then  
6     BuyAtMarket();  
7     Plot(BuyPrice);  
8     XRay("pBuy", (pBuy > 0), "Buy", pBuy);  
9 end;
```

Exemplo Gráfico:





14.34 Função NumberOfPendingOrders

Disponível em todas versões.

A função `NumberOfPendingOrders()` permite obter o número atual de ordens abertas da sua Automação de Estratégias, sendo útil para tomadas de decisão dentro de uma estratégia automatizada.

Sintaxe:

```
Função NumberOfPendingOrders(): Integer;
```

Parâmetros:

Sem parâmetros.

Observações:

Recomenda-se utilizar a função apenas no modo de execução **Ao Satisfazer Condição**, pois em outros modos o valor retornado pode estar desatualizado, gerando comportamento imprevisível na estratégia;

Exemplo de uso:

No exemplo abaixo, a estratégia utiliza a função `NumberOfPendingOrders()` para garantir que sempre sejam mantidas até 10 ordens limitadas de compra pendentes, posicionadas abaixo do preço médio de entrada.

```
1  input
2    TicksGradiente(5);
3  var
4    nIndex,nFaltando : Integer;
5    dPrecoOrdem      : Float;
6  begin
7    if not HasPosition then
8      BuyAtMarket(1);
9    nFaltando := 10 - NumberOfPendingOrders();
10   if nFaltando > 0 then
11     begin
12       for nIndex := 1 to nFaltando do
13         begin
14           dPrecoOrdem := BuyPrice - (nIndex * MinPriceIncrement * TicksGradiente);
15           BuyLimit(dPrecoOrdem);
16         end;
17       end;
18   end;
```

Exemplo Gráfico:



15 Funções de Calendário

15.1 Função BarAnnualization

Disponível em todas versões.

A função **BarAnnualization()** retorna o fator de anualização (raiz quadrada) baseado no intervalo do candle (diário = 365, semanal = 52, mensal = 12).

Sintaxe:

```
Função BarAnnualization(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo abaixo, a variável **temp** irá receber o dado de retorno da função **BarAnnualization()** de PETR4 diário.

```
1 var
2   temp : Float;
3 begin
4   temp := BarAnnualization();
5   Plot(temp);
6 end;
```

Exemplo Gráfico:



15.2 Função BarDuration

Disponível em todas versões.

A função `BarDuration()` retorna, em minutos, a duração da barra atual.

Sintaxe:

```
Função BarDuration(): Integer;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo abaixo, a variável `temp` irá receber o dado da função `BarDuration()`.

```
1 var
2   temp : Integer;
3 begin
4   temp := BarDuration();
5   Plot(temp);
6 end;
```

Exemplo Gráfico:



15.3 Função BarDurationF

Disponível em todas versões.

A função **BarDurationF()** retorna, em minutos, a duração da barra atual com valores não inteiros.

Sintaxe:

Função **BarDurationF()**: **Float**;

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo abaixo, a variável **temp** irá receber o dado da função **BarDurationF()**.

```
1 var
2   temp : Float;
3 begin
4   temp := BarDurationF();
5   Plot(temp);
6 end;
```

Exemplo Gráfico:



15.4 Função Bartype

Disponível em todas versões.

A função `Bartype()` retorna um código numérico referente ao período utilizado.

Sintaxe:

Função `Bartype()`: **Integer**;

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, a variável `numCode` irá receber um inteiro referente ao tempo determinado.

```
1 var
2   numCode : Integer;
3 begin
4   numCode := Bartype();
5   Plot(numCode);
6 end;
```

Exemplo Gráfico:





15.5 Função CalcDate

Disponível em todas versões.

A função `CalcDate()` retorna um valor o qual representa uma data deslocada, obtida ao adicionar ou subtrair dias de uma data de referência. Datas são representadas pelo tipo de dado **Integer**, no formato: **AnoMêsDia**.

Sintaxe:

```
Função CalcDate(RefDate: Integer; DaysChange: Integer): Integer;
```

Parâmetros:

RefDate: Integer; Determina a data que será utilizada como base para o deslocamento.

DaysChange: Integer; Determina quantos dias serão adicionados ou subtraídos da data de referência.

Exemplo de uso:

No exemplo abaixo, será visualizada, graficamente, a data resultante ao subtrair **14** dias da data de 16/04/2025 (**1250416**).

```
1 begin
2   Plot(CalcDate(1250416, -14));
3 end;
```

Exemplo Gráfico:



15.6 Funcao CalcTime

Disponível em todas versões.

A função `CalcTime()` retorna um valor o qual representa uma hora deslocada, obtida ao adicionar ou subtrair minutos de uma hora de referência. Horas são representadas pelo tipo de dado **Integer**, no formato (24 horas): **HorasMinutos**.

Sintaxe:

```
Função CalcTime(RefTime: Integer; MinuteChange: Integer): Integer;
```

Parâmetros:

RefTime: Integer; Determina a hora que será utilizada como base para o deslocamento.

MinuteChange: Integer; Determina quantos minutos serão adicionados ou subtraídos da hora de referência.

Exemplo de uso:

No exemplo abaixo, será visualizada, graficamente, a hora resultante ao deslocar **65** minutos a partir das **9h**.

```
1 begin
2   Plot(CalcTime(900, 65));
3 end;
```

Exemplo Gráfico:



15.7 Função CloseD

Disponível em todas versões.

A função `CloseD()` tem como finalidade retornar o valor de fechamento de um número determinado de dias atrás.

Sintaxe:

```
Função CloseD(DaysAgo: Integer): Float;
```

Parâmetros:

DaysAgo: `Integer`; Determina a quantidade desejada de dias anteriores.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor de fechamento de dois dias anteriores ao dia atual.

```
1 begin
2   Plot(CloseD(2));
3 end;
```

Exemplo Gráfico:





15.8 Função CloseM

Disponível em todas versões.

A função `CloseM()` tem como finalidade retornar o valor de fechamento de um número determinado de meses atrás.

Sintaxe:

```
Função CloseM(MonthsAgo: Integer): Float;
```

Parâmetros:

MonthsAgo: Integer; Determina a quantidade desejada de meses anteriores.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor de fechamento de 3 meses anteriores ao mês atual.

```
1 begin
2   Plot(CloseM(3));
3 end;
```

Exemplo Gráfico:





15.9 Função CloseW

Disponível em todas versões.

A função `CloseW()` tem como finalidade retornar o valor de fechamento de um número determinado de semanas atrás.

Sintaxe:

```
Função CloseW(PeriodsAgo: Integer): Float;
```

Parâmetros:

PeriodsAgo: **Integer**; Determina a quantidade desejada de semanas anteriores.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor de fechamento **2** semanas anteriores à semana atual.

```
1 begin
2   Plot(CloseW(2));
3 end;
```

Exemplo Gráfico:





15.10 Função CloseY

Disponível em todas versões.

A função `CloseY()` tem como finalidade retornar o valor de fechamento de um número determinado de anos atrás.

Sintaxe:

```
Função CloseY(YearsAgo: Integer): Float;
```

Parâmetros:

YearsAgo: **Integer**; Determina a quantidade desejada de anos anteriores.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor de valor de fechamento do ano anterior.

```
1 begin
2   Plot(CloseY(1));
3 end;
```

Exemplo Gráfico:



15.11 Função CurrentAssetDate

Disponível em todas versões.

A função `CurrentAssetDate()` retorna a data atual do ativo no gráfico. Datas são representadas pelo tipo de dado `Integer`, no formato: **1AnoMêsDia**.

Sintaxe:

```
Função CurrentAssetDate(): Integer;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a data do ativo.

```
1 begin
2   Plot(CurrentAssetDate);
3 end;
```

Exemplo Gráfico:





15.12 Função CurrentDate

Disponível em todas versões.

A função `CurrentDate()` possui como finalidade retornar a data atual do sistema. Datas são representadas pelo tipo de dado `Integer`, no formato: **1AnoMêsDia**.

Sintaxe:

```
Função CurrentDate(): Integer;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a data do dia atual.

```
1 begin
2   Plot(CurrentDate);
3 end;
```

Exemplo Gráfico:





15.13 Função CurrentTime

Disponível em todas versões.

A função `CurrentTime()` possui como finalidade retornar a hora atual do sistema. Datas são representadas pelo tipo de dado `Integer`, no formato: `HHMM`.

Sintaxe:

```
Função CurrentTime(): Integer;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a hora atual.

```
1 begin
2   Plot(CurrentTime);
3 end;
```

Exemplo Gráfico:



15.14 Função Date

Disponível em todas versões.

A função `Date()` possui como finalidade retornar a data do candle que está sendo analisado. Datas são representadas pelo tipo de dado `Integer`, no formato: `1AnoMêsDia`.

Sintaxe:

```
Função Date(): Integer;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a data do candle atual.

```
1 begin
2   Plot(Date);
3 end;
```

Exemplo Gráfico:



15.15 Função DayOfMonth

Disponível em todas versões.

A função `DayOfMonth()` retorna o dia do mês de uma data específica. Datas são representadas pelo tipo de dado `Integer`, no formato: **1AnoMêsDia**.

Sintaxe:

```
Função DayOfMonth(Date: Integer): Integer;
```

Parâmetros:

Date: `Integer`; Data para obter o dia.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o dia do mês dia 01/04/25 (**1250401**).

```
1 begin
2   Plot(DayOfMonth(1250401));
3 end;
```

Exemplo Gráfico:



15.16 Função DayOfWeek

Disponível em todas versões.

A função **DayOfWeek()** retorna o dia da semana de uma data específica. Datas são representadas pelo tipo de dado **Integer**, no formato: **1AnoMêsDia**.

Sintaxe:

```
Função DayOfWeek(Date: Integer): Integer;
```

Parâmetros:

Date: **Integer**; Data para obter o dia da semana.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o dia da semana relativo ao dia 01/04/25 (**1250401**), que é 2 (terça-feira).

```
1 begin
2   Plot(DayOfWeek(1250401));
3 end;
```

Exemplo Gráfico:



15.17 Função DaysToExpiration

Disponível em todas versões.

A função **DaysToExpiration()** retorna a quantidade de dias úteis restantes até a terceira sexta-feira de um determinado mês e ano.

Sintaxe:

```
Função DaysToExpiration(ExpMonth: Integer; ExpYear: Integer): Integer;
```

Parâmetros:

ExpMonth: **Integer**; Determina o mês que se deseja a informação.

ExpYear: **Integer**; Determina o ano desejado para análise, onde deverá estar no formato: 1AnoDesejado.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico quantidade de dias úteis até a terceira sexta-feira de dezembro/2025.

```
1 begin
2   Plot(DaysToExpiration(12, 125));
3 end;
```

Exemplo Gráfico:



15.18 Função ELDate

Disponível em todas versões.

A função **ELDate()** possui como finalidade retornar uma data em EasyLanguage format (YYYYMMDD). Datas são representadas pelo tipo de dado **Integer**, no formato: **1AnoMêsDia**.

Sintaxe:

```
Função ELDate(Year: Integer; Month: Integer; Day: Integer): Integer;
```

Parâmetros:

Year: **Integer**; Determina o ano no formato YYYY.

Month: **Integer**; Determina mês no formato MM.

Day: **Integer**; Determina o dia no formato DD.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a data de 25/12/2024 no formato: **1241225**.

```
1 begin
2   Plot(ELDate(2024, 12, 25));
3 end;
```

Exemplo Gráfico:



15.19 Função ELDate_Conso

Disponível em todas versões.

A função `ELDate_Conso()` possui como finalidade converter uma data `YYYYMMDD` em EasyLanguage format (`YYYYM-MDD`).

Sintaxe:

```
Função ELDate_Conso(Date: Integer): Integer;
```

Parâmetros:

Date: Integer; Data no formato YYYYMMDD.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a data de 25/12/2024 no formato: `20241225`.

```
1 begin
2   Plot(ELDate_Conso(20241225));
3 end;
```

Exemplo Gráfico:



15.20 Função FindBar

Disponível em todas versões.

A função **FindBar**() retorna o índice de uma barra através de uma data e hora, onde a contagem é iniciada a partir do candle atual (índice 0). Datas e horas são representadas, respectivamente, pelo tipo de dado **Integer**, no formato: **1AnoMêsDia** e **HHMM**.

Sintaxe:

```
Função FindBar(TargetDate: Integer; TargetTime: Integer): Integer;
```

Parâmetros:

TargetDate: **Integer**; Data do candle.

TargetTime: **Integer**; Hora do candle específico.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o índice do candle relacionado com o horário **11h00** do dia atual.

```
1 begin
2   Plot(FindBar(CurrentDate, 1100));
3 end;
```

Exemplo Gráfico:



15.21 Função Friday

Disponível em todas versões.

A função `Friday()` retorna o número **5**, representando o dia da semana: sexta-feira.

Sintaxe:

```
Função Friday(): Integer;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o número **5**, referente ao dia específico da semana.

```
1 begin
2   Plot(Friday);
3 end;
```

Exemplo Gráfico:





15.22 Função HighD

Disponível em todas versões.

A função `HighD()` tem como finalidade retornar o valor de máxima de um número determinado de dias atrás.

Sintaxe:

```
Função HighD(DaysAgo: Integer): Float;
```

Parâmetros:

DaysAgo: Integer; Determina a quantidade desejada de dias anteriores.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor de máxima de dois dias anteriores ao dia atual.

```
1 begin
2   Plot(HighD(2));
3 end;
```

Exemplo Gráfico:





15.23 Função HighM

Disponível em todas versões.

A função `HighM()` tem como finalidade retornar o valor de máxima de um número determinado de meses atrás.

Sintaxe:

```
Função HighM(MonthsAgo: Integer): Float;
```

Parâmetros:

MonthsAgo: `Integer`; Determina a quantidade desejada de meses anteriores.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor de máxima de cinco meses anteriores ao mês atual.

```
1 begin
2   Plot(HighM(5));
3 end;
```

Exemplo Gráfico:



15.24 Função HighW

Disponível em todas versões.

A função `HighW()` tem como finalidade retornar o valor de máxima de um número determinado de semanas atrás.

Sintaxe:

```
Função HighW(PeriodsAgo: Integer): Float;
```

Parâmetros:

`PeriodsAgo`: **Integer**; Determina a quantidade desejada de semanas anteriores.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor de máxima de sete semanas anteriores à semana atual.

```
1 begin
2   Plot(HighW(7));
3 end;
```

Exemplo Gráfico:



15.25 Função HighY

Disponível em todas versões.

A função **HighY()** tem como finalidade retornar o valor de máxima de um número determinado de anos atrás.

Sintaxe:

```
Função HighY(YearsAgo: Integer): Float;
```

Parâmetros:

YearsAgo: **Integer**; Determina a quantidade desejada de anos anteriores.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor de máxima de dois anos anteriores ao atual.

```
1 begin
2   Plot(HighY(2));
3 end;
```

Exemplo Gráfico:



15.26 Função LastCalcDate

Disponível em todas versões.

A função `LastCalcDate()` retorna a data do último candle completo formado dentro do gráfico. Datas são representadas pelo tipo de dado `Integer`, no formato: **1AnoMêsDia**.

Sintaxe:

Função `LastCalcDate()`: `Integer`;

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a data dos candles de acordo com o andamento do mercado.

```
1 begin
2   Plot>LastCalcDate();
3 end;
```

Exemplo Gráfico:





15.27 Função LastCalcTime

Disponível em todas versões.

A função `LastCalcTime()` retorna a hora do último candle completo formado dentro do gráfico, no formato 24h (HHMM).

Sintaxe:

```
Função LastCalcTime(): Integer;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a hora de fechamento dos candles, de acordo com o andamento do mercado.

```
1 begin
2   Plot>LastCalcTime();
3 end;
```

Exemplo Gráfico:



15.28 Função LastDayOfMonth

Disponível em todas versões.

A função `LastDayOfMonth()` retorna o último dia do mês de referência.

Sintaxe:

```
Função LastDayOfMonth(Month: Integer): Integer;
```

Parâmetros:

Month: Integer; Determina o mês de referência, de Janeiro (1) a Dezembro (12).

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o último dia (30) do mês de abril.

```
1 begin
2   Plot( LastDayOfMonth(4));
3 end;
```

Exemplo Gráfico:



15.29 Função LowD

Disponível em todas versões.

A função **LowD()** tem como finalidade retornar o valor de mínima de um número determinado de dias atrás.

Sintaxe:

```
Função LowD(DaysAgo: Integer): Float;
```

Parâmetros:

DaysAgo: Integer; Determina a quantidade desejada de dias anteriores.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor de mínima de três dias anteriores ao dia atual.

```
1 begin
2   Plot(LowD(3));
3 end;
```

Exemplo Gráfico:



15.30 Função LowM

Disponível em todas versões.

A função `LowM()` tem como finalidade retornar o valor de mínima de um número determinado de meses atrás.

Sintaxe:

```
Função LowM(MonthsAgo: Integer): Float;
```

Parâmetros:

MonthsAgo: `Integer`; Determina a quantidade desejada de meses anteriores.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor de mínima de três meses (3) atrás.

```
1 begin
2   Plot(LowM(3));
3 end;
```

Exemplo Gráfico:



15.31 Função LowW

Disponível em todas versões.

A função `LowW()` tem como finalidade retornar o valor de mínima de um número determinado de semanas atrás.

Sintaxe:

```
Função LowW(PeriodsAgo: Integer): Float;
```

Parâmetros:

PeriodsAgo: **Integer**; Determina a quantidade desejada de semanas anteriores.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor de mínima de sete semanas anteriores à semana atual.

```
1 begin
2   Plot(LowW(7));
3 end;
```

Exemplo Gráfico:



15.32 Função LowY

Disponível em todas versões.

A função **LowY()** tem como finalidade retornar o valor de mínima de um número determinado de anos atrás.

Sintaxe:

```
Função LowY(YearsAgo: Integer): Float;
```

Parâmetros:

YearsAgo: **Integer**; Determina a quantidade desejada de anos anteriores.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor de mínima de dois anos anteriores ao atual.

```
1 begin
2   Plot(LowY(2));
3 end;
```

Exemplo Gráfico:



15.33 Função MinutesIntoWeek

A função `MinutesIntoWeek()` retorna o número de minutos entre domingo (Dia: 0 – Hora: 0h00) até o dia e hora determinados por parâmetro.

Sintaxe:

```
Função MinutesIntoWeek(XDay: Integer; XTime: Integer): Integer;
```

Parâmetros:

XDay: **Integer**; Dia de limite para a conversão em minutos.

- 0: Domingo;
- 1: Segunda;
- 2: Terça;
- 3: Quarta;
- 4: Quinta;
- 5: Sexta;
- 6: Sábado.

XTime: **Integer**; Hora de limite para a conversão.

Exemplo de uso:

No exemplo a seguir, será atribuído à variável `tMin` o total de **8640** minutos entre domingo (Dia 0, 00h00) e sábado (Dia 6, 00h00).

```
1 var
2   tMin : Integer;
3 begin
4   tMin := MinutesIntoWeek(6 , 0000);
5   Plot(tMin);
6 end;
```

Exemplo Gráfico:





15.34 Função Monday

A função `Monday()` retorna o número **1**, representando o dia da semana: segunda-feira.

Sintaxe:

```
Função Monday(): Integer;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor **1**, referente ao dia específico da semana.

```
1 begin
2   Plot(Monday);
3 end;
```

Exemplo Gráfico:





15.35 Função Month

Disponível em todas versões.

A função `Month()` retorna o mês de uma data específica. Datas são representadas pelo tipo de dado **Integer**, no formato: **1AnoMêsDia**.

Sintaxe:

```
Função Month(Date: Integer): Integer;
```

Parâmetros:

Date: **Integer**; Data para obter o mês.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o mês atual.

```
1 begin
2   Plot(Month(CurrentDate));
3 end;
```

Exemplo Gráfico:



15.36 Função Next3rdFriday

Disponível em todas versões.

A função `Next3rdFriday()` retorna a quantidade de dias úteis restantes até a terceira sexta-feira de determinado mês subsequente.

Sintaxe:

```
Função Next3rdFriday(Series: Integer): Integer;
```

Parâmetros:

Series: Integer; Determina o mês que se deseja a informação, onde a contagem inicia-se a partir do mês atual (0).

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a quantidade de dias úteis até a terceira sexta-feira do mês seguinte.

```
1 begin
2   Plot(Next3rdFriday(1));
3 end;
```

Exemplo Gráfico:



15.37 Função OpenD

Disponível em todas versões.

A função `OpenD()` tem como finalidade retornar o valor de abertura de um número determinado de dias atrás.

Sintaxe:

```
Função OpenD(DaysAgo: Integer): Float;
```

Parâmetros:

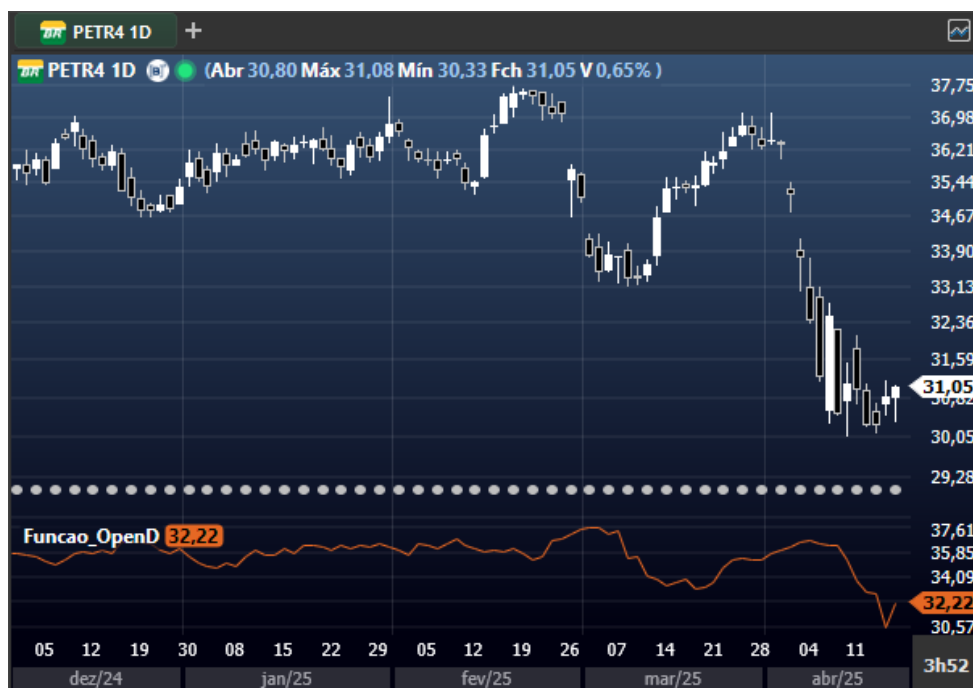
DaysAgo: **Integer**; Determina a quantidade desejada de dias anteriores.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor de abertura de seis dias anteriores ao dia atual.

```
1 begin
2   Plot(OpenD(6));
3 end;
```

Exemplo Gráfico:





15.38 Função OpenM

Disponível em todas versões.

A função `OpenM()` tem como finalidade retornar o valor de abertura de um número determinado de meses atrás.

Sintaxe:

```
Função OpenM(MonthsAgo: Integer): Float;
```

Parâmetros:

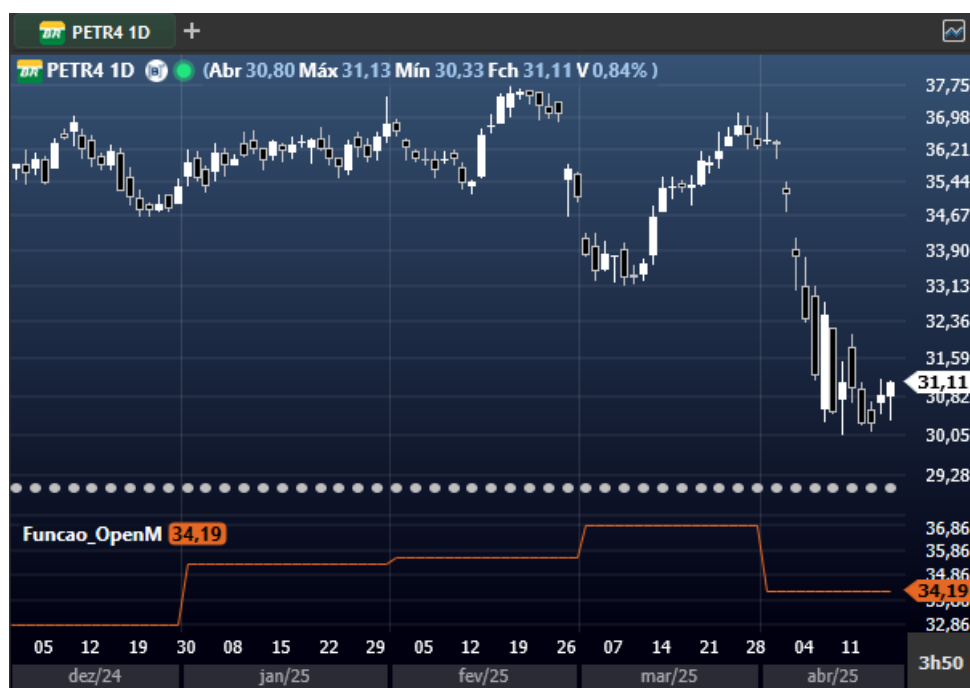
MonthsAgo: Integer; Determina a quantidade desejada de meses anteriores.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor de abertura do mês anterior ao atual.

```
1 begin
2   Plot(OpenM(1));
3 end;
```

Exemplo Gráfico:



15.39 Função OpenW

Disponível em todas versões.

A função `OpenW()` tem como finalidade retornar o valor de abertura de um número determinado de semanas atrás.

Sintaxe:

```
Função OpenW(PeriodsAgo: Integer): Float;
```

Parâmetros:

PeriodsAgo: **Integer**; Determina a quantidade desejada de semanas anteriores.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor de abertura de cinco semanas anteriores à semana atual.

```
1 begin
2   Plot(OpenW(5));
3 end;
```

Exemplo Gráfico:





15.40 Função OpenY

Disponível em todas versões.

A função `OpenY()` tem como finalidade retornar o valor de abertura de um número determinado de anos atrás.

Sintaxe:

```
Função OpenY(YearsAgo: Integer): Float;
```

Parâmetros:

YearsAgo: **Integer**; Determina a quantidade desejada de anos anteriores.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor de abertura do ano anterior ao atual.

```
1 begin
2   Plot(OpenY(1));
3 end;
```

Exemplo Gráfico:



15.41 Função RS_BarsPerDay

Disponível em todas versões.

A função **RS_BarsPerDay()** retorna o número estimado de candles de determinada periodicidade (em minutos). Caso o período seja diário, será obtido 1 como retorno, e, para períodos em minutos, ocorrerá a divisão do número total de minutos em um dia (1440) pela periodicidade selecionada.

Sintaxe:

```
Função RS_BarsPerDay(): Integer;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da chamada de função.

```
1 begin
2   Plot(RS_BarsPerDay);
3 end;
```

Exemplo Gráfico:



15.42 Função Saturday

Disponível em todas versões.

A função `Saturday()` retorna o número **6**, representando o dia da semana: sábado.

Sintaxe:

```
Função Saturday(): Integer;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o número **6**, referente ao dia específico da semana.

```
1 begin
2   Plot(Saturday);
3 end;
```

Exemplo Gráfico:



15.43 Função Sunday

Disponível em todas versões.

A função `Sunday()` retorna o número `0`, representando o dia da semana: domingo.

Sintaxe:

```
Função Sunday(): Integer;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o número `0`, referente ao dia específico da semana.

```
1 begin
2   Plot(Sunday);
3 end;
```

Exemplo Gráfico:



15.44 Função Thursday

Disponível em todas versões.

A função `Thursday()` retorna o número **4**, representando o dia da semana: quinta-feira.

Sintaxe:

```
Função Thursday(): Integer;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o número **4**, referente ao dia específico da semana.

```
1 begin
2   Plot(Thursday);
3 end;
```

Exemplo Gráfico:



15.45 Função Time

Disponível em todas versões.

A função `Time()` possui como finalidade retornar a hora de abertura do candle atual.

Sintaxe:

```
Função Time(): Integer;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a hora de abertura do candle.

```
1 begin
2   Plot(Time);
3 end;
```

Exemplo Gráfico:



15.46 Função TimeExchange

Disponível em todas versões.

A função `TimeExchange()` retorna o tempo do candle de acordo com o timezone da bolsa.

Sintaxe:

```
Função TimeExchange(): Integer;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a data de acordo com o timezone da bolsa.

```
1 begin
2   Plot(TimeExchange);
3 end;
```

Exemplo Gráfico:



15.47 Função TimeToMinutes

Disponível em todas versões.

A função `TimeToMinutes()` tem como propósito converter um valor de hora no formato inteiro (HHMM) para a sua representação correspondente em minutos decorridos desde 00h00.

Sintaxe:

```
Função TimeToMinutes(XTime: Integer): Integer;
```

Parâmetros:

XTime: Integer; Hora para a conversão.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a conversão em minutos da hora: **12h30** (750 minutos).

```
1 begin
2   Plot(TimeToMinutes(1230));
3 end;
```

Exemplo Gráfico:



15.48 Função Today

Disponível em todas versões.

A função **Today()** possui como finalidade retornar a data atual do sistema. Datas são representadas pelo tipo de dado **Integer**, no formato: **1AnoMêsDia**.

Sintaxe:

```
Função Today(): Integer;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a data do dia atual.

```
1 begin
2   Plot(Today);
3 end;
```

Exemplo Gráfico:



15.49 Função Tuesday

Disponível em todas versões.

A função **Tuesday()** retorna o número **2**, representando o dia da semana: terça-feira.

Sintaxe:

```
Função Tuesday(): Integer;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o número **2**, referente ao dia específico da semana.

```
1 begin
2   Plot(Tuesday);
3 end;
```

Exemplo Gráfico:



15.50 Função VolumeD

Disponível em todas versões.

A função **VolumeD()** tem como finalidade retornar o volume financeiro de um número determinado de dias atrás.

Sintaxe:

```
Função VolumeD(DaysAgo: Integer): Float;
```

Parâmetros:

DaysAgo: **Integer**; Determina a quantidade desejada de dias anteriores.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o volume de **6** dias anteriores ao dia atual.

```
1 begin
2   Plot(VolumeD(6));
3 end;
```

Exemplo Gráfico:



15.51 Função VolumeM

Disponível em todas versões.

A função **VolumeM()** tem como finalidade retornar o volume financeiro de um número determinado de meses atrás.

Sintaxe:

```
Função VolumeM(MonthsAgo: Integer): Float;
```

Parâmetros:

MonthsAgo: Integer; Determina a quantidade desejada de meses anteriores.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o volume do mês anterior ao atual.

```
1 begin
2   Plot(VolumeM(1));
3 end;
```

Exemplo Gráfico:



15.52 Função VolumeW

Disponível em todas versões.

A função `VolumeW()` tem como finalidade retornar o volume financeiro de um número determinado de semanas atrás.

Sintaxe:

```
Função VolumeW(WeeksAgo: Integer): Float;
```

Parâmetros:

`WeeksAgo`: **Integer**; Determina a quantidade desejada de semanas anteriores.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor volume de cinco semanas anteriores à semana atual.

```
1 begin
2   Plot(VolumeW(5));
3 end;
```

Exemplo Gráfico:



15.53 Função VolumeY

Disponível em todas versões.

A função **VolumeY()** tem como finalidade retornar o volume financeiro de um número determinado de anos atrás.

Sintaxe:

```
Função VolumeY(YearsAgo: Integer): Float;
```

Parâmetros:

YearsAgo: **Integer**; Determina a quantidade desejada de anos anteriores.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o volume do ano anterior ao atual.

```
1 begin
2   Plot(VolumeY(1));
3 end;
```

Exemplo Gráfico:





15.54 Função Wednesday

Disponível em todas versões.

A função `Wednesday()` retorna o número **3**, representando o dia da semana: quarta-feira.

Sintaxe:

```
Função Wednesday(): Integer;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o número **3**, referente ao dia específico da semana.

```
1 begin
2   Plot(Wednesday);
3 end;
```

Exemplo Gráfico:





15.55 Função Year

Disponível em todas versões.

A função `Year()` retorna o ano de uma data específica. Datas são representadas pelo tipo de dado **Integer**, no formato: **1AnoMêsDia**.

Sintaxe:

```
Função Year(Date: Integer): Integer;
```

Parâmetros:

Date: **Integer**; Data para obter o ano.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o ano atual.

```
1 begin
2   Plot(Year(CurrentDate));
3 end;
```

Exemplo Gráfico:





15.56 Função Yesterday

Disponível em todas versões.

A função `Yesterday()` retorna a data do dia de ontem. Datas são representadas pelo tipo de dado **Integer**, no formato: **1AnoMêsDia**.

Sintaxe:

```
Função Yesterday(): Integer;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a data de ontem.

```
1 begin
2   Plot(Yesterday);
3 end;
```

Exemplo Gráfico:





16 Funções de Candlestick

16.1 Função C_3WhSolds_3BlkCrows

Disponível em todas versões.

A função `C_3WhSolds_3BlkCrows()` identifica a ocorrência de dois tipos de candles: 3 White Soldiers e 3 Black Crows.

Sintaxe:

```
Função C_3WhSolds_3BlkCrows(Length : Integer; Factor : Integer; o3WhiteSoldiers : Integer; o3BlackCrows : Integer): Integer;
```

Parâmetros:

Length: Integer; Tamanho utilizado para calcular a média do corpo do candle.

Factor: Integer; Determina quantas vezes a sombra do candle deve ser maior que o seu corpo.

o3WhiteSoldiers: Integer [var]; Variável para identificação de padrão (3 White Soldiers).

o3BlackCrows: Integer [var]; Variável para identificação de padrão (3 Black Crows).

Exemplo de uso:

No exemplo a seguir, caso seja identificado algum dos padrões entre 3 White Soldiers ou 3 Black Crows, considerando 20 (Length) e 20 (Factor), será aplicada uma coloração (vermelho).

```
1 var
2   o3WhiteSoldiers, o3BlackCrows: Integer;
3 begin
4   if(C_3WhSolds_3BlkCrows(20, 20, o3WhiteSoldiers, o3BlackCrows) = 1) then
5     PaintBar(clRed);
6 end
```



Exemplo Gráfico:



16.2 Função C_BullEng_BearEng

Disponível em todas versões.

A função `C_BullEng_BearEng()` identifica a ocorrência de dois tipos de candles: Bullish Engulfing e Bearish Engulfing.

Sintaxe:

```
Função C_BullEng_BearEng(Length : Integer; oBullishEngulfing : Integer;
    oBearishEngulfing : Integer): Integer;
```

Parâmetros:

Length: Integer; Tamanho utilizado para calcular a média do corpo do candle.

oBullishEngulfing: Integer [var]; Variável para identificação de padrão (Bullish Engulfing).

oBearishEngulfing: Integer [var]; Variável para identificação de padrão (Bearish Engulfing).

Exemplo de uso:

No exemplo a seguir, caso seja identificado algum dos padrões entre Bullish Engulfing ou Bearish Engulfing, considerando 20 (Length), será aplicada uma coloração (amarelo).

```
1 var
2     oBullishEngulfing, oBearishEngulfing: Integer;
3 begin
4     if (C_BullEng_BearEng(20, oBullishEngulfing, oBearishEngulfing) = 1) then
5         PaintBar(c1Yellow);
6 end
```

Exemplo Gráfico:





16.3 Função C_Doji

Disponível em todas versões.

A função `C_Doji()` identifica a ocorrência de um candle tipo Doji.

Sintaxe:

```
Função C_Doji(Percentual : Integer): Integer;
```

Parâmetros:

Percentual: **Integer**; Limiar para (Abertura - Fechamento) que seria uma percentagem do intervalo do candle.

Exemplo de uso:

No exemplo a seguir, caso o padrão seja identificado, considerando percentual de **5%**, será aplicada uma coloração (vermelho).

```
1 begin
2   if(C_Doji(5) = 1) then
3     PaintBar(c1Red);
4   end
```

Exemplo Gráfico:



16.4 Função C_Hammer_HangingMan

Disponível em todas versões.

A função `C_Hammer_HangingMan()` identifica a ocorrência de dois tipos de candles: Hammer e Hanging Man.

Sintaxe:

```
Função C_Hammer_HangingMan(Length : Integer; Factor : Integer; oHammer : Integer;
    oHangingMan : Integer): Integer;
```

Parâmetros:

Length: Integer; Tamanho utilizado para calcular a média do corpo do candle.

Factor: Integer; Determina quantas vezes a sombra do candle deve ser maior que o seu corpo.

oHammer: Integer [var]; Variável para identificação de padrão (Hammer).

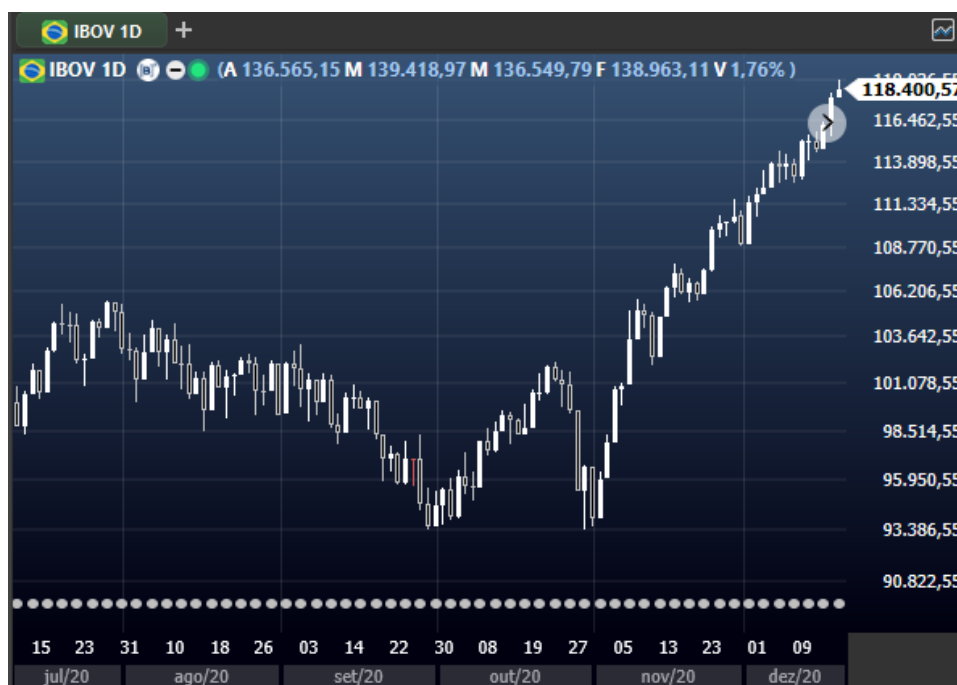
oHangingMan: Integer [var]; Variável para identificação de padrão (Hanging Man).

Exemplo de uso:

No exemplo a seguir, caso seja identificado algum dos padrões entre Hammer ou Hanging Man, considerando **30** (Length) e **30** (Factor), será aplicada uma coloração (vermelho).

```
1 var
2   oHammer, oHangingMan: Integer;
3 begin
4   if (C_Hammer_HangingMan(30, 30, oHammer, oHangingMan) = 1) then
5     PaintBar(clRed);
6 end
```

Exemplo Gráfico:





16.5 Função C_MornDoji_EveDoji

Disponível em todas versões.

A função `C_MornDoji_EveDoji()` identifica a ocorrência de dois tipos de candles: Morning Doji Star e Evening Doji Star.

Sintaxe:

```
Função C_MornDoji_EveDoji(Length : Integer; Percentual : Float; oMorningDojiStar : Integer; oEveningDojiStar : Integer): Integer;
```

Parâmetros:

Length: Integer; Tamanho utilizado para calcular a média do corpo do candle.

Percentual: Float; Doji limiar para o (abrir - fechar) como uma percentagem do intervalo da barra.

oMorningDojiStar: Integer [var]; Variável para identificação de padrão (Morning Doji Star).

oEveningDojiStar: Integer [var]; Variável para identificação de padrão (Evening Doji Star).

Exemplo de uso:

No exemplo a seguir, caso seja identificado algum dos padrões entre Morning Doji Star ou Evening Doji Star, considerando 30 (Length) e 30 (Percentual), será aplicada uma coloração (vermelho).

```
1 var
2   oMorningDojiStar, oEveningDojiStar : Integer;
3 begin
4   if (C_MornDoji_EveDoji(30, 30, oMorningDojiStar, oEveningDojiStar) = 1) then
5     PaintBar(c1Red);
6 end
```

Exemplo Gráfico:



16.6 Função C_MornStar_EveStar

Disponível em todas versões.

A função `C_MornStar_EveStar()` identifica a ocorrência de dois tipos de candles: Morning Star e Evening Star.

Sintaxe:

```
Função C_MornStar_EveStar(Length : Integer; oMorningStar : Integer; oEveningStar : Integer): Integer;
```

Parâmetros:

Length: Integer; Tamanho utilizado para calcular a média do corpo do candle.

oMorningStar: Integer [var]; Variável para identificação de padrão (Morning Star).

oEveningStar: Integer [var]; Variável para identificação de padrão (Evening Star).

Exemplo de uso:

No exemplo a seguir, caso seja identificado algum dos padrões entre Evening Star e Morning Star, considerando 30 (Length), será aplicada uma coloração (verde).

```
1 var
2   oMorningStar, oEveningStar : Integer;
3 begin
4   if (C_MornStar_EveStar(30, oMorningStar, oEveningStar) = 1) then
5     PaintBar(c1Green);
6   end
```

Exemplo Gráfico:





16.7 Função C_PierceLine_DkCloud

Disponível em todas versões.

A função `C_PierceLine_DkCloud()` identifica a ocorrência de dois tipos de candles: Piercing Line e Dark Cloud.

Sintaxe:

```
Função C_PierceLine_DkCloud(Length : Integer; oPiercingLine : Integer; oDarkCloud : Integer): Integer;
```

Parâmetros:

Length: Integer; Tamanho utilizado para calcular a média do corpo do candle.

oPiercingLine: Integer [var]; Variável para identificação de padrão (Piercing Line).

oDarkCloud: Integer [var]; Variável para identificação de padrão (Dark Cloud).

Exemplo de uso:

No exemplo a seguir, caso seja identificado algum dos padrões entre Dark Cloud e Piercing Line, considerando 30 (Length), será aplicada uma coloração (verde).

```
1 var
2   oPiercingLine, oDarkCloud : Integer;
3 begin
4   if (C_PierceLine_DkCloud(30, oPiercingLine, oDarkCloud) = 1) then
5     PaintBar(clGreen);
6   end
```

Exemplo Gráfico:



16.8 Função C_ShootingStar

Disponível em todas versões.

A função `C_ShootingStar()` identifica a ocorrência de candles tipo Shooting Star.

Sintaxe:

```
Função C_ShootingStar(Length : Integer; Factor : Integer; oDarkCloud : Integer):  
    Integer;
```

Parâmetros:

Length: Integer; Tamanho utilizado para calcular a média do corpo do candle.

Factor: Integer; Determina quantas vezes a sombra do candle deve ser maior que o seu corpo.

Exemplo de uso:

No exemplo a seguir, caso o padrão seja identificado, considerando 30 (Length) e 30 (Factor), será aplicada uma coloração (verde).

```
1 begin  
2   if (C_ShootingStar(30, 1) = 1) then  
3     PaintBar(clGreen);  
4   end
```

Exemplo Gráfico:



17 Exemplos

17.1 Função DiMaisDiMenos

Disponível em todas versões.

O indicador "Diferencial(DI+/DI-)" é chamado pela função `DiMaisDiMenos()`, sendo um indicador de momento, tenta qualificar a tendência ou comportamento direcional de um mercado. O exemplo com o código fonte está disponível no editor de estratégias, para visualizá-lo, acesse o menu: "abrir > exemplos".

Sintaxe:

```
Função DiMaisDiMenos(Período : Integer): Float;
```

Parâmetros:

Período: **Integer**; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o indicador criado.

```
1 begin
2   Plot(DiMaisDiMenos(14));
3 end;
```

Exemplo Gráfico:



17.2 Função IFR

Disponível em todas versões.

O "Índice de Força Relativo" **IFR()** mede a evolução da relação de forças entre compradores e vendedores ao longo do tempo. o exemplo com o código fonte está disponível no editor de estratégias, para visualizá-lo, acesse o menu: "abrir > exemplos".

Sintaxe:

```
Função IFR(Período : Integer): Float;
```

Parâmetros:

Período: **Integer**; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "IFR", considerando **2** períodos para o cálculo.

```
1 begin
2   Plot(IFR(2));
3 end;
```

Exemplo Gráfico:



17.3 Função PaintVar

Disponível em todas versões.

A função `PaintVar()` possui a implementação de uma estratégia de coloração, a qual compara se o fechamento do último candle é positivo ou negativo em relação ao fechamento do (candle) anterior, e, como indicador, apenas retorna o último preço do ativo. o exemplo com o código fonte está disponível no editor de estratégias, para visualizá-lo, acesse o menu: "abrir > exemplos".

Sintaxe:

Função `PaintVar()`: **Float**;

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado o último preço, onde a regra de coloração correspondente poderá ser aplicada sobre o indicador.

```
1 begin
2   Plot(PaintVar);
3 end;
```

Exemplo Gráfico:





18 Funções Gráficas

18.1 Função AvgPrice

Disponível em todas versões.

A função `AvgPrice()` retorna o valor da média entre Abertura, Fechamento, Máxima e Mínima, de determinado candle.

Sintaxe:

```
Função AvgPrice(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, a variável `AvgPricePlot` recebe o preço médio da barra atual com a função `AvgPrice()` e plota esse valor no gráfico.

```
1 var
2   AvgPricePlot : Float;
3 begin
4   AvgPricePlot := AvgPrice();
5   Plot(AvgPricePlot);
6 end;
```

Exemplo Gráfico:





18.2 Função BarCount

Disponível em todas versões.

A função `BarCount()` retorna a quantidade total de barras.

Sintaxe:

```
Função BarCount(): Integer;
```

Parâmetros:

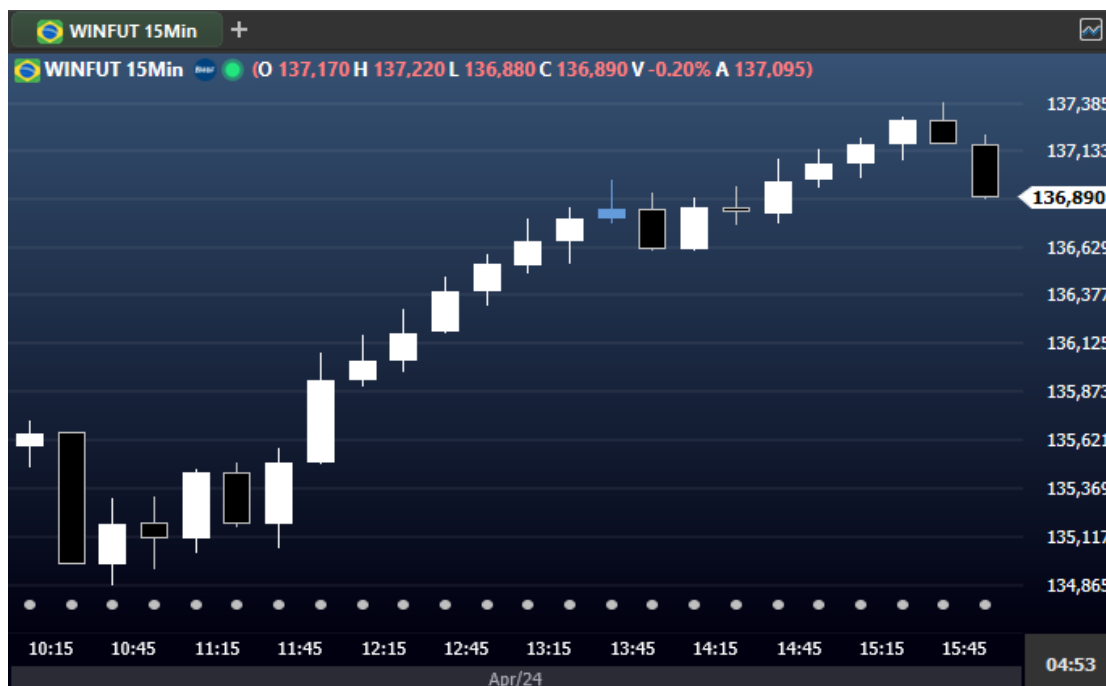
Sem parâmetros.

Exemplo de uso:

No exemplo, será pintado de azul (`c1Blue`) a barra que está **10** posições antes da última no gráfico.

```
1 begin
2   if CurrentBar() = BarCount() - 10 then
3     PaintBar(c1Blue);
4   end;
```

Exemplo Gráfico:





18.3 Função ConsoleLog

Disponível em todas versões.

A função `ConsoleLog()` imprime um texto no terminal de console para ajudar na depuração do código.

Sintaxe:

```
Função ConsoleLog(Content: String; Color: Integer): Void;
```

Parâmetros:

Content: `String`; Texto que irá ser impresso no console de depuração.

Color: `Integer` [Opcional]; Cor do texto impresso.

Exemplo de uso:

No exemplo abaixo, a função `ConsoleLog()` irá imprimir os dias e as variações do WDOFUT em que teve variação maior de 4%.

```
1 begin
2   if Close[1] > Close * 1.04 then
3     ConsoleLog("Day: " + Date + " Variation: " +
4       (Close[1] / Close - 1) * 100 + "%", clRed);
5   end;
```

Exemplo Gráfico:

Compilation Menu

Running Code

```
Day: 1200226 Variation: 5.38%
Day: 1200306 Variation: 4.26%
Day: 1200309 Variation: 10.73%
Day: 1200312 Variation: 5.52%
Day: 1200312 Variation: 5.75%
Day: 1200316 Variation: 7.53%
Day: 1200316 Variation: 5.78%
Day: 1200318 Variation: 8.38%
Day: 1200319 Variation: 4.63%
Day: 1200327 Variation: 4.42%
Day: 1200401 Variation: 4.33%
```



18.4 Função CurrentBar

Disponível em todas versões.

A função `CurrentBar()` tem como finalidade retornar ao usuário o índice do candle atual.

Sintaxe:

```
Função CurrentBar(): Integer;
```

Parâmetros:

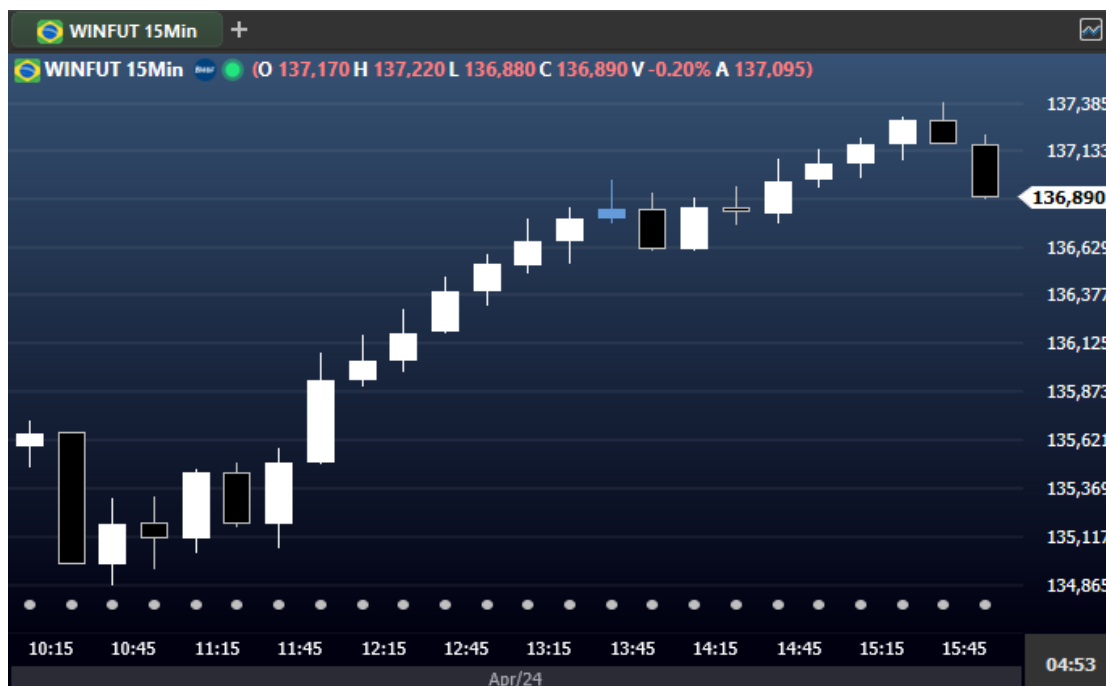
Sem parâmetros.

Exemplo de uso:

No exemplo, será pintado de azul (`clBlue`) a barra que está **10** posições antes da última no gráfico.

```
1 begin
2   if CurrentBar() = BarCount() - 10 then
3     PaintBar(clBlue);
4   end;
```

Exemplo Gráfico:





18.5 Função GetPlotColor

Disponível em todas versões.

A função **GetPlotColor** possui como funcionalidade retornar o valor numérico da cor de determinado plot.

Sintaxe:

```
Função GetPlotColor(PlotNum: Integer): Integer;
```

Parâmetros:

PlotNum: Integer; Número do Plot para obter a cor.

Exemplo de uso:

No exemplo, será aplicada a coloração do Plot 1 ao Plot 2.

```
1 begin
2   SetPlotColor(1, RGB(100, 100, 255));
3   SetPlotColor(2, GetPlotColor(1));
4   PlotN(1, close);
5   PlotN(2, Open);
6 end;
```

Exemplo Gráfico:





18.6 Função GetPlotWidth

Disponível em todas versões.

A função `GetPlotWidth` possui como finalidade retornar o valor da espessura de determinado plot.

Sintaxe:

```
Função GetPlotWidth(PlotNum: Integer): Integer;
```

Parâmetros:

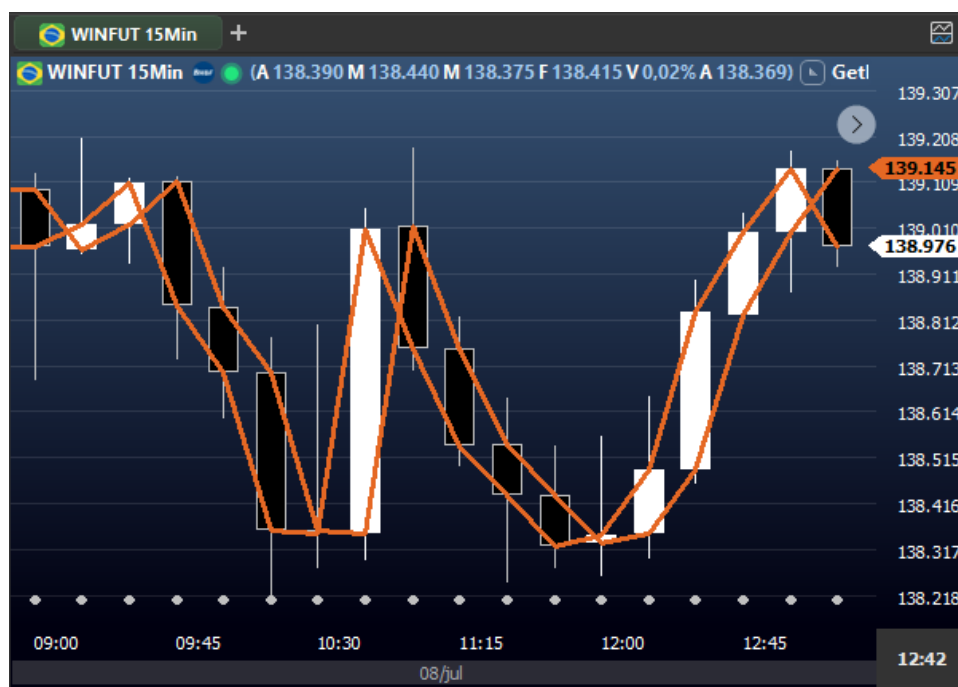
PlotNum: Integer; Número do Plot para obter a espessura

Exemplo de uso:

No exemplo, será aplicada a espessura do Plot 1 ao Plot 2.

```
1 begin
2   SetPlotWidth(1, 5);
3   SetPlotWidth(2, GetPlotWidth(1));
4   PlotN(1,close);
5   PlotN(2,Open);
6 end;
```

Exemplo Gráfico:



18.7 Função GraphicInterval

Disponível em todas versões.

A função "`GraphicInterval()`" possui como finalidade retornar o intervalo do gráfico.

Sintaxe:

```
Função GraphicInterval(): Integer;
```

Parâmetros:

Não possui parâmetros.

Exemplo de uso:

No exemplo, será atribuído à variável `gInterval` o retorno da função.

```
1  var
2    gInterval : Integer;
3  begin
4    gInterval := GraphicInterval();
5    Plot(gInterval);
6  end;
```

Exemplo Gráfico:





18.8 Função GraphicOffset

Disponível em todas versões.

A função `GraphicOffset()` possui como finalidade retornar o offset do gráfico.

Sintaxe:

```
Função GraphicOffset(): Integer;
```

Parâmetros:

Não possui parâmetros.

Exemplo de uso:

No exemplo, será atribuído à variável `gOffset` o retorno da função.

```
1  var
2    gOffset : Integer;
3  begin
4    gOffset := GraphicOffset();
5    Plot(gOffset);
6  end;
```

Exemplo Gráfico:





18.9 Função HorizontalLine

Disponível em todas versões.

A função `HorizontalLine` tem como finalidade adicionar um estudo horizontal em um indicador.

Sintaxe:

```
Função HorizontalLine(Y: Float; Color: Integer): Void;
```

Parâmetros:

Y: **Float**; Valor do estudo.

Color: **Integer** [Optional]; Cor da linha.

Exemplo de uso:

No exemplo abaixo, será adicionada uma linha vermelha, no nível **5288**.

```
1 begin
2   HorizontalLine(5288, RGB(200, 0, 0));
3 end;
```

Exemplo Gráfico:



18.10 Função HorizontalLineCustom

Disponível em todas versões.

A função `HorizontalLineCustom` tem como finalidade adicionar um estudo horizontal customizável em um indicador.

Sintaxe:

```
Função HorizontalLineCustom(Price: Float; Color: Integer; LineWidth: Integer;  
    Style: Integer; Text: String; FontSize: Integer; TextPosition: Integer;  
    EndDate: Integer; PriceVariation: Float; StartTime: Integer; EndTime: Integer):  
    Void;
```

Parâmetros:

Price: **Float**; Valor do estudo.

Color: **Integer**; Cor da linha.

LineWidth: **Integer**; Espessura da linha.

Style: **Integer**; Estilo da linha. Ver `SetPlotStyle()` para mais detalhes.

Text: **String**; Texto da linha.

FontSize: **Integer** [Opcional]; Tamanho da fonte.

TextPosition: **Integer** [Opcional]; Posição do texto na linha.

- tpTopLeft;
- tpTopRight;
- tpBottomLeft;
- tpBottomRight.

StartDate: **Integer** [Opcional]; Data inicial da linha.

EndDate: **Integer** [Opcional]; Data final da linha.

PriceVariation: **Float** [Opcional]; Variação de preço para preencher acima e abaixo da linha principal.

StartTime: **Integer** [Opcional]; Hora inicial da linha.

EndTime: **Integer** [Opcional]; Hora final da linha.

Exemplo de uso:

No exemplo abaixo, será adicionada uma linha amarela, com espessura **3**, estilo linha, com fonte de tamanho **10**, texto no topo direito, do início ao fim do gráfico e com uma variação de **1000** no preço.

```
1 begin  
2     HorizontalLineCustom(Close + 1000, clYellow, 3, 0, "Teste",  
        10, tpTopRight, 0, 0, 1000);  
3 end;
```

Exemplo Gráfico:





18.11 Função LastBarOnChart

Disponível em todas versões.

A função `LastBarOnChart` tem como função retornar um valor Booleano mostrando se o candle atual é o último candle do gráfico.

Sintaxe:

```
Função LastBarOnChart(): Boolean;
```

Parâmetros:

Não possui parâmetros.

Exemplo de uso:

No exemplo a seguir, o candle atual será identificado com a coloração amarela.

```
1 begin
2   if (LastBarOnChart) then
3     PaintBar(c1Yellow);
4 end;
```

Exemplo Gráfico:



18.12 Função Leader

Disponível em todas versões.

A função **Leader()** retorna o valor **0** quando o ponto médio for menor que a mínima anterior ou **1** quando o ponto médio for maior que a máxima anterior.

Sintaxe:

```
Função Leader(): Integer;
```

Parâmetros:

Não possui parâmetros.

Exemplo de uso:

No exemplo abaixo, caso a função **Leader()** seja igual a **1**, o candle analisado será identificado pela cor verde.

```
1 begin
2   if (Leader() = 1) then
3     PaintBar(c1Green);
4 end;
```

Exemplo Gráfico:



18.13 Função LineSegment

Disponível em todas versões.

A função `LineSegment()` desenha um segmento de reta no gráfico com base nos parâmetros fornecidos. A linha começa no ponto definido por `PriceStart` e `DateStart` e termina em `PriceEnd` e `DateEnd`. Se o parâmetro `Extrapolate` estiver definido como verdadeiro, a linha será estendida além dos limites especificados, mantendo sua angulação original.

Sintaxe:

```
Função LineSegment (PriceStart: Float; PriceEnd: Float; StartDate: Integer; EndDate: Integer; Extrapolate: Boolean; Text: String; Color: Integer; LineWidth: Integer; Style: Integer; FontSize: Integer; TextPosition: Integer; StartTime: Integer; EndTime: Integer): Void;
```

Parâmetros:

`PriceStart`: **Float**; Valor inicial da linha no eixo de preço.

`PriceEnd`: **Float**; Valor final da linha no eixo de preço.

`StartDate`: **Integer**; Data inicial da linha.

`EndDate`: **Integer**; Data final da linha.

`Extrapolate`: **Boolean**; Define se a linha deve ser extrapolada além dos limites definidos.

`Text`: **String**; Texto a ser exibido próximo à linha.

`Color`: **Integer** [Opcional]; Cor da linha.

`LineWidth`: **Integer** [Opcional]; Espessura da linha.

`Style`: **Integer** [Opcional]; Estilo da linha.

`FontSize`: **Integer** [Opcional]; Tamanho da fonte do texto.

`TextPosition`: **Integer** [Opcional]; Posição do texto.

`StartTime`: **Integer** [Opcional]; Hora de início (caso aplicável).

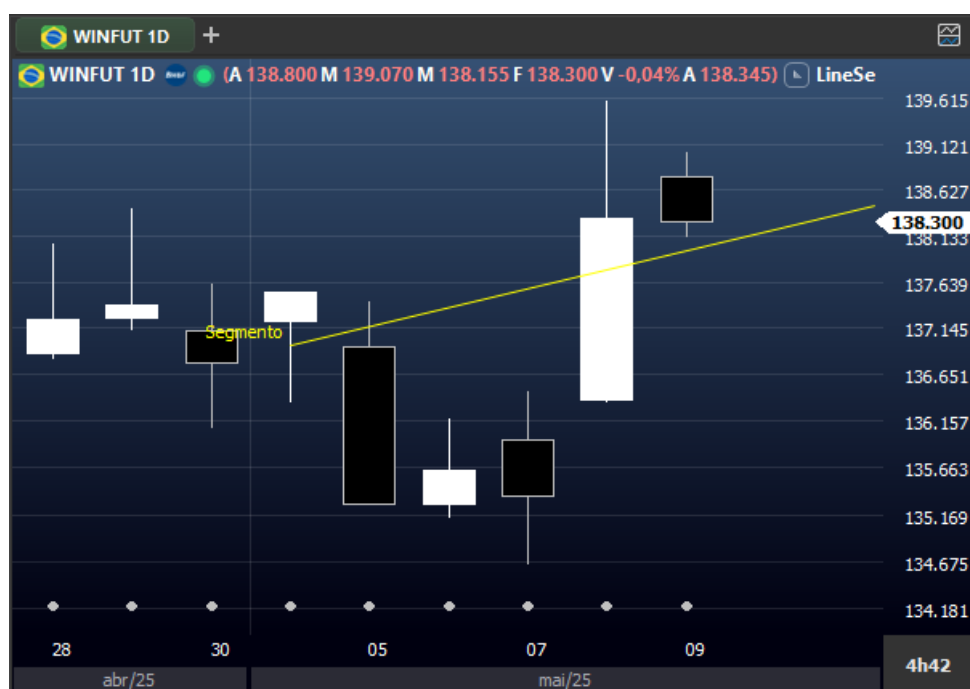
`EndTime`: **Integer** [Opcional]; Hora de término (caso aplicável).

Exemplo de uso:

No exemplo abaixo, será desenhado um segmento de reta do valor **137.000** para **138.000**, entre as datas **01/05** e **09/05**, com extrapolação ativada e texto "Segmento".

```
1 begin
2   LineSegment(137000, 138000, 1250501, 1250509, true, "Segmento");
3 end;
```


Exemplo Gráfico:





18.14 Função MaxBarsBack

Disponível em todas versões.

A função `MaxBarsBack()` tem como finalidade percorrer a lista da série, iniciando do primeiro candle criado (índice 0).

Sintaxe:

```
Função MaxBarsBack(): Integer;
```

Parâmetros:

Não possui parâmetros.

Exemplo de uso:

No exemplo, será aplicada uma coloração no segundo candle criado.

```
1 begin
2   if (MaxBarsBack() = 1) then
3     PaintBar(c1Green);
4 end;
```

Exemplo Gráfico:





18.15 Função MaxBarsForward

Disponível em todas versões.

A função `MaxBarsForward()` tem como finalidade percorrer a lista da série, iniciando do último candle criado (índice 0, candle atual).

Sintaxe:

```
Função MaxBarsForward(): Integer;
```

Parâmetros:

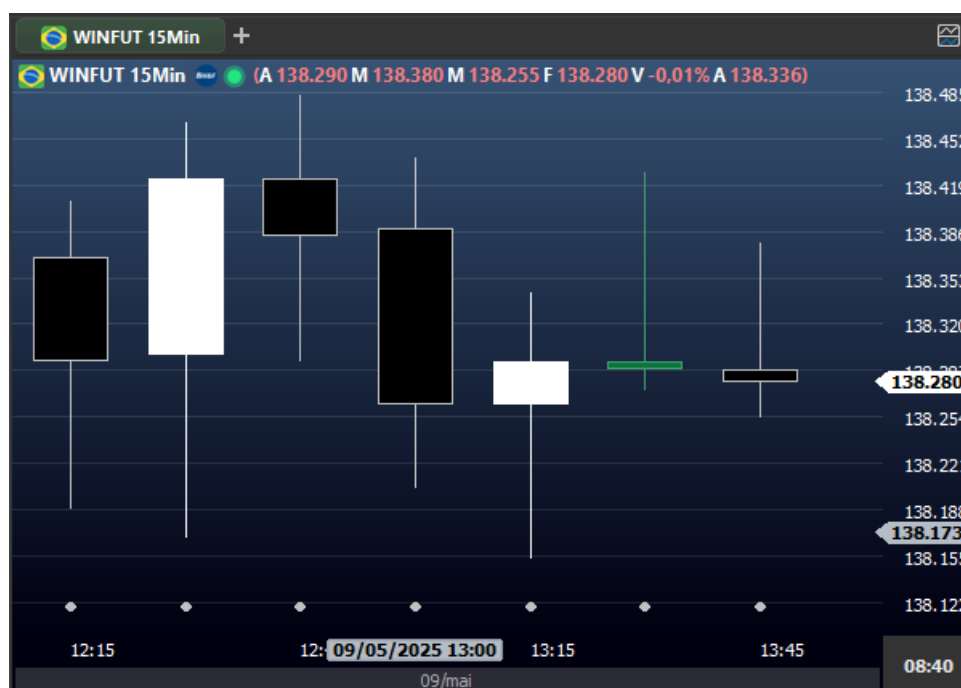
Não possui parâmetros.

Exemplo de uso:

No exemplo, será aplicada uma coloração no candle anterior ao atual.

```
1 begin
2   if (MaxBarsForward() = 1) then
3     PaintBar(c1Green);
4   end;
```

Exemplo Gráfico:





18.16 Função MedianPrice

Disponível em todas versões.

A função `MedianPrice()` retorna a média entre a máxima e a mínima de cada candle.

Sintaxe:

```
Função MedianPrice(): Float;
```

Parâmetros:

Não possui parâmetros.

Exemplo de uso:

No exemplo a seguir, caso o valor da função `MedianPrice()` do candle atual seja maior que o do candle anterior, os candles serão destacados com a cor `clGreen`.

```
1 begin
2   if ((MedianPrice() > MedianPrice(1))) then
3     PaintBar(clGreen);
4     Plot(MedianPrice);
5 end;
```

Exemplo Gráfico:





18.17 Função NoPlot

Disponível em todas versões.

A função **NoPlot()** tem como finalidade efetuar a remoção de determinado Plot.

Sintaxe:

```
Função NoPlot(PlotNum: Integer): Void;
```

Parâmetros:

PlotNum: **Integer**; Número do Plot (**1**, **2**, **3** ou **4**) a ser removido.

Exemplo de uso:

Conforme o exemplo a seguir, será removida a linha referente ao Plot **1**, e será exibida apenas a linha vinculada ao Plot **2**.

```
begin
1   Plot(Close);
2   Plot2(Open);
3   NoPlot(1);
end;
```

Exemplo Gráfico:





18.18 Função PaintBar

Disponível em todas versões.

A função `PaintBar()` permite a aplicação de colorações em indicadores ou candles.

Sintaxe:

```
Função PaintBar(Cor: Integer): Void;
```

Parâmetros:

Cor: Integer; Determina a coloração.

Exemplo de uso:

No exemplo, será aplicada a coloração verde quando o histograma de `MACD()` for maior que zero.

```
1 begin
2   if (MACD(23, 12, 9)|1| > 0) then
3     PaintBar(c1Green);
4   end;
```

Exemplo Gráfico:



18.19 Funções Plot

Disponível em todas versões.

A função **Plot()** realiza a ligação dos valores passados por parâmetro e cria gráficos de linhas.

É possível inserir até 99 linhas, utilizando a versão numerada da função: **Plot1()**, **Plot2()**, **Plot3()** ... **Plot99()**.

Sintaxe:

```
Função Plot(Valor: Float): Void;
Função Plot2(Valor: Float): Void;
Função Plot3(Valor: Float): Void;
...
Função Plot99(Valor: Float): Void;
```

Parâmetros:

Valor: Float; Pode-se utilizar variáveis, funções ou constantes para realizar o desenho do indicador.

Exemplo de uso:

Conforme o exemplo a seguir, são plotados os dados de fechamento do dia anterior e a abertura do dia atual.

```
1 begin
2   Plot(Closed(1));
3   Plot99(OpenD(0));
4 end;
```

Exemplo Gráfico:





18.20 Função PlotCandle

Disponível em todas versões.

A função `PlotCandle()` plota um candle personalizado no gráfico, permitindo definir manualmente os valores de abertura, máxima, mínima e fechamento, além das cores do corpo e da borda.

Sintaxe:

```
Função PlotCandle(Open: Float; High: Float; Low: Float; Close: Float;  
ColorPositive: Integer; ColorLow: Integer; ColorBorder: Integer): Void;
```

Parâmetros:

Open: **Float**; Valor de abertura do candle.

High: **Float**; Valor da máxima do candle.

Low: **Float**; Valor da mínima do candle.

Close: **Float**; Valor de fechamento do candle.

ColorPositive: **Integer** [Opcional]; Cor do corpo do candle positivo (quando `Close > Open`).

ColorLow: **Integer** [Opcional]; Cor do corpo do candle negativo (quando `Close < Open`).

ColorBorder: **Integer** [Opcional]; Cor da borda do candle.

Exemplo de uso:

No exemplo abaixo, são desenhados candles personalizados com cores específicas para candles de alta e baixa. As variáveis de abertura, máxima, mínima e fechamento são obtidas da série principal, e as cores são definidas conforme o tipo do candle.

```
1  var  
2    vOpen, vHigh, vLow, vClose      : Float;  
3    vPositivo, vNegativo, vBorda   : Integer;  
4  begin  
5    vOpen := Open;  
6    vHigh := High;  
7    vLow  := low;  
8    vClose := Close;  
9    vPositivo := ClGreen;  
10   vNegativo := ClRed;  
11   vBorda := clWhite;  
12   PlotCandle(vOpen, vHigh, vLow, vClose, vPositivo, vNegativo, vBorda);  
13 end;
```


Exemplo Gráfico:



18.21 Função PlotN

Disponível em todas versões.

A função **PlotN()** assim como a **Plot()**, desenha o indicador conforme a série de dados informada, porém o número do plot deve ser determinado por parâmetro (entre **0** e **99**).

Sintaxe:

```
Função PlotN(Plot: Integer; Valor: Float): Void;
```

Parâmetros:

Plot: Integer; Número do plot, entre **0** e **99**.

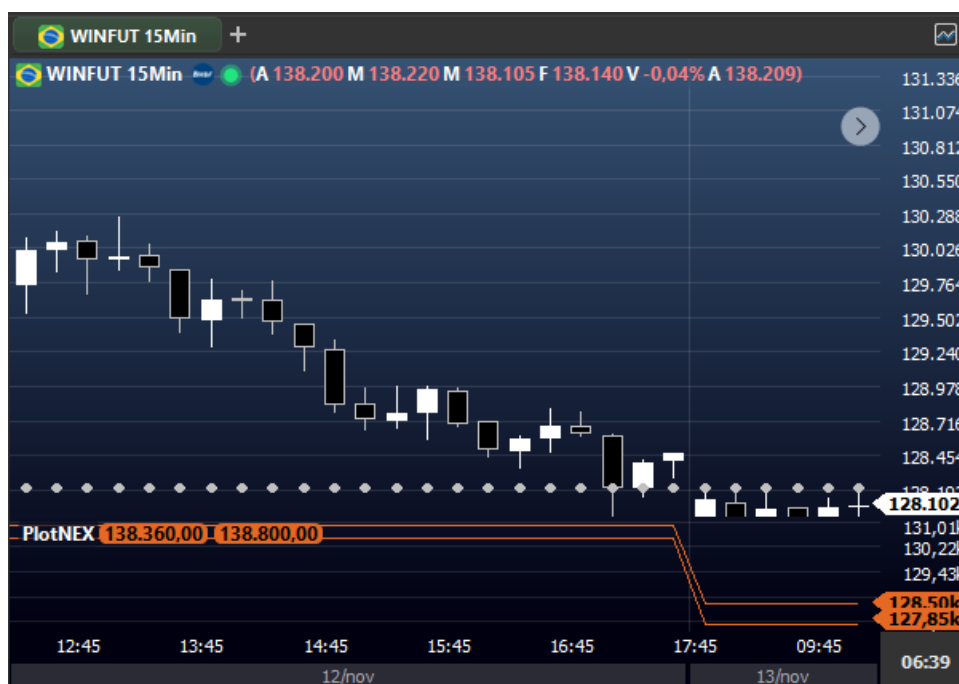
Valor: Float; Pode-se utilizar variáveis, funções ou constantes para realizar o desenho do indicador.

Exemplo de uso:

No exemplo, são plotados os dados de fechamento do dia anterior e a abertura do dia atual.

```
1 begin
2   PlotN(0, Closed(1));
3   PlotN(99, OpenD(0));
4 end;
```

Exemplo Gráfico:





18.22 Função PlotText

Disponível em todas versões.

A função `PlotText()` possui como finalidade adicionar um texto a um indicador.

Sintaxe:

```
Função PlotText(Content: String Color: Integer Position: Integer FontSize: Integer  
Price: Float): Void;
```

Parâmetros:

Content: `String`; Conteúdo do texto.

Color: `Integer`; Cor da fonte.

Position: `Integer` [Opcional]; Posição do texto:

- **0:** Abaixo do candle.
- **1:** Centralizado.
- **2:** Acima do candle.
- **3:** Posiciona no preço definido em `dPrice`.

FontSize: `Integer` [Opcional]; Tamanho da fonte do texto.

Price: `Float` [Opcional]; Quando utilizada `Position = 3`, será utilizado o valor de `dPrice` para posicionar o texto.

Exemplo de uso:

No exemplo, será adicionado o texto "Close > M20" quando o fechamento anterior estiver acima da média exponencial de 20 períodos. Caso contrário, será exibido o texto "Close < M20".

```
1 begin  
2   if (Close[1] < mediaExp(20, Close)[1]) and  
3     (Close > mediaExp(20, Close)) then  
4     PlotText("Close > M20!", clLime, 2, 6)  
5   else if (Close[1] > mediaExp(20, Close)) and  
6     (Close < mediaExp(20, Close)) then  
7     PlotText("Close < M20!", clRed, 0, 6);  
8 end;
```



Exemplo Gráfico:





18.23 Função Range

Disponível em todas versões.

A função `Range()` retorna a diferença entre a máxima e a mínima do candle.

Sintaxe:

```
Função Range(): Float;
```

Parâmetros:

Nenhum parâmetro.

Exemplo de uso:

No exemplo, a variável `nRange` irá receber o valor retornado pela função `Range()`.

```
1 var
2   nRange : Float;
3 begin
4   nRange := Range();
5   Plot(nRange);
6 end;
```

Exemplo Gráfico:





18.24 Função RangeLeader

Disponível em todas versões.

A função `RangeLeader()` compara o candle atual e anterior, analisando duas condições: se o ponto médio do candle atual é maior que a máxima anterior ou menor que a mínima anterior; se o intervalo do candle atual é maior que o intervalo da barra anterior. A função retornará 1 caso ambas as condições forem atendidas, mas retornará 0 caso nenhuma ou apenas uma das condições for atendida.

Sintaxe:

```
Função RangeLeader(): Float;
```

Parâmetros:

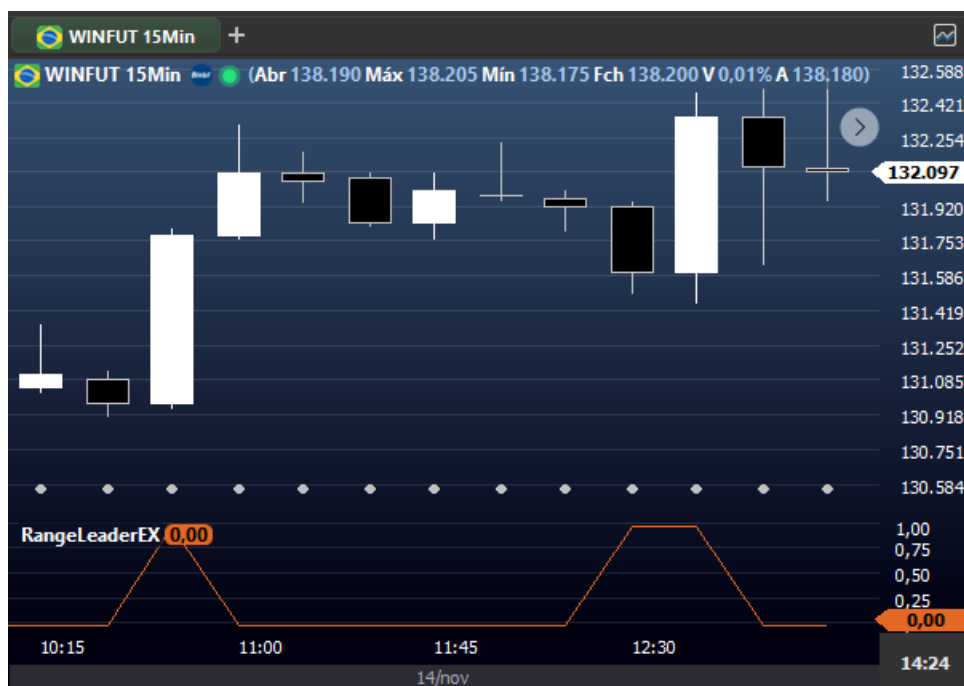
Nenhum parâmetro.

Exemplo de uso:

No exemplo, a variável `nRL` irá receber o valor retornado pela função `RangeLeader()`.

```
1 var
2   nRL : Float;
3 begin
4   nRL := RangeLeader();
5   Plot(nRL);
6 end;
```

Exemplo Gráfico:





18.25 Função RGB

Disponível em todas versões.

A função **RGB()** permite ao usuário customizar cores, a partir da aplicação de intensidade dos parâmetros vermelho, verde e azul.

Sintaxe:

```
Função RGB(Red: Integer; Green: Integer; Blue: Integer): Integer;
```

Parâmetros:

Red: **Integer**; Intensidade da cor vermelha, variando de 0 a 255.

Green: **Integer**; Intensidade da cor verde, variando de 0 a 255.

Blue: **Integer**; Intensidade da cor azul, variando de 0 a 255.

Exemplo de uso:

No exemplo a seguir, será aplicada a coloração azul, conforme os parâmetros passados à função **RGB()**.

```
1 begin
2   PaintBar(RGB(0, 0, 230));
3 end;
```

Exemplo Gráfico:



18.26 Função SetPlotColor

Disponível em todas versões.

A função `SetPlotColor()` possui como finalidade alterar a coloração de determinado Plot de 1 a 4.

Sintaxe:

```
Função SetPlotColor(NúmeroPlot: Integer; Cor: Integer): Void;
```

Parâmetros:

NúmeroPlot: **Integer**; Número do plot que terá a cor alterada:

- **1**: Plot;
- **2**: Plot2;
- **3**: Plot3;
- **4**: Plot4.

Cor: **Integer**; Determina a coloração. Pode ser uma constante de cor ou uma chamada à função `RGB()`.

Exemplo de uso:

No exemplo abaixo, será alterada a coloração para vermelha referente à linha do `Plot()`.

```
1 begin
2   Plot(Close);
3   SetPlotColor(1, c1Red);
4 end;
```

Exemplo Gráfico:





18.27 Função SetPlotStyle

Disponível em todas versões.

A partir da função `SetPlotStyle()`, é possível alterar o estilo da linha de um plot específico.

Sintaxe:

```
Função SetPlotStyle(Number: Integer; Style: Integer): Void;
```

Parâmetros:

Number: Integer; Número do plot a ser modificado.

Style: Integer; Estilo da linha:

- 0: Contínua;
- 1: Tracejada (Traço longo);
- 2: Tracejada (Traço curto);
- 3: Tracejada (Traço longo e curto);
- 4: Tracejada (Traço longo e dois curtos).

Exemplo de uso:

No exemplo abaixo, serão customizados os estilos dos plots: o primeiro terá linha tracejada com traços longos, e o segundo, com traços curtos.

```
1 begin
2   Plot(OpenD(0));
3   Plot2(PriorCote(4));
4
5   SetPlotStyle(1, 1);
6   SetPlotStyle(2, 2);
7
8   SetPlotWidth(2, 2);
9 end;
```

Exemplo Gráfico:



18.28 Função SetPlotType

Disponível em todas versões.

A função `SetPlotType()` possui como finalidade alterar o tipo de gráfico de determinado plot.

Sintaxe:

Função `SetPlotType`(Number: **Integer**; Type: **Integer**): **Void**;

Parâmetros:

Number: **Integer**; Número do plot específico.

Type: **Integer**; Tipo de visualização:

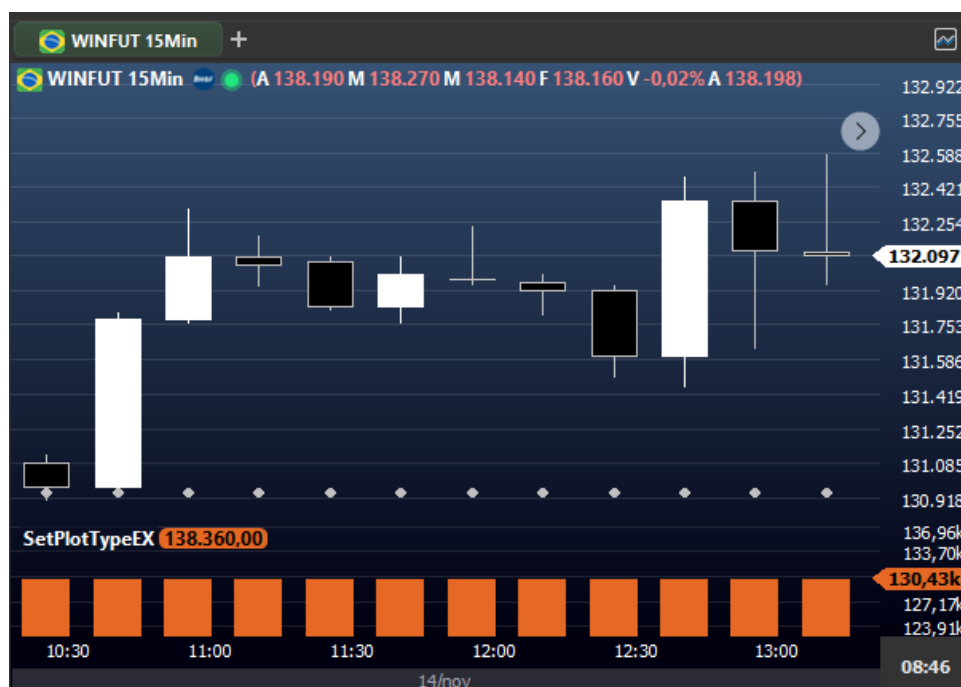
- **0**: Linha;
- **1**: Histograma.

Exemplo de uso:

No exemplo abaixo, será alterado o tipo do Plot 1 para histograma.

```
1 begin
2   PlotN(1, closed(1));
3   SetPlotType(1, 1);
4 end;
```

Exemplo Gráfico:



18.29 Função SetPlotWidth

Disponível em todas versões.

A função `SetPlotWidth()` possui como finalidade alterar a espessura de determinado Plot (1 a 4).

Sintaxe:

```
Função SetPlotWidth(NúmeroPlot: Integer; Espessura: Integer): Void;
```

Parâmetros:

NúmeroPlot: **Integer**; Número do plot a ser modificado:

- **1**: Plot;
- **2**: Plot2;
- **3**: Plot3;
- **4**: Plot4.

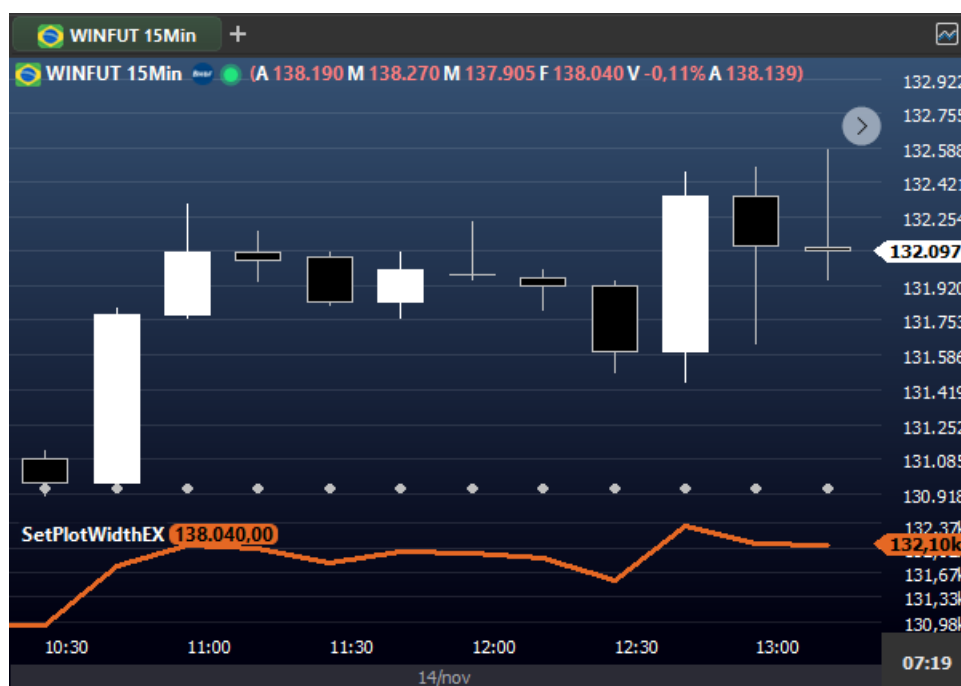
Espessura: **Integer**; Valor da nova espessura da linha.

Exemplo de uso:

No exemplo abaixo, será alterada a espessura da linha do Plot 1 para 3.

```
1 begin
2   Plot(Close);
3   SetPlotWidth(1, 3);
4 end;
```

Exemplo Gráfico:





18.30 Função TrueHigh

Disponível em todas versões.

A função `TrueHigh()` retorna o maior valor entre o máximo da barra atual e o fechamento da barra anterior.

Sintaxe:

```
Função TrueHigh(): Float;
```

Parâmetros:

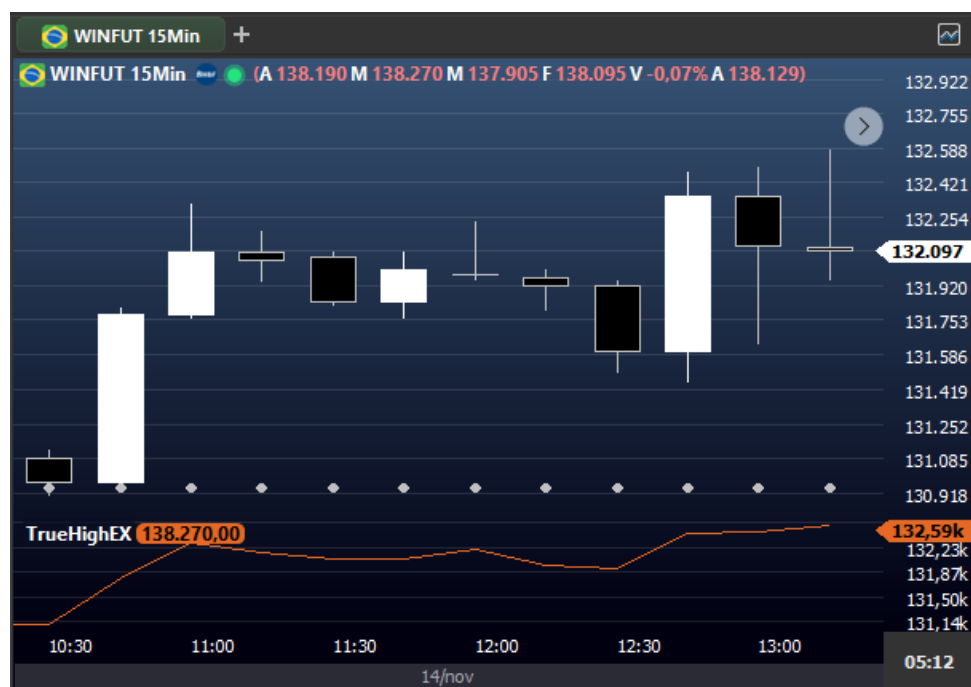
Nenhum parâmetro.

Exemplo de uso:

No exemplo abaixo, a variável TH receberá o valor retornado pela `TrueHigh()`.

```
1 var
2   TH : Float;
3 begin
4   TH := TrueHigh();
5   Plot(TH);
6 end;
```

Exemplo Gráfico:





18.31 Função TrueLow

Disponível em todas versões.

A função `TrueLow()` retorna o menor valor entre a mínima da barra atual e o fechamento da barra anterior.

Sintaxe:

```
Função TrueLow(): Float;
```

Parâmetros:

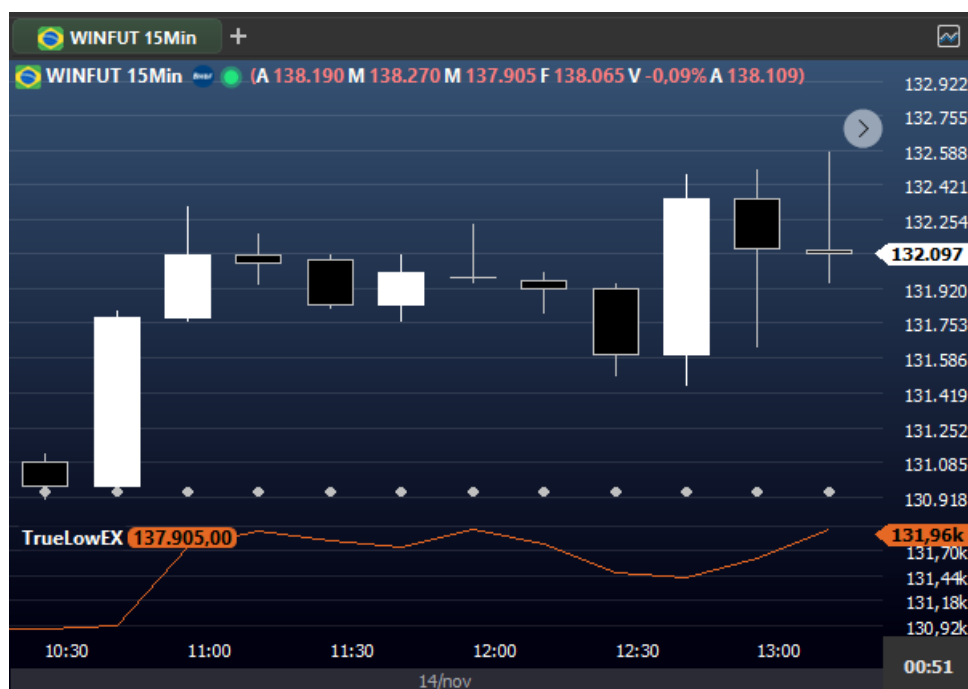
Nenhum parâmetro.

Exemplo de uso:

No exemplo abaixo, a variável TL receberá o valor retornado pela função `TrueLow()`.

```
1 var
2   TL : Float;
3 begin
4   TL := TrueLow();
5   Plot(TL);
6 end;
```

Exemplo Gráfico:





18.32 Função TrueRange

Disponível em todas versões.

A função `TrueRange()` retorna a diferença entre os valores de `TrueHigh()` e `TrueLow()`.

Sintaxe:

```
Função TrueRange(): Float;
```

Parâmetros:

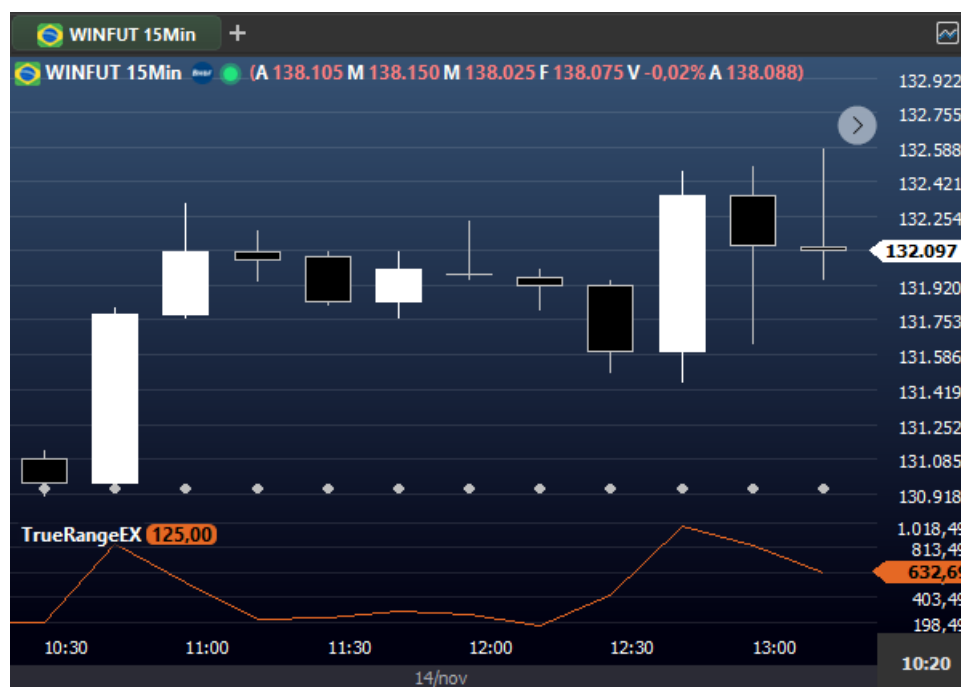
Nenhum parâmetro.

Exemplo de uso:

No exemplo abaixo, a variável TR receberá o valor retornado pela função `TrueRange()`.

```
1 var
2   TR : Float;
3 begin
4   TR := TrueRange();
5   Plot(TR);
6 end;
```

Exemplo Gráfico:





18.33 Função TrueRangeCustom

Disponível em todas versões.

A função `TrueRangeCustom()` retorna o valor de TrueRange com base nos dados fornecidos pelo usuário.

Sintaxe:

```
Função TrueRangeCustom(Maxima: Float; Minima: Float; Fechamento: Float): Float;
```

Parâmetros:

Maxima: **Float**; Valor de máxima para o cálculo do indicador.

Minima: **Float**; Valor de mínima para o cálculo do indicador.

Fechamento: **Float**; Valor de fechamento de referência.

Exemplo de uso:

No exemplo abaixo, a variável TRC receberá o valor retornado pela função `TrueRangeCustom()`.

```
1 var
2   TRC : Float;
3 begin
4   TRC := TrueRangeCustom(3763.5, 3761, 3761);
5   Plot(TRC);
6 end;
```

Exemplo Gráfico:





18.34 Função TypicalPrice

Disponível em todas versões.

A função `TypicalPrice()` retorna a média aritmética entre os valores de máxima, mínima e fechamento do candle.

Sintaxe:

```
Função TypicalPrice(): Float;
```

Parâmetros:

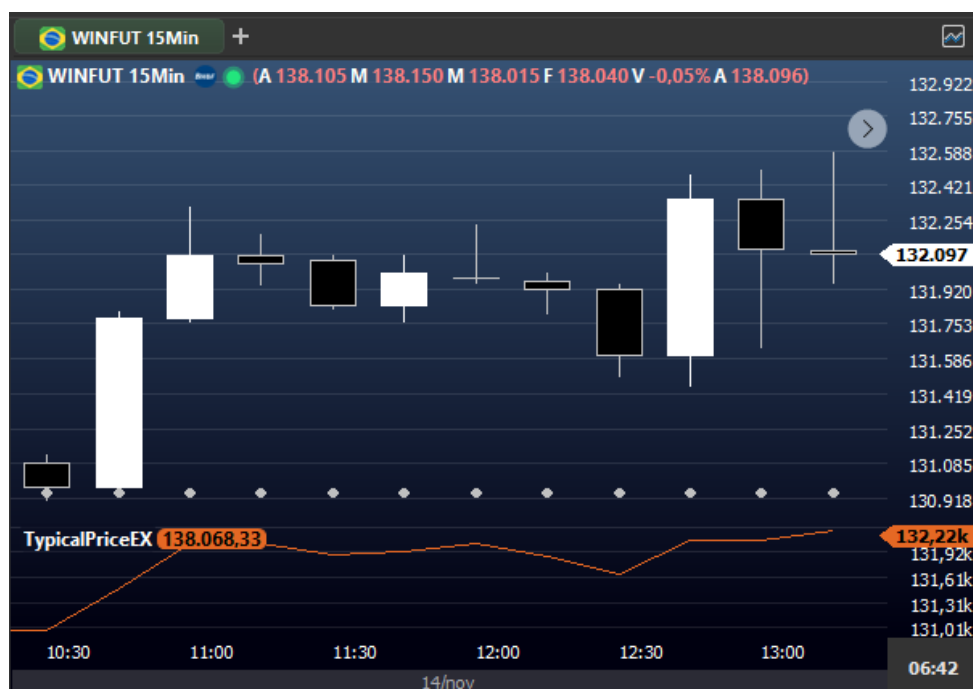
Nenhum parâmetro.

Exemplo de uso:

No exemplo abaixo, a variável `aux` receberá o valor retornado pela função `TypicalPrice()`.

```
1 var
2   aux : Float;
3 begin
4   aux := TypicalPrice();
5   Plot(aux);
6 end
```

Exemplo Gráfico:



18.35 Função VerticalLine

Disponível em todas versões.

A função `VerticalLine()` adiciona uma linha vertical no gráfico, destacando um ponto específico no eixo do tempo.

Sintaxe:

```
Função VerticalLine(Date: Integer Color: Integer;): Void;
```

Parâmetros:

Date: Integer; Data no formato 1AAMDD.

Color: Integer; Cor da linha.

Exemplo de uso:

No exemplo abaixo, será inserida uma linha vertical amarela na data do dia anterior.

```
1 begin
2   VerticalLine(CurrentDate - 1, clYellow);
3 end;
```

Exemplo Gráfico:





18.36 Função VerticalLineCustom

Disponível em todas versões.

A função `VerticalLineCustom()` permite desenhar múltiplas linhas verticais com configurações personalizadas, como texto, estilo, espessura e região pintada.

Sintaxe:

```
Função VerticalLineCustom (Color: Color; LineWidth: Integer; Style: Integer; Text: String; FontSize: Integer; TextPosition: Integer; PriceTop: Float; PriceBottom: Float; PaintRegion: Boolean): Void;
```

Parâmetros:

Color: **Color**; Cor da linha.

LineWidth: **Integer**; Espessura da linha.

Style: **Integer**; Estilo da linha. Ver `SetPlotStyle()` para mais detalhes.

Text: **String**; Texto exibido na linha.

FontSize: [Opcional] **Integer**; Tamanho da fonte do texto.

TextPosition: [Opcional] **Integer**; Posição do texto.

PriceTop: [Opcional] **Float**; Preço inicial da linha (se 0, começa do topo do gráfico).

PriceBottom: [Opcional] **Float**; Preço final da linha (se 0, vai até o fundo do gráfico).

PaintRegion: [Opcional] **Boolean**; Define se será pintada uma região até a próxima linha ou até o fim.

Exemplo de uso:

No exemplo abaixo, a função `VerticalLineCustom()`, será plotado uma linha vertical personalizada no gráfico.

```
1 begin
2   VerticalLineCustom(CLRED,1,1,"1234");
3 end;
```



Exemplo Gráfico:





18.37 Função WeightedClose

Disponível em todas versões.

A função `WeightedClose()` retorna a média entre o ponto médio da barra e dois fechamentos.

Sintaxe:

```
Função WeightedClose(): Float;
```

Parâmetros:

Nenhum parâmetro.

Exemplo de uso:

No exemplo abaixo, a variável `aux` irá receber o valor retornado pela função `WeightedClose()`.

```
1 var
2   aux : Float;
3 begin
4   aux := WeightedClose();
5   Plot(aux);
6 end;
```

Exemplo Gráfico:





19 Indicadores

As funções de indicadores não poderão conter variáveis em seus parâmetros, apenas constantes. O único tipo de parâmetro que foge à esta regra são os parâmetros do tipo Serie que contém um valor para cada candle, esses valores podem ser considerados "variáveis" dado que a série muda ao longo do tempo.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a quantidade de dias úteis até a terceira sexta-feira do mês seguinte.

```
1 begin
2   Plot(Media(21, Close));
3 end
```

Veja abaixo as descrições das funções de indicadores disponíveis nas plataformas da Nelogica:

19.1 Função TWOMV VOLATILITY

Disponível em todas versões.

A função **TWOMV VOLATILITY()** foi criada para auxiliar na visualização no funcionamento do modo volatilidade do robô 2MV.

Sintaxe:

```
Função TWOMV VOLATILITY(factorpos : Float; factorneg : Float): Float;
```

Parâmetros:

factorpos: **Float**; Fator positivo usado para o cálculo da volatilidade em movimentos de alta.

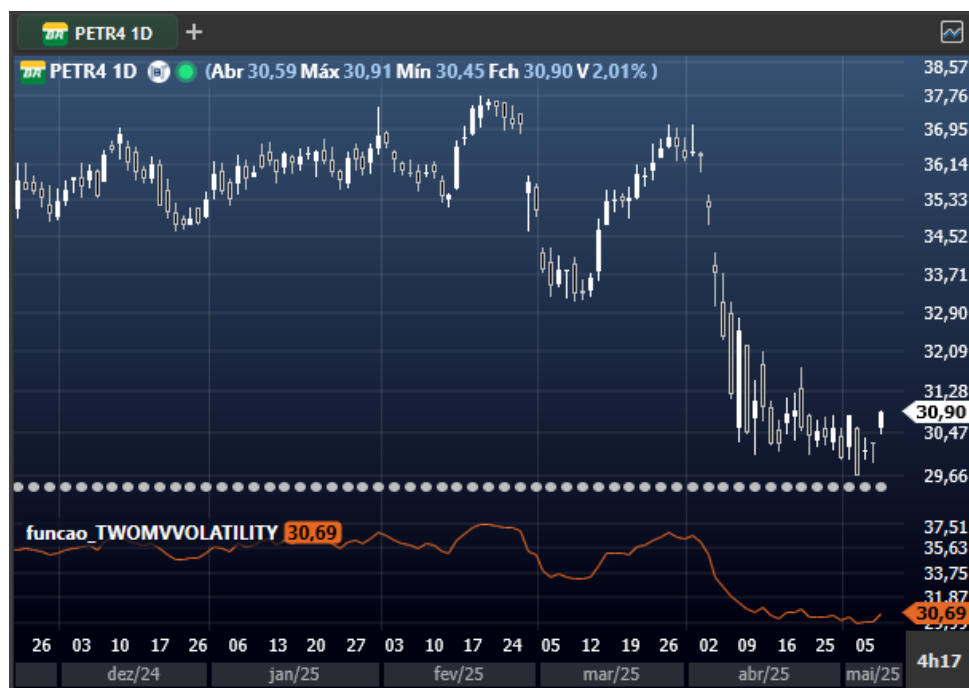
factorneg: **Float**; Fator negativo usado para o cálculo da volatilidade em movimentos de baixa.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor da função **TWOMV VOLATILITY()**.

```
1 begin
2   Plot(TWOMV VOLATILITY(5.5, 5.5));
3 end
```

Exemplo Gráfico:





19.2 Função ADX

Disponível em todas versões.

A função `ADX()` retorna o valor do indicador de acordo com os períodos desejados. O `ADX()` mede a força do movimento não importando a direção da tendência.

Sintaxe:

```
Função ADX(Período : Integer; Media : Integer): Float;
```

Parâmetros:

Período: Integer; Período utilizado no momento do cálculo do indicador:

Media: Integer; Período utilizado no momento do cálculo da média utilizada no indicador:

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "ADX", considerando Período de 9 e Média de 9 para o cálculo.

```
1 begin
2   Plot(ADX(9,9));
3 end
```

Exemplo Gráfico:



19.3 Função AccAgressSaldo

Disponível nos produtos Pro ou Ultra ou módulo Scalper.

AccAgressSaldo() é um indicador de "TR - Acúmulo de Agressão - Saldo" desenvolvido pela Scalper Trader.

Sintaxe:

```
Função AccAgressSaldo(Tipo : Integer): Float;
```

Parâmetros:

Tipo: **Integer**; Tipo de volume para o cálculo:

- **0**: Financeiro.
- **1**: Quantidade.
- **2**: Negócios.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o volume de quantidade do indicador "TR - Acúmulo de Agressão - Saldo".

```
1 begin
2   Plot(AccAgressSaldo(1));
3 end
```

Exemplo Gráfico:



19.4 Função AccuDistr

Disponível em todas versões.

O indicador **AccuDistr()** relaciona preço e volume, tendo por base a idéia de que movimentos significativos são acompanhados pelo volume, sendo muito utilizado através de divergências.

Sintaxe:

```
Função AccuDistr(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador.

```
1 begin
2   Plot(AccuDistr);
3 end
```

Exemplo Gráfico:





19.5 Função AccuDistrW

Disponível para produtos da BlackArrow ou Scalper Global.

A função **AccuDistrW()** é um indicador de momentum criado por Larry Williams que avalia a relação entre preço e volume. Sua função é indicar a direção da pressão do mercado, seja de acumulação (alta) ou distribuição (baixa).

Sintaxe:

```
Função AccuDistrW(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador.

```
1 begin
2   Plot(AccuDistrW);
3 end
```

Exemplo Gráfico:





19.6 Função AdaptiveMovingAverage

Disponível em todas versões.

Descoberto e desenvolvido por Perry Kaufman, A função `AdaptiveMovingAverage()` é utilizada para construir uma média móvel com pouca sensibilidade ao ruído das series de preços e é caracterizado por levar pouco tempo para detectar uma tendência.

Sintaxe:

```
Função AdaptiveMovingAverage(Período : Integer; FastSC : Integer; SlowSC : Integer)  
: Float;
```

Parâmetros:

Período: Integer; Período utilizado no momento do cálculo do indicador.

FastSC: Integer; Determina o período para o indicador FastStochastic.

SlowSC: Integer; Determina o período para o indicador SlowStochastic.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador, considerando **10** períodos para o cálculo, com **2** períodos para FastStochastic, e **30** para SlowStochastic.

```
1 begin  
2   Plot(AdaptiveMovingAverage(10, 2, 30));  
3 end
```

Exemplo Gráfico:



19.7 Função AgressionVolBalance

Disponível nos produtos Pro ou Ultra ou módulo Scalper.

AgressionVolBalance() é um indicador de "TR - Acúmulo de Agressão - Saldo" desenvolvido pela Scalper Trader.

Sintaxe:

```
Função AgressionVolBalance(): Float;
```

Parâmetros:

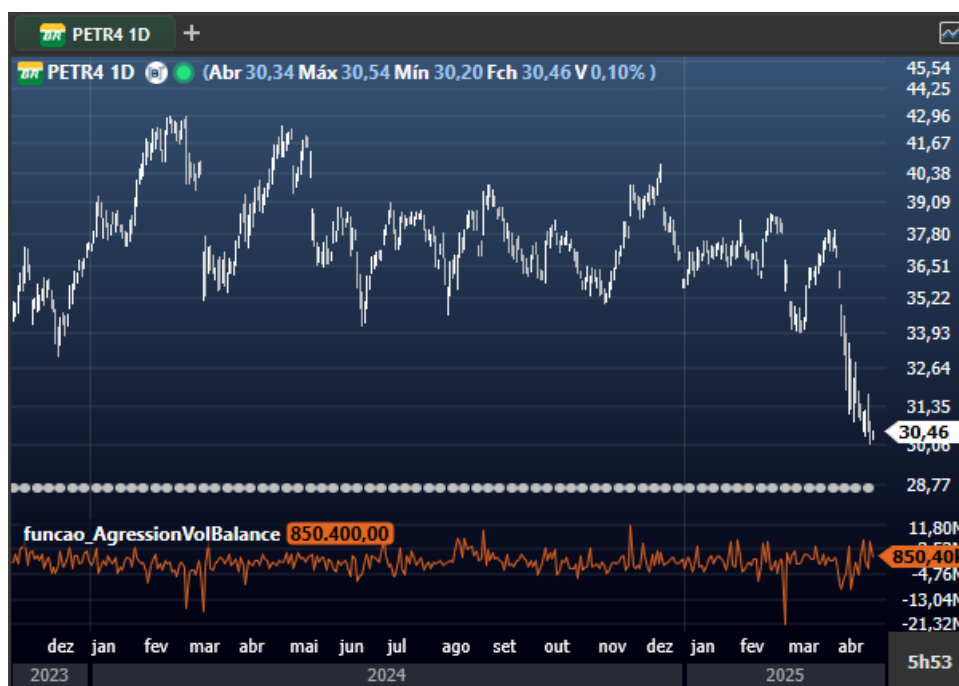
Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "TR - Acúmulo de Agressão - Saldo".

```
1 begin
2   Plot(AgressionVolBalance);
3 end
```

Exemplo Gráfico:





19.8 Função AgressionVolBuy

Disponível nos produtos Pro ou Ultra ou módulo Scalper.

AgressionVolBuy() é um indicador de "TR - Volume de Agressão - Compra" desenvolvido pela Scalper Trader.

Sintaxe:

```
Função AgressionVolBuy(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "TR - Volume de Agressão - Compra".

```
1 begin
2   Plot(AgressionVolBuy);
3 end
```

Exemplo Gráfico:





19.9 Função AgressionVolSell

Disponível nos produtos Pro ou Ultra ou módulo Scalper.

AgressionVolSell() é um indicador de "TR - Volume de Agressão - Venda" desenvolvido pela Scalper Trader.

Sintaxe:

```
Função AgressionVolSell(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "TR - Volume de Agressão - Venda".

```
1 begin
2   Plot(AgressionVolSell);
3 end
```

Exemplo Gráfico:





19.10 Função AllTimeHigh

Disponível em todas versões.

A função `AllTimeHigh()` retorna o valor máximo histórico alcançado pelo ativo.

Sintaxe:

```
Função AllTimeHigh(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador.

```
1 begin
2   Plot(AllTimeHigh);
3 end
```

Exemplo Gráfico:





19.11 Função AllTimeLow

Disponível em todas versões.

A função **AllTimeLow()** retorna o valor mínimo histórico alcançado pelo ativo.

Sintaxe:

```
Função AllTimeLow(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador.

```
1 begin
2   Plot(AllTimeLow);
3 end
```

Exemplo Gráfico:





19.12 Função ArmsEaseOfMov

Disponível em todas versões.

A função `ArmsEaseOfMov()` retorna o valor do indicador "Arms Ease of Movement" de acordo com o período e tipo de média desejados. O indicador combina preço e volume para indicar o nível de facilidade com o qual os preços estão se movimentando.

Sintaxe:

```
Função ArmsEaseOfMov(Media : Integer; Tipo: Integer): Float;
```

Parâmetros:

Media: Integer; Período da média utilizada no momento do cálculo do indicador:

Tipo: Integer; Determina o tipo da média utilizada:

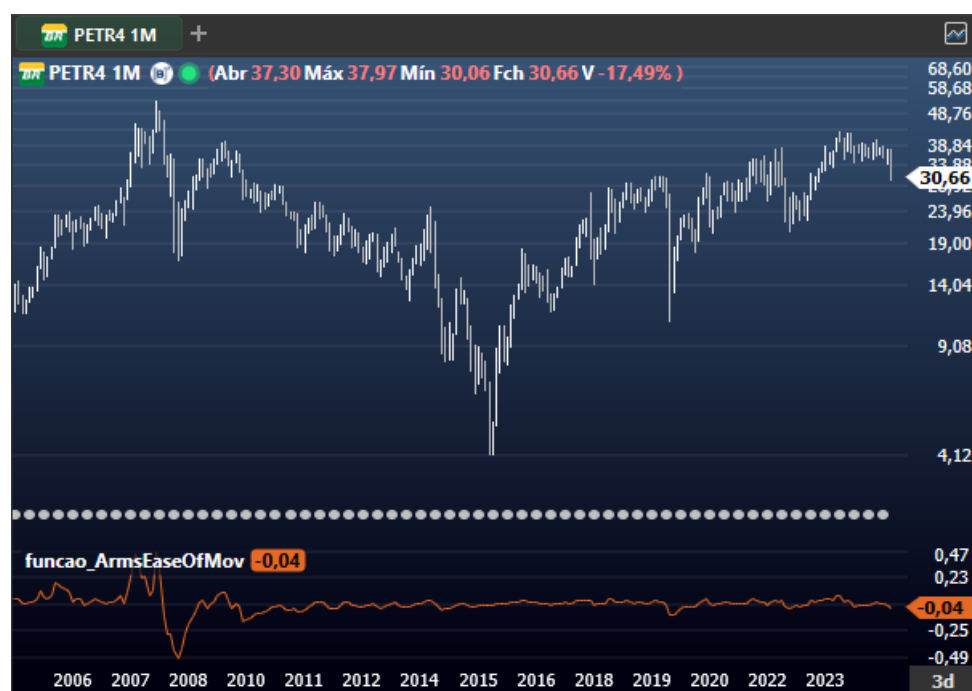
- **kaArithmetic:** Aritmética;
- **kaExponential:** Exponencial;
- **kaWelles:** Welles Wilder;
- **kaWeighted:** Ponderada;
- **kaADX:** ADX;

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Arms Ease of Movement", considerando **9** períodos para o tipo de média exponencial (**1**).

```
1 begin
2   Plot(ArmsEaseOfMov(9, kaExponential));
3 end
```

Exemplo Gráfico:





19.13 Função AroonLin

Disponível em todas versões.

A função `AroonLin()` retorna o valor do indicador de acordo com o período desejado. O indicador é formado por dois componentes que ajudam a identificar a existência de uma tendência e também a medir a sua força.

Sintaxe:

```
Função AroonLin(Período : Integer): Float;
```

Parâmetros:

Período: Integer; Período utilizado no momento do cálculo do indicador:

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador, considerando 9 períodos para o cálculo.

```
1 begin
2   Plot(AroonLin(9));
3 end
```

Exemplo Gráfico:





19.14 Função AroonOsc

Disponível em todas versões.

A função `AroonOsc()` retorna o valor do indicador de acordo com o período desejado. O `AroonOsc()` sinaliza uma tendência de alta quando está acima de zero e uma tendência de baixa quando está abaixo do valor zero. A distância da linha zero indica a força da tendência.

Sintaxe:

```
Função AroonOsc(Período : Integer): Float;
```

Parâmetros:

Período: Integer; Período utilizado no momento do cálculo do indicador:

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador, considerando 9 períodos para o cálculo.

```
1 begin
2   Plot(AroonOsc(9));
3 end
```

Exemplo Gráfico:



19.15 Função AutoFibonacci

Disponível em todas versões.

A função `AutoFibonacci()` retorna o valor do indicador de acordo com o período desejado. O indicador é uma ferramenta clássica nas plataformas mundialmente conhecidas usando por padrão 144 períodos clássicos e pode ser personalizado. A ferramenta oferece conforto e praticidade ao analisar muitos ativos no mesmo layout.

Sintaxe:

```
Função AutoFibonacci(Período : Integer; Níveis: Integer): Float;
```

Parâmetros:

Período: **Integer**; Período utilizado no momento do cálculo do indicador:

Níveis: **Integer**; Determina quantos níveis serão gerados para o indicador, limitado em 8, e seguindo a ordem: (0%, 23.6%, 38.2%, 50%, 61.8%, 78.6%, 100%, 161.8%).

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador, considerando um período de **144** candles, e gerando oito níveis. Ao não especificar o nível desejado, o padrão será sempre retornar o nível **0** (0%).

```
1 begin
2   Plot(AutoFibonacci(144, 8));
3 end
```

Exemplo Gráfico:



Também é possível escolher qual o nível desejado passando como parâmetro no final da chamada. No exemplo a seguir, será plotado no gráfico o valor do indicador, considerando um período de **144** candles, gerando oito níveis e retornando o valor do nível **4** (50%).



```
1 begin
2   Plot(AutoFibonacci(144, 8)|3|);
3 end
```

Exemplo Gráfico:



19.16 Função AutoFibonacciCustom

Disponível em todas versões.

A função `AutoFibonacciCustom()` retorna o valor do indicador "Auto Fibonacci" de acordo com o período desejado. O indicador é uma ferramenta clássica nas plataformas mundialmente conhecidas usando por padrão 144 períodos clássicos e pode ser personalizado. A ferramenta oferece conforto e praticidade ao analisar muitos ativos no mesmo layout.

Sintaxe:

```
Função AutoFibonacciCustom(Período : Integer; Valor do Nível: Float): Float;
```

Parâmetros:

Período: Integer; Período utilizado no momento do cálculo do indicador:

Valor do Nível: Integer; Determina qual o valor do nível utilizado para o cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador, considerando um período de **144** candles e o valor do nível em **75.5%**.

```
1 begin
2   Plot(AutoFibonacciCustom(144, 75.5));
3 end
```

Exemplo Gráfico:



19.17 Função Averan

Disponível em todas versões.

Criado por Eduardo Garufi, do SST, a função `Averan()` permite medir a amplitude e volatilidade do ativo.

Sintaxe:

```
Função Averan(Period : Integer; nStdDevs: Integer): Float;
```

Parâmetros:

Period: Integer; Período utilizado no momento do cálculo do indicador

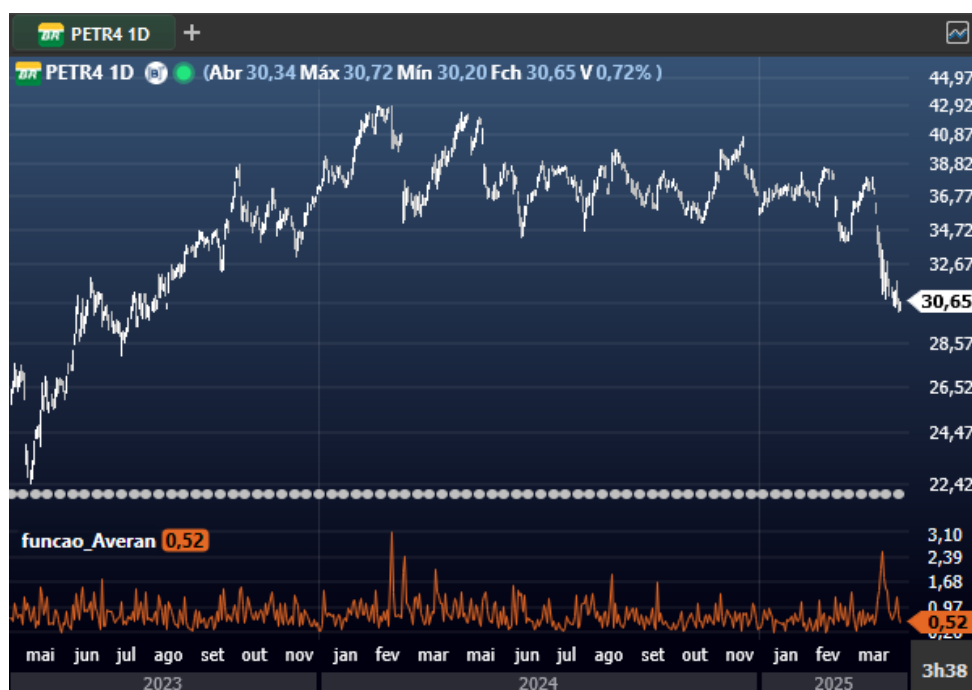
nStdDevs: Integer; Número de desvios padrão utilizados para definir a faixa de variação.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador.

```
1 begin
2   Plot(Averan(200, 2));
3 end
```

Exemplo Gráfico:



19.18 Função AvgAgent

Disponível nas versões Ultra ou com a Automação de Estratégias 2.

A função `AvgAgent()` retorna o preço médio filtrado pelo agente.

Sintaxe:

```
Função AvgAgent(AgentID : Integer; Período : Integer): Float;
```

Parâmetros:

AgentID: **Integer**; ID do agente escolhido.

Período: **Integer**; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função `AvgAgent()`.

```
1 begin
2   Plot(AvgAgent(baItau,9));
3 end
```

Exemplo Gráfico:





19.19 Função AvgAgrBuySell

Disponível nos produtos Pro ou Ultra ou módulo Scalper.

Desenvolvido pela Scalper Trader, a função `AvgAgrBuySell()` retorna o valor do indicador "TR - Agressão Média - Compra e Venda" de acordo com os parâmetros desejados.

Sintaxe:

```
Função AvgAgrBuySell(AlertaVariacoes : Integer; TipoDeVolume: Integer;  
    TipoDeDesenho: Integer): Float;
```

Parâmetros:

AlertaVariacoes: Integer; Período utilizado no momento do cálculo do indicador:

TipoDeVolume: Integer; Determina qual tipo de volume será obtido:

- 0: Financeiro.
- 1: Quantidade.

TipoDeDesenho: Integer; Relação entre compra e venda:

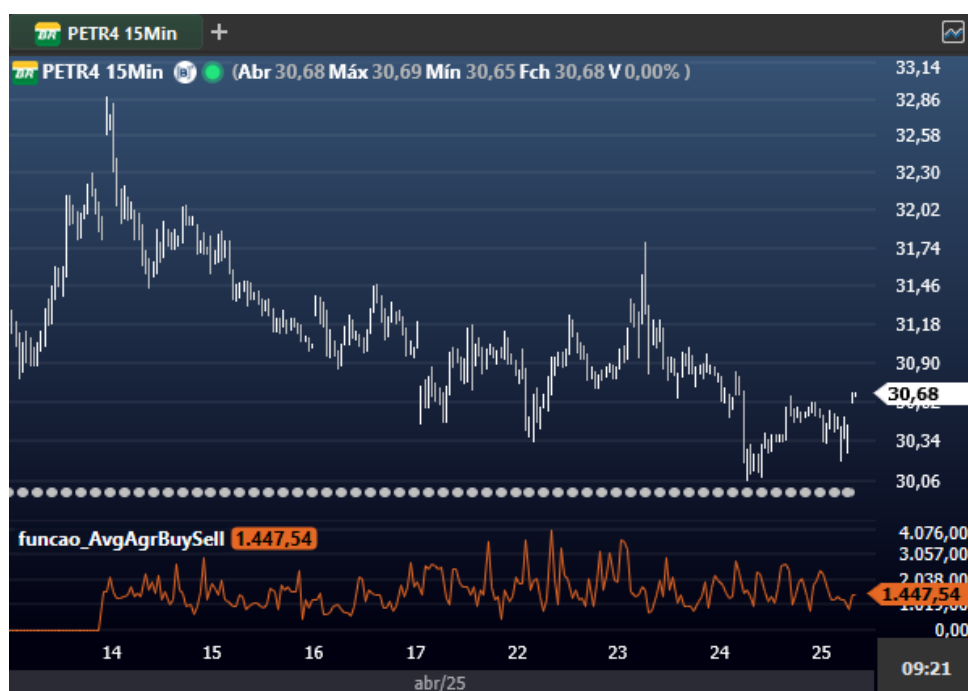
- 0: Compra e Venda.
- 1: Compra/Venda.
- 2: Compra-Venda.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador, considerando 3 variações, a quantidade e tipo de desenho.

```
1 begin  
2   Plot(AvgAgrBuySell(3, 1, 0));  
3 end
```

Exemplo Gráfico:





19.20 Função AvgAgrTotal

Disponível nos produtos Pro ou Ultra ou módulo Scalper.

Desenvolvido pela Scalper Trader, a função **AvgAgrTotal()** retorna o valor do indicador "TR - Agressão Média - Total" de acordo com os parâmetros desejados.

Sintaxe:

```
Função AvgAgrTotal(AlertaVariacoes : Integer; TipoDeVolume: Integer; TipoDeDesenho: Integer): Float;
```

Parâmetros:

AlertaVariacoes: Integer; Número de variações.

TipoDeVolume: Integer; Determina qual tipo de volume será obtido:

- 0: Financeiro.
- 1: Quantidade.

TipoDeDesenho: Integer; Relação entre compra e venda:

- 0: Compra e Venda.
- 1: Compra/Venda.
- 2: Compra-Venda.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador, considerando 3 variações, a quantidade e tipo de desenho.

```
1 begin
2   Plot(AvgAgrTotal(3, 1, 0));
3 end
```

Exemplo Gráfico:





19.21 Função AvgSeparation

Disponível em todas versões.

A função **AvgSeparation()** retorna o valor do indicador "Afastamento Médio" de acordo com os parâmetros desejados. Este indicador, mensura os afastamentos proporcionais dos preços em relação à uma média móvel previamente selecionada.

Sintaxe:

```
Função AvgSeparation(Media : Integer; Tipo: Integer): Float;
```

Parâmetros:

Media: Integer; Período utilizado no momento do cálculo do indicador.

Tipo: Integer; Determina qual tipo de média será considerada:

- **kaArithmetic:** Aritmética;
- **kaExponential:** Exponencial;
- **kaWelles:** Welles Wilder;
- **kaWeighted:** Ponderada;
- **kaADX:** ADX;

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Afastamento Médio", considerando **21** períodos e tipo exponencial para o cálculo.

```
1 begin
2   Plot(AvgSeparation(21, kaExponential));
3 end
```

Exemplo Gráfico:



19.22 Função AvgTrueRange

Disponível em todas as versões.

A função `AvgTrueRange()` retorna o valor do indicador "True Range" de acordo com os parâmetros desejados. Este indicador ajuda a medir a volatilidade de um ativo.

Sintaxe:

```
Função AvgTrueRange(Período : Integer; Tipo: Integer): Float;
```

Parâmetros:

Período: `Integer`; Período utilizado no momento do cálculo do indicador.

Tipo: `Integer`; Determina qual tipo de média será considerada:

- `kaArithmetic`: Aritmética;
- `kaExponential`: Exponencial;
- `kaWelles`: Welles Wilder;
- `kaWeighted`: Ponderada;
- `kaADX`: ADX;

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "True Range", considerando **5** períodos, e tipo de média ponderada para o cálculo.

```
1 begin
2   Plot(AvgTrueRange(5, kaWeighted));
3 end
```

Exemplo Gráfico:



19.23 Função BTCPrice

Disponível com o pacote opcional NeloCripto.

A função **BTCPrice()** retorna o valor do indicador que mostra a comparação entre BITFUT (Bitcoin na B3) e o Bitcoin em exchanges de criptomoedas. Indicador é formado por duas linhas: uma delas é o preço do ativo BITFUT (B3) e a outra é o preço do BTC das exchanges de criptomoedas. Quando ocorre um alargamento distanciando esses 2 preços, normalmente indica momentos de oportunidade.

Sintaxe:

```
Função BTCPrice(SignalType : Integer): Float;
```

Parâmetros:

SignalType: **Integer**; Determina o signal do indicador.

- **0**: BTC/BRL.
- **1**: BTC/USD.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador em BTC/USD.

```
1 begin
2   Plot(BTCPrice(1));
3 end
```

Exemplo Gráfico:



19.24 Função BTCSpread

Disponível com o pacote opcional NeloCripto.

A função **BTCSpread()** retorna o valor do indicador que calcula o spread entre o preço do BITFUT (Bitcoin na B3) e o preço de Bitcoin em exchanges de criptomoedas. Foi desenvolvido para identificar momentos de distorção nos preços, que aumenta o spread e pode indicar sobrecompra ou sobrevenda.

Sintaxe:

```
Função BTCSpread(SignalType : Integer): Float;
```

Parâmetros:

SignalType: **Integer**; Determina o sinal do indicador.

- 0: BTC/BRL.
- 1: BTC/USD.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador em BTC/USD.

```
1 begin
2   Plot(BTCSpread(1));
3 end
```

Exemplo Gráfico:



19.25 Função BalanceAgent

Disponível nas versões Ultra ou com a Automação de Estratégias 2.

A função `BalanceAgent()` retorna o saldo financeiro filtrado por agente. Retorna os dados em tempo real, não executando em backtest.

Sintaxe:

```
Função BalanceAgent(AgentID : Integer; Período : Integer): Float;
```

Parâmetros:

AgentID: `Integer`; ID do agente escolhido.

Período: `Integer`; Determina qual tipo de cálculo do saldo será obtido:

- `0`: Retorna o saldo diário, não importando o tempo gráfico onde está executando.
- `1`: Retorna o saldo candle a candle.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o saldo do agente retornado pela função `BalanceAgent()`.

```
1 begin
2   Plot(BalanceAgent(baItau,1));
3 end
```

Exemplo Gráfico:





19.26 Função BalanceOfPower

Disponível em todas versões.

A função **BalanceOfPower**() retorna o valor do indicador "Balança do Poder" de acordo com os parâmetros desejados. Este indicador mede a força de compradores vs vendedores, avaliando a capacidade de cada um dos lados de levar os preços a seu extremo.

Sintaxe:

```
Função BalanceOfPower(Media : Integer; Tipo: Integer): Float;
```

Parâmetros:

Media: **Integer**; Período utilizado no momento do cálculo do indicador.

Tipo: **Integer**; Determina qual tipo de média será considerada:

- **kaArithmetic**: Aritmética;
- **kaExponential**: Exponencial;
- **kaWelles**: Welles Wilder;
- **kaWeighted**: Ponderada;
- **kaADX**: ADX;

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Balança do Poder", considerando **14** períodos, e o tipo de média exponencial para o cálculo.

```
1 begin  
2   Plot(BalanceOfPower(14, kaExponential));  
3 end
```

Exemplo Gráfico:





19.27 Função BearPower

Disponível em todas versões.

A função **BearPower()** retorna o valor do indicador "Bear Power" de acordo com os parâmetros desejados. Este indicador procura mostrar a força dos vendedores. Quanto mais negativo for, maior é a pressão de venda sendo exercida sobre o ativo.

Sintaxe:

```
Função BearPower(Período : Integer; Tipo Média: Integer; Período Média: Integer):  
Float;
```

Parâmetros:

Período: **Integer**; Período utilizado no momento do cálculo do indicador.

Tipo Média: **Integer**; Determina qual tipo de média será considerada:

- **kaArithmetic**: Aritmética;
- **kaExponential**: Exponencial;
- **kaWelles**: Welles Wilder;
- **kaWeighted**: Ponderada;
- **kaADX**: ADX;

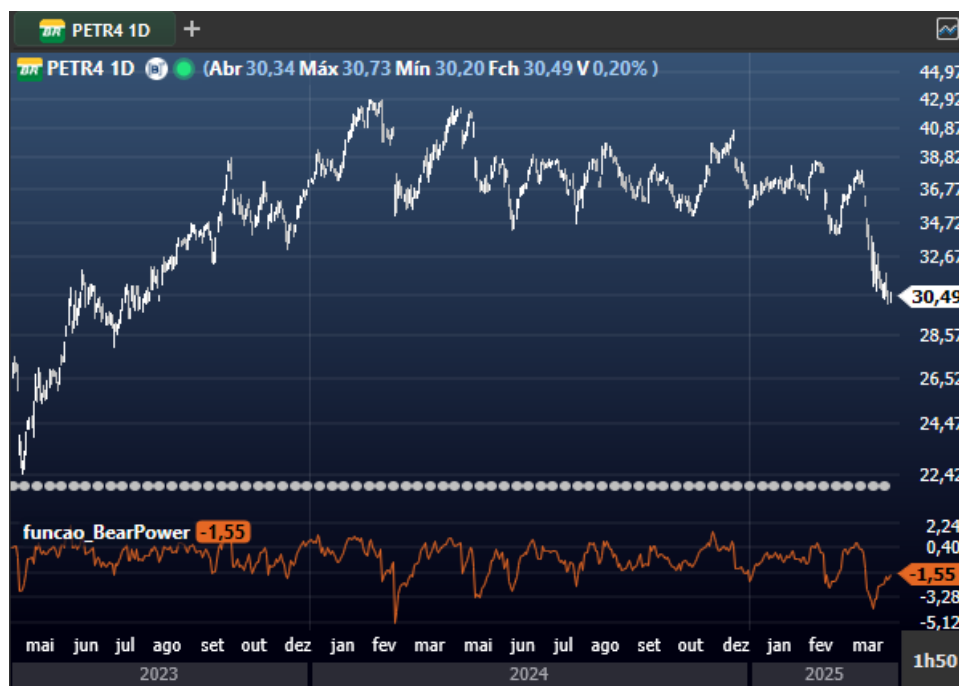
Período Média: **Integer**; Período da média utilizada no momento do cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Balança do Poder", considerando **13** períodos, e o tipo de média aritmética para o cálculo.

```
1 begin  
2   Plot(BearPower(13,kaArithmetic,0));  
3 end
```

Exemplo Gráfico:



19.28 Função BidAsk

Disponível apenas para a plataforma BlackArrow.

A função `BidAsk()` retorna os valores históricos dos preços de compra (Bid) e de venda (Ask) de um ativo. Permite visualizar individualmente cada um desses preços no gráfico

Sintaxe:

```
Função BidAsk(): Float;
```

Parâmetros:

Linha: **Integer** [Opcional]; Determina qual dado será obtido:

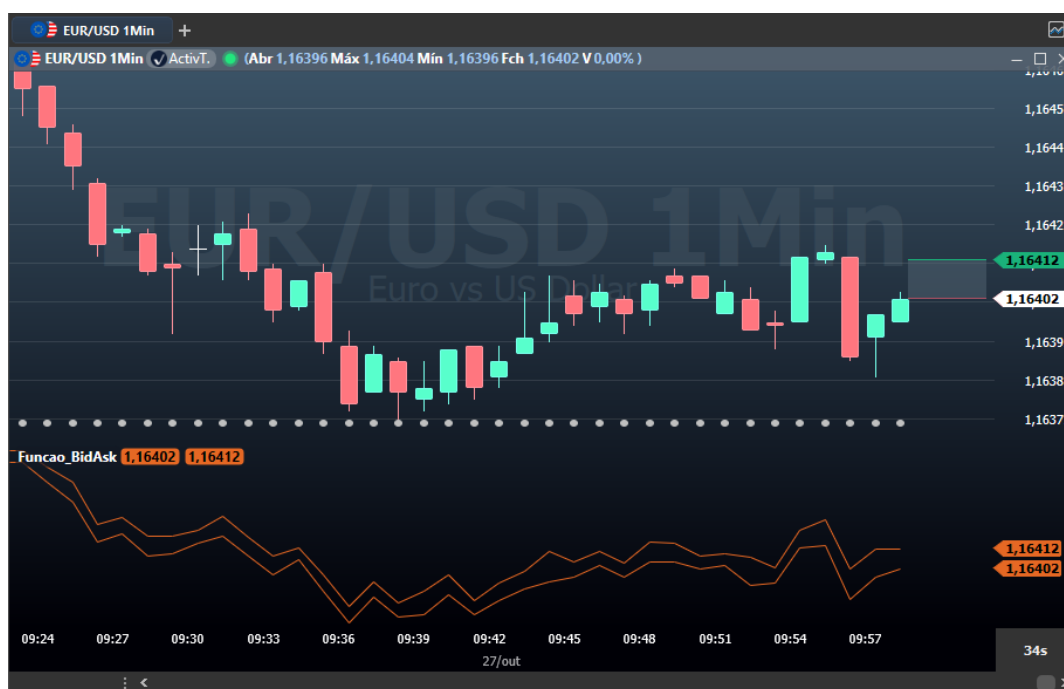
- |0| Preço de compra (Bid).
- |1| Preço de venda (Ask).

Exemplo de uso:

No exemplo a seguir, serão plotados os valores do preço de compra e venda do ativo, utilizando a função `BidAsk()` com os parâmetros `0` e `1`.

```
1 begin
2   Plot(BidAsk|0|);
3   Plot2(BidAsk|1|);
4 end
```

Exemplo Gráfico:





```
EXPORTSYMBOL(groupsfree);  
/* export the groupinfo to  
objects.active = modifier  
if(modifier==0) it's modified
```

19.29 Função BollingerBPerc

Disponível em todas versões.

A função **BollingerBPerc()** retorna o valor do indicador "Bollinger b%" de acordo com o período e tipo de média desejados. Neste indicador das "Bandas de Bollinger, quando o valor maior que 100, ele sinaliza que os preços estão sobre a banda superior e, quando menor que 0, mostra que os preços estão sob a banda inferior.

Sintaxe:

```
Função BollingerBPerc(Desvio : Float; PeríodoMédia: Integer; Tipo: Integer): Float;
```

Parâmetros:

Desvio: Float; Desvio utilizado no momento do cálculo do indicador.

PeríodoMédia: Integer; Período da média utilizada no momento do cálculo do indicador:

Tipo: Integer; Determina o tipo da média utilizada:

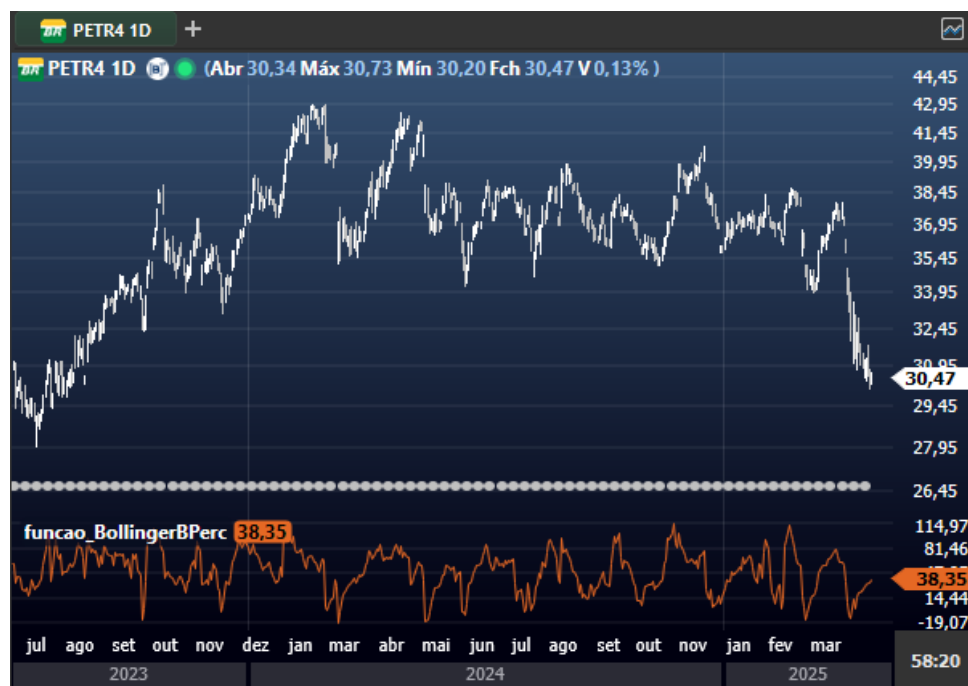
- **kaArithmetic**: Aritmética;
- **kaExponential**: Exponencial;
- **kaWelles**: Welles Wilder;
- **kaWeighted**: Ponderada;
- **kaADX**: ADX;

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Bollinger Band Width", considerando **2** como desvio, **20** períodos e tipo de média Ponderada (**3**).

```
1 begin  
2   Plot(BollingerBPerc(2,20,kaWeighted));  
3 end
```

Exemplo Gráfico:





19.30 Função BollingerBandW

Disponível em todas versões.

A função **BollingerBandW()** retorna o valor do indicador "Bollinger Band Width" de acordo com o período e tipo de média desejados. Este indicador derivado das "Bandas de Bollinger" mede a largura das bandas como um percentual de sua média móvel. A ocorrência de estritamentos, normalmente, antecede grandes movimentos.

Sintaxe:

```
Função BollingerBandW(Desvio : Float; PeríodoMédia: Integer; Tipo: Integer): Float;
```

Parâmetros:

Desvio: Float; Desvio utilizado no momento do cálculo do indicador.

PeríodoMédia: Integer; Período da média utilizada no momento do cálculo do indicador:

Tipo: Integer; Determina o tipo da média utilizada:

- **kaArithmetic**: Aritmética;
- **kaExponential**: Exponencial;
- **kaWelles**: Welles Wilder;
- **kaWeighted**: Ponderada;
- **kaADX**: ADX;

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Bollinger Band Width", considerando **2** como desvio, **20** períodos e tipo de média exponencial (**1**).

```
1 begin  
2   Plot(BollingerBandW(2,20,kaExponential));  
3 end
```

Exemplo Gráfico:





19.31 Função BollingerBands

Disponível em todas versões.

A função `BollingerBands()` retorna o valor do indicador "Bandas de Bollinger" de acordo com o período e tipo de média desejados. Este indicador é formado por duas linhas, uma superior e outra inferior. São traçadas a um desvio padrão de distância a partir de uma média móvel.

Sintaxe:

```
Função BollingerBands(Desvio : Float; PeríodoMédia: Integer; Tipo: Integer): Float;
```

Parâmetros:

Desvio: `Float`; Desvio utilizado no momento do cálculo do indicador.

PeríodoMédia: `Integer`; Período da média utilizada no momento do cálculo do indicador:

Tipo: `Integer`; Determina o tipo da média utilizada:

- `kaArithmetic`: Aritmética;
- `kaExponential`: Exponencial;
- `kaWelles`: Welles Wilder;
- `kaWeighted`: Ponderada;
- `kaADX`: ADX;

Linha: `Integer` [Opcional]; Determina qual linha será obtida:

- `|0|` Superior;
- `|1|` Inferior.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor da linha superior do indicador "Bandas de Bollinger", considerando `2` como desvio, `20` períodos e tipo de média aritmética (`0`).

```
1 begin  
2   Plot(BollingerBands(2, 20, kaArithmetic)|0|);  
3 end
```



```
EXPORTSYMBOL(groupsfree);  
/* export the groupinfo to  
objects.active = modifier  
set(modifier-ob))#modifier(ga
```

Exemplo Gráfico:





19.32 Função BullPower

Disponível em todas versões.

A função **BullPower**() retorna o valor do indicador "BullPower" de acordo com o período e tipo de média desejados. Neste indicador das bandas de Bollinger, quando o valor maior que 100, ele sinaliza que os preços estão sobre a banda superior e, quando menor que 0, mostra que os preços estão sob a banda inferior.

Sintaxe:

```
Função BullPower(Período : Integer; Tipo Média: Integer; Período Média: Integer):  
    Float;
```

Parâmetros:

Período: Integer; Período utilizado no momento do cálculo do indicador.

Tipo Média: Integer; Determina o tipo da média utilizada:

- **kaArithmetic:** Aritmética;
- **kaExponential:** Exponencial;
- **kaWelles:** Welles Wilder;
- **kaWeighted:** Ponderada;
- **kaADX:** ADX;

Período Média: Integer; Período utilizado no momento do cálculo da média utilizada no indicador.

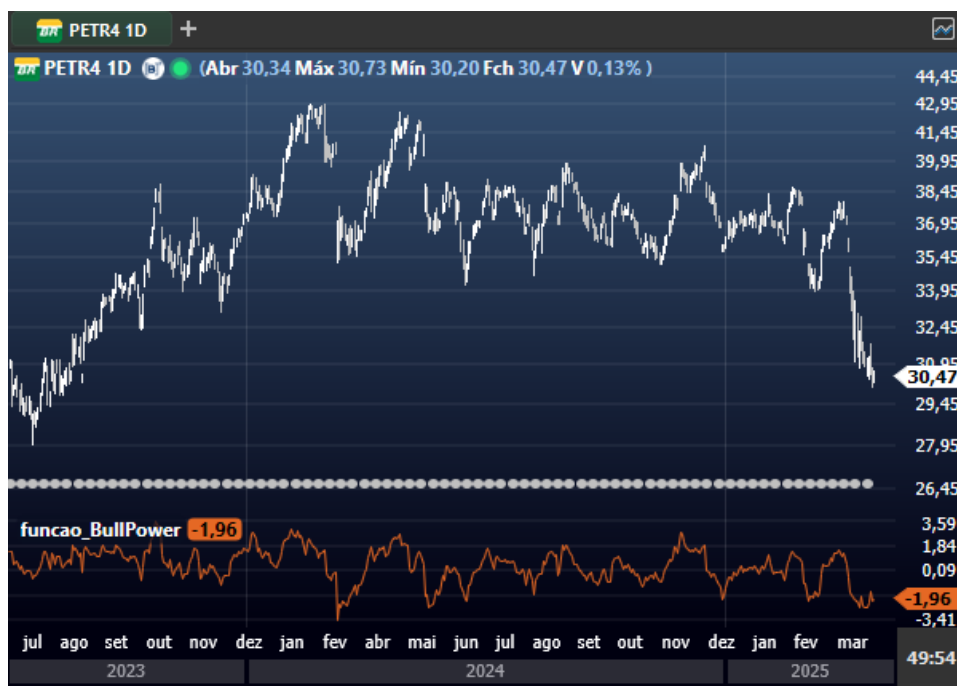
Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Bull Power", considerando 21 períodos, e 9 períodos para a média aritmética (0).

```
1 begin  
2   Plot(BullPower(21,kaArithmetic,9));  
3 end
```




Exemplo Gráfico:



19.33 Função CCI

Disponível em todas versões.

A função `CCI()` retorna o valor do indicador "CCI" de acordo com o período desejado. Este indicador mede a variação do preço de um ativo em relação a sua média. Bastante utilizado com divergências e níveis de sobrecompra/sobrevenda.

Sintaxe:

```
Função CCI(Período : Integer): Float;
```

Parâmetros:

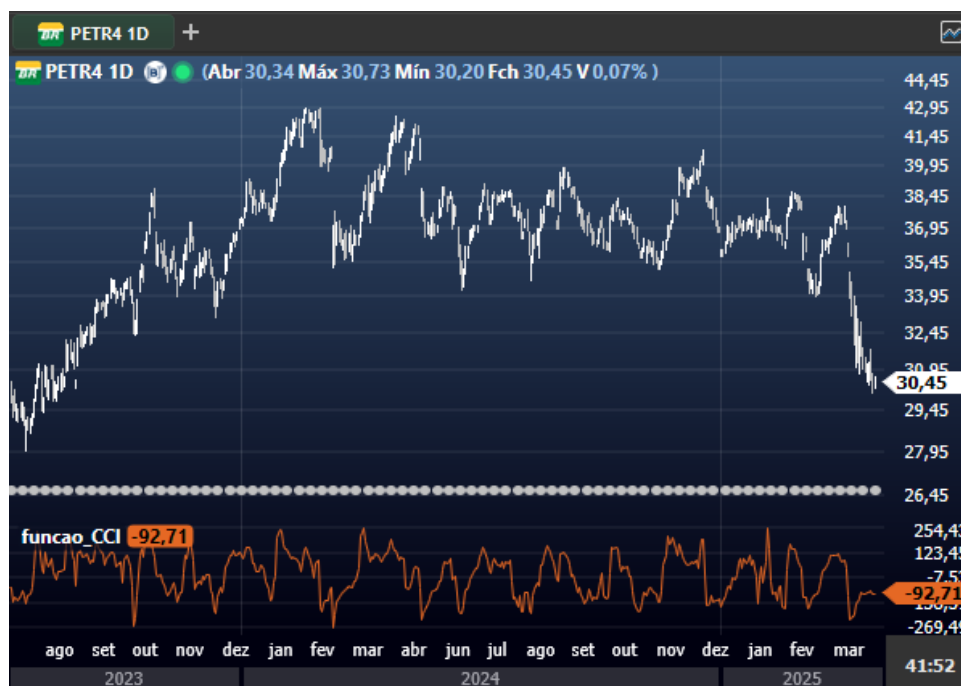
Período: `Integer`; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "CCI", considerando `14` períodos para o cálculo.

```
1 begin
2   Plot(CCI(14));
3 end
```

Exemplo Gráfico:





19.34 Função ChaikinMoneyFlow

Disponível em todas versões.

A função **ChaikinMoneyFlow()** retorna o valor do indicador “Chaikin Money Flow” de acordo com o período desejado. Este indicador utiliza relações de preço e volume. Quando menor que zero, indica maior pressão vendedora. Quando maior, indica força compradora.

Sintaxe:

```
Função CCI(Período : Integer): Float;
```

Parâmetros:

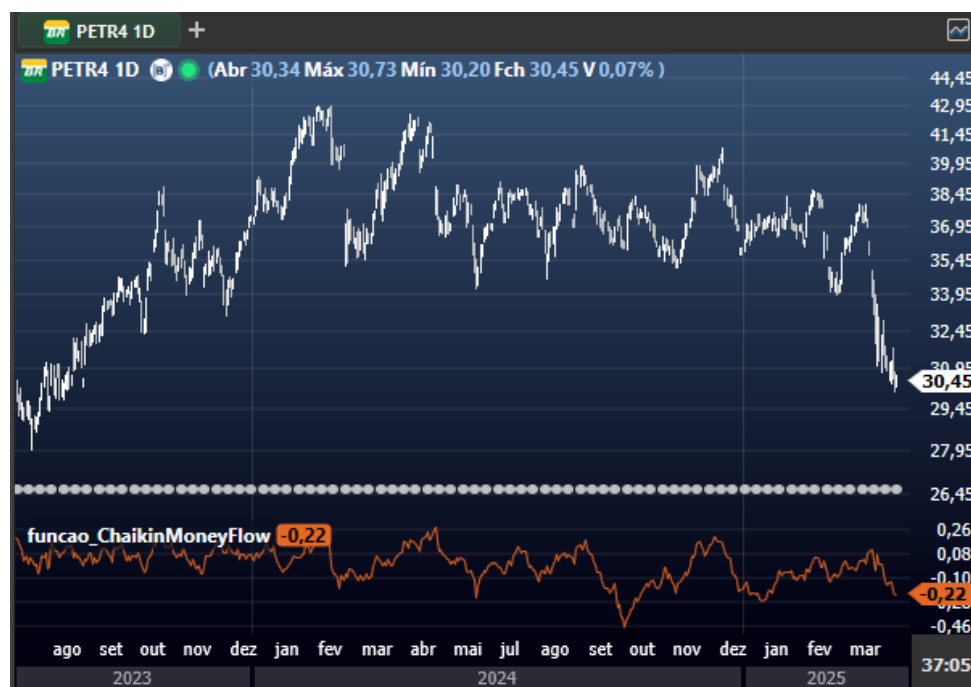
Período: Integer; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador “Chaikin Money Flow”, considerando **21** período para o cálculo.

```
1 begin
2   Plot(ChaikinMoneyFlow(21));
3 end
```

Exemplo Gráfico:



19.35 Função ChaikinOsc

Disponível em todas versões.

A função `ChaikinOsc()` retorna o valor do indicador "Oscilador Chaikin" de acordo com as médias desejadas. Este indicador relaciona preço e volume. Sua principal utilização é através da identificação de divergências.

Sintaxe:

```
Função ChaikinOsc(MediaLonga : Integer; MediaCurta : Integer): Float;
```

Parâmetros:

MediaLonga: **Integer**; Determina o período da Média Longa para formação do cálculo.

MediaCurta: **Integer**; Determina o período da Média Curta.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Oscilador Chaikin", considerando **10** períodos para a média longa, e **3** para a curta.

```
1 begin
2   Plot(ChaikinOsc(10, 3));
3 end
```

Exemplo Gráfico:





19.36 Função ChainSetup

Disponível em todas versões.

A função **ChainSetup()** retorna o valor do indicador "ChainSetup" de acordo com as médias desejadas. Este indicador do "Café com Trade" consolida os indicadores Trix, Macd, Tilson + volume e agressão financeira, como forma de ampliar a análise do Tape Reading, indicando a tendência do ativo e verificando o melhor ' + ' momento de entrar, pois indica baixa e alta liquidez + explosão de volatilidades.

Sintaxe:

```
Função ChainSetup(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o dado do indicador "ChainSetup".

```
1 begin
2   Plot(ChainSetup);
3 end
```

Exemplo Gráfico:





19.37 Função COHENPRICEWAVE

Disponível em todas versões.

A função **COHENPRICEWAVE**() retorna o valor do indicador "Cohen - Price Wave". Este indicador indica a diferença absoluta de preços de cada swing no atico. Junto com o "WeisWave", serve como um indicador de exaustão.

Sintaxe:

```
Função COHENPRICEWAVE(Ticks : Integer): Float;
```

Parâmetros:

Ticks: **Integer**; Níveis de preço.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno do indicador, considerando **15** ticks para o cálculo.

```
1 begin
2   Plot(COHENPRICEWAVE(15));
3 end
```

Exemplo Gráfico:





19.38 Função COHENWEISWAVE

Disponível em todas versões.

A função **COHENWEISWAVE()** retorna o valor do indicador "Cohen - Price Wave". Este indicador indica a soma do volume de cada swing no ativo. Serve como um indicador de exaustão.

Sintaxe:

```
Função COHENWEISWAVE(Ticks : Integer): Float;
```

Parâmetros:

Ticks: Integer; Relacionado ao valor de entrada do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno do indicador, considerando **15** ticks para o cálculo.

```
1 begin
2   Plot(COHENWEISWAVE(15));
3 end
```

Exemplo Gráfico:





19.39 Função ContadorDeCandle

Disponível em todas versões.

A função `ContadorDeCandle()` contabiliza e sinaliza de forma numérica e organizada no gráfico o número de cada candle.

Sintaxe:

```
Função ContadorDeCandle(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o número do candle atual.

```
1 begin
2   Plot(ContadorDeCandle());
3 end
```

Exemplo Gráfico:





19.40 Função DT Oscillator

Disponível em todas versões.

A função **DT Oscillator**() retorna o valor do indicador "DT Oscillator", de acordo com o período desejado. Este indicador é formado traçando-se um canal com os valores de máxima alta e da mínima baixa do período escolhido. É uma versão suavizada do IFR Estocástico. Consiste em duas linhas que produzem sinais de compra e venda baseados nos seus cruzamentos.

Sintaxe:

```
Função DT Oscillator(PeríodoEstocastico : Integer; PeríodoSK : Integer; TipoSK :  
  Integer; PeríodoSD : Integer; TipoSD : Integer): Float;
```

Parâmetros:

PeríodoEstocastico: **Integer**; Período utilizado no momento do cálculo do indicador.

PeríodoSK: **Integer**; Período referente ao parâmetro "Período SK".

TipoSK: **Integer**; Determina qual média será considerada:

- **kaArithmetic**: Aritmética;
- **kaExponential**: Exponencial;
- **kaWelles**: Welles Wilder;
- **kaWeighted**: Ponderada;
- **kaADX**: ADX;

PeríodoSD: **Integer**; Período referente ao parâmetro "Período SD".

TipoSD: **Integer**; Determina qual média será considerada:

- **kaArithmetic**: Aritmética;
- **kaExponential**: Exponencial;
- **kaWelles**: Welles Wilder;
- **kaWeighted**: Ponderada;
- **kaADX**: ADX;

Linha: **Integer** [Opcional]; Determina qual linha será obtida:

- |**0**| Linha 1.
- |**1**| Linha 2.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "DT Oscillator", considerando **12** (PeríodoEstocastico), **8** (PeríodoSK), **5** (PeríodoSD) períodos, e tipo aritmética (**0**) para o cálculo.

```
1 begin  
2   Plot(DT Oscillator(12, 8, kaArithmetic, 5, kaArithmetic)|1|);  
3 end
```

Exemplo Gráfico:



19.41 Função DarvasBox

Disponível em todas versões.

A função `DarvasBox()` retorna o valor do indicador "Darvas Box".

Sintaxe:

```
Função DarvasBox(): Float;
```

Parâmetros:

Linha: **Integer** [Opcional]; Determina qual dado (Compra ou Venda) será obtido:

- |0| Compra.
- |1| Venda.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico os valores do indicador "Darvas Box" com parâmetro de linha **1** (Venda).

```
1 begin
2   Plot(DarvasBox|1|);
3 end
```

Exemplo Gráfico:





19.42 Função DecisionPoints

Disponível nos produtos Pro ou Ultra ou módulo Scalper.

A função `DecisionPoints()` retorna o valor do indicador "Pontos de Decisão".

Sintaxe:

```
Função DecisionPoints(Tipo : Integer; Linha : Integer): Float;
```

Parâmetros:

Tipo: **Integer**; Determina o tipo:

- **0**: Preço;
- **1**: Volume;
- **2**: Faixas de Volume;
- **3**: Variação.

Linha: **Integer**; Retorna o dado de um tipo específico:

Para Tipo = **0**:

- **0**: Abertura;
- **1**: Máxima;
- **2**: Mínima;
- **3**: Fechamento;
- **4**: Ajuste.

Para Tipo = **1**:

- **0**: Dado volume;
- **1**: Dado volume;
- **2**: Dado volume.

Para Tipo = **2**:

- **0**: Dado volume.

Para Tipo = **3**:

- **0**: Dado variação linha inferior (-2%);
- **1**: Dado variação linha inferior (-1%);
- **2**: Dado variação linha superior (1%);
- **3**: Dado variação linha superior (2%).

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o dado referente à máxima (**1**).

```
1 begin
2   Plot(DecisionPoints(0, 1));
3 end
```

Exemplo Gráfico:



19.43 Função DiPDiM

Disponível em todas versões.

A função `DiPDiM()` retorna o valor do indicador "DI+/DI-", de acordo com o período desejado. Este indicador utiliza o conceito de movimento direcional, onde sinais de compras são gerados quando DI+ cruza para cima DI-, sinais de venda são gerados quando DI- cruza para baixo DI+. Deve-se esperar por confirmações nos preços.

Sintaxe:

```
Função DiPDiM(Período : Integer): Float;
```

Parâmetros:

Período: `Integer`; Período utilizado no momento do cálculo do indicador.

Linha: `Integer` [Opcional]; Determina qual linha será obtida:

- |0| DI+;
- |1| DI-.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor DI- (1) do indicador "DI+/DI-", considerando 14 período para o cálculo.

```
1 begin
2   Plot(DiPDiM(14)|1|);
3 end
```

Exemplo Gráfico:





19.44 Função DiDiIndex

Disponível em todas versões.

A função **DiDiIndex()** retorna o valor do indicador "Didi Index", de acordo com o período e tipos de médias desejados.

Sintaxe:

```
Função DiDiIndex(PeriodoMediaR : Integer; TipoR : Integer; PeriodoMedia1 : Integer;  
    Tipo1 : Integer; PeriodoMedia2 : Integer; Tipo2 : Integer): Float;
```

Parâmetros:

PeriodoMediaR: Integer; Parâmetro para o período utilizado no cálculo da média de referência do indicador.

TipoR: Integer; Determina qual média será considerada:

- **kaArithmetic:** Aritmética;
- **kaExponential:** Exponencial;
- **kaWelles:** Welles Wilder;
- **kaWeighted:** Ponderada;
- **kaADX:** ADX;

PeriodoMedia1: Integer; Período utilizado no cálculo da média1 do indicador.

Tipo1: Integer; Determina qual média será considerada:

- **kaArithmetic:** Aritmética;
- **kaExponential:** Exponencial;
- **kaWelles:** Welles Wilder;
- **kaWeighted:** Ponderada;
- **kaADX:** ADX;

PeriodoMedia2: Integer; Período utilizado no cálculo da média2 do indicador.

Tipo2: Integer; Determina qual média será considerada:

- **kaArithmetic:** Aritmética;
- **kaExponential:** Exponencial;
- **kaWelles:** Welles Wilder;
- **kaWeighted:** Ponderada;
- **kaADX:** ADX;

Linha: Integer [Opcional]; Determina qual linha será obtida:

- **|0|** Linha;
- **|1|** Linha 2.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor da **linha 2**, considerando **8** (Média de Referência), **3** (Média1) e **20** (Média2) períodos, aplicando o tipo de média aritmética (**0**) para o cálculo.

```
1 begin  
2   Plot(DidiIndex(8,kaArithmetic,3,kaArithmetic,20,kaArithmetic)|1|);  
3 end
```

Exemplo Gráfico:





19.45 Função DivergenceDetector

Disponível em todas versões.

A função **DivergenceDetector()** retorna os pontos de alta ou baixa (PH ou PL) do ativo de acordo com o número de pivôs à esquerda e direita definidos nos parâmetros ou retorna a quantidade de indicadores considerados naquele candle relacionados aos indicadores considerados na divergência. Esse indicador retorna os pontos identificados pelo indicador "Detector de Divergências".

Sintaxe:

```
Função DivergenceDetector(NumLeftCandles : Integer; NumRigthCandles : Integer; MACD : Boolean; MACDHistogram : Boolean; RSI : Boolean; Stochastic : Boolean; CCI : Boolean; Momentum : Boolean; OBV : Boolean; Diosc : Boolean; VWmacd : Boolean; ChaikinMoneyFlow : Boolean; MoneyFlowIndex : Boolean): Float;
```

Parâmetros:

NumLeftCandles: **Integer**; Número de candles à considerar como pivô à esquerda (apenas influencia cálculo de PH e PL).

NumRigthCandles: **Integer**; Número de candles à considerar como pivô à direita (apenas influencia cálculo de PH e PL).

MACD: **Boolean** [Opcional]; Parâmetro opcional que indica se deve considerar o indicador MACD para cálculo de divergência dos indicadores.

MACDHistogram: **Boolean** [Opcional]; Parâmetro opcional que indica se deve considerar o indicador MACD Histograma para cálculo de divergência dos indicadores.

RSI: **Boolean** [Opcional]; Parâmetro opcional que indica se deve considerar o indicador RSI para cálculo de divergência dos indicadores.

Stochastic: **Boolean** [Opcional]; Parâmetro opcional que indica se deve considerar o indicador Estocástico para cálculo de divergência dos indicadores.

CCI: **Boolean** [Opcional]; Parâmetro opcional que indica se deve considerar o indicador CCI para cálculo de divergência dos indicadores.

Momentum: **Boolean** [Opcional]; Parâmetro opcional que indica se deve considerar o indicador Momentum para cálculo de divergência dos indicadores.

OBV: **Boolean** [Opcional]; Parâmetro opcional que indica se deve considerar o indicador OBV para cálculo de divergência dos indicadores.

Diosc: **Boolean** [Opcional]; Parâmetro opcional que indica se deve considerar o indicador Diosc para cálculo de divergência dos indicadores.

VWmacd: **Boolean** [Opcional]; Parâmetro opcional que indica se deve considerar o indicador VWmacd para cálculo de divergência dos indicadores.

ChaikinMoneyFlow: **Boolean** [Opcional]; Parâmetro opcional que indica se deve considerar o indicador Chaikin Money Flow para cálculo de divergência dos indicadores.

MoneyFlowIndex: **Boolean** [Opcional]; Parâmetro opcional que indica se deve considerar o indicador MoneyFlowIndex para cálculo de divergência dos indicadores.

Index: **Integer** [Opcional]; Determina qual tipo de cálculo será obtido:

- |0| Retorna os valores de PH e PL para o ativo, onde os resultados são da seguinte forma:
 - 1: caso o candle analisado seja PL.



Documentação Módulo Estratégias

- 0: caso não seja nem PL nem PH.
- 1: caso o candle analisado seja PH.

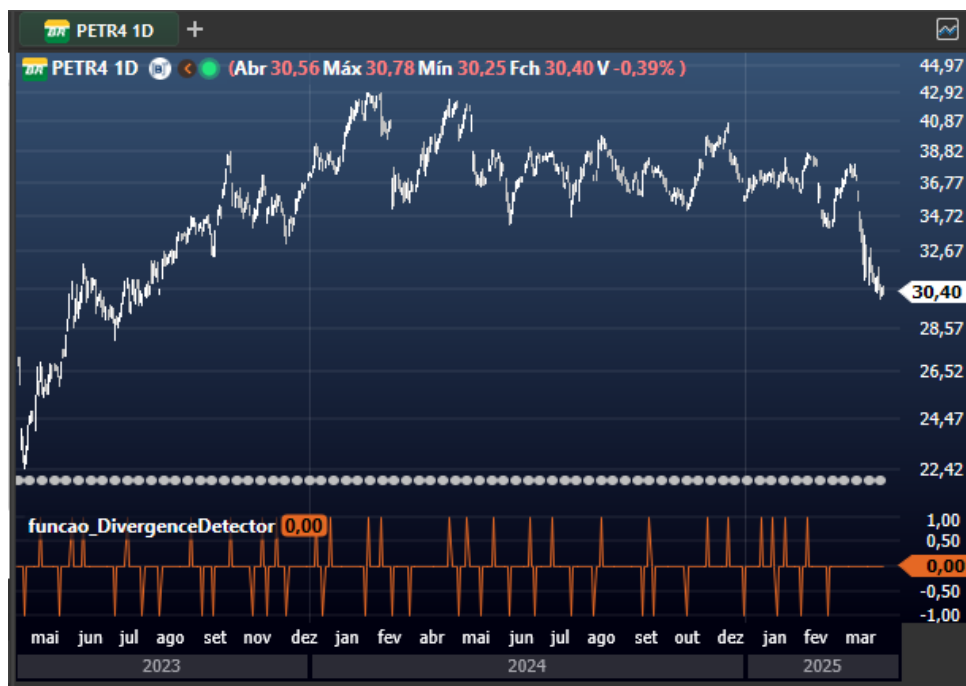
– |1| Retorna os valores relacionados à divergência dos indicadores definidos da seguinte forma:
Soma a quantidade de indicadores que indicam divergência naquele candle e retorna o valor positivo caso seja um candle positivo, ou negativo caso seja um candle negativo.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor relacionado a PH/PL do candle a ser analisado considerando os valores passados para o NumLeftCandles e NumRightCandles (note que esse valor é independente dos indicadores passados opcionalmente por parâmetro).

```
1 begin
2   Plot(DivergenceDetector(5, 5)|0|);
3 end
```

Exemplo Gráfico:



No exemplo a seguir, será plotado no gráfico o valor relacionado à soma dos indicadores que detectaram alguma divergência, ou nenhuma, no candle analisado. Nesse exemplo abaixo, estamos considerando apenas os indicadores MACD, MACD Histograma e RSI para o cálculo.

```
1 begin
2   Plot(DivergenceDetector(5, 5, True, True, True)|1|);
3 end
```



Exemplo Gráfico:





19.46 Função DonchianCH

Disponível em todas versões.

A função `DonchianCH()` retorna o valor do indicador "Canal Donchian", de acordo com o período desejado. Este indicador é formado traçando-se um canal com os valores de máxima alta e da mínima baixa do período escolhido.

Sintaxe:

```
Função DonchianCh(Período : Integer): Float;
```

Parâmetros:

Período: Integer; Período utilizado no momento do cálculo do indicador.

Linha: Integer [Opcional]; Determina qual linha será obtida:

- |0| Média;
- |1| Superior;
- |2| Inferior.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor da linha inferior do indicador "Canal Donchian", considerando 20 períodos para o cálculo.

```
1 begin
2   Plot(DonchianCh(20)|2|);
3 end
```

Exemplo Gráfico:



19.47 Função EMA

Disponível em todas versões.

A função `EMA()` calcula a média móvel exponencial com base em um número definido de períodos. Diferente da média simples, a EMA dá maior peso aos valores mais recentes, tornando-se mais sensível a mudanças na tendência de preço.

Sintaxe:

```
Função EMA(Período : Integer; TipoValor : Integer): Float;
```

Parâmetros:

Período: Integer; Quantidade de períodos utilizada no cálculo da média exponencial.

TipoValor: Integer[Opcional]; Define qual valor do candle será considerado no cálculo:

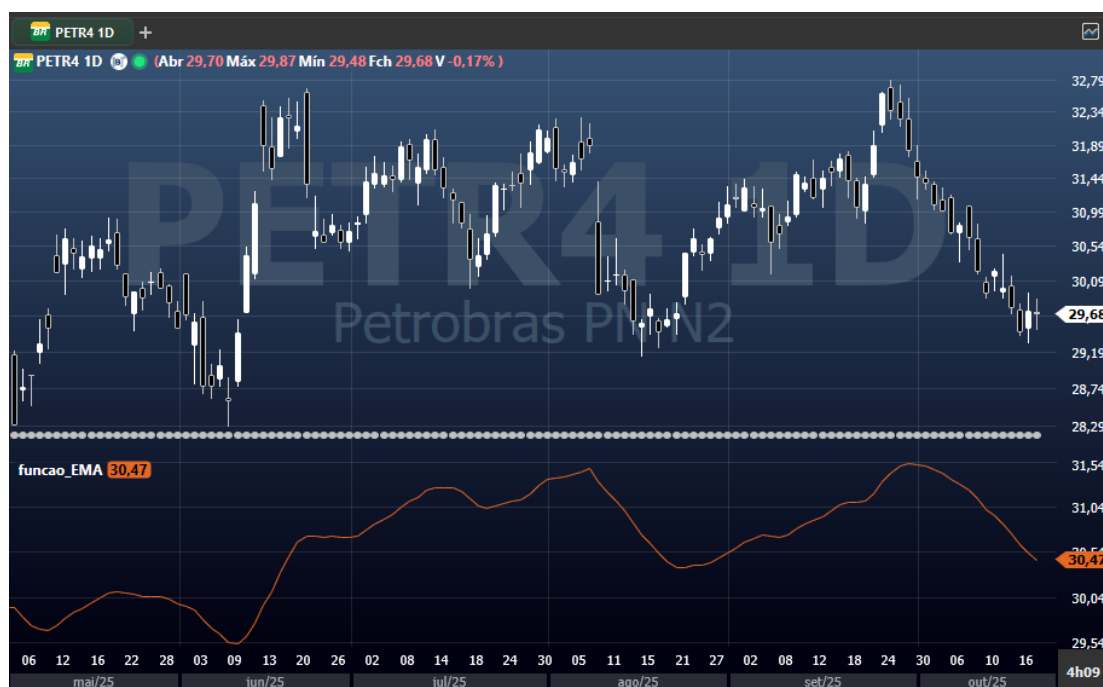
- **tvClose:** Fechamento (Close);
- **tvOpen:** Abertura (Open);
- **tvMax:** Máxima (High);
- **tvMin:** Mínima (Low).

Exemplo de uso:

No exemplo abaixo, é plotada uma média móvel exponencial de **20** períodos, considerando o valor de fechamento.

```
1 begin
2   Plot(EMA(20, tvClose));
3 end
```

Exemplo Gráfico:





19.48 Função Envelope

Disponível em todas versões.

A função `Envelope()` retorna o valor do indicador "Envelope", de acordo com o período e média desejados. São linhas traçadas a uma distância percentual fixa de uma média móvel. Variando-se os parâmetros do envelope é possível identificar níveis de preços nos quais correções/revisões são prováveis.

Sintaxe:

```
Função Envelope(Desvio : Float; PeriodoMedia : Integer; Tipo : Integer): Float;
```

Parâmetros:

Desvio: `Float`; Percentual utilizado no momento do cálculo do indicador.

PeriodoMedia: `Integer`; Período utilizado para o cálculo da média.

Tipo: `Integer`; Determina qual média será considerada:

- `kaArithmetic`: Aritmética;
- `kaExponential`: Exponencial;
- `kaWelles`: Welles Wilder;
- `kaWeighted`: Ponderada;
- `kaADX`: ADX;

Linha: `Integer` [Opcional]; Determina qual linha será obtida:

- `|0|` Ponto médio;
- `|1|` Superior;
- `|2|` Inferior.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o dado da linha superior do indicador "Envelope", considerando `5.0` como percentual, `20` períodos e tipo de média aritmética (`0`) para o cálculo.

```
1 begin
2   Plot(Envelope(5.0, 20 , kaArithmetic)|1|);
3 end
```

Exemplo Gráfico:





19.49 Função Euroinvest

Disponível em todas versões.

A função **Euroinvest()** retorna o valor do indicador "Euroinvest", conforme os parâmetros determinados. Este indicador de tendência baseado em Trend Following é utilizado para verificação de estado do robô EuroinvestTrend.

Sintaxe:

```
Função Euroinvest(Risco : Integer; Modo de Cálculo : Integer; Período : Integer;  
Desvio : Float; Usar VWAP : Boolean; Usar Atr : Boolean): Float;
```

Parâmetros:

Risco: **Integer**; Determina o tipo de perfil:

- **0**: Zero;
- **1**: Um;
- **2**: Dois;
- **3**: Três.

Modo de Cálculo: **Integer**; Determina o tipo de média.

- **kaArithmetic**: Aritmética;
- **kaExponential**: Exponencial;
- **kaWelles**: Welles Wilder;
- **kaWeighted**: Ponderada;
- **kaADX**: ADX;

Período: **Integer**; Período considerado para o cálculo da média.

Desvio: **Float**; Desvio da média.

Usar VWAP: **Boolean**; Determina se o VWAP será utilizado.

Usar Atr: **Boolean**; Determina a habilitação do StopATR.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função, considerando o perfil Zero (**0**), tipo de média aritmética (**0**), **21** períodos, com **2** de desvio, onde o VWAP e StopAtr estão habilitados.

```
1 begin  
2   Plot(Euroinvest(0, kaArithmetic, 21, 2.0, True, True));  
3 end
```


Exemplo Gráfico:



19.50 Função FastStochastic

Disponível em todas versões.

A função **FastStochastic()** retorna o valor do indicador "Estocástico Rápido", de acordo com o período desejado. A família de indicadores estocástico (lento, pleno e rápido) ajuda a identificar situações de compra e venda excessiva. Normalmente, acima de 80 e abaixo de 20 o mercado torna-se vulnerável a correções.

Sintaxe:

```
Função FastStochastic(Período : Integer, PeríodoMedia : Integer, TipoMedia :  
Integer): Float;
```

Parâmetros:

Período: **Integer**; Determina o tipo de perfil:

PeríodoMedia: **Integer** [**var**] [**Opcional**]; Determina o tipo de média. Em caso de não utilização, o valor padrão será "14".

TipoMedia: **Integer** [**var**] [**Opcional**]; Período considerado para o cálculo da média. Em caso de não utilização, o valor padrão será "1":

- **kaArithmetic**: Aritmética;
- **kaExponential**: Exponencial;
- **kaWelles**: Welles Wilder;
- **kaWeighted**: Ponderada;
- **kaADX**: ADX;

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Estocástico Rápido", considerando **14** períodos para o cálculo.

```
1 begin  
2   Plot(FastStochastic(14));  
3 end
```

Exemplo Gráfico:



19.51 Função FastWMA

Disponível em todas versões.

A função **FastWMA()** retorna o valor do indicador "Weighted Moving Average (WMA) - Média Móvel Ponderada Rápida", aplicando uma média móvel com peso maior para períodos mais recentes e pesos menores para períodos mais antigos dentro da janela amostragem.

Sintaxe:

```
Função FastWMA(Período : Integer): Float;
```

Parâmetros:

Período: Integer; Período considerado para o cálculo da média.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador período igual a **15**.

```
1 begin
2   Plot(FastWMA(15));
3 end
```

Exemplo Gráfico:





19.52 Função FgzRobot

Disponível em todas versões.

A função `FgzRobot()` retorna o valor do "Robô Fgz".

Sintaxe:

```
Função FgzRobot(): Float;
```

Parâmetros:

Sem parâmetros.



19.53 Função FinancialVol

Disponível em todas versões.

A função **FinancialVol()** retorna o valor do indicador "Volume Financeiro", podendo-se incluir ou desconsiderar os dados: "volume projetado" e "leilão e trades diretos". Este indicador mostra o volume financeiro gerado pelas negociações. É um importante elemento na confirmação de movimentos. Nesta função o parâmetro **SoAgressores** só será considerado para o cálculo do indicador caso o seja assinado o Opcional "Plugin Tape Reading", caso não possua o opcional o indicador será sempre calculado como False, independente do que for inserido no campo.

Sintaxe:

```
Função FinancialVol(VolumeProjetado : Boolean; SoAgressores : Boolean): Float;
```

Parâmetros:

VolumeProjetado: **Boolean**; Determina se o volume irá considerar o dado projetado.

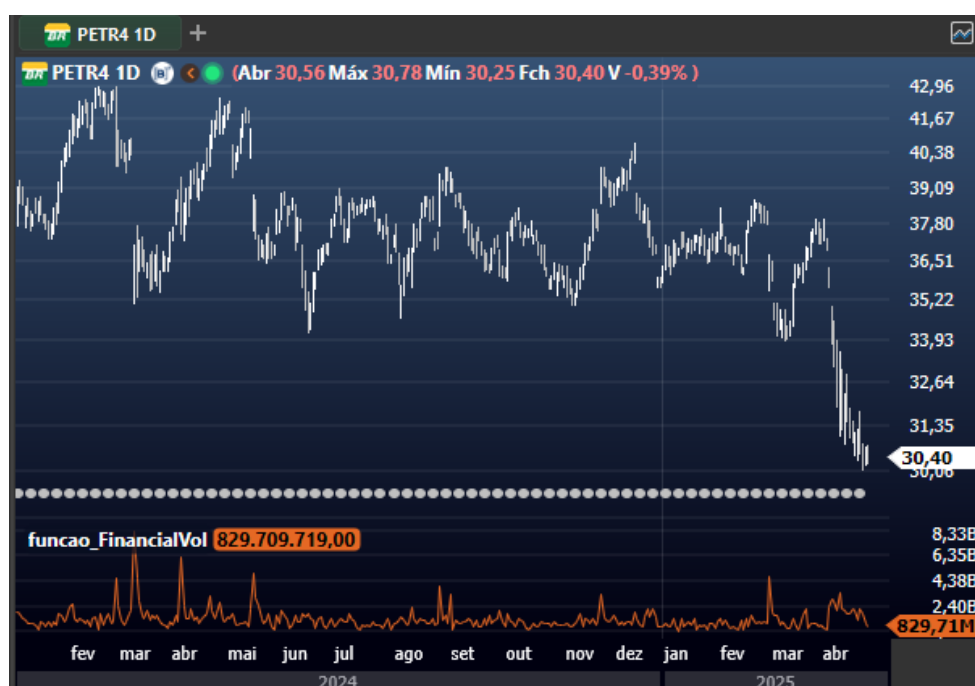
SoAgressores: **Boolean** [Opcional]; Determina se o volume irá desconsiderar o leilão e trades diretos.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Volume Financeiro", desconsiderando os dados de "volume projetado" e "leilão e trades diretos".

```
1 begin
2   Plot(FinancialVol(False, False));
3 end
```

Exemplo Gráfico:





19.54 Função ForceIndex

Disponível em todas versões.

A função `ForceIndex()` retorna o valor do indicador "Force Index", de acordo com o período e tipo de média desejados. Este indicador relaciona preço e volume. É utilizado para identificar pontos de reversão através de movimentos excessivos e divergências.

Sintaxe:

```
Função ForceIndex(Período : Integer; Tipo : Integer): Float;
```

Parâmetros:

Período: `Integer`; Período utilizado no momento do cálculo do indicador.

Tipo: `Integer`; Determina qual média será considerada:

- `kaArithmetic`: Aritmética;
- `kaExponential`: Exponencial;
- `kaWelles`: Welles Wilder;
- `kaWeighted`: Ponderada;
- `kaADX`: ADX;

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Force Index", considerando `13` períodos e tipo de média exponencial (`1`).

```
1 begin
2   Plot(ForceIndex(13, kaExponential));
3 end
```

Exemplo Gráfico:





19.55 Função FrassonATR

Disponível em todas versões.

A função **FrassonATR()** retorna o valor do indicador "Frasson ATR", de acordo com o fator e períodos desejados. O **FrassonATR()** é uma variação do indicador "ATR", funcionando como um stop tanto para posições compradas como para posições vendidas.

Sintaxe:

```
Função FrassonATR(Fator : Float; PerodoMaxMin : Integer; PerodoATR : Integer):  
    Float;
```

Parâmetros:

Fator: **Float**; Fator de multiplicação do ATR utilizado no momento do cálculo do indicador.

PerodoMaxMin: **Integer**; Determina o período de Máxima de Mínima.

PerodoATR: **Integer**; Determina o período do cálculo do ATR.

Linha: **Integer** [Opcional]; Determina qual linha será obtida:

- |0| Superior;
- |1| Inferior.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor da linha inferior do indicador "Frasson ATR", considerando **0,03** (Fator), **15** (Período Máxima/Mínima) e **50** (Período ATR) para o cálculo.

```
1 begin  
2   Plot(FrassonATR(0.03, 15, 50)|1|);  
3 end
```

Exemplo Gráfico:





19.56 Função FrassonVH

Disponível em todas versões.

A função **FrassonVH()** retorna o valor do indicador "Frasson VH", de acordo com o fator e períodos desejados. O **FrassonVH()** é uma variação do indicador "ATR", funcionando como um stop tanto para posições compradas como para posições vendidas.

Sintaxe:

```
Função FrassonVH(Fator : Float; PerodoMaxMin : Integer; PerodoVH : Integer):  
    Float;
```

Parâmetros:

Fator: **Float**; Fator de multiplicação do ATR utilizado no momento do cálculo do indicador.

PerodoMaxMin: **Integer**; Determina o período de Máxima de Mínima.

PerodoVH: **Integer**; Determina o período do cálculo do VH.

Linha: **Integer** [Opcional]; Determina qual linha será obtida:

- |0| Superior;
- |1| Inferior.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor da linha superior do indicador "Frasson VH", considerando **0,03** (Fator), **15** (Período Máxima/Mínima) e **50** (Período VH) para o cálculo.

```
1 begin  
2   Plot(FrassonVH(0.03, 15, 50)|0|);  
3 end
```



Exemplo Gráfico:



19.57 Função FullStochastic

Disponível em todas versões.

A função **FullStochastic()** retorna o valor do indicador "Estocástico Pleno", de acordo com o período desejado. A família de indicadores estocástico (lento, pleno e rápido) ajuda a identificar situações de compra e venda excessiva. Normalmente, acima de 80 e abaixo de 20 o mercado torna-se vulnerável a correções.

Sintaxe:

```
Função FullStochastic(Período : Integer): Float;
```

Parâmetros:

Período: Integer; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Estocástico Pleno", considerando **14** períodos para o cálculo.

```
1 begin
2   Plot(FullStochastic(14));
3 end
```

Exemplo Gráfico:





19.58 Função FuraChao

Disponível em todas versões.

A função **FuraChao()** retorna o valor do indicador "Fura-Chão", de acordo com o coeficiente e deslocamento desejados. Sendo uma técnica de confirmação de rompimento de um suporte do semanal, a função procura refinar o melhor ponto de entrada para venda no prazo de position.

Sintaxe:

```
Função FuraChao(Coeficiente : Float; Deslocamento : Integer): Float;
```

Parâmetros:

Coeficiente: **Float**; Coeficiente utilizado no momento do cálculo do indicador.

Deslocamento: **Integer**; Determina quantos períodos anteriores serão utilizados como base no indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Fura-Chão", considerando **0,14** (Coeficiente) e **1** período (Deslocamento) para o cálculo.

```
1 begin
2   Plot(FuraChao(0.14, 1));
3 end
```

Exemplo Gráfico:



19.59 Função FuraTeto

Disponível em todas versões.

A função `FuraTeto()` retorna o valor do indicador "Fura-Teto", de acordo com o coeficiente e deslocamento desejados. Sendo uma técnica de confirmação de rompimento de resistência do semanal, a função procura refinar o melhor ponto de entrada na compra no prazo de postion de forma a torná-lo mais efetivo.

Sintaxe:

```
Função FuraTeto(Coeficiente : Float; Deslocamento : Integer): Float;
```

Parâmetros:

Coeficiente: **Float**; Coeficiente utilizado no momento do cálculo do indicador.

Deslocamento: **Integer**; Determina quantos períodos anteriores serão utilizados como base no indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Fura-Teto", considerando **0,14** (Coeficiente) e **1** período (Deslocamento) para o cálculo.

```
1 begin
2   Plot(FuraTeto(0.14, 1));
3 end
```

Exemplo Gráfico:





19.60 Função HeikinAshi

Disponível em todas versões.

A função **HeikinAshi()** retorna o valor do indicador "Heikin Ashi", de acordo com o período e tipo de média desejados. É um tipo de gráfico de candles que compartilha muitas semelhanças com o gráfico de candles padrão, mas difere nos cálculos usados para criar cada candle.

Sintaxe:

```
Função HeikinAshi(Período : Integer; Tipo : Integer): Float;
```

Parâmetros:

Período: **Integer**; Período utilizado no momento do cálculo do indicador.

Tipo: **Integer**; Determina qual média será considerada:

- **kaArithmetic**: Aritmética;
- **kaExponential**: Exponencial;
- **kaWelles**: Welles Wilder;
- **kaWeighted**: Ponderada;
- **kaADX**: ADX;

Linha: **Integer** [Opcional]; Determina qual dado será obtido:

- **|0|** Fechamento (Close);
- **|1|** Abertura (Open);
- **|2|** Máxima (High);
- **|3|** Mínima (Low).

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor de fechamento do indicador "Heikin Ashi", considerando **1** (Período) e **0** (Aritmética) para o cálculo.

```
1 begin
2   Plot(HeikinAshi(tvClose, kaArithmetic)|1|);
3 end
```


Exemplo Gráfico:





19.61 Função HighestVolume

Disponível nas versões Ultra ou com a Automação de Estratégias 2.

A função **HighestVolume**() retorna o maior volume identificado de acordo com o tipo especificado.

Os dados do Volume Profile não fornecem um histórico extenso, o que limita o backtest de estratégias que utilizam essa função.

O parâmetro **Interval** define o período de análise e pode assumir os valores **itDaily** (intervalo diário — com **Offset** limitado a 1 nesse caso) ou **itMinute** (intervalo em minutos). O **Offset** indica quantos períodos atrás o cálculo deve considerar, sendo **0** o período atual e até **720** para períodos anteriores.

O parâmetro **Type** determina o tipo de volume a ser analisado: **vtFinancial** (volume financeiro), **vtQuantity** (quantidade de papéis) ou **vtTrade** (número de negócios). Os parâmetros **Standard**, **Auction** e **Cross** controlam a inclusão, respectivamente, de negócios padrão, leilões e operações cruzadas. Todos assumem **True** por padrão.

A função deve ser finalizada com o parâmetro **|Rank|**, onde **0** corresponde ao maior volume identificado na análise.

Sintaxe:

```
Função HighestVolume(Interval: Integer; Offset: Integer; Type: Integer; Standard: Boolean; Auction: Boolean; Cross: Boolean): Float;
```

Parâmetros:

Interval: **Integer**; Intervalo da análise.

Offset: [Opcional] **Integer**; Deslocamento da barra. [Padrão = 1]

Type: [Opcional] **Integer**; Tipo de volume a considerar. [Padrão = 1]

Standard: [Opcional] **Boolean**; Considera volume padrão. [Padrão = True]

Auction: [Opcional] **Boolean**; Considera volume de leilão. [Padrão = True]

Cross: [Opcional] **Boolean**; Considera volume cruzado. [Padrão = True]

Exemplo de uso:

No exemplo a seguir, será retornado o maior volume financeiro dos últimos **60** minutos.

```
1 begin
2   HighestVolume(itMinute, 60, vtFinancial)|0|;
3 end;
```



Exemplo Gráfico:



19.62 Função HighestVolumePrice

Disponível nas versões Ultra ou com a Automação de Estratégias 2.

A função `HighestVolumePrice()` retorna o preço associado ao maior volume, de acordo com o tipo especificado.

Os dados do Volume Profile não fornecem um histórico extenso, o que limita o backtest de estratégias que utilizam essa função.

O parâmetro `Interval` define o período de análise e pode assumir os valores `itDaily` (intervalo diário — com `Offset` limitado a 1 nesse caso) ou `itMinute` (intervalo em minutos). O `Offset` indica quantos períodos atrás o cálculo deve considerar, sendo `0` o período atual e até `720` para períodos anteriores.

O parâmetro `Type` determina o tipo de volume a ser analisado: `vtFinancial` (volume financeiro), `vtQuantity` (quantidade de papéis) ou `vtTrade` (número de negócios). Os parâmetros `Standard`, `Auction` e `Cross` controlam a inclusão, respectivamente, de negócios padrão, leilões e operações cruzadas. Todos assumem `True` por padrão.

A função deve ser finalizada com o parâmetro `|Rank|`, onde `0` corresponde ao maior volume identificado na análise.

Sintaxe:

```
Função HighestVolumePrice(Interval: Integer; Offset: Integer; Type: Integer;  
Standard: Boolean; Auction: Boolean; Cross: Boolean): Float;
```

Parâmetros:

`Interval: Integer`; Intervalo da análise.

`Offset: [Opcional] Integer`; Deslocamento da barra. [Padrão = 1]

`Type: [Opcional] Integer`; Tipo de volume a considerar. [Padrão = 1]

`Standard: [Opcional] Boolean`; Considera volume padrão. [Padrão = True]

`Auction: [Opcional] Boolean`; Considera volume de leilão. [Padrão = True]

`Cross: [Opcional] Boolean`; Considera volume cruzado. [Padrão = True]

Exemplo de uso:

No exemplo a seguir, será retornado o preço do maior volume em quantidade nos últimos `5` minutos.

```
1 begin  
2   HighestVolumePrice(itMinute, 5, vtQuantity)|0|;  
3 end;
```



Exemplo Gráfico:





19.63 Função HiLoActivator

Disponível em todas versões.

A função **HiLoActivator**() retorna o valor do indicador "HiLo Activator", de acordo com o período desejado. Este indicador é um seguidor de tendência muito utilizado para sinalizar saída de trades. O sinal ocorre quando o preço fecha abaixo do indicador.

Sintaxe:

```
Função HiLoActivator(Período : Integer): Float;
```

Parâmetros:

Período: **Integer**; Período utilizado no momento do cálculo do indicador.

Linha: **Integer**; Define qual informação será retornada pelo indicador:

- |0| Valor do indicador (linha principal);
- |1| Tendência atual.

Retorno para identificação da tendência (quando **Linha** = 1):

- 0: Tendência de baixa;
- 1: Tendência de alta.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "HiLo Activator", considerando 3 períodos para o cálculo.

```
1 begin
2   Plot(HiLoActivator(3)|0|);
3 end
```

Exemplo Gráfico:





19.64 Função HistVolatility

Disponível em todas versões.

A função **HistVolatility()** retorna o valor do indicador "Volatilidade Histórica", de acordo com o período e tipo de média desejados. O Historical Volatility Index é baseado no livro de Don Fishback, "Odds: The Key to 90% Winners". Esta fórmula irá resultar em um índice de volatilidade histórica de 30 dias, plotando valores de 0 e 1.

Sintaxe:

```
Função HistVolatility(Media : Integer; Tipo : Integer): Float;
```

Parâmetros:

Media: Integer; Média utilizada no momento do cálculo do indicador.

Tipo: Integer; Determina qual média será considerada:

- **kaArithmetic:** Aritmética;
- **kaExponential:** Exponencial;
- **kaWelles:** Welles Wilder;
- **kaWeighted:** Ponderada;
- **kaADX:** ADX;

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Volatilidade Histórica", considerando **22** (Períodos) e tipo de média Welles Wilder (**kaWelles**).

```
1 begin
2   Plot(HistVolatility(22, kaWelles));
3 end
```


Exemplo Gráfico:





19.65 Função HullMovingAverage

Disponível em todas versões.

A função `HullMovingAverage()` retorna o valor do indicador "Hull Moving Average", de acordo com o período desejado. Busca identificar a tendência do mercado, possuindo uma suavização maior que a média móvel exponencial e acompanha com maior proximidade o gráfico de preços.

Sintaxe:

```
Função HullMovingAverage(Período : Integer): Float;
```

Parâmetros:

Período: Integer; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Hull Moving Average", considerando 8 períodos para o cálculo.

```
1 begin
2   Plot(HullMovingAverage(8));
3 end
```

Exemplo Gráfico:



19.66 Função IchimokuCloud

Disponível em todas versões.

A função `IchimokuCloud()` retorna o valor do indicador "Ichimoku Cloud", de acordo com os parâmetros desejados. Este foi desenvolvido para que o trader possa avaliar a tendência e a força do ativo e seus pontos de suporte e de resistência, se a necessidade de qualquer outro indicador técnico.

Sintaxe:

```
Função IchimokuCloud(Tenkan-Sen : Integer; Kijun-Sen : Integer; SenkouSpanB : Integer): Float;
```

Parâmetros:

Tenkan-Sen: Integer; Utilizado no momento do cálculo do indicador.

Kijun-Sen: Integer; Utilizado no momento do cálculo do indicador.

SenkouSpanB: Integer; Utilizado no momento do cálculo do indicador.

Linha: Integer; Determina qual linha será obtida:

- |0| Tenkan-Sen;
- |1| Kijun-Sen;
- |2| Chikou Span;
- |3| Senkou Span A;
- |4| Senkou Span B.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor da linha Senkou Span B (4) do indicador "Ichimoku Cloud", considerando 9 (Tenkan-Sen), 26 (Kijun-Sen) e 52 (Senkou Span B) para o cálculo.

```
1 begin
2   Plot(IchimokuCloud(9, 26, 52)|4|);
3 end
```

Exemplo Gráfico:





19.67 Função ImpliedVolatility

Disponível em todas versões.

A função `ImpliedVolatility()` retorna o valor do indicador “Volatilidade Implícita”, de acordo com o período desejado. Este indicador é utilizado para calcular a volatilidade implícita de derivativos (opções). A volatilidade implícita pode ser definida como a volatilidade que o mercado inclui para um ativo através dos preços das opções.

Sintaxe:

```
Função ImpliedVolatility(ModeloTeorico : Boolean; TipoOpcao : Boolean): Float;
```

Parâmetros:

ModeloTeorico: **Boolean**; Determina o modelo para o cálculo:

- True: Black & Scholes;
- False: Binomial.

TipoOpcao: **Boolean**; Determina o tipo da opção:

- True: Americana;
- False: Européia.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador “Volatilidade Implícita”, utilizando o modelo Black & Scholes (**True**).

```
1 begin
2   Plot(ImpliedVolatility(True, False));
3 end
```

Exemplo Gráfico:





19.68 Função KVO

Disponível em todas versões.

A função `KVO()` retorna o valor do indicador "KVO", de acordo com os períodos desejados. O indicador "Klinger Volume Oscillator (KVO)" é um oscilador baseado em preço e volume projetado para medir o fluxo financeiro tanto em longo prazo como em curto prazo.

Sintaxe:

```
Função KVO(MediaLonga : Integer; MediaCurta : Integer; Sinal : Integer): Float;
```

Parâmetros:

MediaLonga: Integer; Determina o período da Média Longa para formação do cálculo.

MediaCurta: Integer; Determina o período da Média Curta.

Sinal: Integer; Determina o sinal para a formação do cálculo.

Linha: Integer [Opcional]; Determina qual linha será obtida:

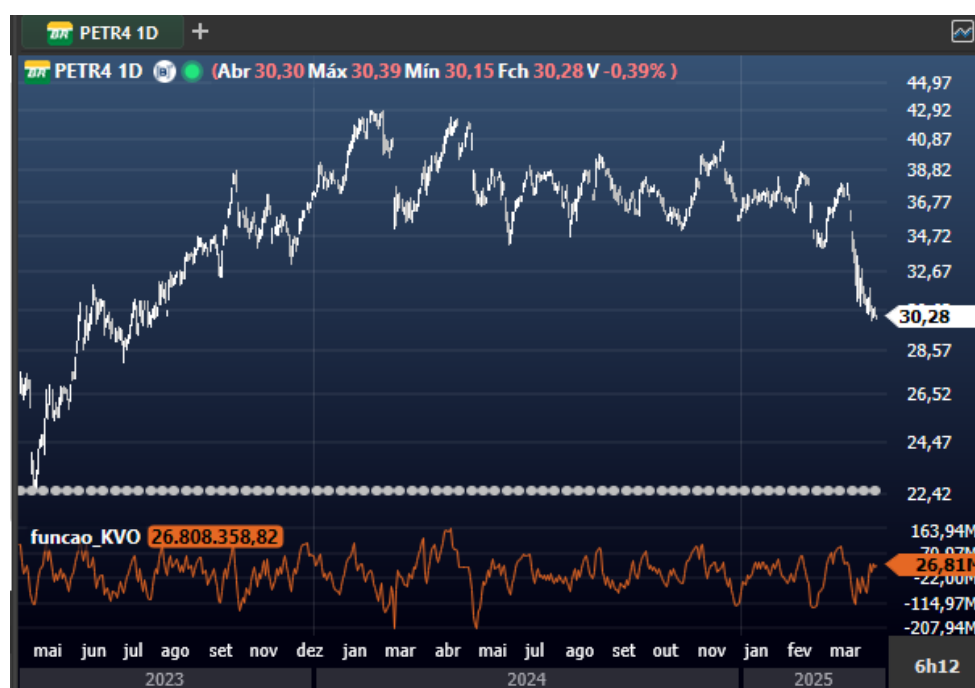
- |0| Linha.
- |1| Histograma.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do histograma do indicador "KVO", considerando 55 (Média Longa), 34 (Média Curta), 13 (Sinal) para o cálculo.

```
1 begin
2   Plot(KVO(55, 34, 13)|1|);
3 end
```

Exemplo Gráfico:





19.69 Função KeltnerCH

Disponível em todas versões.

A função `KeltnerCH()` retorna o valor do indicador "Keltner Channels", de acordo com o período e tipo de média desejados. É utilizado para definir zonas de atividade. O mercado está sobrecomprado quando estiver acima da banda superior, e sobrevendido quando estiver abaixo da banda inferior.

Sintaxe:

```
Função KeltnerCH(Desvio : Float; Período : Integer; Tipo : Integer): Float;
```

Parâmetros:

Desvio: `Float`; Desvio utilizado no momento do cálculo do indicador.

Período: `Integer`; Período utilizado para o cálculo do indicador.

Tipo: `Integer`; Determina qual média será considerada:

- `kaArithmetic`: Aritmética;
- `kaExponential`: Exponencial;
- `kaWelles`: Welles Wilder;
- `kaWeighted`: Ponderada;
- `kaADX`: ADX;

Linha: `Integer` [Opcional]; Determina qual linha será obtida:

- `|0|` Superior;
- `|1|` Inferior.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor da linha inferior do indicador "Keltner Channels", considerando `2.0` (Desvio), `20` (Períodos) e tipo de média exponencial (`1`).

```
1 begin
2   Plot(KeltnerCH(2.0, 20, kaExponential)|1|);
3 end
```



Exemplo Gráfico:





19.70 Função LSVolatilityIndex

Disponível em todas versões.

A função **LSVolatilityIndex()** retorna o valor do indicador "L&S Volatility Index". Indica o quão afastado o preço está da mma21 calculada como uma proporção da média da volatilidade histórica calculada sobre os últimos 21 candles. Esse indicador é utilizado como ferramenta em uma série de abordagens de trade.

Sintaxe:

```
Função LSVolatilityIndex(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "L&S Volatility Index".

```
1 begin
2   Plot(LSVolatilityIndex);
3 end
```

Exemplo Gráfico:



19.71 Função LinearRegressionChannel

Disponível em todas versões.

A função `LinearRegressionChannel()` retorna o valor do indicador "Canal de Regressão Linear". O canal de regressão linear é um indicador técnico de três linhas, que descreve a alta, a baixa e meio de uma tendência ou movimento de preço que está sendo analisado usando filtro com desvio padrão na tentativa de encontrar suportes e resistências compatíveis com a tendência do tempo gráfico analisado.

Sintaxe:

```
Função LinearRegressionChannel(Período : Integer; Usar Desvio Superior : Boolean;  
    Desvio Superior : Float; Usar Desvio Inferior : Boolean; Desvio Inferior : Float  
): Float;
```

Parâmetros:

Período: `Integer`; Determina o período da Média Longa para formação do cálculo.

Usar Desvio Superior: `Boolean`; Define se o desvio superior será utilizado.

Desvio Superior: `Float`; Desvio superior considerado.

Usar Desvio Inferior: `Boolean`; Define se o desvio inferior será utilizado.

Desvio Inferior: `Float`; Desvio inferior considerado.

Linha: `Integer` [Opcional]; Determina qual linha será obtida:

- `|0|` Central;
- `|1|` Superior;
- `|2|` Inferior.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a linha central.

```
1 begin  
2   Plot(LinearRegressionChannel(100, True, 2, True, 2));  
3 end
```

Exemplo Gráfico:



19.72 Função MACD

Disponível em todas versões.

A função `MACD()` retorna o valor do indicador "MACD Linha e Histograma" de acordo com os períodos desejados. Este indicador é uma combinação dos indicadores "MACD Histograma" e "MACD Linha".

Sintaxe:

```
Função MACD(MediaLonga : Integer; MediaCurta : Integer; Sinal : Integer): Float;
```

Parâmetros:

MediaLonga: Integer; Determina o período da Média Longa para formação do cálculo.

MediaCurta: Integer; Determina o período da Média Curta.

Sinal: Integer; Determina o sinal para a formação do cálculo.

Linha: Integer [Opcional]; Determina qual linha será obtida:

- |0| Linha;
- |1| Histograma.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do histograma, considerando 26 (Média Longa), 12 (Média Curta), 9 (Sinal).

```
1 begin
2   Plot(MACD(26, 12, 9)|1|);
3 end
```

Exemplo Gráfico:



19.73 Função MFI

Disponível em todas versões.

A função **MFI()** retorna o valor do indicador "Market Facilitation Index". É uma medida de eficiência com a qual o ativo se move para cada unidade de volume. Analisado junto com o volume ele permite classificar o mercado em 4 estágios.

Sintaxe:

```
Função MFI(): Float;
```

Parâmetros:

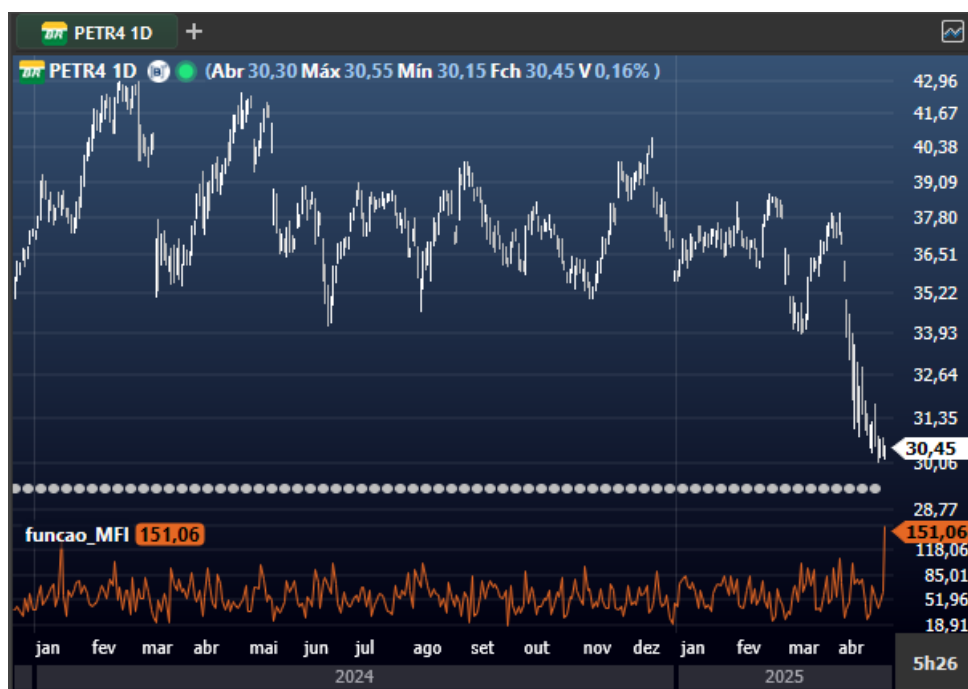
Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "MFI".

```
1 begin
2   Plot(MFI);
3 end
```

Exemplo Gráfico:



19.74 Função MIMA

Disponível nas versões com o pacote PhiCube.

A função `MIMA()` retorna o indicador cumulativo que aproxima uma média móvel convencional. Sua fórmula e calibragem proprietária o deixa mais rápido, porém com menos sinais falsos. MIMA é aplicável tanto na tendência quanto na consolidação.

Sintaxe:

```
Função MIMA(Período : Integer): Float;
```

Parâmetros:

Período: `Integer`; Determina o período a ser utilizado.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função `MIMA()` com intervalo `9`.

```
1 begin
2   Plot(MIMA(9));
3 end
```

Exemplo Gráfico:





19.75 Função MIMAROC

Disponível nas versões com o pacote PhiCube.

A função **MIMAROC()** a medição da inclinação (Rate of Change) do "PhiCube MIMA". Um valor acima de zero indica subida, enquanto o valor abaixo de zero indica queda.

Sintaxe:

```
Função MIMAROC(Período : Integer): Float;
```

Parâmetros:

Período: Integer; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função **MIMAROC()** com intervalo 9.

```
1 begin
2   Plot(MIMAROC(9));
3 end
```

Exemplo Gráfico:



19.76 Função MADRIDTRENDSCQUEEZE

Disponível em todas versões.

A função **MADRIDTRENDSCQUEEZE()** retorna o valor do indicador "Madrid Trend Squeeze". Busca medir graficamente a forla em 6 cores e tem objetivo de indicar força e direção até que a cor amarela seja visualizada, dando sinal de saída de trading.

Sintaxe:

Função **MADRIDTRENDSCQUEEZE**(Period : **Integer**): **Float**;

Parâmetros:

Period: **Integer**; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Madrid Trend Squeeze" requisitando um período de 34 unidades.

```
1 begin
2   Plot(MADRIDTRENDSCQUEEZE(34));
3 end
```

Exemplo Gráfico:





19.77 Função MAGOMACD

Disponível em todas versões.

A função **MAGOMACD()** é baseada nos períodos "MACD" em conjunto com "Média Móveis" e suas variações conforme o tempo gráfico.

Sintaxe:

```
Função MAGOMACD(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador.

```
1 begin  
2   Plot(MAGOMACD);  
3 end
```

Exemplo Gráfico:





19.78 Função MAGOMEDIAS

Disponível em todas versões.

A função **MAGOMEDIAS**() exibe médias móveis exponenciais baseadas nas médias utilizadas pelo trader Igor Rodrigues, utilizando de parâmetros do Ajuste e VWAP.

Sintaxe:

```
Função MAGOMEDIAS(PlotarMediasMago : Boolean; PlotarAjuste : Boolean; PlotarVWAP : Boolean): Float;
```

Parâmetros:

PlotarMediasMago: **Boolean**; Determina se as médias serão plotadas no gráfico:

- True: Verdadeiro;
- False: Falso.

PlotarAjuste: **Boolean**; Determina se as médias terão ajustes:

- True: Verdadeiro;
- False: Falso.

PlotarVWAP: **Boolean**; Determina se a VWAP será aplicada às médias:

- True: Verdadeiro;
- False: Falso.

Exemplo de uso:

```
1 begin
2   Plot(MAGOMEDIAS(True, True, True));
3 end
```





19.79 Função Momentum

Disponível em todas versões.

A função `Momentum()` retorna o valor do indicador "Momentum" de acordo com o período e tipo de média desejados. Este indicador é uma medida da velocidade de mudança dos preços. O momentum oscila em torno de uma linha zero e alterações na declividade da linha pode indicar topos ou fundos.

Sintaxe:

```
Função Momentum(Período : Integer; Média : Integer; Tipo : Integer): Float;
```

Parâmetros:

Período: Integer; Período utilizado no momento do cálculo do indicador.

Média: Integer; Média utilizada no momento do cálculo do indicador.

Tipo: Integer; Determina qual média será considerada:

- **kaArithmetic:** Aritmética;
- **kaExponential:** Exponencial;
- **kaWelles:** Welles Wilder;
- **kaWeighted:** Ponderada;
- **kaADX:** ADX;

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Momentum", considerando **10** (Períodos), **3** (Média) e tipo de média exponencial (**kaExponential**) para o cálculo.

```
1 begin
2   Plot(Momentum(10, 3, kaExponential));
3 end
```

Exemplo Gráfico:



19.80 Função MomentumStochastic

Disponível em todas versões.

A função `MomentumStochastic()` retorna o valor do indicador "Momento Estocástico" de acordo com o período desejado. O "Momento Estocástico" é baseado no "Oscilador Estocástico", onde relaciona o fechamento atual com o ponto médio da diferença entre a mínima e a máxima do período determinado.

Sintaxe:

```
Função MomentumStochastic(Período : Integer): Float;
```

Parâmetros:

Período: `Integer`; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Momento Estocástico", considerando **14** período para o cálculo.

```
1 begin
2   Plot(MomentumStochastic(14));
3 end
```

Exemplo Gráfico:



19.81 Função MoneyFlow

Disponível em todas versões.

A função `MoneyFlow()` retorna o valor do indicador "Money Flow". Este indicador momentum tiliza os valores máximos e mínimos atuais e anteriores para determinar a direção dos preços.

Sintaxe:

```
Função MoneyFlow(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Money Flow".

```
1 begin
2   Plot(MoneyFlow);
3 end
```

Exemplo Gráfico:



19.82 Função MoneyFlowIndex

Disponível em todas versões.

A função `MoneyFlowIndex()` retorna o valor do indicador "Money Flow Index", de acordo com o período desejado. O "Money Flow Index (MFI)" oscila entre 0-100, e é usado para determinar a força de uma tendência atual, à partir de análise de preço e volume de período anterior.

Sintaxe:

```
Função MoneyFlowIndex(Período : Integer): Float;
```

Parâmetros:

Período: Integer; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Money Flow Index", considerando **14** período para o cálculo.

```
1 begin
2   Plot(MoneyFlowIndex(14));
3 end
```

Exemplo Gráfico:



19.83 Função MADRIDMOVINGAVERAGE

Disponível em todas versões.

O indicador "Madrid Moving Average Ribbon" trabalha com 4 cores, onde verde limão indica tendência de alta, ótimo para fazer negociações longas; Verde escuro, perfeito para fazer reentradas, comprar na queda seguindo a tendência, ou um aviso de reversão de tendência de baixa; Vermelho claro indica tendência de baixa, ótimo pra Negociações curtas; Já a cor Marrom encontra indícios de venda no pico do movimento ou aviso de reversão de tendência de alta para baixa longa.

Sintaxe:

Função **MADRIDMOVINGAVERAGE()**: **Float**;

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Madrid Moving Average Ribbon".

```
1 begin
2   Plot(MADRIDMOVINGAVERAGE);
3 end
```

Exemplo Gráfico:



19.84 Função NelogicaBottomFinder

Disponível em todas versões.

A função `NelogicaBottomFinder()` retorna o valor do indicador "Nelogica Bottom Finder", conforme o dado desejado (linha e histograma). É um indicador de compra contra a tendência que utiliza uma fórmula proprietária para identificar momentos de teversão. Baseia-se na hipótese de "reversão à média" após um período de sobrevida amplificado por volatilidade.

Sintaxe:

```
Função NelogicaBottomFinder(): Float;
```

Parâmetros:

Linha: **Integer** [Opcional]; Determina qual dado será obtido:

- |0| Linha;
- |1| Histograma.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o dado do histograma.

```
1 begin
2   Plot(NelogicaBottomFinder|1|);
3 end
```

Exemplo Gráfico:



19.85 Função NelogicaPullBackFinder

Disponível em todas versões.

A função **NelogicaPullBackFinder()** retorna o valor do indicador “Nelogica Pullback Finder”, conforme o dado desejado (linha e histograma). É um indicador de compra e venda a favor da tendência, identificando os principais pontos de pullback da mesma. Baseia-se em uma fórmula proprietária de identificação de identificação de ciclos há tendência definida.

Sintaxe:

Função NelogicaPullBackFinder(): Float;

Parâmetros:

Linha: **Integer** [Opcional]; Determina qual dado será obtido:

- |0| Linha;
- |1| Histograma.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o dado do histograma.

```
1 begin
2   Plot(NelogicaPullBackFinder|1|);
3 end
```

Exemplo Gráfico:





19.86 Função NelogicaWeisWave

Disponível em todas versões.

A função `NelogicaWeisWave()` retorna o valor do indicador "Nelogica Weis Wave", de acordo com o período desejado. O indicador é baseado no método de ondas de Wyckoff, levando em consideração o volume das barras e sua direção.

Sintaxe:

```
Função NelogicaWeisWave(Período : Integer): Float;
```

Parâmetros:

Período: Integer; Determina o período para o cálculo do indicador:

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função, considerando 3 períodos para o cálculo.

```
1 begin
2   Plot(NelogicaWeisWave(3));
3 end
```

Exemplo Gráfico:



19.87 Função OBV

Disponível em todas versões.

A função **OBV()** retorna o valor do indicador "OBV", de acordo com o período desejado. O indicador relaciona preço e volume, além de divergências, pode-se aplicar métodos da análise técnica como suportes/resistências e linhas de tendência.

Sintaxe:

```
Função OBV(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "OBV".

```
1 begin
2   Plot(OBV);
3 end
```

Exemplo Gráfico:



19.88 Função OBVAvg

Disponível em todas versões.

A função **OBVAvg()** retorna o valor do indicador "OBV Ponderado". O indicador relaciona preço e volume, além de divergências, pode-se aplicar métodos da análise técnica como suportes/resistências e linhas de tendência.

Sintaxe:

```
Função OBVAvg(Período : Integer; Tipo : Integer): Float;
```

Parâmetros:

Período: Integer; Período utilizado no momento do cálculo do indicador:

Tipo: Integer; Determina qual média será considerada:

- **kaArithmetic:** Aritmética;
- **kaExponential:** Exponencial;
- **kaWelles:** Welles Wilder;
- **kaWeighted:** Ponderada;
- **kaADX:** ADX;

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função **OBVAvg()**, considerando **10** (Períodos) e tipo de média ponderada (**kaWeighted**).

```
1 begin
2   Plot(OBVAvg(10, kaWeighted));
3 end
```

Exemplo Gráfico:



19.89 Função OnBalanceTR

Disponível em todas versões.

A função `OnBalanceTR()` retorna o valor do indicador "On-Balance True Range". Oferece confirmações para sinais de divergência em outros indicadores.

Sintaxe:

```
Função OnBalanceTR(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "On-Balance True Range".

```
1 begin
2   Plot(OnBalanceTR);
3 end
```

Exemplo Gráfico:



19.90 Função OpenDaily

Disponível em todas versões.

A função `OpenDaily()` retorna o dado de abertura, conforme o deslocamento especificado.

Sintaxe:

```
Função OpenDaily(DaysBack : Integer): Float;
```

Parâmetros:

DaysBack: **Integer**; Número de dias para o deslocamento.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a abertura do dia anterior.

```
1 begin
2   Plot(OpenDaily(1));
3 end
```

Exemplo Gráfico:





19.91 Função OpenInterest

Disponível em todas versões.

A função `OpenInterest()` retorna o valor do indicador "Contratos em Aberto". O indicador mostra o número de contratos em aberto dos ativos da BM&F.

Sintaxe:

```
Função OpenInterest(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o dado do indicador.

```
1 begin
2   Plot(OpenInterest);
3 end
```

Exemplo Gráfico:



19.92 Função PTAX

Disponível nos produtos Pro ou Ultra.

A função **PTAX()** retorna o valor do indicador "TR - PTAX". PTAX é uma taxa de câmbio calculada durante o dia pelo Banco Central do Brasil. Consiste na média das taxas informadas pelos dealers de dolar durante 4 janelas do dia.

Sintaxe:

```
Função PTAX(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o indicador "PTAX".

```
1 begin
2   Plot(PTAX);
3 end
```

Exemplo Gráfico:





19.93 Função PTAXFuturo

Disponível nos produtos Pro ou Ultra.

A função **PTAXFuturo()** retorna o valor do indicador "TR - PTAX Futuro". PTAX é uma taxa de câmbio calculada durante o dia pelo Banco Central do Brasil. Consiste na média das taxas informadas pelos dealers de dolar durante 4 janelas do dia.

Sintaxe:

```
Função PTAXFuturo(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o indicador "PTAXF".

```
1 begin
2   Plot(PTAXFuturo);
3 end
```

Exemplo Gráfico:



19.94 Função ParabolicSAR

Disponível em todas versões.

A função `ParabolicSAR()` retorna o valor do indicador "SAR Parabólico", de acordo com os parâmetros desejados. O "SAR parabólico" é usado principalmente para gerar sinais de saída. Quando durante uma alta, por exemplo, os preços caem para baixo do SAR, um sinal de venda é gerado.

Sintaxe:

```
Função ParabolicSAR(Fator : Float; Limite : Float): Float;
```

Parâmetros:

Fator: **Float**; Determina o Fator de Aceleração para formação do cálculo.

Limite: **Float**; Determina o Limite de Aceleração.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "SAR Parabólico", considerando **0,02** (Fator) e **0,2** (Limite) para o cálculo.

```
1 begin
2   Plot(ParabolicSAR(0.02, 0.2));
3 end
```

Exemplo Gráfico:





19.95 Função Phibo

Disponível nas versões com o pacote Harmonic Trading.

A função **Phibo()** retorna o valor do indicador "PhiCube - Phibo Line".

Sintaxe:

```
Função Phibo(Período : Integer): Float;
```

Parâmetros:

Período: **Integer**; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função **Phibo()**, considerando **17** (Períodos) para o cálculo.

```
1 begin  
2   Plot(Phibo(17));  
3 end
```

Exemplo Gráfico:





19.96 Função Pivot

Disponível em todas versões.

A função **Pivot()** retorna o valor do indicador "Pivot", de acordo com os parâmetros específicos desejados. Com base nos valores de abertura, máxima e fechamento de um dia determina possíveis suportes e resistências para o próximo dia.

Sintaxe:

```
Função Pivot(Normal : Boolean; TresLinhas : Boolean): Float;
```

Parâmetros:

Normal: **Boolean**; Determina o tipo de cálculo que será efetuado para o Pivot:

- True: (Máxima + Mínima + Fechamento) / 3;
- False: (Abertura + Máxima + Mínima + Fechamento) / 4.

TresLinhas: **Boolean**; Determina quantas linhas serão consideradas:

- True: Três Linhas;
- False: Duas Linhas.

Linha: **Integer**; Determina qual linha será obtida:

Para True em TresLinhas:

- |0| Pivot;
- |1| R3;
- |2| S3;
- |3| R2;
- |4| S2;
- |5| R1;
- |6| S1.

Para False em TresLinhas:

- |0| Pivot;
- |1| R2;
- |2| S2;
- |3| R1;
- |4| S1.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a linha **S3**, considerando para o "Pivot" o tipo de cálculo: (Máxima + Mínima + Fechamento) / 3.

```
1 begin
2   Plot(Pivot(True, True)|2|);
3 end
```

Exemplo Gráfico:





19.97 Função PowerMeter

Disponível em todas versões.

A função **PowerMeter**() retorna o valor do indicador "PowerMeter (Medidor de Pressão)", informando a quantidade do Agressor Comprador ou do Agressor Vendedor, dependendo dos parâmetros fornecidos. Onde side deve ser 0 para compra e 1 para venda. O padrão do indicador é retornar os dados do dia de hoje, mas, ao adicionar uma janela de minutos, ele retornará baseado nos últimos X minutos.

Sintaxe:

```
Função PowerMeter(Side : Integer; Minutes : Integer; InitialDate : Integer; EndDate : Integer): Float;
```

Parâmetros:

Side: Integer; Parâmetro do tipo inteiro, informa o lado de ordem que considera para devolver os valores, passando a constante **osBuy** para valores de Compra e a constante **osSell** para valores de Venda.

Minutes: Integer; Informa a janela de tempo, em minutos, que o indicador deve considerar, dos últimos X minutos. Quando não informado considera valores diários. Valor padrão é "0".

InitialDate: Integer [var]; Utilizado para informar períodos customizados de análise, data inicial da janela de análise, onde o padrão é "0".

EndDate: Integer [var]; Utilizado para informar períodos customizados de análise, data final da janela de análise, onde o padrão é "0".

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da chamada, considerando o lado de Compra para o valor diário sem utilizar um período customizado.

```
1 begin  
2   Plot(PowerMeter(osBuy));  
3 end
```


Exemplo Gráfico:



19.98 Função PRICENERY

Disponível em todas versões.

A função **PRICENERY()** retorna o valor do indicador "PriceNery". O indicador realiza a divisão do dia anterior em zonas proporcionais, projetando distâncias equivalentes acima e abaixo do dia anterior.

Sintaxe:

```
Função PriceNery(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da chamada.

```
1 begin
2   Plot(PriceNery);
3 end
```

Exemplo Gráfico:





19.99 Função PriceOsc

Disponível em todas versões.

A função `PriceOsc()` retorna o valor do indicador "Oscilador de Preços", de acordo com os períodos e tipos de médias desejados. O indicador pe de seguimento de tendência que reflete a diferença entre duas médias móveis exponenciais para períodos diferentes em um gráfico.

Sintaxe:

```
Função PriceOsc(Media1 : Integer; TipoMedia1 : Integer; Media2 : Integer;  
    TipoMedia2 : Integer): Float;
```

Parâmetros:

Media1: **Integer**; Período utilizado para a média 1.

TipoMedia1: **Integer**; Determina qual média será considerada:

- **kaArithmetic**: Aritmética;
- **kaExponential**: Exponencial;
- **kaWelles**: Welles Wilder;
- **kaWeighted**: Ponderada;
- **kaADX**: ADX;

Media2: **Integer**; Período utilizado para a média 2.

TipoMedia2: **Integer**; Determina qual média será considerada:

- **kaArithmetic**: Aritmética;
- **kaExponential**: Exponencial;
- **kaWelles**: Welles Wilder;
- **kaWeighted**: Ponderada;
- **kaADX**: ADX;

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Oscilador de Preços", considerando **12** (Média 1) e **21** (Média 2) períodos, e tipo exponencial (**1**) para o cálculo.

```
1 begin  
2   Plot(PriceOsc(12, kaExponential, 21, kaExponential));  
3 end
```

Exemplo Gráfico:



19.100 Função PriceVolumeTrend

Disponível em todas versões.

A função **PriceVolumeTrend()** retorna o valor do indicador "Tendência Preço/Volume". É um indicador que procura relacionar o preço e o volume de uma determinada ação. O indicador Tendência de Preço/Volume pode ser utilizado para confirmar tendências, bem como identificar sinais de negociação.

Sintaxe:

```
Função PriceVolumeTrend(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o dado do indicador "Tendência Preço/Volume".

```
1 begin
2   Plot(PriceVolumeTrend);
3 end
```

Exemplo Gráfico:



19.101 Função PriorCote

Disponível em todas versões.

A função **PriorCote()** retorna o valor do indicador "Prior Cote", de acordo com o dado desejado. O indicador "Prior Cote" mostra os valores de Fechamento, Abertura, Máximo ou Mínimo correspondentes ao período anterior. Este período de referência é definido pela periodicidade do gráfico e pode ser dia, mês ou ano.

Sintaxe:

```
Função PriorCote(Tipo : Integer): Float;
```

Parâmetros:

Tipo: **Integer**; Determina o dado que será obtido:

- **0**: Fechamento;
- **1**: Abertura;
- **2**: Máxima;
- **3**: Mínima;
- **4**: Ajuste.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor de fechamento do período anterior.

```
1 begin
2   Plot(PriorCote(0));
3 end
```

Exemplo Gráfico:



19.102 Função QuantityVol

Disponível em todas versões.

A função `QuantityVol()` retorna o valor do indicador "Volume Quantidade". Este indicador mostra o volume em quantidade de títulos negociados, atuando de forma similar ao volume financeiro, podendo-se incluir ou desconsiderar os dados: "volume projetado" e "leilão e trades diretos". Nesta função o parâmetro `SoAgressores` só será considerado para o cálculo do indicador caso o seja assinado o Opcional "Plugin Tape Reading", caso não possua o opcional o indicador será sempre calculado como False, independente do que for inserido no campo.

Sintaxe:

```
Função QuantityVol(VolumeProjetado : Boolean; SoAgressores : Boolean): Float;
```

Parâmetros:

VolumeProjetado: **Boolean**; Determina se o volume irá considerar o dado projetado.

SoAgressores: **Boolean** [Opcional]; Determina se o volume irá desconsiderar o leilão e trades diretos.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Volume Quantidade", desconsiderando os dados de "volume projetado" e "leilão e trades diretos".

```
1 begin
2   Plot(QuantityVol(False, False));
3 end
```

Exemplo Gráfico:





19.103 Função RBG

Disponível em todas versões.

A função **RBG()** retorna o valor do indicador "RBG". O indicador é baseado no comportamento do ativo no pregão anterior, ele funciona identificando falsos rompimentos de máximas, e pullbacks no 50% do dia anterior, indicado para day trade.

Sintaxe:

```
Função RBG(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da chamada de função.

```
1 begin
2   Plot(RBG);
3 end
```

Exemplo Gráfico:





19.104 Função ROC

Disponível em todas versões.

A função `ROC()` retorna o valor do indicador "ROC", de acordo com os períodos e tipo de média desejados. No "ROC", o preço atual é comparado com um anterior para medir a magnitude da mudança do preço sobre um período de tempo.

Sintaxe:

```
Função ROC(Período : Integer; Media : Integer; Tipo : Integer): Float;
```

Parâmetros:

Período: Integer; Período utilizado no momento do cálculo do indicador.

Media: Integer; Período da média utilizada.

Tipo: Integer; Determina qual média será considerada:

- **kaArithmetic:** Aritmética;
- **kaExponential:** Exponencial;
- **kaWelles:** Welles Wilder;
- **kaWeighted:** Ponderada;
- **kaADX:** ADX;

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "ROC", considerando **3** (Período), **9** (Média) e tipo de aritmética (**kaArithmetic**).

```
1 begin
2   Plot(ROC(3, 9, kaArithmetic));
3 end
```

Exemplo Gráfico:



19.105 Função RSI

Disponível em todas versões.

A função `RSI()` retorna o valor do indicador "IFR" (RSI), de acordo com o período e tipo desejados. Ajuda os investidores a avaliar a força atual de um ativo, sinalizando níveis de compra excessiva e venda excessiva. Ferramentas como linhas de suporte podem ser usadas sobre o indicador.

Sintaxe:

```
Função RSI(Período : Integer; Tipo : Integer): Float;
```

Parâmetros:

Período: Integer; Período utilizado no momento do cálculo do indicador.

Tipo: Integer; Determina qual média será considerada:

- 0: Clássico;
- 1: Simples.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "IFR" (RSI), considerando 2 períodos e tipo clássico (0) para o cálculo.

```
1 begin
2   Plot(RSI(2, 0));
3 end
```

Exemplo Gráfico:



19.106 Função Rafi

Disponível em todas versões.

A função **Rafi()** retorna o valor do indicador "Rafi". É um indicador de "market breadth" e tem como objetivo identificar a forla de determinado rompimento de suporte e/ou resistência e verificar se o rompimento poderá ser falso ou não.

Sintaxe:

```
Função Rafi(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o dado do indicador específico.

```
1 begin
2   Plot(Rafi);
3 end
```

Exemplo Gráfico:





19.107 Função TENDENCYTRACKER

Disponível em todas versões.

A função **TENDENCYTRACKER()** retorna o valor do indicador "Rastreador de Tendências". É um indicador com foco em rastreabilidade de tendência para mercados futuros.

Sintaxe:

```
Função TENDENCYTRACKER(Dias : Integer): Float;
```

Parâmetros:

Dias: Integer; Período utilizado.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Rastreador de Tendências", considerando **30** (dias).

```
1 begin
2   Plot(TENDENCYTRACKER(30));
3 end
```

Exemplo Gráfico:





19.108 Função Ravi

Disponível em todas versões.

A função `Ravi()` retorna o valor do indicador "Ravi", de acordo com os períodos desejados. É um indicador de momentum similar ao "ADX", mas que gera sinais mais rápidos.

Sintaxe:

```
Função Ravi(MediaLonga : Integer; MediaCurta : Integer): Float;
```

Parâmetros:

MediaLonga: **Integer**; Período utilizado na média longa.

MediaCurta: **Integer**; Período utilizado na média curta para o cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador Ravi, considerando **7** (Média Curta) e **65** (Média Longa) períodos para o cálculo.

```
1 begin
2   Plot(Ravi(65, 7));
3 end
```

Exemplo Gráfico:





19.109 Função RenkoVTwo

Disponível em todas versões.

A função `RenkoVTwo()` retorna o valor do indicador "RenkoV2", de acordo com os parâmetros desejados. Cria um envelope nas áreas de consolidação e destaca os candles a partir do momento que saem da área de consolidação.

Sintaxe:

```
Função RenkoVTwo(Período : Integer; Abertura : Float; Deslocamento : Integer;  
    UseCustomDate : Boolean; CustomDate : Integer; CustomTime : Integer; Linha :  
    Integer): Integer;
```

Parâmetros:

Período: Integer; Período utilizado no momento do cálculo do indicador.

Abertura: Float; Referente ao parâmetro "Abertura" do indicador.

Deslocamento: Integer; Relacionado ao parâmetro "Deslocamento" do indicador.

UseCustomDate: Boolean [var] [Opcional]; Determina se a data será utilizada ou não:

- True: Usar data personalizada;
- False: Não usar data personalizada.

CustomDate: Integer [var] [Opcional]; Determina a data utilizada.

CustomTime: Integer [var] [Opcional]; Determina o tempo utilizado.

Linha: Integer [Opcional]; Determina qual linha será obtida:

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor da linha RenkoV2- (1) do indicador RenkoV2, considerando 20 (Período), 1.5 (Abertura) e 0 (Deslocamento) para o cálculo.

```
1 begin  
2   Plot(RenkoVTwo(20, 1.5, 0)|1|);  
3 end
```

Exemplo Gráfico:



19.110 Função RsiStochastic

Disponível em todas versões.

A função **RsiStochastic()** retorna o valor do indicador "IFR Estocástico", de acordo com o período desejado. Sendo um indicador derivado de outro indicador, a fórmula do estocástico é aplicada ao valor calculado do "IFR", variando entre 0 e 1. Pode ser utilizado com o IFR simples para medir sua sensibilidade.

Sintaxe:

```
Função RsiStochastic(Período : Integer): Float;
```

Parâmetros:

Período: Integer; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "IFR Estocástico", considerando 2 períodos para o cálculo.

```
1 begin
2   Plot(RsiStochastic(2));
3 end
```

Exemplo Gráfico:





19.111 Função SMA

Disponível em todas versões.

A função **SMA()** cria uma média móvel simples com base em um número definido de períodos. O indicador calcula a média dos dados entre a cotação atual e um período anterior, sendo amplamente utilizado na identificação de tendências.

Sintaxe:

```
Função SMA(Período : Integer; TipoValor : Integer; TipoMedia : Integer): Float;
```

Parâmetros:

Período: Integer; Quantidade de períodos utilizada no cálculo da média.

TipoValor: Integer[Opcional]; Define qual valor do candle será considerado no cálculo:

- **tvClose:** Fechamento (Close);
- **tvOpen:** Abertura (Open);
- **tvMax:** Máxima (High);
- **tvMin:** Mínima (Low).

TipoMedia: Integer[Opcional]; Define o tipo de média móvel utilizada:

- **kaArithmetic:** Aritmética;
- **kaExponential:** Exponencial;
- **kaWelles:** Welles Wilder;
- **kaWeighted:** Ponderada;
- **kaADX:** ADX;

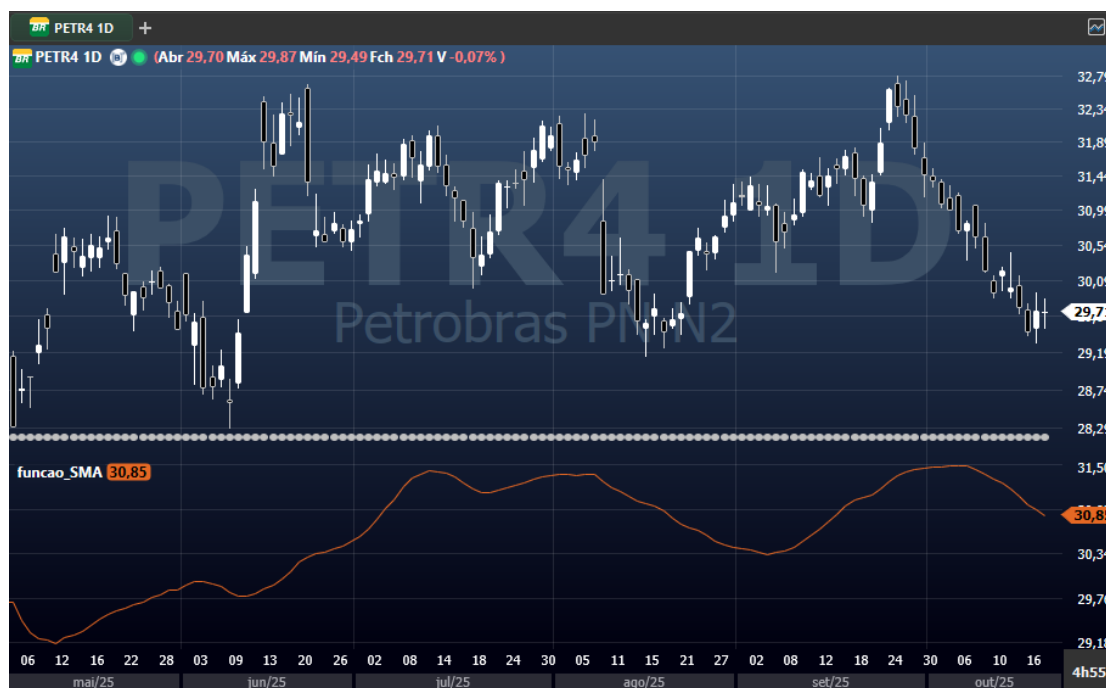
Exemplo de uso:

No exemplo abaixo, é plotada uma média móvel simples de 20 períodos, considerando o valor de fechamento e o tipo de média aritmética.

```
1 begin
2   Plot(SMA(20, tvClose, kaArithmetic));
3 end
```



Exemplo Gráfico:





19.112 Função SafeZoneDownTrend

Disponível em todas versões.

A função `SafeZoneDownTrend()` retorna o valor do indicador "Stop SafeZone DownTrend", de acordo com os parâmetros desejados. Visa indicar níveis adequados para posicionamento do stop durante uma tendência de baixa.

Sintaxe:

```
Função SafeZoneDownTrend(Multiplicador : Float; Período : Integer; Deslocamento : Integer): Float;
```

Parâmetros:

Multiplicador: Float; Valor de Multiplicador utilizado no cálculo do indicador.

Período: Integer; Período considerado.

Deslocamento: Integer; Deslocamento de períodos.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o dado do indicador "Stop SafeZone DownTrend", considerando **2,0** (Multiplicador), **10** (Período) e **0** (Deslocamento) para o cálculo.

```
1 begin
2   Plot(SafeZoneDownTrend(2.0, 10, 0));
3 end
```

Exemplo Gráfico:



19.113 Função SafeZoneUpTrend

Disponível em todas versões.

A função `SafeZoneUpTrend()` retorna o valor do indicador "Stop SafeZone UpTrend", de acordo com os parâmetros desejados. Visa indicar níveis adequados para posicionamento do stop durante uma tendência de alta.

Sintaxe:

```
Função SafeZoneUpTrend(Multiplicador : Float; Período : Integer; Deslocamento : Integer): Float;
```

Parâmetros:

Multiplicador: **Float**; Valor de Multiplicador utilizado no cálculo do indicador.

Período: **Integer**; Período considerado.

Deslocamento: **Integer**; Deslocamento de períodos.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o dado do indicador "Stop SafeZone UpTrend", considerando **2,0** (Multiplicador), **10** (Período) e **0** (Deslocamento) para o cálculo.

```
1 begin
2   Plot(SafeZoneUpTrend(2.0, 10, 0));
3 end
```

Exemplo Gráfico:



19.114 Função SampleAverage

Disponível em todas versões.

A função `SampleAverage()` retorna a média com base em dois períodos distintos.

Sintaxe:

```
Função SampleAverage(shortPeriod : Integer; longPeriod : Integer): Float;
```

Parâmetros:

`shortPeriod`: **Integer**; Usado para calcular uma média móvel de curto prazo.

`longPeriod`: **Integer**; Usado para calcular uma média móvel de longo prazo.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função `SampleAverage`, com **9** períodos de curto e longo prazo.

```
1 begin
2   Plot(SampleAverage(9, 9));
3 end
```

Exemplo Gráfico:



19.115 Função SampleBollinger

Disponível em todas versões.

A função `SampleBollinger()` retorna valor relacionado às "Bandas de Bollinger".

Sintaxe:

```
Função SampleBollinger(nPeriod : Integer; fStandardDeviation : Float): Float;
```

Parâmetros:

`nPeriod`: Integer; Define o número de períodos utilizados para calcular a média móvel.

`fStandardDeviation`: Float; Representa o multiplicador de desvio padrão.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função `SampleBollinger`, com 9 períodos e 9,5 como multiplicador do desvio padrão.

```
1 begin
2   Plot(SampleBollinger(9, 9.5));
3 end
```

Exemplo Gráfico:



19.116 Função SampleMACD

Disponível em todas versões.

A função `SampleMACD()` retorna valor relacionado às "MACD".

Sintaxe:

```
Função SampleMACD(shortPeriod : Integer; longPeriod : Integer; signalPeriod : Integer): Float;
```

Parâmetros:

shortPeriod: **Integer**; Define o número de períodos da média móvel exponencial curta.

longPeriod: **Integer**; Define o número de períodos da média móvel exponencial longa.

signalPeriod: **Integer**; Define o número de períodos da linha de sinal, que é uma média móvel do MACD.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função `SampleMACD`, com **12** períodos de média móvel exponencial curta, **26** períodos de média móvel exponencial longa e **9** períodos da linha de sinal.

```
1 begin
2   Plot(SampleMACD(12, 26, 9));
3 end
```

Exemplo Gráfico:



19.117 Função SampleRSI

Disponível em todas versões.

A função `SampleRSI()` retorna valor baseado no "RSI".

Sintaxe:

```
Função SampleRSI(dSellThreshold : Integer; dBuyThreshold : Integer): Float;
```

Parâmetros:

`dSellThreshold`: **Integer**; Representa o limite superior (nível de sobrecompra).

`dBuyThreshold`: **Integer**; Representa o limite inferior (nível de sobrevenda).

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função `SampleRSI`, com limite superior **25** e inferior **10**.

```
1 begin
2   Plot(SampleRSI(25, 10));
3 end
```

Exemplo Gráfico:



19.118 Função SampleStochastic

Disponível em todas versões.

A função `SampleStochastic()` retorna valores relacionados ao indicador estocástico.

Sintaxe:

```
Função SampleStochastic(nPeriod : Integer; nLimitBuy : Integer; nLimitSell : Integer): Float;
```

Parâmetros:

`nPeriod`: **Integer**; Número de períodos usados para calcular o Estocástico.

`nLimitBuy`: **Integer**; Limite inferior para indicar condição de compra.

`nLimitSell`: **Integer**; Limite superior para indicar condição de venda.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função `SampleStochastic()`, com **14** períodos, limite inferior de compra **20** e **80** de limite superior de venda.

```
1 begin
2   Plot(SampleStochastic(14, 20, 80));
3 end
```

Exemplo Gráfico:



19.119 Função SampleVWAP

Disponível em todas versões.

A função `SampleVWAP()` retorna o cálculo do "VWAP" (Volume Weighted Average Price).

Sintaxe:

```
Função SampleVWAP(fStandardDeviation : Float): Float;
```

Parâmetros:

`fStandardDeviation: Float`; Determina o número de desvios padrão a serem aplicados ao VWAP.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função `SampleVWAP()` com desvio padrão de `2`.

```
1 begin
2   Plot(SampleVWAP(2));
3 end
```

Exemplo Gráfico:





19.120 Função Santo

Disponível nas versões com o pacote PhiCube.

A função **Santo()** retorna o indicador cumulativo que mede direção, força e aceleração do preço. Sendo aplicável em tendência e consolidação, sua fórmula proprietária é uma combinação de vários indicadores convencionais.

Sintaxe:

```
Função Santo(Período : Integer): Float;
```

Parâmetros:

Período: Integer; Determina o período a ser utilizado.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função **Santo()** com intervalo 9.

```
1 begin
2   Plot(Santo(9));
3 end
```

Exemplo Gráfico:





```
EXPORTSYMBOL(groupsfree);  
/* export the groupinfo to  
objects.active = modifier  
if(modifier==0) it's modifier=0
```

19.121 Função SlowStochastic

Disponível em todas versões.

A função **SlowStochastic()** retorna o valor do indicador "Estocástico Lento", de acordo com o período desejado. A família de indicadores estocástico (lento, pleno e rápido) ajuda a identificar situações de compra e venda excessiva. Normalmente, acima de 80 e abaixo de 20 o mercado torna-se vulnerável a correções. Os parâmetros **PeriodoMedia** e **TipoMedia** são opcionais, caso não sejam determinados, serão utilizados os valores 14 e 1 respectivamente.

Sintaxe:

```
Função SlowStochastic(Período : Integer; PeríodoMedia : Integer; TipoMedia :  
Integer): Float;
```

Parâmetros:

Período: Integer; Período utilizado no momento do cálculo do indicador.

PeríodoMedia: Integer [var] [Opcional]; Período utilizado no momento do cálculo da média.

TipoMedia: Integer [var] [Opcional]; Tipo da média a ser calculada pelo indicador:

- **kaArithmetic:** Aritmética;
- **kaExponential:** Exponencial;
- **kaWelles:** Welles Wilder;
- **kaWeighted:** Ponderada;
- **kaADX:** ADX;

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Estocástico Lento", considerando **14** períodos para o cálculo.

```
1 begin  
2   Plot(SlowStochastic(14));  
3 end
```



Exemplo Gráfico:





19.122 Função SOMOSRENKOTRENDMPONTO

Disponível nas versões com o pacote SomosRenko.

A função **SOMOSRENKOTRENDMPONTO()** auxilia no envio de ordens. Tem como função sinalizar 0,5 ponto acima ou abaixo do fechamento do último box, indicando o ponto exato de onde o Trade deve colocar sua ordem de compra ou venda, caso entenda que o último fechamento é um ponto de entrada relevante.

Sintaxe:

Função **SOMOSRENKOTRENDMPONTO()**: **Float**;

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função **SOMOSRENKOTRENDMPONTO()**.

```
1 begin
2   Plot(SOMOSRENKOTRENDMPONTO());
3 end
```

Exemplo Gráfico:



19.123 Função StdDevConstPeriod

Disponível em todas versões.

A função `StdDevConstPeriod()` retorna o valor do indicador "StdDevConstPeriod". Calcula o desvio padrão de uma série de dados em um determinado período constante.

Sintaxe:

```
Função StdDevConstPeriod(Dados : Serie; Período : Integer): Float;
```

Parâmetros:

Dados: **Serie**; Série que será considerada para o cálculo.

Período: **Integer**; Período utilizado no momento do cálculo da média.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "StdDevConstPeriod", considerando **30** períodos, e a série de fechamento (Close) para o cálculo.

```
1 begin
2   Plot(StdDevConstPeriod(Close, 30));
3 end
```

Exemplo Gráfico:





19.124 Função StopATR

Disponível em todas versões.

A função **StopATR()** retorna o valor do indicador "Stop ATR", de acordo com os parâmetros desejados.

Sintaxe:

```
Função StopATR(Desvio : Float; Período : Integer; Tipo : Integer): Float;
```

Parâmetros:

Desvio: **Float**; Desvio utilizado para o cálculo do indicador.

Período: **Integer**; Período que será considerado.

Tipo: **Integer**; Determina qual média será considerada:

- **kaArithmetic**: Aritmética;
- **kaExponential**: Exponencial;
- **kaWelles**: Welles Wilder;
- **kaWeighted**: Ponderada;
- **kaADX**: ADX;

Dado: **Integer** [Opcional]; Determina o dado que será obtido:

- **|0|** Valor indicador;
- **|1|** Flag para informar a ocorrência (retorno): "0" para ATR+ e "1" para ATR.

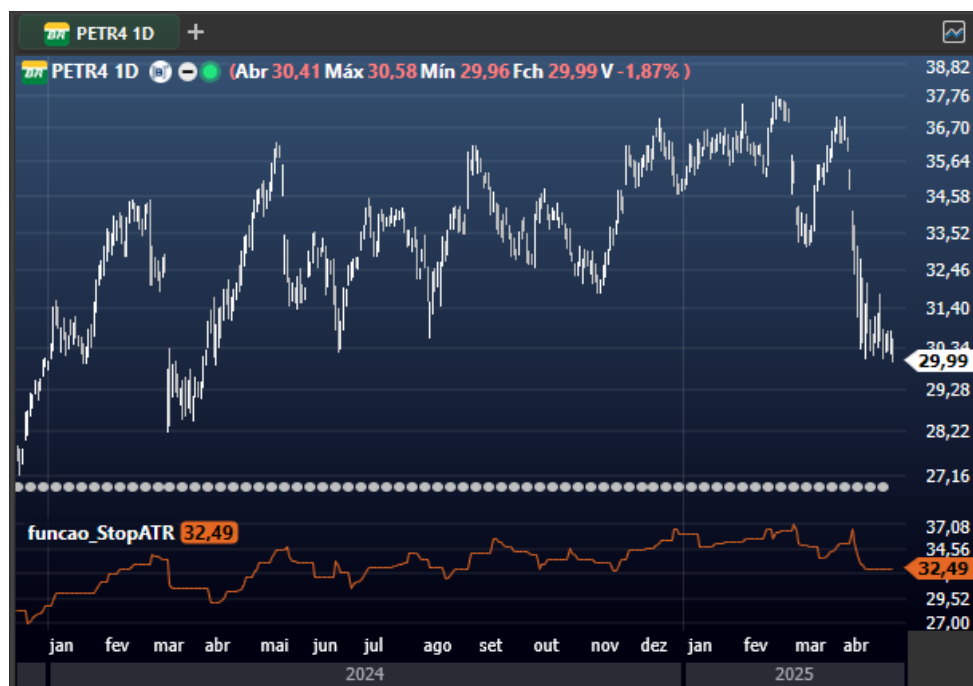
Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Stop ATR", considerando **2,0** (Desvio), **20** (Períodos) para o cálculo.

```
1 begin
2   Plot(StopATR(2.0, 20, kaArithmetic)|0|);
3 end
```



Exemplo Gráfico:





```
add_data(the_data)
EXPORTSYMBOL(groupsfree);
/* export the groupinfo to
objects.active = modifier_
(modifier_obj))*/#modifierexport(a
```

19.125 Função TOSCANABRUNELLO

Disponível nas versões com o pacote *Toscana*.

A função **TOSCANABRUNELLO()** trata-se da região de fibo compreendida entre 0% e 23% do movimento, na cor vermelha com linhas de demarcação da região e linha de demarcação de 10% do movimento. Esta região estará na parte de cima com mercado de venda e na parte de baixo com mercado de compra. Ela é quem define a direção da operação, e consequentemente faz com que os dados de negociação funcionem na direção correta do movimento. Usabilidade: Técnica desenvolvida por Felipe Vasconcellos, indica a região de retração de Fibonacci entre 0% e 23% do movimento do dia.

Sintaxe:

```
Função TOSCANABRUNELLO(Dias_Retroativos : Integer; Exibir_linhas : Boolean;
    Exibir_10_Porcento : Boolean; Estilo_da_Linha : Integer; Espessura_da_Linha :
    Integer; Filtro_de_Horario : Integer; Limite_de_Horario : Integer;
    Ativar_Reversão_Avançada : Boolean; Incluir_GAP_na_Abertura : Boolean): Float;
```

Parâmetros:

Dias_Retroativos: Integer; Número de dias passados considerados para o cálculo.

Exibir_linhas: Boolean; Mostra ou oculta as linhas da região de Fibo.

Exibir_10_Porcento: Boolean; Mostra ou oculta a linha de 10% do movimento.

Estilo_da_Linha: Integer; Define o estilo visual da linha.

Espessura_da_Linha: Integer; Define a espessura (largura) da linha.

Filtro_de_Horario: Integer; Define se há filtro por horário.

Limite_de_Horario: Integer; Define o horário-limite para considerar os dados.

Ativar_Reversão_Avançada: Boolean; Ativa lógica avançada de reversão de tendência.

Incluir_GAP_na_Abertura: Boolean; Considera ou não gaps na abertura do mercado.



19.126 Função TOSCANAPICCOLA

Disponível nas versões com o pacote Toscana.

A função **TOSCANAPICCOLA()** mostra regiões de operação que precedem as RTs, como um fibo menor traçado entre o Topo ou Fundo e a região das RTs. Estas regiões também se invertem a depender da direção do mercado, se compra ou venda. Usabilidade: Técnica desenvolvida por Felipe Vasconcellos, indica as micros regiões de retração de Fibonacci em relação à máxima e a mínima, com relação a retração de 61%.

Sintaxe:

```
Função TOSCANAPICCOLA(Dias_Retroativos : Integer; Exibir_linhas : Boolean;  
    Estilo_da_Linha : Integer; Espessura_da_Linha : Integer; Filtro_de_Horario :  
    Integer; Limite_de_Horario : Integer; Ativar_Reversão_Avançada : Boolean;  
    Incluir_GAP_na_Abertura : Boolean): Float;
```

Parâmetros:

Dias_Retroativos: **Integer**; Número de dias passados considerados para o cálculo.

Exibir_linhas: **Boolean**; Mostra ou oculta as linhas da região de Fibo.

Estilo_da_Linha: **Integer**; Define o estilo visual da linha.

Espessura_da_Linha: **Integer**; Define a espessura (largura) da linha.

Filtro_de_Horario: **Integer**; Define se há filtro por horário.

Limite_de_Horario: **Integer**; Define o horário-limite para considerar os dados.

Ativar_Reversão_Avançada: **Boolean**; Ativa lógica avançada de reversão de tendência.

Incluir_GAP_na_Abertura: **Boolean**; Considera ou não gaps na abertura do mercado.



19.127 Função TOSCANAPICCOLO

Disponível nas versões com o pacote Toscana.

A função **TOSCANAPICCOLO()** mostra regiões de operação que precedem as RTs, como um fibo menor traçado entre o Topo ou Fundo e a região das RTs. Estas regiões também se invertem a depender da direção do mercado, se compra ou venda. Usabilidade: Técnica desenvolvida por Felipe Vasconcellos, indica as micros regiões de retração de Fibonacci em relação à máxima e a mínima, com relação a retração de 61%.

Sintaxe:

```
Função TOSCANAPICCOLO(Dias_Retroativos : Integer; Exibir_linhas : Boolean;  
    Exibir_10_Porcento : Boolean; Estilo_da_Linha : Integer; Espessura_da_Linha :  
    Integer; Filtro_de_Horario : Integer; Limite_de_Horario : Integer;  
    Ativar_Reversão_Avançada : Boolean; Incluir_GAP_na_Abertura : Boolean): Float;
```

Parâmetros:

Dias_Retroativos: **Integer**; Número de dias passados considerados para o cálculo.

Exibir_linhas: **Boolean**; Mostra ou oculta as linhas da região de Fibo.

Exibir_10_Porcento: **Boolean**; Mostra ou oculta a linha de 10% do movimento.

Estilo_da_Linha: **Integer**; Define o estilo visual da linha.

Espessura_da_Linha: **Integer**; Define a espessura (largura) da linha.

Filtro_de_Horario: **Integer**; Define se há filtro por horário.

Limite_de_Horario: **Integer**; Define o horário-limite para considerar os dados.

Ativar_Reversão_Avançada: **Boolean**; Ativa lógica avançada de reversão de tendência.

Incluir_GAP_na_Abertura: **Boolean**; Considera ou não gaps na abertura do mercado.



19.128 Função TOSCANAPRODUTIVIDADE

Disponível nas versões com o pacote Toscana.

TOSCANAPRODUTIVIDADE() é um indicador com uma série de ferramentas para uso, com base no comportamento da toscana que podem ser ativados ou desativados pelo usuário que podem ser ativados ou desativados pelo usuário, sendo estes: A amplitude do dia mostrada em texto à frente do Gráfico. Linhas de demarcação da máxima e mínima do dia. Alvos referentes ao fivo medidos do topo ao fundo no momento do sinal de entrada. Alarmes que podem ser configurados por região onde o usuário deseja que o alarme seja exibido. Os alvos e Alarmes podem ser configurados com os mesmos filtros presentes na Regra de Coloração. Usabilidade: Técnica desenvolvida por Felipe Vasconcellos, indica no gráfico pontos relevantes para a operação, tais como: Amplitude do dia, Máxima e mínima do dia, alvos de Fibonacci e Alarmes.

Sintaxe:

```
Função TOSCANAPRODUTIVIDADE(Dias_Retroativos : Integer; Exibir_Amplitude : Boolean;  
    Exibir_Maxima_e_Minima : Boolean; Exibir_Alvos_de_Fibo : Boolean;  
    Espessura_Maxima_e_Minima : Integer; Espessura_Alvos : Integer;  
    Fonte_da_Amplitude : Integer; Exibir_Alvo_138 : Boolean; Exibir_Alvo_150 :  
    Boolean; Exibir_Alvo_161 : Boolean; Exibir_Alvo_200 : Boolean; Ativar_RT61 :  
    Boolean; Ativar_RT50 : Boolean; Ativar_RT38 : Boolean; Ativar_Brunello : Boolean;  
    Filtro_de_Horario : Integer; Filtro_de_Amplitude : Integer;  
    Filtro_de_Rompimento : Integer; Ativar_Reversão_Avançada : Boolean;  
    Incluir_GAP_na_Abertura : Boolean; Filtro_de_Horario_Dolar : Integer;  
    Limite_de_Horario_Dolar : Integer; Filtro_de_Amplitude_Dolar : Integer;  
    Filtro_de_Rompimento_Dolar : Integer; Filtro_de_Horario_Indice : Integer;  
    Limite_de_Horario_Indice : Integer; Filtro_de_Amplitude_Indice : Integer;  
    Filtro_de_Rompimento_Indice : Integer; Filtro_de_Horario_Ações : Integer;  
    Limite_de_Horario_Ações : Integer; Filtro_de_Amplitude_Ações : Integer;  
    Filtro_de_Rompimento_Ações : Integer): Float;
```

Parâmetros:

Dias_Retroativos: **Integer**; Número de dias passados considerados para o cálculo.

Exibir_Amplitude: **Boolean**; Define se a amplitude do dia (diferença entre máxima e mínima) será exibida como texto no gráfico.

Exibir_Maxima_e_Minima: **Boolean**; Define se as linhas de demarcação da máxima e mínima do dia serão exibidas.

Exibir_Alvos_de_Fibo: **Boolean**; Habilita ou desabilita a exibição dos alvos baseados em níveis de retração de Fibonacci no gráfico.

Espessura_Maxima_e_Minima: **Integer**; Define a espessura (largura) das linhas da máxima e mínima do dia.

Espessura_Alvos: **Integer**; Define a espessura (largura) das linhas dos alvos de Fibonacci.

Fonte_da_Amplitude: **Integer**; Define o tamanho ou estilo da fonte usada para exibir o texto da amplitude do dia.

Exibir_Alvo_138: **Boolean**; Controla a exibição do alvo baseado no nível de Fibonacci 138%.

Exibir_Alvo_150: **Boolean**; Controla a exibição do alvo baseado no nível de Fibonacci 150%.

Exibir_Alvo_161: **Boolean**; Controla a exibição do alvo baseado no nível de Fibonacci 161%.

Exibir_Alvo_200: **Boolean**; Controla a exibição do alvo baseado no nível de Fibonacci 200%.

Ativar_RT61: **Boolean**; Ativa a retração de Fibonacci de 61% para análise e exibição.



Ativar_RT50: **Boolean**; Ativa a retração de Fibonacci de 50% para análise e exibição.

Ativar_RT38: **Boolean**; Ativa a retração de Fibonacci de 38% para análise e exibição.

Ativar_Brunello: **Boolean**; Ativa a lógica ou método chamado "Brunello" (técnica ou filtro específico desenvolvido pelo autor).

Filtro_de_Horario: **Integer**; Define se e qual filtro de horário será aplicado para considerar os dados na análise.

Filtro_de_Amplitude: **Integer**; Define um filtro baseado na amplitude para incluir ou excluir dados da análise.

Filtro_de_Rompimento: **Integer**; Define um filtro para considerar rompimentos no preço, influenciando a análise.

Ativar_Reversão_Avançada: **Boolean**; Ativa uma lógica avançada para detectar reversões de tendência

Incluir_GAP_na_Abertura: **Boolean**; Determina se gaps (diferenças de preço na abertura) devem ser considerados na análise.

Filtro_de_Horario_Dolar: **Integer**; Filtro de horário específico para dados de mercado de dólar.

Limite_de_Horario_Dolar: **Integer**; Define o horário limite para considerar dados do mercado de dólar.

Filtro_de_Amplitude_Dolar: **Integer**; Filtro de amplitude específico para o mercado de dólar.

Filtro_de_Rompimento_Dolar: **Integer**; Filtro de rompimento específico para o mercado de dólar.

Filtro_de_Horario_Indice: **Integer**; Filtro de horário específico para dados de mercado de índice.

Limite_de_Horario_Indice: **Integer**; Define o horário limite para considerar dados do mercado de índice.

Filtro_de_Amplitude_Indice: **Integer**; Filtro de amplitude específico para o mercado de índice.

Filtro_de_Rompimento_Indice: **Integer**; Filtro de rompimento específico para o mercado de índice.

Filtro_de_Horario_Ações: **Integer**; Filtro de horário específico para dados de mercado de ações.

Limite_de_Horario_Ações: **Integer**; Define o horário limite para considerar dados do mercado de ações.

Filtro_de_Amplitude_Ações: **Integer**; Filtro de amplitude específico para o mercado de ações.

Filtro_de_Rompimento_Ações: **Integer**; Filtro de rompimento específico para o mercado de ações.



19.129 Função TOSCANART

Disponível nas versões com o pacote Toscana.

TOSCANART() São Regiões da Toscana, área verde mostra a região central do fibo, que se inicia em 38% e se estende até 61% do movimento, com linhas de demarcação externas, a linha central marca os 50% do movimento. Usabilidade: Técnica desenvolvida por Felipe Vasconcellos, indica as regiões de retração de Fibonacci em relação à máxima do mercado no dia, mostrando as regiões 61%, 50% e 38%.

Sintaxe:

```
Função TOSCANART(Dias_Retroativos : Integer; Exibir_Linhas : Boolean;  
    Estilo_da_Linha : Integer; Espessura_da_Linha : Integer; Filtro_de_Horario :  
    Integer; Limite_de_Horario : Integer; Ativar_Reversão_Avançada : Boolean;  
    Incluir_GAP_na_Abertura : Boolean): Float;
```

Parâmetros:

Dias_Retroativos: Integer; Número de dias passados considerados para o cálculo.

Exibir_Linhas: Boolean; Define se as linhas que demarcam as regiões de retração serão exibidas no gráfico.

Estilo_da_Linha: Integer; Determina o estilo visual das linhas no gráfico.

Espessura_da_Linha: Integer; Define a espessura (largura) das linhas que representam as retrações de Fibonacci no gráfico.

Filtro_de_Horario: Integer; Define se há filtro por horário.

Limite_de_Horario: Integer; Define o horário limite para considerar os dados no cálculo e na exibição das regiões.

Ativar_Reversão_Avançada: Boolean; Ativa lógica avançada de reversão de tendência.

Incluir_GAP_na_Abertura: Boolean; Considera ou não gaps na abertura do mercado.

19.130 Função TRIX

Disponível em todas versões.

A função `TRIX()` retorna o valor do indicador "TRIX", de acordo com o período e tipo de média desejados. Utilizado para identificar reversões, é um indicador momentum projetado para filtrar ciclos irrelevantes.

Sintaxe:

```
Função TRIX(Media : Integer; Tipo : Integer): Float;
```

Parâmetros:

Media: `Integer`; Período utilizado no momento do cálculo do indicador.

Tipo: `Integer`; Determina qual média será considerada:

- `kaArithmetic`: Aritmética;
- `kaExponential`: Exponencial;
- `kaWelles`: Welles Wilder;
- `kaWeighted`: Ponderada;
- `kaADX`: ADX;

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "TRIX", considerando **9** (Média) períodos e tipo exponencial para o cálculo.

```
1 begin  
2   Plot(TRIX(9, kaExponential));  
3 end
```

Exemplo Gráfico:



19.131 Função TRIXM

Disponível em todas versões.

A função **TRIXM()** retorna o valor do indicador "TRIXM", de acordo com o período e tipo de média desejados. Este indicador é a própria medida da taxa de variação do TRIX. Por representar o momentum possui curva mais agressiva.

Sintaxe:

```
Função TRIXM(Media : Integer; Tipo : Integer): Float;
```

Parâmetros:

Media: **Integer**; Período utilizado no momento do cálculo do indicador.

Tipo: **Integer**; Determina qual média será considerada:

- **kaArithmetic**: Aritmética;
- **kaExponential**: Exponencial;
- **kaWelles**: Welles Wilder;
- **kaWeighted**: Ponderada;
- **kaADX**: ADX;

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "TRIXM", considerando **9** (Média) períodos e tipo exponencial (**1**) para o cálculo.

```
1 begin  
2   Plot(TRIXM(9, kaExponential));  
3 end
```

Exemplo Gráfico:





19.132 Função TRPTAX

Disponível nos produtos Pro ou Ultra.

A função **TRPTAX()** retorna o valor do indicador "TR - PTAX". Este indicador é uma taxa de câmbio calculada durante o dia pelo Banco Central do Brasil. Consiste na média das taxas informadas pelos dealers de dolar durante 4 janelas do dia.

Sintaxe:

```
Função TRPTAX(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "TR - PTAX".

```
1 begin
2   Plot(TRPTAX);
3 end
```

Exemplo Gráfico:



19.133 Função TRPTAXFuturo

Disponível em todas versões.

A função **TRPTAXFuturo()** retorna o valor do indicador "TR - PTAX futuro". Este indicador é uma taxa de câmbio calculada durante o dia pelo Banco Central do Brasil. Consiste na média das taxas informadas pelos dealers de dolar durante 4 janelas do dia.

Sintaxe:

```
Função TRPTAXFuturo(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "TR - PTAX futuro".

```
1 begin
2   Plot(TRPTAXFuturo);
3 end
```

Exemplo Gráfico:



19.134 Função Tilson

Disponível em todas versões.

A função `Tilson()` retorna o valor do indicador "Tillson's T3 Moving Average", de acordo com os parâmetros desejados. O "T Moving Average" é um tipo de média móvel que se tornou um indicador popular na análise técnica. Conta com um baixo atraso em relação ao gráfico de preços, e uma linha mais suavizada.

Sintaxe:

```
Função Tilson(Fator : Float; Media : Integer): Float;
```

Parâmetros:

Fator: **Float**; Determina o Fator para formação do cálculo.

Media: **Integer**; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Tillson's T3 Moving Average", considerando **0,7** (Fator) e **3** (Media) para o cálculo.

```
1 begin
2   Plot(Tilson(0.7, 3));
3 end
```

Exemplo Gráfico:



19.135 Função TimeAgrBuySell

Disponível nos produtos Pro ou Ultra ou módulo Scalper.

A função `TimeAgrBuySell()` retorna o valor do indicador "TR - Tempo Agressão - Compra". É um indicador de Tape Reading desenvolvido pela Scalper Trader.

Sintaxe:

```
Função TimeAgrBuySell(AlertaVariacoes : Integer): Float;
```

Parâmetros:

AlertaVariacoes: **Integer**; Quantidade de variações que serão consideradas.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função, considerando **3** variações.

```
1 begin
2   Plot(TimeAgrBuySell(3));
3 end
```

Exemplo Gráfico:



19.136 Função TimeAgrSellBuy

Disponível nos produtos Pro ou Ultra ou módulo Scalper.

A função **TimeAgrSellBuy()** retorna o valor do indicador "TR - Tempo Agressão - Venda". É um indicador de Tape Reading desenvolvido pela Scalper Trader.

Sintaxe:

```
Função TimeAgrSellBuy(AlertaVariacoes : Integer): Float;
```

Parâmetros:

AlertaVariacoes: **Integer**; Quantidade de variações que serão consideradas.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função, considerando **3** variações.

```
1 begin
2   Plot(TimeAgrSellBuy(3));
3 end
```

Exemplo Gráfico:



19.137 Função TimeAgrTotal

Disponível nos produtos Pro ou Ultra ou módulo Scalper.

A função `TimeAgrTotal()` retorna o valor do indicador "TR - Tempo Agressão - Total". É um indicador de Tape Reading desenvolvido pela Scalper Trader.

Sintaxe:

```
Função TimeAgrTotal(AlertaVariacoes : Integer): Float;
```

Parâmetros:

AlertaVariacoes: **Integer**; Quantidade de variações que serão consideradas.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função, considerando **3** variações.

```
1 begin
2   Plot(TimeAgrTotal(3));
3 end
```

Exemplo Gráfico:



19.138 Função TopBottomDetector

Disponível em todas versões.

A função **TopBottomDetector()** retorna o valor do indicador “Detector de Topos e Fundos”, de acordo com o período desejado. Identifica valores extremos no gráfico de um ativo para facilitar a observação do movimento dos preços, filtrando as oscilações menos significativas.

Sintaxe:

```
Função TopBottomDetector(Período : Integer): Float;
```

Parâmetros:

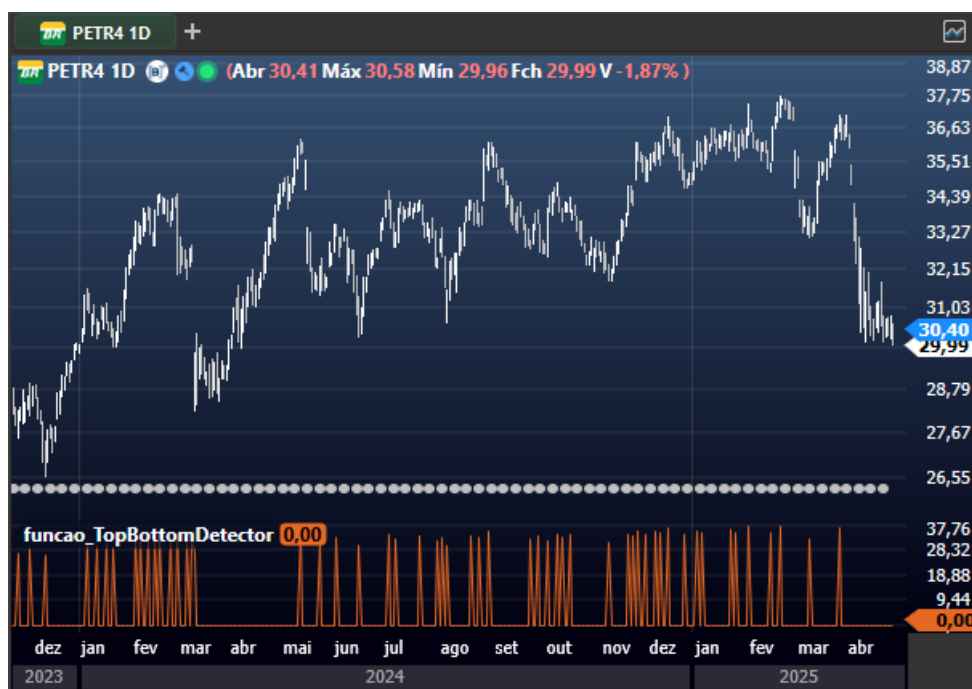
Período: Integer; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador “Detector de Topos e Fundos”, considerando **2** período para o cálculo.

```
1 begin
2   Plot(TopBottomDetector(2));
3 end
```

Exemplo Gráfico:





19.139 Função TREND AVERAGE

Disponível em todas versões.

A função **TREND AVERAGE**() determina a volatilidade de qualquer ativo de forma visual.

Sintaxe:

```
Função TREND AVERAGE(Dias_Retroativos : Integer; Cor_Compra_1Verde_2Azul : Integer;  
Cor_Venda_3Red_4Fucsia : Integer; Cor_Reversão_1Sim_2Não : Integer;  
CorRev_5Amarelo_6Branco : Integer): Float;
```

Parâmetros:

Dias_Retroativos: **Integer**; Período utilizado no momento do cálculo do indicador.

Cor_Compra_1Verde_2Azul: **Integer**; Determina a coloração de compra:

- **1**: Verde;
- **2**: Azul.

Cor_Venda_3Red_4Fucsia: **Integer**; Determina a coloração de venda:

- **3**: Vermelho;
- **4**: Fucsia.

Cor_Reversão_1Sim_2Não: **Integer**; Determina se a reversão será aplicada ou não:

- **1**: Sim;
- **2**: Não.

CorRev_5Amarelo_6Branco: **Integer**; Determina a coloração de reversão:

- **5**: Amarelo;
- **6**: Branco.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função **TREND AVERAGE**(), com **90** dias retroativos, cor verde (**1**) para compras, vermelho (**3**) para vendas, com reversão (**1**) e cor amarela (**5**).

```
1 begin  
2   Plot(TREND AVERAGE(90,1,3,1,5));  
3 end
```

Exemplo Gráfico:





19.140 Função TRENDCLOUD

Disponível em todas versões.

A função **TRENDCLOUD()** retorna o valor do indicador "Tradencia - TrendCloud". Indicador de Tendência de "nuvem" com base no cruzamento de duas médias tilson (T3) e regra de coloração específica. Utilizado pelo Tradencia Traders Clube.

Sintaxe:

```
Função TRENDCLOUD(Dias_Retroativos : Integer; Cor_Compra_1Verde_2Azul : Integer;  
Vwap_1Ligado_2Desligado : Integer; Cor_Venda_3Red_4Fucsia : Integer): Float;
```

Parâmetros:

Dias_Retroativos: **Integer**; Determina os dias retroativos.

Cor_Compra_1Verde_2Azul: **Integer**; Determina a coloração de compra:

- **1**: Verde;
- **2**: Azul.

Vwap_1Ligado_2Desligado: **Integer**; Determina a utilização ou não de VWAP:

- **1**: Ligado;
- **2**: Desligado.

Cor_Venda_3Red_4Fucsia: **Integer**; Determina a coloração de venda:

- **3**: Vermelho;
- **4**: Fucsia.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Tradencia - TrendCloud", considerando **90** dias retroativos sem VWAP.

```
1 begin  
2   Plot(TRENDCLOUD(90,2,2,3));  
3 end
```

Exemplo Gráfico:





19.141 Função TRENDIFUSOR

Disponível em todas versões.

A função **TRENDIFUSOR()** determina regiões de reversão de mercado para qualquer ativo, tempo e gráfico, bem como lateralidade.

Sintaxe:

```
Função TRENDIFUSOR(Ligar_Linhas_1Sim_2Nao : Integer; Valor : Integer;  
Cor_compra_1Verde_2Azul : Integer): Float;
```

Parâmetros:

Ligar_Linhas_1Sim_2Nao: Integer; Período utilizado no momento do cálculo do indicador.

- 1: Sim;
- 2: Não.

Valor: Integer; Determina valor para o calculo:

Cor_compra_1Verde_2Azul: Integer; Determina a coloração de venda:

- 1: Verde;
- 2: Azul.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função **TRENDIFUSOR()**, com ligamentos de linhas 1, valor (30), e cor verde (1) para compras.

```
1 begin  
2   Plot(TRENDIFUSOR(1,30,1));  
3 end
```

Exemplo Gráfico:





19.142 Função TRENDFLOW

Disponível em todas versões.

A função **TRENDFLOW()** registra o volume de agressão que está ocorrendo no mercado em forma de histograma e análise de reversão dos movimentos e sua continuidade.

Sintaxe:

```
Função TRENDFLOW(Dias_Retroativos : Integer; Strong_1Ligado_2Desligado : Integer;  
CorIndecisao_1Branco_2Preto : Integer; Cor_compra_1Verde_2Azul : Integer): Float  
;
```

Parâmetros:

Dias_Retroativos: **Integer**; Período utilizado no momento do cálculo do indicador.

Strong_1Ligado_2Desligado: **Integer**;

- **1**: Ligado;
- **2**: Desligado.

CorIndecisao_1Branco_2Preto: **Integer**; Determina a coloração de indecisão:

- **1**: Branco;
- **2**: Preto.

Cor_compra_1Verde_2Azul: **Integer**; Determina a coloração de compra:

- **1**: Verde;
- **2**: Azul.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função **TRENDFLOW()**, com **30** dias retroativos, strong ligado (**1**), cor de indecisão branca (**1**) e cor de compra verde (**1**).

```
1 begin  
2   Plot(TRENDFLOW(30,1,1,1));  
3 end
```


Exemplo Gráfico:





19.143 Função TRENDSNIPER

Disponível em todas versões.

A função **TRENDSNIPER()** retorna o valor do indicador "Tradencia - TrendSniper". Regra de coloração de Tendência baseado no cruzamento de duas médias com volatilidade. Utilizado pelo Tradencia Traders Clube.

Sintaxe:

```
Função TRENDSNIPER(Dias_Retroativos : Integer; Cor_Compra_1Verde_2Azul : Integer;  
  Vwap_1Ligado_2Desligado : Integer; Cor_Venda_3Red_4Fucsia : Integer;  
  Reversão_1Ligado_2Desligado : Integer): Float;
```

Parâmetros:

Dias_Retroativos: **Integer**; Determina os dias retroativos.

Cor_Compra_1Verde_2Azul: **Integer**; Determina a coloração de compra:

- **1**: Verde;
- **2**: Azul.

Vwap_1Ligado_2Desligado: **Integer**; Determina a utilização ou não de VWAP:

- **1**: Ligado;
- **2**: Desligado.

Cor_Venda_3Red_4Fucsia: **Integer**; Determina a coloração de venda:

- **3**: Vermelho;
- **4**: Fucsia.

Reversão_1Ligado_2Desligado: **Integer**; Determina se a reversão será aplicada ou não:

- **1**: Ligado;
- **2**: Desligado.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Tradencia - TrendCloud", considerando **30** dias retroativos sem VWAP.

```
1 begin  
2   Plot(TRENDSNIPER(30,2,2,3,1));  
3 end
```

Exemplo Gráfico:





19.144 Função TREND SOLDIER

Disponível em todas versões.

A função **TREND SOLDIER()** é uma regra de coloração definitiva com base nos 3 dados fornecidos pela B3: volume, preço e agressão.

Sintaxe:

```
Função TREND SOLDIER(Dias_Retroativos : Integer; Strong_1Ligado_2Desligado : Integer  
; CorIndecisao_1Branco_2Preto : Integer; Cor_compra_1Verde_2Azul : Integer):  
Float;
```

Parâmetros:

Dias_Retroativos: **Integer**; Período utilizado no momento do cálculo do indicador.

Strong_1Ligado_2Desligado: **Integer**;

- **1**: Ligado;
- **2**: Desligado.

CorIndecisao_1Branco_2Preto: **Integer**; Determina a coloração de indecisão:

- **1**: Branco;
- **2**: Preto.

Cor_compra_1Verde_2Azul: **Integer**; Determina a coloração de compra:

- **1**: Verde;
- **2**: Azul.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função **TREND SOLDIER()**, com **30** dias retroativos, strong ligado (**1**), cor de indecisão branca (**1**) e cor de compra verde (**1**).

```
1 begin  
2   Plot(TREND SOLDIER(30,1,1,1));  
3 end
```

Exemplo Gráfico:



19.145 Função TRADENCIATRENDVARIATION

Disponível em todas versões.

A função **TRADENCIATRENDVARIATION()** retorna o valor do indicador "Tradencia - TrendVariation". Estratégia de operações contra a tendência com base em variação percentual do ativo, que podem ser personalizadas pelo usuário escolhendo o valor do percentual e também de qual tipo de valor decorrerá a variação, podendo escolher entre fechamento, ajuste, abertura, máxima, mínima, vwap diária, semanal ou mensal.

Sintaxe:

```
Função TRADENCIATRENDVARIATION(Tipo : Integer; Dias_Retroativos : Integer; Pct1 : Float; Pct2 : Float; Pct3 : Float; Pct4 : Float; HoraInicio : Integer; HoraFechamento : Integer; Alvo : Integer; Stop : Integer): Float;
```

Parâmetros:

Tipo: **Integer**; Determina o tipo de variação.

- 0: Fechamento;
- 1: Ajuste;
- 2: Abertura;
- 3: Máxima;
- 4: Mínima;
- 5: VWAP diária;
- 6: VWAP semanal;
- 7: VWAP mensal.

Dias_Retroativos: **Integer**; Determina os dias retroativos.

Pct1: **Float**; Percentual definido pelo usuário para compor o limite das faixas de variação.

Pct2: **Float**; Percentual definido pelo usuário para compor o limite das faixas de variação.

Pct3: **Float**; Percentual definido pelo usuário para compor o limite das faixas de variação.

Pct4: **Float**; Percentual definido pelo usuário para compor o limite das faixas de variação.

HoraInicio: **Integer**; Determina o horário de inicio.

HoraFechamento: **Integer**; Determina o horário de fechamento.

Alvo: **Integer**; Determina o alvo.

Stop: **Integer**; Determina um stop.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Tradencia - TrendCloud", considerando 30 dias retroativos.

```
1 begin
2   Plot( TRADENCIATRENDVARIATION(2,30,1.5,1.5,1.5,1.5,0900,1730,10,10));
3 end
```

Exemplo Gráfico:



19.146 Função TRENDVOLAG

Disponível em todas versões.

A função **TRENDVOLAG()** é uma regra de coloração para análise de fluxo de candle a candle, registrando o volume visualmente.

Sintaxe:

Função **TRENDVOLAG**(voIM : **Integer**): **Float**;

Parâmetros:

voIM: **Integer**; Define o número de candles anteriores (período retroativo) considerados na análise de volume.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função **TRENDVOLAG()**.

```
1 begin
2   Plot(TRENDVOLAG(30));
3 end
```

Exemplo Gráfico:



19.147 Função Trades

Disponível em todas versões.

A função `Trades()` retorna o valor do indicador "Negócios". Mostra os negócios realizados durante as negociações. É um importante elemento na confirmação de movimentos.

Sintaxe:

```
Função Trades(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o dado do indicador "Negócios".

```
1 begin
2   Plot(Trades);
3 end
```

Exemplo Gráfico:



19.148 Função TwoMVAggression

Disponível nas versões com o pacote 2MV.

A função `TwoMVAggression()` retorna o indicador 2MV Agressão, sendo elaborado para apontar os pontos de rompimento e apoiar suas entradas de compra ou venda, fazendo uma leitura de ordens.

Sintaxe:

```
Função TwoMVAggression(): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função `TwoMVAggression()`.

```
1 begin
2   Plot(TwoMVAggression());
3 end
```

Exemplo Gráfico:



19.149 Função TwoMVPower

Disponível nas versões com o pacote 2MV.

A função `TwoMVPower()` retorna o indicador 2MV, sendo criado para identificar os melhores pontos de entradas para Scalp e Tendência em operações DayTrade. Pode ser aplicado para Swing Trade e Position em gráficos maiores.

Sintaxe:

```
Função TwoMVPower(Período1 : Integer; Período2 : Integer; Período3 : Integer; Media : Integer): Float;
```

Parâmetros:

Período1: Integer; Determina o primeiro período para o calculo da função.

Período2: Integer; Determina o segundo período para o calculo da função.

Período3: Integer; Determina o terceiro período para o calculo da função.

Media: Integer; Define o período para o cálculo da média utilizada na função.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função `TwoMVPower()`.

```
1 begin
2   Plot(TwoMVPower(9,9,9,9));
3 end
```

Exemplo Gráfico:



19.150 Função VSS

Disponível em todas versões.

A função **VSS()** retorna o valor do indicador "VSS", de acordo com os parâmetros desejados. Utiliza a volatilidade para definir bons pontos para a colocação de stop.

Sintaxe:

```
Função VSS(Multiplicador : Float; Media : Integer; Deslocamento : Integer): Float;
```

Parâmetros:

Multiplicador: **Float**; Valor de Multiplicador utilizado no cálculo do indicador.

Media: **Integer**; Período da média utilizada.

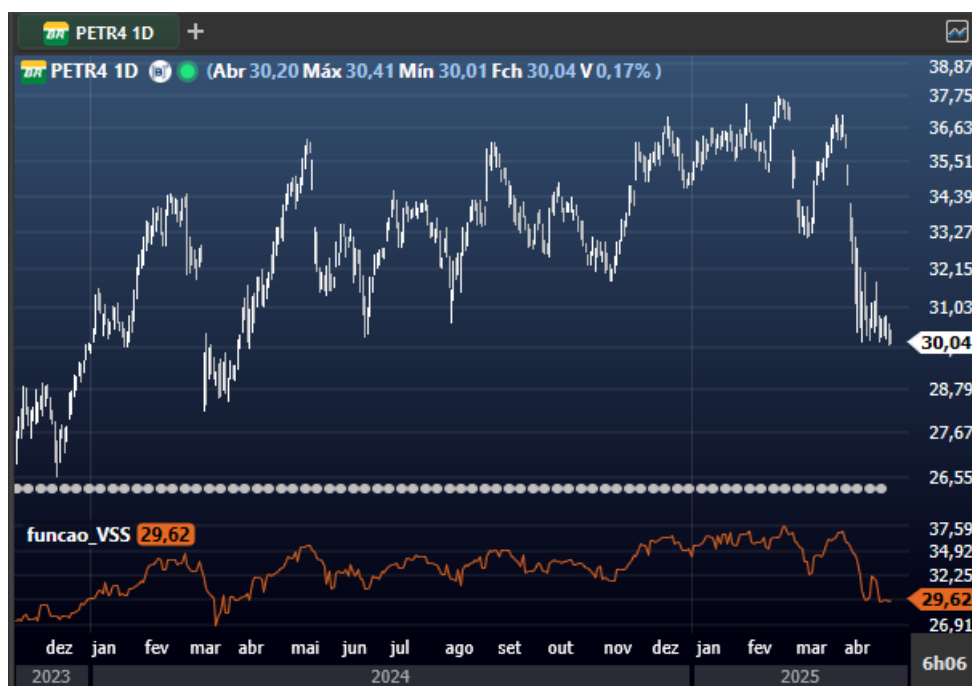
Deslocamento: **Integer**; Deslocamento de períodos.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "VSS", considerando **1,5** (Multiplicador), **5** (Média) períodos e **0** (Deslocamento) para o cálculo.

```
1 begin
2   Plot(VSS(1.5, 5, 0));
3 end
```

Exemplo Gráfico:



19.151 Função VWAP

Disponível em todas as versões, exceto Vector Basic e BA One.

A função `VWAP()` retorna o valor do indicador "VWAP", de acordo com a periodicidade desejada. O VWAP é o preço médio ponderado pelo volume de uma cotação. Pode ser utilizado para medir se o preço atual está sobrecomprado ou sobrevendido ao preço médio.

Sintaxe:

```
Função VWAP(Período : Integer): Float;
```

Parâmetros:

Período: **Integer**; Período para obter o dado do indicador:

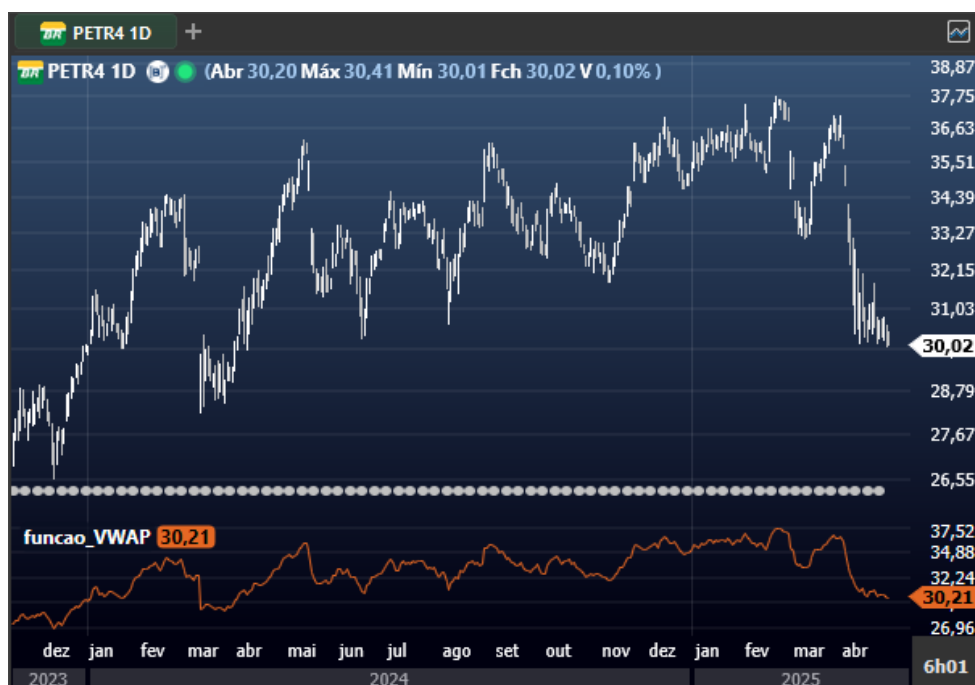
- **0**: Barra;
- **1**: Diário;
- **2**: Semanal;
- **3**: Mensal.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "VWAP", considerando a periodicidade diária.

```
1 begin
2   Plot(VWAP(1));
3 end
```

Exemplo Gráfico:



19.152 Função VWAPDate

Disponível em todas as versões, exceto Vector Basic e BA One.

A função **VWAPDate()** retorna o preço médio ponderado pelo volume, a partir de uma data e horário específicos. O VWAP Data é o preço médio ponderado pelo volume de uma cotação, começando a partir de uma data específica. Pode ser utilizado para medir se o preço atual está sobrecomprado ou sobrevendido em relação ao preço médio.

Sintaxe:

```
Função VWAPDate(Data : Integer; Time : Integer): Float;
```

Parâmetros:

Data: **Integer**; Data de referência, para início do cálculo. A Data precisa ser uma constante:

Time: **Integer**; Horário inicial.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da chamada, considerando a partir do dia 01/01/2025 (1250101) às 9h00.

```
1 begin
2   Plot(VWAPDate(1250502, 900));
3 end
```

Exemplo Gráfico:



19.153 Função VWAPMonthly

Disponível em todas as versões, exceto Vector Basic e BA One.

A função **VWAPMonthly()** retorna o valor do indicador “VWAP Mensal”. O VWAP é o preço médio ponderado pelo volume de uma cotação. Pode ser utilizado para medir se o preço atual está sobrecomprado ou sobrevendido ao preço médio.

Sintaxe:

Função **VWAPMonthly()**: **Float**;

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador “VWAPMonthly”.

```
1 begin
2   Plot(VWAPMonthly);
3 end
```

Exemplo Gráfico:



19.154 Função VWAPWeekly

Disponível em todas as versões, exceto Vector Basic e BA One.

A função **VWAPWeekly()** retorna o valor do indicador “VWAP Semanal”. O VWAP é o preço médio ponderado pelo volume de uma cotação. Pode ser utilizado para medir se o preço atual está sobrecomprado ou sobrevendido ao preço médio.

Sintaxe:

Função **VWAPWeekly()**: **Float**;

Parâmetros:

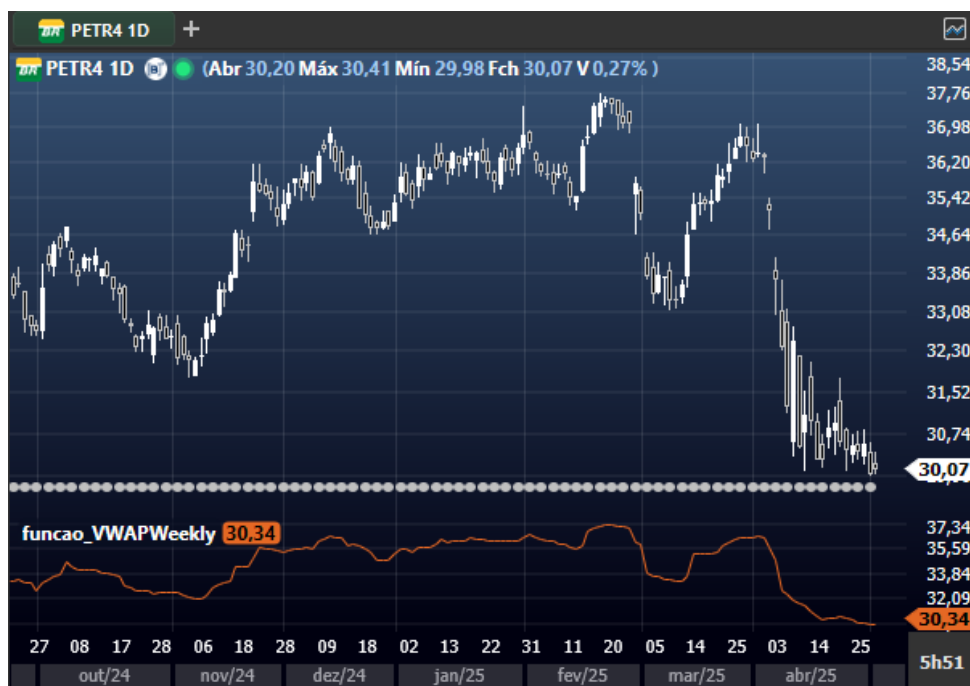
Sem parâmetros.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador “VWAPWeekly”.

```
1 begin
2   Plot(VWAPWeekly);
3 end
```

Exemplo Gráfico:



19.155 Função VWMA

Disponível em todas versões.

A função `VWMA()` retorna o valor do indicador "VWMA", de acordo com o período desejado. A média móvel ponderada por volume (VWMA) é uma variação da média móvel tradicional, onde aplica o volume de cada candle como multiplicador no cálculo ponderado da média.

Sintaxe:

```
Função VWMA(Período : Integer): Float;
```

Parâmetros:

Período: `Integer`; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "VWMA", considerando `10` períodos para o cálculo.

```
1 begin
2   Plot(VWMA(10));
3 end
```

Exemplo Gráfico:





19.156 Função Valerie

Disponível em todas versões.

A função **Valerie()** retorna o valor do indicador "Válerie", conforme os parâmetros desejados. Criado pela Válerie com objetivo de identificar as correções de mercado com base em cálculo estatístico. Observar os pontos como regiões de inversão de fluxo ou realização de lucro.

Sintaxe:

```
Função Valerie(Período : Integer; Offset : Integer): Float;
```

Parâmetros:

Período: Integer; Período utilizado no cálculo do indicador.

Offset: Integer; Offset considerado.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da chamada, considerando 45 períodos, e 5 para o Offset.

```
1 begin
2   Plot(Valerie(45, 5));
3 end
```

Exemplo Gráfico:



19.157 Função VolumeAgent

Disponível nas versões Ultra ou com a Automação de Estratégias 2.

A função `VolumeAgent()` retorna o volume financeiro filtrado pelo agente.

Sintaxe:

```
Função VolumeAgent(AgentID : Integer; Período : Integer): Float;
```

Parâmetros:

AgentID: Integer; ID do agente escolhido.

Período: Integer; Determina qual tipo de cálculo do volume será obtido:

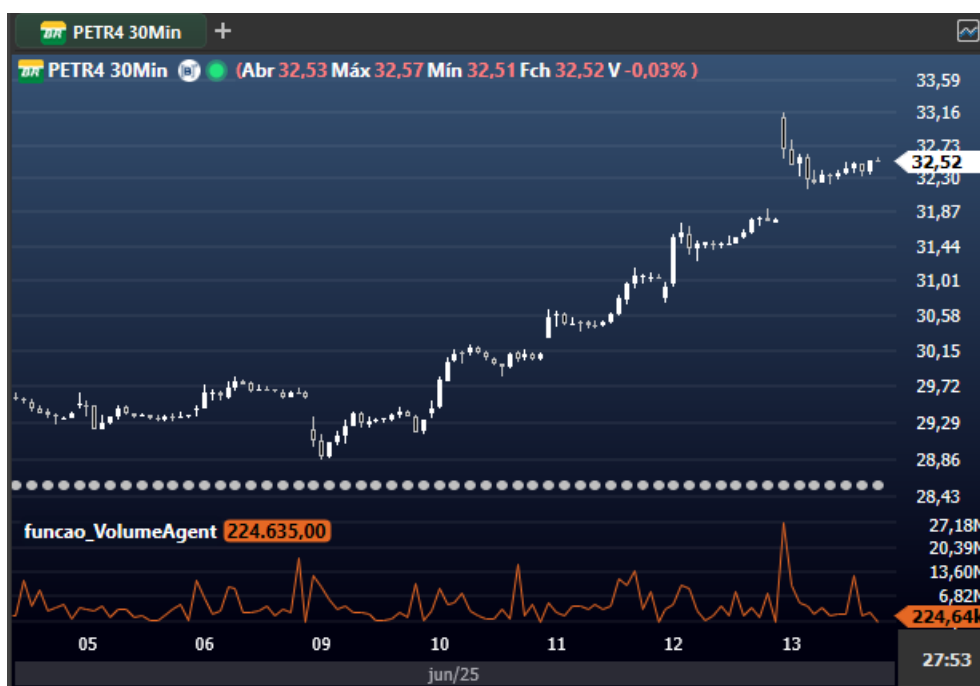
- 1: Retorna o volume diário, não importando o tempo gráfico onde está executando;
- 2: Retorna o volume candle a candle.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função `VolumeAgent()`.

```
1 begin
2   Plot(VolumeAgent(baItau,1));
3 end
```

Exemplo Gráfico:



19.158 Função VolumeAtPrice

Exclusivo da versão Profit Ultra e Automações Premium ou superior

A função **VolumeAtPrice()** retorna o volume negociado exatamente no preço especificado, considerando o intervalo e tipo definidos.

Os dados do Volume Profile não fornecem um histórico extenso, o que limita o backtest de estratégias que utilizam essa função.

O parâmetro **Interval** define o período de análise e pode assumir os valores **itDaily** (intervalo diário) ou **itMinute** (intervalo em minutos). O **Offset** indica quantos períodos atrás o cálculo deve considerar, sendo **0** o período atual e até **720** para períodos anteriores.

O parâmetro **Type** determina o tipo de volume a ser analisado: **vtFinancial** (volume financeiro), **vtQuantity** (quantidade de papéis) ou **vtTrade** (número de negócios). Os parâmetros **Standard**, **Auction** e **Cross** controlam a inclusão, respectivamente, de negócios padrão, leilões e operações cruzadas. Todos assumem **True** por padrão.

A função deve ser finalizada com o preço desejado entre barras verticais, no formato **|Price|**, onde **Price** representa o valor exato no qual o volume será consultado.

Sintaxe:

```
Função VolumeAtPrice(Interval: Integer; Offset: Integer; Type: Integer; Standard: Boolean; Auction: Boolean; Cross: Boolean): Float;
```

Parâmetros:

Interval: Integer; Intervalo da análise.

Offset: Integer [Opcional]; Deslocamento da barra. [Padrão = 1]

Type: Integer [Opcional]; Tipo de volume a considerar. [Padrão = 1]

Standard: Boolean [Opcional]; Considera volume padrão. [Padrão = True]

Auction: Boolean [Opcional]; Considera volume de leilão. [Padrão = True]

Cross: Boolean [Opcional]; Considera volume cruzado. [Padrão = True]

Price: Float; Preço exato no qual se deseja consultar o volume.

Exemplo de uso:

No exemplo a seguir, será atribuído à variável **dVolumeAtPrice** o volume no preço **38,60** para os últimos **60** minutos.

```
1 var
2   dVolumeAtPrice: Float;
3 begin
4   dVolumeAtPrice := VolumeAtPrice(itMinute, 60, vtTrade, True, True, True)|38.60|;
5 end;
```



19.159 Função Williams

Disponível em todas versões.

A função `Williams()` retorna o valor do indicador "Williams %R", de acordo com o período desejado. É parecido com o estocástico, compara o fechamento com o valor mais alto de determinado período.

Sintaxe:

```
Função Williams(Período : Integer): Float;
```

Parâmetros:

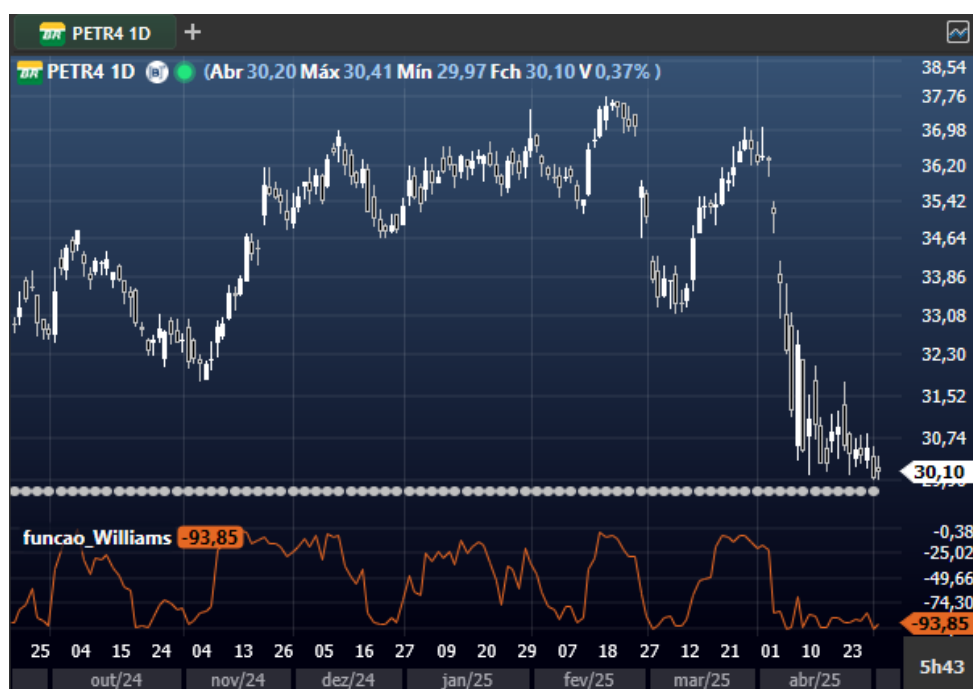
Período: **Integer**; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor do indicador "Williams %R", considerando **14** períodos para o cálculo.

```
1 begin
2   Plot(Williams(14));
3 end
```

Exemplo Gráfico:



20 Funções de Livro

20.1 Função AskPrice

Disponível em todas versões.

A função `AskPrice()` retorna o preço da melhor oferta de venda.

Sintaxe:

```
Função AskPrice(Asset: Ativo): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, caso não tenha posição aberta, será atribuído à variável `ask` o valor do topo (melhor oferta de venda) do livro.

```
1 var
2   ask : Float;
3 begin
4   if not HasPosition then
5     ask := AskPrice;
6   Plot(ask);
7 end;
```

Exemplo Gráfico:





20.2 Função AskSize

Disponível em todas versões.

A função `AskSize()` retorna a quantidade da melhor oferta de venda.

Ao utilizar o modo de `CrossOrder` na automação, o `AskSize` irá retornar a quantidade do `AskSize` da série histórica, e não no ativo de negociação.

Sintaxe:

```
Função AskSize(Asset: Ativo): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, caso não tenha posição aberta, será atribuído à variável `qtdask` a quantidade do topo (melhor oferta de venda) do livro.

```
1 var
2   qtdask : Float;
3 begin
4   if not HasPosition then
5     qtdask := AskPrice;
6   Plot(qtdask);
7 end;
```

Exemplo Gráfico:





20.3 Função BidPrice

Disponível em todas versões.

A função **BidPrice()** retorna o preço da melhor oferta de compra.

Sintaxe:

```
Função BidPrice(Asset: Ativo): Float;
```

Parâmetros:

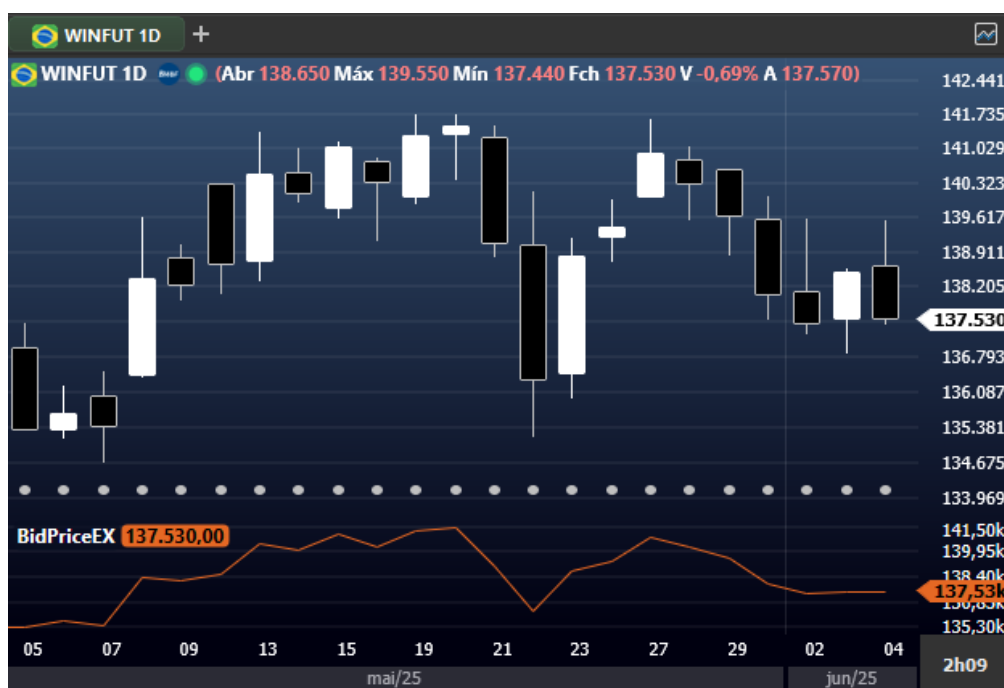
Sem parâmetros.

Exemplo de uso:

No exemplo, caso não tenha posição aberta, será atribuído à variável **bid** o valor de topo (melhor oferta de compra) do livro.

```
1 var
2   bid : Float;
3 begin
4   if not HasPosition then
5     bid := BidPrice;
6   Plot(bid);
7 end;
```

Exemplo Gráfico:





20.4 Função BidSize

Disponível em todas versões.

A função **BidSize()** retorna a quantidade da melhor oferta de compra.
Ao utilizar o modo de CrossOrder na automação, o BidSize irá retornar a quantidade do BidSize da série histórica, e não no ativo de negociação.

Sintaxe:

```
Função BidSize(Asset: Ativo): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, caso não tenha posição aberta será atribuído à variável **qtdbid**. A quantidade do topo (melhor oferta de compra) do livro.

```
1 var
2   qtdbid : Float;
3 begin
4   if not HasPosition then
5     bid := BidSize;
6   Plot(qtdbid);
7 end;
```

Exemplo Gráfico:



20.5 Função BookSpread

Disponível em todas versões.

A função `BookSpread()` retorna a diferença entre os melhores preços de compra e venda, no topo do livro.

Sintaxe:

```
Função BookSpread(Asset: Ativo): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, caso não tenha posição aberta, será atribuído à variável `spread` a diferença entre os valores das melhores ofertas de compra e venda.

```
1 var
2   spread : Float;
3 begin
4   if not HasPosition then
5     spread := BookSpread;
6   Plot(spread);
7 end;
```

Exemplo Gráfico:





20.6 Função BuyOfferCount

Disponível em todas versões.

A função `BuyOfferCount()` retorna as ofertas de compra do ativo

Sintaxe:

```
Função BuyOfferCountd(Asset: Ativo; Níveis considerados: Integer): Integer;
```

Parâmetros:

Asset: **Ativo** [Opcional]; Parâmetro Opcional declarado a partir do recurso Asset, valor padrão é "".

Nível considerado: **Integer** [Opcional]; Parâmetro Opcional Inteiro, valor padrão é 0.

Exemplo de uso:

No exemplo 1, caso não tenha posição aberta, será atribuído à variável `buyCount` as ofertas de compra do ativo.

No exemplo 2, caso não tenha posição aberta, será atribuído à variável `buyCount` as ofertas de compra do ativo VALE3.

Exemplo 1:

```
1 var
2   buyCount : Integer;
3 begin
4   if not HasPosition then
5     buyCount := BuyOfferCount;
6   Plot(buyCount);
7 end;
```



Exemplo Gráfico 1:



Exemplo 2:

```
1 const
2     VALE3 = Asset("VALE3", feedBovespa);
3 var
4     buyCount : Integer;
5 begin
6     if not HasPosition then
7         buyCount := BuyOfferCount(VALE3);
8     Plot(buyCount);
9 end;
```

Exemplo Gráfico 2:





20.7 Função GetAsset

Disponível em todas versões.

A função `GetAsset()` retorna o ticker do ativo, onde a chamada pode ser efetuada realizando a passagem do parâmetro opcional, para retornar o ticker de determinado Asset, ou declarar a função sem parametrização, onde será retornado o ticker do gráfico selecionado.

Para retornar o ticker (String) do Asset declarado, basta acessar de forma análoga às séries de dados: `A1GetAsset`.

Sintaxe:

```
Função GetAsset(Asset: Ativo): String;
```

Parâmetros:

Asset: **Ativo** [Opcional]; Parâmetro Opcional declarado a partir do recurso Asset, valor padrão é "".

Exemplo de uso:

No exemplo 1, para o último candle, será visualizado o nome do ativo selecionado no gráfico, com os preços de bid e ask.

No exemplo 2, a distinção está na especificação do Asset que será exibido (WDOFUT), independentemente do ativo selecionado no gráfico.

Exemplo 1:

```
1  var
2    bid : Float;
3    ask : Float;
4    get : String;
5  begin
6    if (MaxBarsForward = 0) then
7      begin
8        get := GetAsset;
9        bid := BidPrice;
10       ask := AskPrice;
11       plotText((get + ": " + "Bid =" + bid + " Ask=" + ask), clGreen, 2, 8);
12     end;
13 end;
```



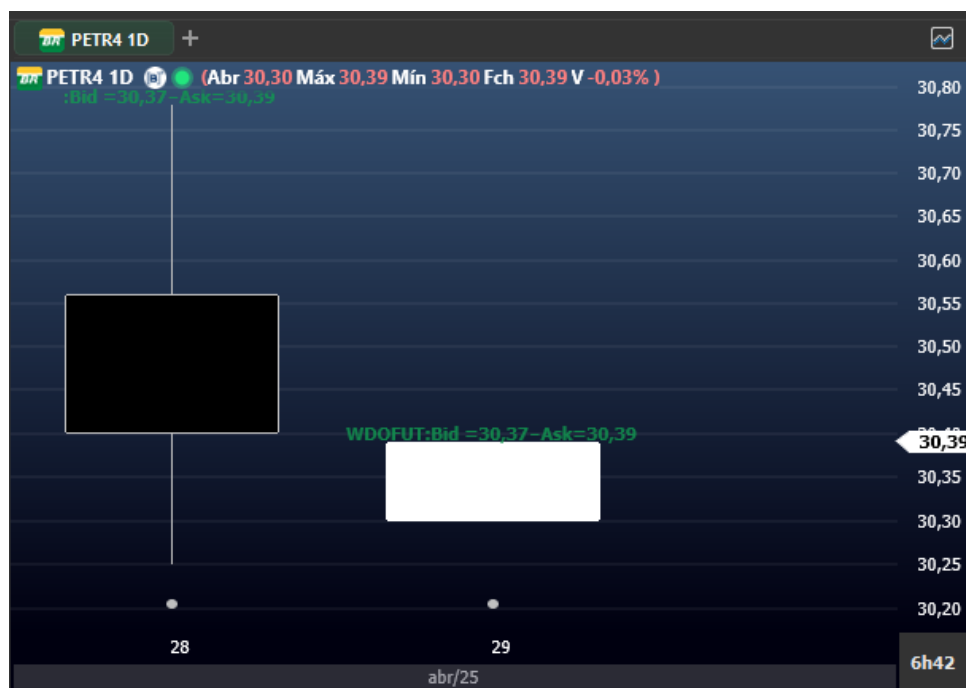
Exemplo Gráfico 1:



Exemplo 2:

```
1  const
2    WDO = Asset("WDOFUT", feedBMF);
3  var
4    bid : Float;
5    ask : Float;
6    get : String;
7  begin
8    if (MaxBarsForward = 0) then
9      begin
10         get := GetAsset(WDO);
11         bid := BidPrice;
12         ask := AskPrice;
13         plotText((get + ":" + " Bid =" + bid + " Ask=" + ask), c1Green, 2, 8);
14       end
15     end;
```

Exemplo Gráfico 2:





20.8 Função GetFeed

Disponível em todas versões.

A função `GetFeed()` retorna o código textual do Feed do ativo.

A parametrização é opcional, caso não seja determinado o Asset, será retornado o ticker do gráfico selecionado.

Para retornar a bolsa relacionada (String) do Asset específico, pode-se acessar de forma análoga às séries de dados: `A1.GetFeed`.

Sintaxe:

```
Função GetFeed(Asset: Ativo): String;
```

Parâmetros:

Asset: **Ativo** [Opcional]; Constante declarada a partir do recurso Asset.

Exemplo de uso:

No exemplo 1, para o último candle, será visualizado o nome do ativo selecionado no gráfico, a bolsa a qual pertence, e o spread do book.

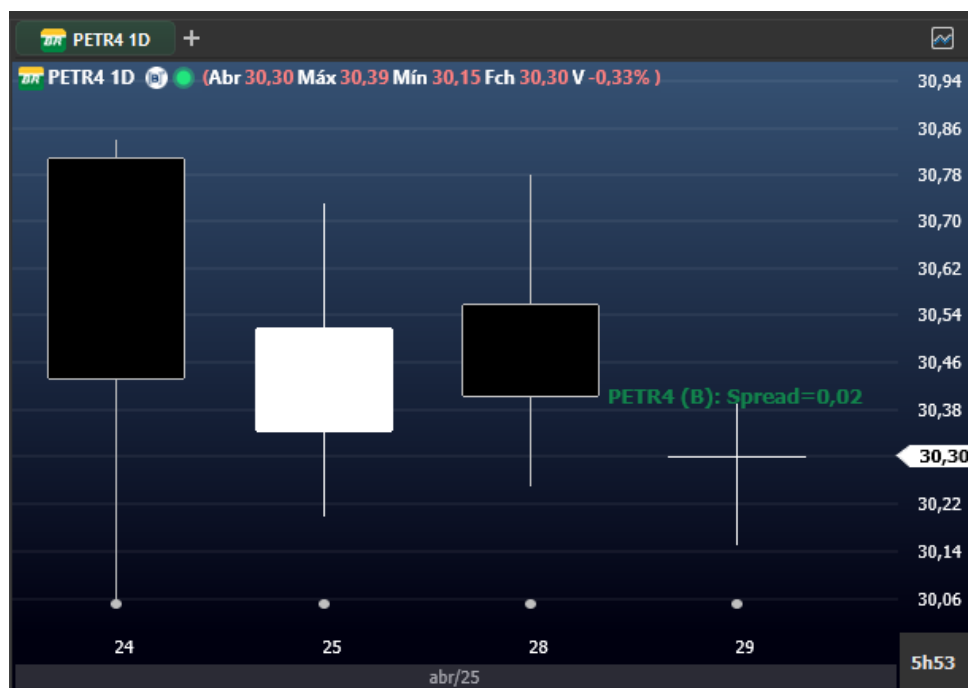
No exemplo 2, será identificada a bolsa (F) vinculada ao Asset definido (WDO), independentemente do ativo selecionado.

Exemplo 1:

```
1  var
2    geta : String;
3    getf : String;
4    spread: Float;
5  begin
6    if (MaxBarsForward = 0) then
7      begin
8        geta := GetAsset;
9        spread := BookSpread;
10       getf := GetFeed;
11       plotText((geta + "("+getf+"): "+"Spread=" +spread), clGreen, 2, 10);
12     end;
13 end;
```



Exemplo Gráfico 1:



Exemplo 2:

```
1  const
2    WDO = Asset("WDOFUT", feedBMF);
3  var
4    geta : String;
5    getf : String;
6    spread : Float;
7  begin
8    if (MaxBarsForward = 0) then
9      begin
10         geta := GetAsset(WDO);
11         spread := BookSpread;
12         getf := GetFeed(WDO);
13         plotText((geta + "(" + getf + "): " + "Spread=" + spread), c1Green, 2, 10);
14       end;
15     end;
```

Exemplo Gráfico 2:





20.9 Função IsBMF

Disponível em todas versões.

A função `IsBMF()` retorna se o ativo pertence ao segmento BMF.

Sintaxe:

```
Função IsBMF(): Boolean;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, caso o ativo pertença ao BMF, será plotado o preço da melhor oferta de venda.

```
1 begin
2   if (isBMF) then
3     Plot(AskPrice);
4 end;
```

Exemplo Gráfico:





20.10 Função Lote

Disponível em todas versões.

A função **Lote()** retorna a quantidade mínima de contratos referente ao lote do ativo.

Ao utilizar o modo de CrossOrder na automação, o Lote irá retornar a quantidade do Lote da série histórica, e não no ativo de negociação.

Sintaxe:

```
Função Lote(Asset: Ativo): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, caso não haja posição, ocorrerá simulada uma ordem de compra, então será atribuído à variável **qtd** a quantidade do lote.

```
1 var
2   qtd : Float;
3 begin
4   if not HasPosition then
5     BuyAtMarket;
6   qtd := Lote;
7   Plot(qtd);
8 end;
```

Exemplo Gráfico:



20.11 Função MinPriceIncrement

Disponível em todas versões.

A função `MinPriceIncrement()` retorna o incremento mínimo do preço do ativo.

Sintaxe:

```
Função MinPriceIncrement(Asset: Ativo): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, caso não haja posição, ocorrerá simulada uma ordem de compra, então será atribuído à variável `incMin` o valor do incremento mínimo.

```
1 var
2   incMin : Float;
3 begin
4   if not HasPosition then
5     BuyAtMarket;
6   incMin := MinPriceIncrement;
7   Plot(incMin);
8 end;
```

Exemplo Gráfico:





20.12 Função MinOrderIncrement

Disponível em todas versões.

A função `MinOrderIncrement()` retorna o incremento mínimo do preço da ordem.

Sintaxe:

```
Função MinOrderIncrement(Asset: Ativo): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, caso não haja posição, ocorrerá simulada uma ordem de compra, então será atribuído à variável `incMin` o valor do incremento mínimo de ordem.

```
1 var
2   incMin : Float;
3 begin
4   if not HasPosition then
5     BuyAtMarket;
6   incMin := MinOrderIncrement;
7   Plot(incMin);
8 end;
```

Exemplo Gráfico:



20.13 Função MinOrderQty

Disponível em todas versões.

A função `MinOrderIncrement()` retorna a quantidade mínima de ordem.

Sintaxe:

```
Função MinOrderQty(Asset: Ativo): Float;
```

Parâmetros:

Sem parâmetros.

Exemplo de uso:

No exemplo, caso não haja posição, ocorrerá simulada uma ordem de compra, então será atribuído à variável `incMin` a quantidade mínima de ordem.

```
1 var
2   incMin : Float;
3 begin
4   if not HasPosition then
5     BuyAtMarket;
6   incMin := MinOrderQty;
7   Plot(incMin);
8 end;
```

Exemplo Gráfico:



20.14 Função SellOfferCount

Disponível em todas versões.

A função `SellOfferCount()` retorna as Ofertas de Venda do Ativo.

Sintaxe:

```
Função SellOfferCount(Asset: Ativo; Níveis considerados: Integer): Integer;
```

Parâmetros:

Asset: **Ativo** [Opcional]; Parâmetro Opcional declarado a partir do recurso Asset, valor padrão é "".

Nível considerado: **Integer** [Opcional]; Parâmetro Opcional Inteiro, valor padrão é 0.

Exemplo de uso:

No exemplo 1, caso não tenha posição aberta, será atribuído à variável `sellCount` as ofertas de venda do ativo.

No exemplo 2, caso não tenha posição aberta, será atribuído à variável `sellCount` as ofertas de venda do ativo VALE3.

Exemplo 1:

```
1 var
2   sellCount : Integer;
3 begin
4   if not HasPosition then
5     sellCount := sellOfferCount;
6   Plot(sellCount);
7 end;
```



Exemplo Gráfico 1:



Exemplo 2:

```
1  const
2    VALE3 = Asset("VALE3", feedBovespa);
3  var
4    sellCount : Integer;
5  begin
6    if not HasPosition then
7      sellCount := SellOfferCount(VALE3);
8    Plot(sellCount);
9  end;
```



Exemplo Gráfico 2:





20.15 Função TotalBuyQtd

Disponível em todas versões.

A função `TotalBuyQtd()` retorna a quantidade de Compras do Ativo.

Sintaxe:

```
Função TotalBuyQtd(Asset: Ativo; Níveis considerados: Integer): Integer;
```

Parâmetros:

Asset: **Ativo** [Opcional]; Parâmetro Opcional declarado a partir do recurso Asset, valor padrão é "".

Nível considerado: **Integer** [Opcional]; Parâmetro Opcional Inteiro, valor padrão é 0.

Exemplo de uso:

No exemplo 1, caso não tenha posição aberta, será atribuído à variável `buyQtd` a quantidade de compras do ativo.

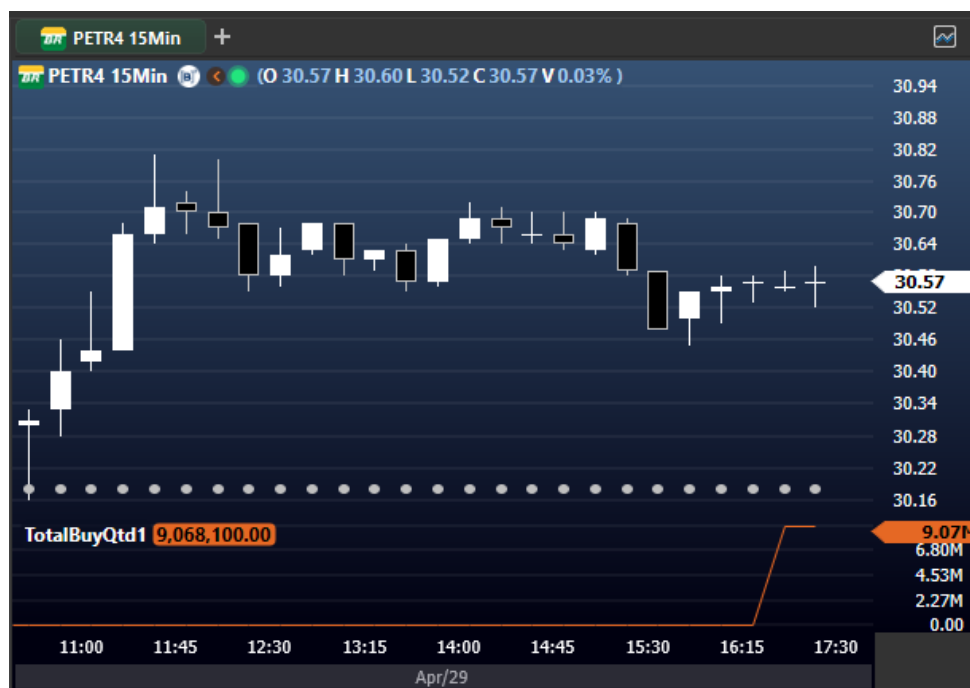
No exemplo 2, caso não tenha posição aberta, será atribuído à variável `buyQtd` a quantidade de compras do ativo VALE3.

Exemplo 1:

```
1 var
2   buyQtd : Integer;
3 begin
4   if not HasPosition then
5     buyQtd := TotalBuyQtd;
6   Plot(buyQtd);
7 end;
```



Exemplo Gráfico 1:



Exemplo 2:

```
1  const
2    VALE3 = Asset("VALE3", feedBovespa);
3  var
4    buyQtyd : Integer;
5  begin
6    if not HasPosition then
7      buyQtyd := TotalBuyQtyd(VALE3);
8    Plot(buyQtyd);
9  end;
```

Exemplo Gráfico 2:





20.16 Função TotalSellQty

Disponível em todas versões.

A função `TotalSellQty()` retorna a quantidade de vendas do Ativo.

Sintaxe:

```
Função TotalSellQty(Asset: Ativo; Níveis considerados: Integer): Integer;
```

Parâmetros:

Asset: **Ativo** [Opcional]; Parâmetro Opcional declarado a partir do recurso Asset, valor padrão é "".

Nível considerado: **Integer** [Opcional]; Parâmetro Opcional Inteiro, valor padrão é 0.

Exemplo de uso:

No exemplo 1, caso não tenha posição aberta, será atribuído à variável `sellQty` a quantidade de vendas do ativo.

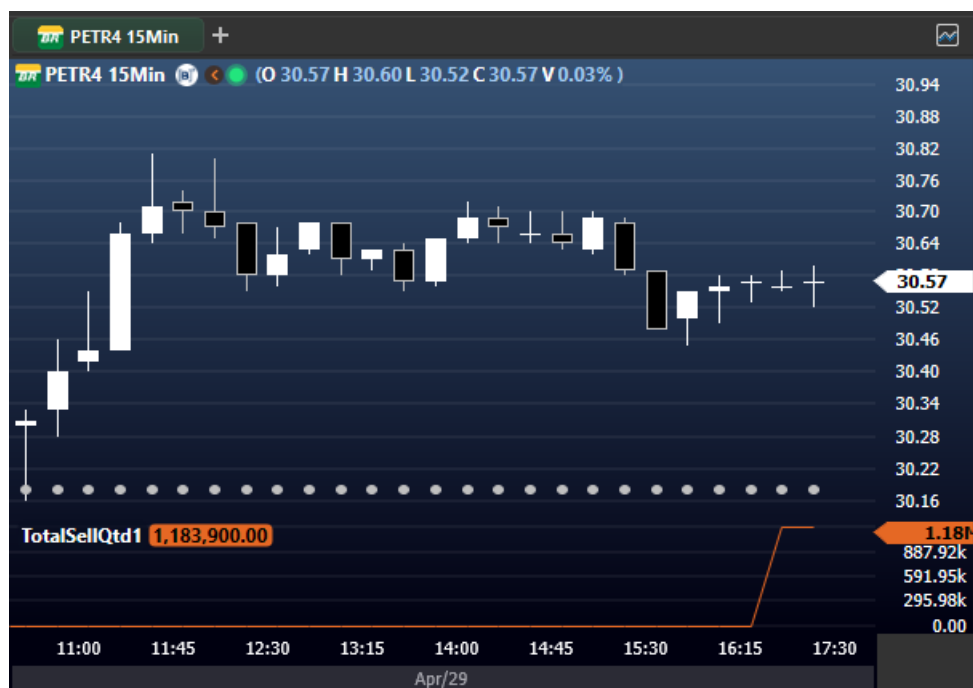
No exemplo 2, caso não tenha posição aberta, será atribuído à variável `sellQty` a quantidade de vendas do ativo VALE3.

Exemplo 1:

```
1 var
2   sellQty : Integer;
3 begin
4   if not HasPosition then
5     sellQty := TotalSellQty;
6   Plot(sellQty);
7 end;
```



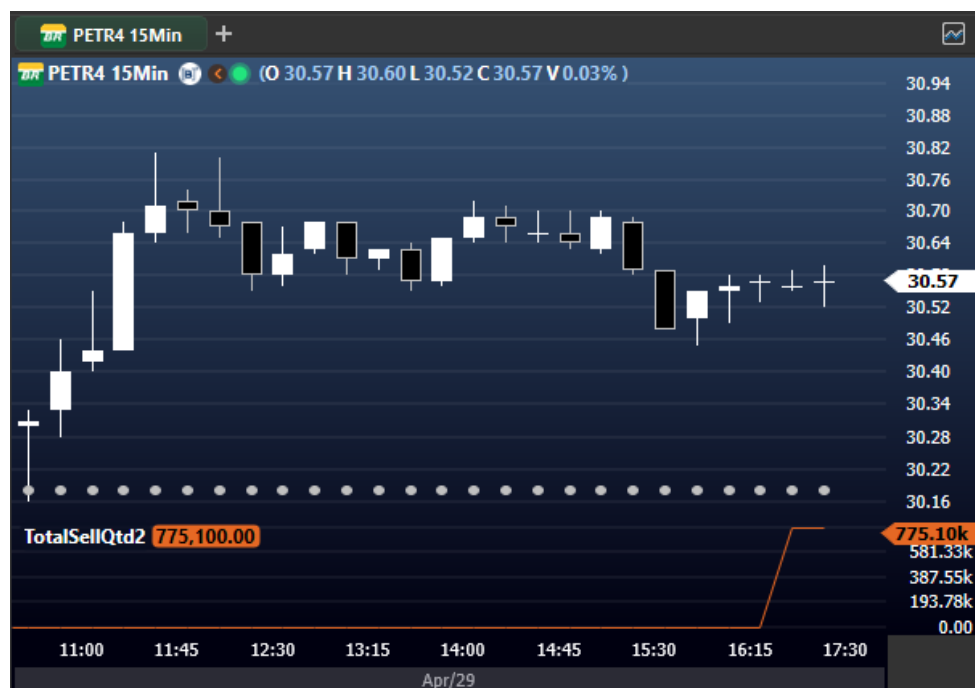
Exemplo Gráfico 1:



Exemplo 2:

```
1 const
2   VALE3 = Asset("VALE3", feedBovespa);
3 var
4   sellQtd : Integer;
5 begin
6   if not HasPosition then
7     sellQtd := TotalSellQtd(VALE3);
8   Plot(sellQtd);
9 end;
```


Exemplo Gráfico 2:



21 Funções Matemáticas

21.1 Função ABS

Disponível em todas versões.

A função **ABS()** retorna o valor absoluto de um número ponto flutuante.

Sintaxe:

```
Função ABS(Valor: Float): Float;
```

Parâmetros:

Valor: **Float** ; Valor ou variável para obter o módulo.

Exemplo de uso:

No exemplo, a variável **n** irá receber o módulo (4) do valor (-4).

```
1 var
2   n : Float;
3 begin
4   n := ABS(-4);
5   Plot(n);
6 end;
```

Exemplo Gráfico:



21.2 Função Arctangent

Disponível em todas versões.

A função `Arctangent()` retorna o arcotangente (em graus) de determinado número

Sintaxe:

```
Função Arctangent(Num: Float): Float;
```

Parâmetros:

Num: **Float** ; Número que será convertido.

Exemplo de uso:

No exemplo, a variável `arc` irá receber o arcotangente, em graus, do número **12**.

```
1 var
2   arc : Float;
3 begin
4   arc := Arctangent(12);
5   Plot(arc);
6 end;
```

Exemplo Gráfico:





21.3 Função Ceiling

Disponível em todas versões.

A função **Ceiling()** efetua um arredondamento, retornando o menor inteiro maior que um número específico.

Sintaxe:

```
Função Ceiling(Num: Float): Float;
```

Parâmetros:

Num: **Float** ; Número que será arredondado.

Exemplo de uso:

No exemplo abaixo, usamos a função **Ceiling()** para retornar o menor inteiro maior que o valor **2.3** (Retorno: 3).

```
1 var
2   number : Float;
3 begin
4   number := Ceiling(2.3);
5   Plot(number);
6 end;
```

Exemplo Gráfico:



21.4 Função Combination

Disponível em todas versões.

A função `Combination()` calcula o número de grupos com combinação única, considerando um conjunto específico de números.

Sintaxe:

```
Função Combination(Num: Integer; NumChosen: Integer): Integer;
```

Parâmetros:

Num: **Integer**; Total de números ou itens a serem considerados.

NumChosen: **Integer**; Número de itens únicos em cada grupo.

Exemplo de uso:

No exemplo, será atribuído à variável `nComb` o número de grupos da combinação **(4,2)**, onde o resultado será 6, conforme o cálculo: (1, 2), (1, 3), (1, 4), (2, 3), (2, 4) e (3, 4).

```
1 var
2   nComb : Integer;
3 begin
4   nComb := Combination(4,2);
5   Plot(NcOMB);
6 end;
```

Exemplo Gráfico:





21.5 Função Cos

Disponível em todas versões.

A função **Cos()** tem como objetivo retornar ao usuário o Cosseno de um valor em radianos.

Sintaxe:

```
Função Cos(Valor: Float): Float;
```

Parâmetros:

Valor: **Float**; Valor ou variável para obter o Cosseno.

Exemplo de uso:

No exemplo, será atribuído à variável **nCos** o Cosseno do valor **180** em radianos(-0,60).

```
1 var
2   nCos : Float;
3 begin
4   nCos := Cos(180);
5   Plot(nCos);
6 end;
```

Exemplo Gráfico:



21.6 Função Cosine

Disponível em todas versões.

A função `Cosine()` tem como objetivo retornar ao usuário o Cosseno de um valor em graus.

Sintaxe:

```
Função Cosine(Num: Float): Float;
```

Parâmetros:

Num: `Float`; Valor ou variável para obter o Cosseno.

Exemplo de uso:

No exemplo, será atribuído à variável `nCos` o Cosseno do valor `45` em graus (0,71).

```
1 var
2   nCos : Float;
3 begin
4   nCos := Cosine(45);
5   Plot(nCos);
6 end;
```

Exemplo Gráfico:





21.7 Função Cotangent

Disponível em todas versões.

A função `Cotangent()` tem como objetivo retornar ao usuário a Cotangente de um valor em graus.

Sintaxe:

```
Função Cotangent(Num: Float): Float;
```

Parâmetros:

Num: **Float**; Valor ou variável para obter o Cosseno.

Exemplo de uso:

No exemplo, será atribuído à variável `nCo` a Cotangente do valor **30** em graus (1,73).

```
1 var
2   nCo : Float;
3 begin
4   nCo := Cotangent(30);
5   Plot(nCo);
6 end;
```

Exemplo Gráfico:





21.8 Função Exp

Disponível em todas versões.

A função `Exp()` tem como objetivo retornar ao usuário a enésima potência do número de Euler.

Sintaxe:

```
Função Exp(Value: Float): Float;
```

Parâmetros:

Value: **Float**; Valor ou uma variável para obter a enésima potência (Euler).

Exemplo de uso:

No exemplo, será atribuído à variável `n` o valor de **(2)** e irá retornar o valor 7,39

```
1 var
2   n : Float;
3 begin
4   n := Exp(2);
5   Plot(n);
6 end;
```

Exemplo Gráfico:





21.9 Função ExpValue

Disponível em todas versões.

A função `ExpValue()` possui como finalidade retornar o valor exponencial de um determinado número.

Sintaxe:

```
Função ExpValue(Num: Float): Float;
```

Parâmetros:

Num: **Float**; Valor ou uma variável para obter o valor exponencial.

Exemplo de uso:

No exemplo, será atribuído à variável `n` o valor de **(2)** e irá retornar o valor 7,39

```
1 var
2   n : Float;
3 begin
4   n := ExpValue(2);
5   Plot(n);
6 end;
```

Exemplo Gráfico:



21.10 Função ExtremePriceRatio

Disponível em todas versões.

A função `ExtremePriceRatio()` retorna o ratio das extremidades (divide o maior valor no período pelo menor valor) de um número determinado de barras.

Sintaxe:

```
Função ExtremePriceValue(Length: Integer; UseLog: Boolean): Float;
```

Parâmetros:

Length: **Integer**; O número de barras que serão considerados no cálculo.

UseLog: **Boolean**; Determina se o logaritmo de 10 do resultado da divisão será aplicado.

Exemplo de uso:

No exemplo, será atribuído à variável `pRatio()` o retorno da função, considerando **300** períodos para a divisão.

```
1 var
2   pRatio : Float;
3 begin
4   pRatio := ExtremePriceRatio(300, True);
5   Plot(pRatio);
6 end;
```

Exemplo Gráfico:



21.11 Função Factorial

Disponível em todas versões.

A função **Factorial**() tem como finalidade efetuar o cálculo fatorial($n!$) de um número natural n .

Sintaxe:

Função **Factorial**(Value: **Float**): **Float**;

Parâmetros:

Value: **Float**; Número natural para o cálculo do factorial.

Exemplo de uso:

No exemplo, será atribuído à variável **nFactorial** irá receber o fatorial do número **4**.

```
1 var
2   nFactorial : Float;
3 begin
4   nFactorial := Factorial(4);
5   Plot(pFactorial);
6 end;
```

Exemplo Gráfico:



21.12 Função FastD

Disponível em todas versões.

A função **FastD()** retorna o valor de FastD do Oscilador Estocástico, de acordo com o período desejado.

Sintaxe:

Função **FastD**(StochLength: **Integer**): **Float**;

Parâmetros:

StochLength: **Float**; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo, será atribuído à variável `interval_d` irá receber o retorno da função **FastD()**, considerando **4** períodos para o cálculo.

```
1 var
2   interval_d : Float;
3 begin
4   interval_d := FastD(4);
5   Plot(interval_d);
6 end;
```

Exemplo Gráfico:



21.13 Função FastK

Disponível em todas versões.

A função **FastK()** retorna o valor de FastK do Oscilador Estocástico, de acordo com o período desejado.

Sintaxe:

Função **FastK**(StochLength: **Integer**): **Float**;

Parâmetros:

StochLength: **Float**; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo, será atribuído à variável `interval_k` irá receber o retorno da função **FastK()**, considerando **14** períodos para o cálculo.

```
1 var
2   interval_k : Float;
3 begin
4   interval_k := FastK(14);
5   Plot(interval_k);
6 end;
```

Exemplo Gráfico:



21.14 Função Floor

Disponível em todas versões.

A função **Floor()** possui como finalidade retornar o maior valor inteiro menor que um número determinado.

Sintaxe:

```
Função Floor(Num: Float): Float;
```

Parâmetros:

Num: **Float**; Valor de referência para obter o dado específico.

Exemplo de uso:

No exemplo, será atribuído à variável **m** o valor -7.

```
1 var
2   m : Float;
3 begin
4   m := Floor(- 6.1);
5   Plot(m);
6 end;
```

Exemplo Gráfico:



21.15 Função FracPortion

Disponível em todas versões.

A função `FracPortion()` possui como finalidade retornar a parte fracionária de um número.

Sintaxe:

```
Função FracPortion(Num: Float): Float;
```

Parâmetros:

Num: **Float**; Valor de referência para obter o dado específico.

Exemplo de uso:

No exemplo, será atribuído à variável `numFrac` o valor **7.14** e será retornado a parte fracionária de 0,14.

```
1 var
2   numFrac : Float;
3 begin
4   numFrac := FracPortion(7.14);
5   Plot(numFrac);
6 end;
```

Exemplo Gráfico:



21.16 Função GCD

Disponível em todas versões.

A função `GCD()` retorna o maior denominador comum entre dois números.

Sintaxe:

```
Função GCD(Value1: Float; Value2: Float): Float;
```

Parâmetros:

Value1: **Float**; Primeiro valor a ser analisado.

Value2: **Float**; Segundo valor a ser analisado.

Exemplo de uso:

No exemplo a seguir, será atribuído à variável `nDem` o maior denominador comum entre **12** e **9**.

```
1 var
2   nDem : Float;
3 begin
4   nDem := GCD(12, 9);
5   Plot(nDem);
6 end;
```

Exemplo Gráfico:





21.17 Função IntPortion

Disponível em todas versões.

A função `IntPortion()` tem como finalidade retornar a parte inteira de determinado número.

Sintaxe:

```
Função IntPortion(Num: Float): Float;
```

Parâmetros:

Num: **Float**; Número para obter a parte inteira.

Exemplo de uso:

No exemplo abaixo, será atribuído à variável `aux` o valor **7**, que representa a parte inteira de **7.52**.

```
1 var
2   aux : Float;
3 begin
4   aux := IntPortion(7.52);
5   Plot(aux);
6 end;
```

Exemplo Gráfico:





21.18 Função Log

Disponível em todas versões.

A função **Log()** retorna o logaritmo natural (ln) de um número.

Sintaxe:

```
Função Log(Num: Float): Float;
```

Parâmetros:

Num: **Float**; Número para o logaritmo natural (ln).

Exemplo de uso:

No exemplo abaixo, será atribuído à variável **vLn** o valor do logaritmo natural de **5**, aproximadamente **1,61**.

```
1 var
2   vLn : Float;
3 begin
4   vLn := Log(5);
5   Plot(vLn);
6 end;
```

Exemplo Gráfico:





21.19 Função LogN

Disponível em todas versões.

A função **Log()** retorna o logaritmo na base N de um número.

Sintaxe:

```
Função LogN(Base: Float; Número: Float): Float;
```

Parâmetros:

Base: **Float**; Número que deve ser elevado a uma certa potência.

Número: **Float**; Número para o logaritmo natural (ln).

Exemplo de uso:

No exemplo abaixo, será atribuído à variável **vLn** o valor do logaritmo natural de **2** com base **5**, aproximadamente **0,43**.

```
1 var
2   vLn : Float;
3 begin
4   vLn := LogN(5, 2);
5   Plot(vLn);
6 end;
```

Exemplo Gráfico:





21.20 Função Max

Disponível em todas versões.

A função `Max()` retorna o maior valor entre dois números.

Sintaxe:

```
Função Max(Valor1: Float; Valor2: Float): Float;
```

Parâmetros:

Valor1: **Float**; Primeiro número da comparação.

Valor2: **Float**; Segundo número da comparação.

Exemplo de uso:

No exemplo a seguir, será atribuído à variável `dMax` o valor **20.0**, que é o maior entre **10.0** e **20**.

```
1 var
2   dMax : Float;
3 begin
4   dMax := Max(10.0 , 20);
5   Plot(dMax);
6 end;
```

Exemplo Gráfico:





21.21 Função Min

Disponível em todas versões.

A função `Min()` retorna o menor valor entre dois números.

Sintaxe:

```
Função Min(Valor1: Float; Valor2: Float): Float;
```

Parâmetros:

Valor1: **Float**; Primeiro número da comparação.

Valor2: **Float**; Segundo número da comparação.

Exemplo de uso:

No exemplo a seguir, será atribuído à variável `dMin` o valor **10.0**, que é o menor entre **10.0** e **20**.

```
1 var
2   dMin : Float;
3 begin
4   dMin := Min(10.0 , 20);
5   Plot(dMin);
6 end;
```

Exemplo Gráfico:





21.22 Função MinutesToTime

Disponível em todas versões.

A função `MinutesToTime()` retorna a conversão de minutos em hora militar (contagem iniciada à meia-noite).

Sintaxe:

```
Função MinutesToTime(Minutes: Integer): Integer;
```

Parâmetros:

Minutes: Integer; Minutos para a conversão em horas.

Exemplo de uso:

No exemplo a seguir, a variável `nHora` irá receber a conversão de **600** minutos em horas, retornando o valor **1000**, que representa 10h.

```
1 var
2   nHora : Integer;
3 begin
4   nHora := MinutesToTime(600);
5   Plot(nHora);
6 end;
```

Exemplo Gráfico:



21.23 Função Mod

Disponível em todas versões.

A função **Mod()** possui como finalidade retornar o resto da divisão entre dois números inteiros.

Sintaxe:

```
Função Mod(Num: Integer; Divisor: Integer): Integer;
```

Parâmetros:

Num: **Integer**; Número referente ao dividendo.

Divisor: **Integer**; Número que será o divisor.

Exemplo de uso:

No exemplo abaixo, será atribuído à variável **res** o valor **1**, referente ao resto da divisão de **10** por **3**.

```
1 var
2   res : Integer;
3 begin
4   res := Mod(10, 3);
5   Plot(res);
6 end;
```

Exemplo Gráfico:





21.24 Função Neg

Disponível em todas versões.

A função `Neg()` retorna o valor negativo de um determinado número.

Sintaxe:

```
Função Neg(Num: Float): Float;
```

Parâmetros:

Num: **Float**; Valor para obter seu número negativo.

Exemplo de uso:

No exemplo abaixo, a variável `nNeg` irá receber o valor **-5**, que é o retorno da função aplicada sobre **5**.

```
1 var
2   nNeg : Float;
3 begin
4   nNeg := Neg(5);
5   Plot(nNeg);
6 end;
```

Exemplo Gráfico:



21.25 Função NumUnits

Disponível em todas versões.

A função `NumUnits()` retorna o número de contratos/ações de um certo investimento.

Sintaxe:

```
Função NumUnits(Amnt: Integer; MinLot: Integer): Integer;
```

Parâmetros:

Amnt: `Integer`; Valor total de investimento, em reais, por operação.

MinLot: `Integer`; Tamanho mínimo desejado de lote por transação.

Exemplo de uso:

No exemplo abaixo, assumindo que a ação selecionada custa **29.74 BRL**, ao investir **15500** com lote mínimo de **100**, a função `NumUnits()` retornará 500.

```
1 var
2   numU : Integer;
3 begin
4   numU := NumUnits(15500, 100);
5   Plot(numU);
6 end;
```

Exemplo Gráfico:





21.26 Função PercentR

Disponível em todas versões.

A função `PercentR()` retorna uma porcentagem indicando onde o preço atual está em relação à faixa de negociação avaliada.

Sintaxe:

```
Função PercentR(Length: Integer): Float;
```

Parâmetros:

Length: Integer; Comprimento considerado para o cálculo.

Exemplo de uso:

No exemplo abaixo, será atribuído à variável `vPercentR` o retorno da função `PercentR()`, considerando `2` como comprimento.

```
1 var
2   vPercentR : Float;
3 begin
4   vPercentR := PercentR(2);
5   Plot(vPercentR);
6 end;
```

Exemplo Gráfico:



21.27 Função Permutation

Disponível em todas versões.

A função `Permutation()` calcula o número de permutações para um determinado número de objetos.

Sintaxe:

```
Função Permutation(Num: Integer; NumChosen: Integer): Integer;
```

Parâmetros:

Num: `Integer`; Determina o número de candles a serem analisados.

NumChosen: `Integer`; Define o número de objetos dentro do intervalo de candles que podem ser selecionados.

Exemplo de uso:

A variável `n` irá receber a permutação, considerando **4** candles e **2** objetos.

```
1 var
2   n : Integer;
3 begin
4   n := Permutation(4, 2);
5   Plot(n);
6 end;
```

Exemplo Gráfico:





21.28 Função Pos

Disponível em todas versões.

A função **Pos()** retorna o valor absoluto (módulo) de um número ponto flutuante.

Sintaxe:

```
Função Pos(Valor: Float): Float;
```

Parâmetros:

Valor: **Float**; Valor ou variável para obter o módulo.

Exemplo de uso:

No exemplo abaixo, a variável **n** irá receber o valor **4**, que é o módulo do valor **-4**.

```
1 var
2   n : Float;
3 begin
4   n := Pos(-4);
5   Plot(n);
6 end;
```

Exemplo Gráfico:



21.29 Função Power

Disponível em todas versões.

A função **Power**() tem como finalidade retornar a enésima potência de um valor.

Sintaxe:

```
Função Power(Valor: Float; Potência: Integer): Float;
```

Parâmetros:

Valor: **Float**; Valor base da potenciação.

Potência: **Integer**; Valor ao qual a base será elevada.

Exemplo de uso:

No exemplo a seguir, será atribuído à variável x o resultado da operação 2^3 , ou seja, 8.

```
1 var
2   x : Float;
3 begin
4   x := Power(2, 3);
5   Plot(x);
6 end;
```

Exemplo Gráfico:





21.30 Função PowerF

Disponível em todas versões.

A função **Power**() tem como finalidade retornar a enésima potência de um valor.

Sintaxe:

```
Função Power(Valor: Float; Potência: Float): Float;
```

Parâmetros:

Valor: **Float**; Valor base da potenciação.

Potência: **Float**; Valor ao qual a base será elevada.

Exemplo de uso:

No exemplo a seguir, será atribuído à variável x o resultado da operação $2^{3.5}$, ou seja, 11,31.

```
1 var
2   x : Float;
3 begin
4   x := Power(2, 3.5);
5   Plot(x);
6 end;
```

Exemplo Gráfico:



21.31 Função Random

Disponível em todas versões.

Com a função `Random()`, o usuário poderá gerar um número aleatório inteiro dentro de um intervalo iniciado em zero, tendo como limite superior o valor definido por parâmetro.

Sintaxe:

```
Função Random(Limite: Integer): Integer;
```

Parâmetros:

Limite: Integer; Valor ou variável que determina o limite superior do intervalo.

Exemplo de uso:

No exemplo a seguir, a função `Random()` irá gerar números aleatórios entre 0 e 5, que serão atribuídos à variável `aux`.

```
1 var
2   aux : Integer;
3 begin
4   aux := Random(5);
5   Plot(aux);
6 end;
```

Exemplo Gráfico:





21.32 Função Round

Disponível em todas versões.

A função `Round()` possui como finalidade efetuar o arredondamento de um número ponto flutuante.

Sintaxe:

```
Função Round(Valor: Float): Integer;
```

Parâmetros:

Valor: **Float**; Número (variável ou constante) com casas decimais que será arredondado.

Exemplo de uso:

Conforme o exemplo abaixo, será atribuído à variável `aux` o valor arredondado de **2.6**, ao utilizar a função `Round()`.

```
1 var
2   aux : Integer;
3 begin
4   aux := Round(2.6);
5   Plot(aux);
6 end;
```

Exemplo Gráfico:



21.33 Função Round2Fraction

Disponível em todas versões.

A função `Round2Fraction()` efetua o arredondamento de um número para o valor mais próximo de um múltiplo do incremento mínimo de um ativo.

Sintaxe:

```
Função Round2Fraction(Valor: Float): Float;
```

Parâmetros:

Valor: `Float`; Valor desejado para o arredondamento de acordo com o incremento mínimo do ativo.

Exemplo de uso:

No exemplo abaixo (ativos Bovespa), ao aplicar a função para o valor **31.626**, será atribuído o valor **31.63** à variável `nRound2`.

```
1 var
2   nRound2 : Float;
3 begin
4   nRound2 := Round2Fraction(31.626);
5   Plot(nRound2);
6 end;
```

Exemplo Gráfico:





21.34 Função Sign

Disponível em todas versões.

A função `Sign()` possui como definição retornar um número inteiro baseado no sinal de um valor numérico.

Sintaxe:

```
Função Sign(Num: Float): Float;
```

Parâmetros:

Num: **Float**; Número para obter seu sinal.

Exemplo de uso:

No exemplo abaixo, será atribuído à variável `nSinal` o valor **-1**, tendo em vista o valor **-205** passado por parâmetro.

```
1 var
2   nSinal : Float;
3 begin
4   nSinal := Sign(-205);
5   Plot(nSinal);
6 end;
```

Exemplo Gráfico:





21.35 Função Sin

Disponível em todas versões.

A função `Sin()` tem como objetivo retornar o seno de um valor em radianos.

Sintaxe:

```
Função Sin(Valor: Float): Float;
```

Parâmetros:

Valor: **Float**; Valor para obter o seno.

Exemplo de uso:

No exemplo a seguir, será atribuído à variável `nSeno` o valor do seno de **180**, aproximadamente **-0,80**.

```
1 var
2   nSeno : Float;
3 begin
4   nSeno := Sin(180);
5   Plot(nSeno);
6 end;
```

Exemplo Gráfico:



21.36 Função Sine

Disponível em todas versões.

A função `Sine()` retorna o seno de um número em graus.

Sintaxe:

```
Função Sine(Num: Float): Float;
```

Parâmetros:

Sine: **Float**; Determina número escolhido para o cálculo.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o seno de **75**.

```
1 begin
2   Plot(Sine(75));
3 end
```

Exemplo Gráfico:





21.37 Função SlowK

Disponível em todas versões.

A função `SlowK()` retorna o valor de SlowK do oscilador estocástico.

Sintaxe:

```
Função SlowK(StochLength: Integer): Float;
```

Parâmetros:

`StochLength: Integer`; Determina o tamanho escolhido para o cálculo.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função `SlowK()`.

```
1 begin
2   Plot(SlowK(10));
3 end
```

Exemplo Gráfico:





21.38 Função Sqrt

Disponível em todas versões.

A função `Sqrt()` retorna a raiz quadrada de um valor.

Sintaxe:

```
Função Sqrt(Valor: Float): Float;
```

Parâmetros:

Valor: **Float**; Determina o valor escolhido para o cálculo.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a raiz quadrada de **144**.

```
1 begin
2   Plot(Sqrt(144));
3 end
```

Exemplo Gráfico:





21.39 Função Square

Disponível em todas versões.

A função `Square()` retorna o valor de um determinado número ao quadrado.

Sintaxe:

```
Função Square(Num: Float): Float;
```

Parâmetros:

Num: **Float**; Determina o valor escolhido para o cálculo.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função `Square()`, considerando **12** para o cálculo.

```
1 begin
2   Plot(Square(12));
3 end
```

Exemplo Gráfico:



21.40 Função Tangent

Disponível em todas versões.

A função `Tangent()` retorna a tangente de um número específico.

Sintaxe:

```
Função Tangent(Num: Float): Float;
```

Parâmetros:

Num: **Float**; Determina o valor escolhido para o cálculo.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função `Tangent()`, considerando **45** para o cálculo.

```
1 begin
2   Plot(Tangent(45));
3 end
```

Exemplo Gráfico:



21.41 Função UltimateOscillator

Disponível em todas versões.

A função `UltimateOscillator()` retorna o valor do "Ultimate Oscillator" desenvolvido por Larry Williams.

Sintaxe:

```
Função UltimateOscillator(ShortLength: Integer, MidLength: Integer, LongLength: Integer): Float;
```

Parâmetros:

ShortLength: Integer; Número de períodos para a média mais curta utilizada no cálculo do indicador.

MidLength: Integer; Número de períodos para a média intermediária utilizada no cálculo do indicador.

LongLength: Integer; Número de períodos para a média mais longa utilizada no cálculo do indicador.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno do indicador "Ultimate Oscillator" com períodos de **10** candles para as 3 médias.

```
1 begin
2   Plot(UltimateOscillator(10,10,10));
3 end
```

Exemplo Gráfico:



21.42 Função Volatility

Disponível em todas versões.

A função **Volatility()** retorna a volatilidade de um determinado período.

Sintaxe:

```
Função Volatility(Period: Integer): Float;
```

Parâmetros:

Period: **Integer**; Determina o periodo para cálculo.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função **Volatility()** com período **30**.

```
1 begin
2   Plot(Volatility(30));
3 end
```

Exemplo Gráfico:



21.43 Função VolumeOsc

Disponível em todas versões.

A função **VolumeOsc()** retorna a diferença entre a média rápida e a média móvel lenta do volume.

Sintaxe:

```
Função VolumeOsc(FastLength: Integer, SlowLength: Integer): Float;
```

Parâmetros:

FastLength: **Integer**; Valor de média rápida.

SlowLength: **Integer**; Valor de média móvel.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função **VolumeOsc()** com média rápida igual a **1** e média móvel igual a **9**.

```
1 begin
2   Plot(VolumeOsc(1,9));
3 end
```

Exemplo Gráfico:



21.44 Função VolumeROC

Disponível em todas versões.

A função **VolumeROC()** retorna o "VolumeROC" baseado em um número de barras.

Sintaxe:

```
Função VolumeROC(Length: Integer): Float;
```

Parâmetros:

Length: **Integer**; Período definido pela quantidade de barras, usado no cálculo do Volume Rate of Change.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o retorno da função **VolumeROC()**.

```
1 begin
2   Plot(VolumeROC(1));
3 end
```

Exemplo Gráfico:





22 Funções de Opções

22.1 Função Delta

Disponível em todas versões.

A função **Delta()** mede a variação do preço da opção com o preço da ação.

Sintaxe:

```
Função Delta(DaysLeft : Integer; StrikePr : Float; AssetPr : Float; Rate100 :  
Float; Volty100 : Float; PutCall : Integer): Float;
```

Parâmetros:

DaysLeft: **Integer**; Dias uteis até o vencimento da opção.

StrikePr: **Float**; Preço do exercício da opção.

AssetPr: **Float**; Preço da ação.

Rate100: **Float**; Taxa de juros em %.

Volty100: **Float**; Volatilidade em %.

PutCall: **Integer**; Indica se é uma put ou uma call:

- [optPut:] Opção de venda.
- [optCall:] Opção de compra.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a variação, considerando **13** dias para o vencimento, **16.47** como Strike, **17.25** como preço da ação, **0.1** para taxa de juros e **0.25** para volatilidade, aplicados para uma **Call**.

```
1 begin  
2   Plot(Delta(13, 16.47, 17.25, 0.1, 0.25, optCall));  
3 end
```

Exemplo Gráfico:





22.2 Função Gamma

Disponível em todas versões.

A função `Gamma()` retorna o valor de Gamma.

Sintaxe:

```
Função Gamma(DaysLeft : Integer; StrikePr : Float; AssetPr : Float; Rate100 :  
Float; Volty100 : Float; PutCall : Integer): Float;
```

Parâmetros:

DaysLeft: Integer; Dias uteis até o vencimento da opção.

StrikePr: Float; Preço do exercício da opção.

AssetPr: Float; Preço da ação.

Rate100: Float; Taxa de juros em %.

Volty100: Float; Volatilidade em %.

PutCall: Integer; Indica se é uma put ou uma call:

- [optPut:] Opção de venda.
- [optCall:] Opção de compra.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a variação, considerando 9 dias para o vencimento, 11.08 como Strike, 11.94 como preço da ação, 0.2 para taxa de juros e 0.3 para volatilidade, aplicados para uma Call.

```
1 begin  
2   Plot(Gamma(9, 11.08, 11.94, 0.2, 0.3, optCall));  
3 end
```


Exemplo Gráfico:





22.3 Função Rho

Disponível em todas versões.

A função **Rho()** retorna a variação da opção em relação à taxa de juros.

Sintaxe:

```
Função Rho(DaysLeft : Integer; StrikePr : Float; AssetPr : Float; Rate100 : Float;  
Volty100 : Float; PutCall : Integer): Float;
```

Parâmetros:

DaysLeft: Integer; Dias uteis até o vencimento da opção.

StrikePr: Float; Preço do exercício da opção.

AssetPr: Float; Preço da ação.

Rate100: Float; Taxa de juros em %.

Volty100: Float; Volatilidade em %.

PutCall: Integer; Indica se é uma put ou uma call:

- [optPut:] Opção de venda.
- [optCall:] Opção de compra.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a variação, considerando **60** dias para o vencimento, **11.00** como Strike, **12.92** como preço da ação, **0.2** para taxa de juros e **0.3** para volatilidade, aplicados para uma **Call**.

```
1 begin  
2   Plot(Rho(60, 11.00, 12.92, 0.2, 0.3, optCall));  
3 end
```

Exemplo Gráfico:



22.4 Função Strike

Disponível em todas versões.

A função **Strike()** retorna o valor de strike caso o ativo seja uma opção.

Sintaxe:

```
Função Strike(Asset : Ativo): Float;
```

Parâmetros:

Asset: **Ativo** [Opcional]; Determina o ativo para a função.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico o valor da opção **PETRE285W2**, onde a constante **PETR** recebe o nome e bolsa do ativo.

```
1 const
2   PETR = Asset("PETRE285W2", feedBovespa);
3
4 begin
5   Plot(Strike(PETR));
6 end
```

Exemplo Gráfico:





22.5 Função Theta

Disponível em todas versões.

A função **Theta()** retorna a variação da opção em relação à taxa de juros.

Sintaxe:

```
Função Theta(DaysLeft : Integer; StrikePr : Float; AssetPr : Float; Rate100 :  
              Float; Volty100 : Float; PutCall : Integer): Float;
```

Parâmetros:

DaysLeft: **Integer**; Dias uteis até o vencimento da opção.

StrikePr: **Float**; Preço do exercício da opção.

AssetPr: **Float**; Preço da ação.

Rate100: **Float**; Taxa de juros em %.

Volty100: **Float**; Volatilidade em %.

PutCall: **Integer**; Indica se é uma put ou uma call:

- [optPut:] Opção de venda.
- [optCall:] Opção de compra.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a variação, considerando **9** dias para o vencimento, **17.22** como Strike, **11.94** como preço da ação, **0.1** para taxa de juros e **0.32** para volatilidade, aplicados para uma **Put**.

```
1 begin  
2   Plot(Theta(9, 17.22, 11.94, 0.1, 0.32, optPut));  
3 end
```

Exemplo Gráfico:





22.6 Função Vega

Disponível em todas versões.

A função **Vega()** retorna a variação da opção em relação à volatilidade.

Sintaxe:

```
Função Vega(DaysLeft : Integer; StrikePr : Float; AssetPr : Float; Rate100 : Float  
; Volty100 : Float; PutCall : Integer): Float;
```

Parâmetros:

DaysLeft: Integer; Dias uteis até o vencimento da opção.

StrikePr: Float; Preço do exercício da opção.

AssetPr: Float; Preço da ação.

Rate100: Float; Taxa de juros em %.

Volty100: Float; Volatilidade em %.

PutCall: Integer; Indica se é uma put ou uma call:

- [optPut:] Opção de venda.
- [optCall:] Opção de compra.

Exemplo de uso:

No exemplo a seguir, será plotado no gráfico a variação, considerando **28** dias para o vencimento, **11.08** como Strike, **11.94** como preço da ação, **0.2** para taxa de juros e **0.36** para volatilidade, aplicados para uma **Put**.

```
1 begin  
2   Plot(Vega(28, 11.08, 11.94, 0.2, 0.36, optPut));  
3 end
```

Exemplo Gráfico:





23 Screening

23.1 Função Select

Disponível em todas versões.

A função `Select()`, apesar de estar disponível na linguagem, não possui recursos para utilização do usuário, sua implementação é específica para uso do sistema, a fim de habilitação do Screening.

Sintaxe:

```
Função Select(): Void;
```

Parâmetros:

Sem parâmetros.

24 Funções de Usuário

24.1 Função Cum

Disponível em todas versões.

A função `Cum()` acumula o valor de uma série de dados, desde a primeira barra até a atual.

Sintaxe:

```
Função Cum(SerieDeDados: Serie): Float;
```

Parâmetros:

SerieDeDados: **Serie**; Série para efetuar o somatório.

Exemplo de uso:

No exemplo, será atribuído à variável `aux` o somatório da diferença entre as séries Close e Open.

```
1 var
2   aux : Float;
3 begin
4   aux := Cum(Close - Open);
5   Plot(aux);
6 end;
```

Exemplo Gráfico:





24.2 Função FastKCustom

Disponível em todas versões.

A função **FastKCustom()** retorna o valor de FastK do Oscilador Estocástico, de acordo com os preços determinados por parâmetro, e período desejado.

Sintaxe:

```
Função FastKCustom(PriceH: Serie; PriceL: Serie; PriceC: Serie; Período: Serie):  
    Float;
```

Parâmetros:

PriceH: **Serie**; Série de referência para a máxima.

PriceL: **Serie**; Série de referência para a mínima.

PriceC: **Serie**; Série de referência para o fechamento.

Período: **Integer**; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo, será atribuído à variável **kCustom** o retorno da função **FastKCustom**, considerando as séries de máxima(**PriceH**), mínima(**PriceL**), fechamento(**PriceC**), e 14 períodos para o cálculo.

```
1 var  
2   kCustom : Float;  
3 begin  
4   kCustom := FastKCustom(High, Low, Close, 14);  
5   Plot(kCustom);  
6 end;
```

Exemplo Gráfico:





24.3 Função HarmonicMean

Disponível em todas versões.

A função `HarmonicMean()` calcula a média harmônica de uma série de dados, baseada em um determinado período.

Sintaxe:

```
Função HarmonicMean(SerieDeDados: Serie Período: Integer;): Float;
```

Parâmetros:

SerieDeDados: `Serie`; Série utilizada para o cálculo.

Período: `Float`; Período utilizado no momento do cálculo.

Exemplo de uso:

No exemplo a seguir, será atribuído à variável `aux` o retorno da função `HarmonicMean()`, considerando a máxima (Close) e `20` (Períodos) para o cálculo.

```
1 var
2   aux : Float;
3 begin
4   aux := HarmonicMean(High, 20);
5   Plot(aux);
6 end;
```

Exemplo Gráfico:





```
EXPORTSYMBOL(groupsfree);  
/* export the groupinfo to  
objects.active = modifier  
if(modifier=ob))it#modifier(a
```

24.4 Função Highest

Disponível em todas versões.

A função **Highest()** tem como funcionalidade retornar ao usuário o maior valor da série estipulada por ele, dentro de um período determinado.

Sintaxe:

```
Função Highest(SerieDeDados: Serie; Período: Integer): Float;
```

Parâmetros:

SerieDeDados: Serie; Série de dados desejada, podendo ser a abertura, máxima, mínima, fechamento, ou até mesmo indicadores.

Período: Integer; Determina o período que será considerado para a pesquisa.

Exemplo de uso:

No exemplo abaixo, usamos a função **Highest()** para retornar a maior abertura dentro de **9** períodos.

```
1 begin  
2   if not HasPosition then  
3     Plot(Highest(Open, 9));  
4 end;
```

Exemplo Gráfico:





24.5 Função HighestBar

Disponível em todas versões.

A função `HighestBar()` tem como funcionalidade retornar o índice do maior valor da série especificada, dentro de um período determinado.

Sintaxe:

```
Função HighestBar(SerieDeDados: Serie; Período: Integer): Float;
```

Parâmetros:

SerieDeDados: Serie; Série de dados desejada, como abertura, máxima, mínima, fechamento ou indicadores.

Período: Integer; Determina o período que será considerado para a pesquisa.

Exemplo de uso:

No exemplo abaixo, usamos a função `HighestBar()` para retornar o índice da maior mínima dentro de 9 períodos, utilizando a série de abertura.

```
1 var
2   mMinima : Float;
3 begin
4   mMinima := HighestBar(Open, 9);
5   Plot(mMinima);
6 end;
```

Exemplo Gráfico:



24.6 Função Lowest

Disponível em todas versões.

A função `Lowest()` tem como funcionalidade retornar o menor valor da série estipulada, dentro de um período determinado.

Sintaxe:

```
Função Lowest(SerieDeDados: Serie; Período: Integer): Float;
```

Parâmetros:

SerieDeDados: Serie; Série de dados desejada, podendo ser abertura, máxima, mínima, fechamento ou indicadores.

Período: Integer; Determina o período que será considerado para a pesquisa.

Exemplo de uso:

No exemplo abaixo, usamos a função `Lowest()` para retornar o menor fechamento dentro de **50** períodos.

```
1 var
2   low : Float;
3 begin
4   low := Lowest(Close, 50);
5   Plot(low);
6 end;
```

Exemplo Gráfico:



24.7 Função LowestBar

Disponível em todas versões.

A função `LowestBar()` tem como funcionalidade retornar o índice do menor valor da série estipulada, dentro de um período determinado.

Sintaxe:

```
Função LowestBar(SerieDeDados: Serie; Período: Integer): Float;
```

Parâmetros:

SerieDeDados: Serie; Série de dados desejada, podendo ser abertura, máxima, mínima, fechamento ou indicadores.

Período: Integer; Determina o período que será considerado para a pesquisa.

Exemplo de uso:

No exemplo abaixo, usamos a função `LowestBar()` para retornar o índice da menor mínima dentro de **26** períodos, utilizando a série `Low`.

```
1 var
2   lowB : Float;
3 begin
4   lowB := LowestBar(Close, 26);
5   Plot(lowB);
6 end;
```

Exemplo Gráfico:





24.8 Função Media

Disponível em todas versões.

A função `Media()` retorna o valor do indicador Média Móvel, tipo aritmética. o exemplo com o código fonte está disponível no editor de estratégias, para visualizá-lo, acesse o menu: "abrir > exemplos".

Sintaxe:

```
Função Media(Período: Integer; TipoSerie: Serie): Float;
```

Parâmetros:

Período: `Integer`; Período utilizado no momento do cálculo do indicador.

Dados: `TipoSerie`; Série que será considerada para o cálculo.

Exemplo de uso:

No exemplo, será atribuído à variável `vMed` o valor do indicador Média Móvel (Aritmética), considerando **100** períodos, e a série de fechamento (Close) para o cálculo.

```
1 var
2   vMed : Float;
3 begin
4   vMed := Media;
5   Plot(Media(100, Close));
6 end;
```

Exemplo Gráfico:





24.10 Função MidPoint

Disponível em todas versões.

A função `MidPoint()` retorna a média entre o maior e o menor valor encontrados no período.

Sintaxe:

```
Função MidPoint(SerieDados: Serie; Período: Integer): Float;
```

Parâmetros:

SerieDados: `Serie`; Série de referência.

Período: `Integer`; Período utilizado no momento do cálculo.

Exemplo de uso:

No exemplo abaixo, a variável `midP` irá receber o valor retornado pela função `MidPoint()`, utilizando a série `Close` e `15` períodos como base.

```
1 var
2   midP : Float;
3 begin
4   midP := MidPoint(Close, 15);
5   Plot(midP);
6 end;
```

Exemplo Gráfico:



24.11 Função PercentChange

Disponível em todas versões.

A função **PercentChange()** calcula a alteração percentual no preço do candle atual em relação a um determinado deslocamento anterior.

Sintaxe:

```
Função PercentChange(SerieDados: Serie; Período: Integer): Float;
```

Parâmetros:

SerieDados: **Serie**; Série base de referência.

Período: **Integer**; Período anterior para a comparação com o dado da série atual.

Exemplo de uso:

No exemplo abaixo, será atribuído à variável **pc** o retorno da função **PercentChange()**, utilizando a série **Close** e **2** períodos para a comparação.

```
1 var
2   pc : Float;
3 begin
4   pc := PercentChange(Close, 2);
5   Plot(pc);
6 end;
```

Exemplo Gráfico:





24.12 Função PriceOscillator

Disponível em todas versões.

A função `PriceOscillator()` retorna o valor do indicador **PriceOscillator**, de acordo com os parâmetros desejados.

Sintaxe:

```
Função PriceOscillator(SerieDados: Serie; ComprimentoRapido: Integer;  
    ComprimentoLento: Integer): Float;
```

Parâmetros:

SerieDados: Serie; Série utilizada para o cálculo do indicador.

ComprimentoRapido: Integer; Referente ao parâmetro FastLength.

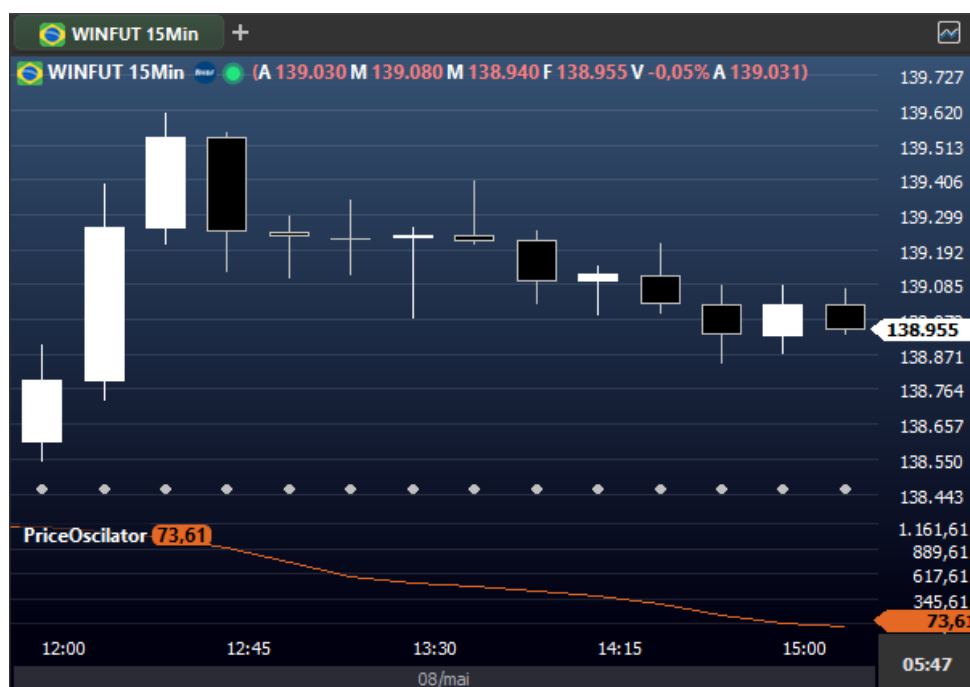
ComprimentoLento: Integer; Referente ao parâmetro SlowLength.

Exemplo de uso:

No exemplo abaixo, a variável `pOsc` irá receber o valor do indicador `PriceOscillator()`, considerando a série `High`, com comprimento rápido igual a **9** e comprimento lento igual a **18**.

```
1 var  
2   pOsc : Float;  
3 begin  
4   pOsc := PriceOscillator(High, 9, 18);  
5   Plot(pOsc);  
6 end;
```

Exemplo Gráfico:



24.13 Função RateOfChange

Disponível em todas versões.

A função `RateOfChange()` retorna a variação percentual de uma série de dados em relação a um valor anterior definido.

Sintaxe:

```
Função RateOfChange(SerieDados: Serie; Período: Integer): Float;
```

Parâmetros:

SerieDados: *Serie*; Série base de referência.

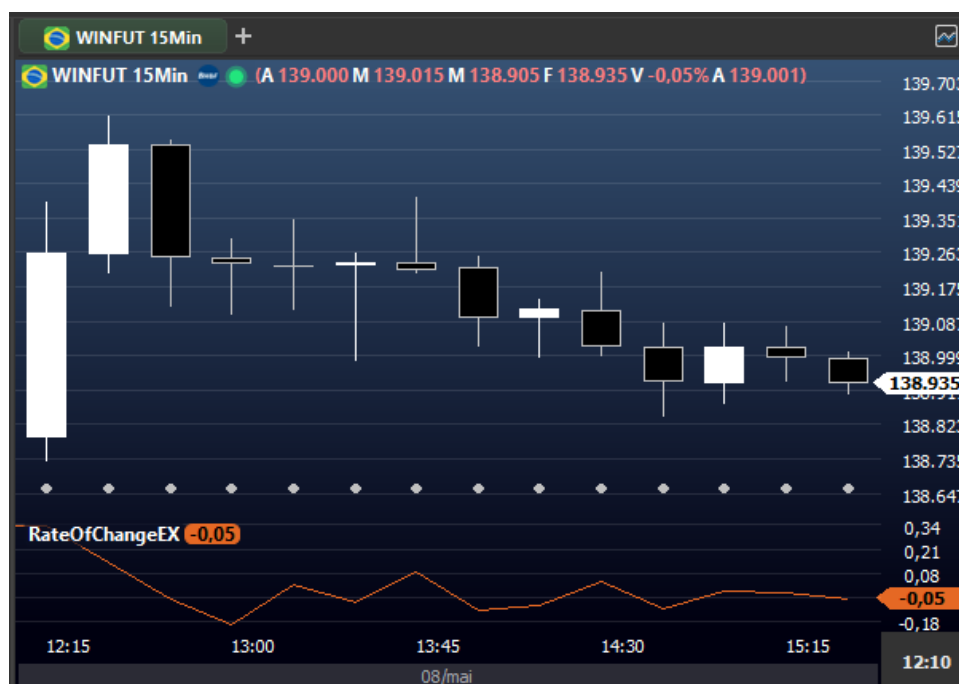
Período: *Integer*; Índice do dado utilizado na comparação com o valor atual da série.

Exemplo de uso:

No exemplo abaixo, será atribuído à variável `rc` o valor retornado pela função `RateOfChange()`, utilizando a série `High` e o candle anterior (`1`) como base de comparação.

```
1 var
2   rc : Float;
3 begin
4   rc := RateOfChange(High, 1);
5   Plot(rc);
6 end;
```

Exemplo Gráfico:



24.14 Função SlowD

Disponível em todas versões.

A função **SlowD()** retorna o valor do **SlowD** (Oscilador Estocástico), de acordo com o período desejado.

Sintaxe:

```
Função SlowD(Período: Integer): Float;
```

Parâmetros:

Período: **Integer**; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo abaixo, a variável **sd** irá receber o valor do indicador **SlowD()**, considerando **14** períodos para o cálculo.

```
1 var
2   sd : Float;
3 begin
4   sd := SlowD(14);
5   Plot(sd);
6 end;
```

Exemplo Gráfico:





24.15 Função StdDevs

Disponível em todas versões.

A função **StdDevs()** retorna o desvio padrão de uma série de dados, em um determinado período.

Sintaxe:

```
Função StdDevs(SerieDados: Serie; Período: Integer): Float;
```

Parâmetros:

SerieDados: **Serie**; Série utilizada para o cálculo.

Período: **Integer**; Período utilizado no momento do cálculo.

Exemplo de uso:

No exemplo abaixo, a variável **sd** irá receber o retorno da função **StdDevs()**, considerando a série **Close** e **20** períodos para o cálculo.

```
1 var
2   sd : Float;
3 begin
4   sd := StdDevs(Close, 20);
5   Plot(sd);
6 end;
```

Exemplo Gráfico:



24.16 Função Summation

Disponível em todas versões.

A função `Summation()` efetua o somatório dos valores de uma série de dados em um determinado número de barras.

Sintaxe:

```
Função Summation(SerieDados: Serie; Período: Integer): Float;
```

Parâmetros:

SerieDados: `Serie`; Série utilizada para o cálculo.

Período: `Integer`; Período utilizado no momento do cálculo.

Exemplo de uso:

No exemplo abaixo, a variável `s` irá receber o retorno da função `Summation()`, considerando a série `Close` e `10` períodos.

```
1 var
2   s : Float;
3 begin
4   s := Summation(Close, 10);
5   Plot(s);
6 end;
```

Exemplo Gráfico:





24.17 Função TriAverage

Disponível em todas versões.

A função `TriAverage()` efetua a média triangular de uma série de dados, dentro de um determinado período.

Sintaxe:

```
Função TriAverage(SerieDados: Serie; Período: Integer): Float;
```

Parâmetros:

SerieDados: `Serie`; Série utilizada para o cálculo.

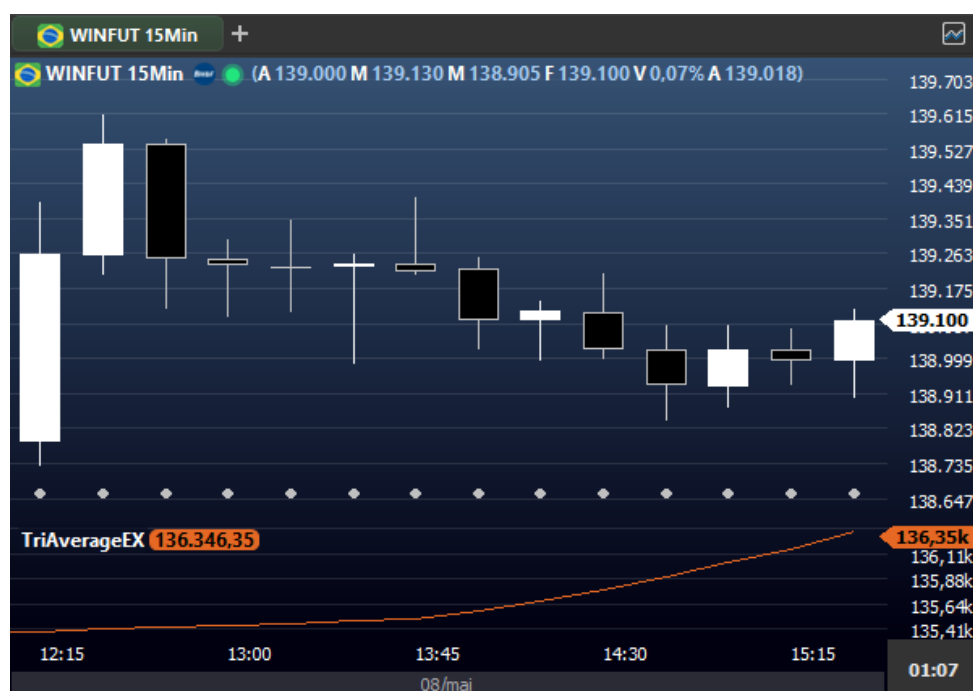
Período: `Integer`; Período utilizado no momento do cálculo.

Exemplo de uso:

No exemplo abaixo, a variável `aux` irá receber o retorno da função `TriAverage()`, considerando a série `High` e `20` períodos para o cálculo.

```
1 var
2   aux : Float;
3 begin
4   aux := TriAverage(High, 20);
5   Plot(aux);
6 end;
```

Exemplo Gráfico:





24.18 Função UlcerIndex

Disponível em todas versões.

A função `UlcerIndex()` mede o nível de estresse do mercado com base na variação negativa dos preços em relação a um topo anterior, ao longo de um período determinado.

Sintaxe:

```
Função UlcerIndex(SerieDados: Serie; Período: Integer): Float;
```

Parâmetros:

SerieDados: `Serie`; Série utilizada para o cálculo.

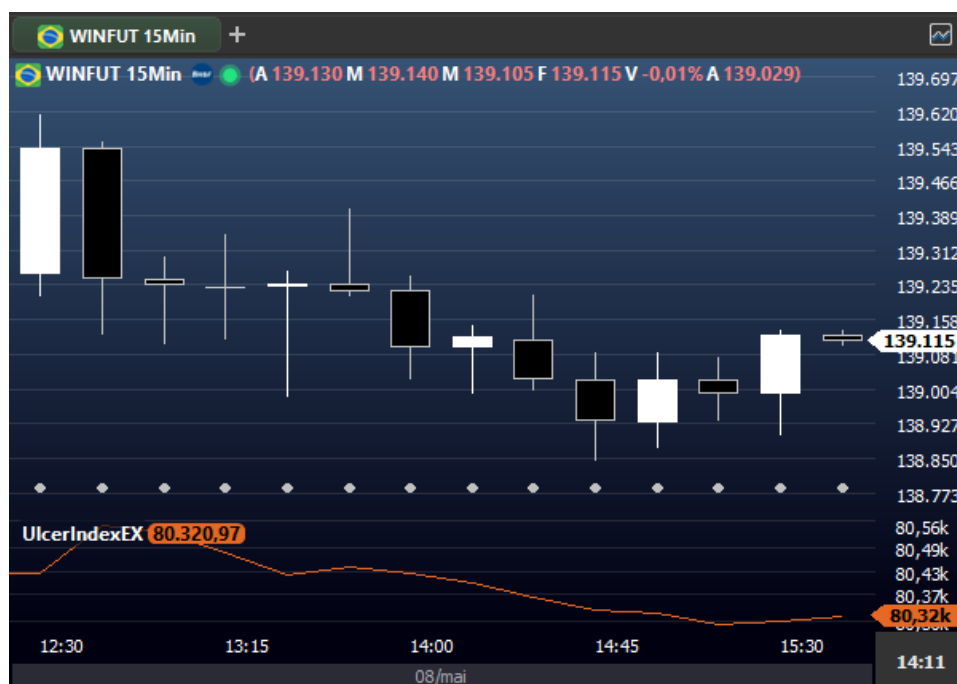
Período: `Integer`; Período utilizado no momento do cálculo.

Exemplo de uso:

No exemplo abaixo, a variável `aux` irá receber o retorno da função `UlcerIndex()`, considerando a série `High` e `3` períodos.

```
1 var
2   aux : Float;
3 begin
4   aux := UlcerIndex(High, 3);
5   Plot(aux);
6 end;
```

Exemplo Gráfico:



24.19 Função WAverage

Disponível em todas versões.

A função **WAverage()** calcula a média ponderada de um determinado período.

Sintaxe:

```
Função WAverage(SerieDeDados: Serie, Período: Integer): Float;
```

Parâmetros:

SerieDeDados: **Serie**; Série utilizada para o cálculo.

Período: **Integer**; Período utilizado no momento do cálculo.

Exemplo de uso:

No exemplo abaixo, será plotado no gráfico o retorno da função **WAverage()**, considerando a série High e **20** períodos para o cálculo.

```
1 begin
2   Plot(WAverage(High, 20));
3 end
```

Exemplo Gráfico:



24.20 Função WellesSum

Disponível em todas versões.

A função `WellesSum()` retorna o valor do indicador "WellesSum", de acordo com os parâmetros desejados. É o método que cria a soma móvel de uma série de dados, onde cada valor é a soma dos últimos *n* períodos. O exemplo com o código fonte está disponível no editor de estratégias, para visualizá-lo, acesse o menu: "abrir > exemplos".

Sintaxe:

```
Função WellesSum(Período: Integer, Dados: Serie, Offset : Integer): Float;
```

Parâmetros:

Período: *Integer*; Período utilizado no momento do cálculo do indicador.

SerieDeDados: *Serie*; Série utilizada para o cálculo.

Offset: *Integer*; Referente ao parâmetro "Offset" do indicador.

Exemplo de uso:

No exemplo abaixo, será plotado no gráfico o dado do indicador "WellesSum", considerando **20** (Período), série de fechamento (SerieReferencia) e **0** (Offset) para o cálculo.

```
1 begin
2   Plot(WellesSum(20, Close, 0));
3 end
```

Exemplo Gráfico:



24.21 Função xAverage

Disponível em todas versões.

A função `xAverage()` retorna a média exponencial de um determinado período.

Sintaxe:

```
Função xAverage(SerieDeDados: Serie, Período: Integer): Float;
```

Parâmetros:

SerieDeDados: `Serie`; Série utilizada para o cálculo.

Período: `Integer`; Período utilizado no momento do cálculo do indicador.

Exemplo de uso:

No exemplo abaixo, será plotado no gráfico o retorno da função `xAverage()`, considerando a série de fechamento e `10` períodos para o cálculo.

```
1 begin
2   Plot(xAverage(Close, 10));
3 end
```

Exemplo Gráfico:



25 Funções Úteis

25.1 Função AssetHasValue

Disponível em todas versões.

A função `AssetHasValue()` retorna se o ativo escolhido tem valor de mercado ou não.

Sintaxe:

```
Função AssetHasValue(Asset: Ativo): Boolean;
```

Parâmetros:

Asset: **Ativo**; Ativo escolhido para a função.

Exemplo de uso:

No exemplo abaixo, será plotado no gráfico o melhor preço de venda do ativo, usando a função `AskPrice()`, caso o ativo **PETR4** tenha valor de mercado.

```
1 const
2   PETR = Asset("PETR4", feedBovespa);
3
4 begin
5   if AssetHasValue(PETR) then
6     Plot(AskPrice);
7 end
```

Exemplo Gráfico:



25.2 Função BoolToString

Disponível em todas versões.

A função `BoolToString()` transforma um valor booleano em um string.

Sintaxe:

```
Função BoolToString(Content: Boolean): String;
```

Parâmetros:

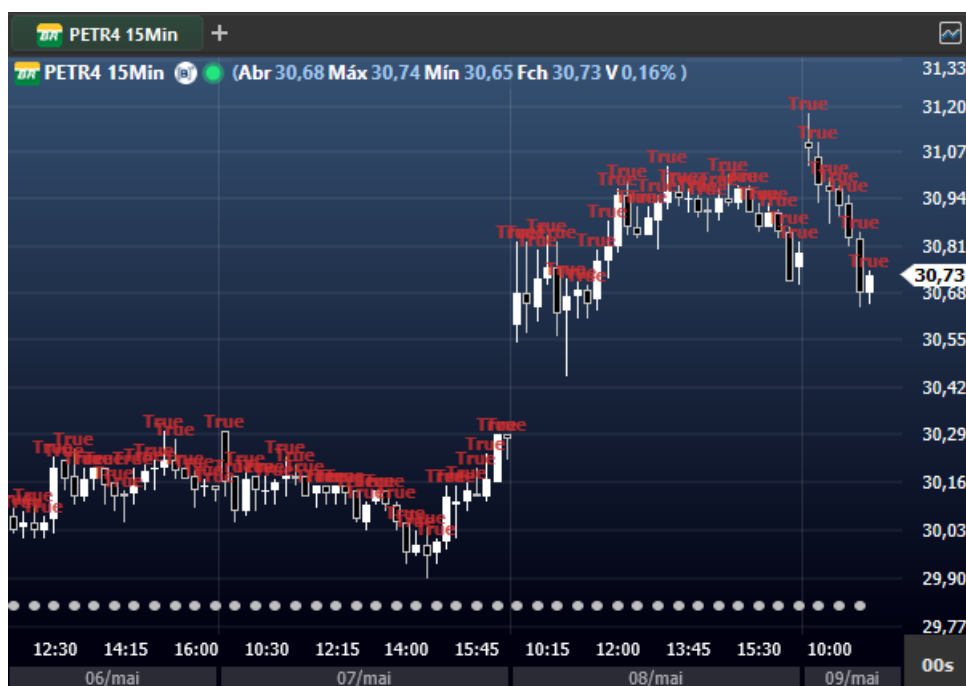
Content: **Boolean**; Valor booleano escolhido para ser transformado em string.

Exemplo de uso:

No exemplo abaixo, será plotado no gráfico o texto da variável `compare` caso o ativo **PETR4** tenha valor de mercado.

```
1 const
2   PETR4 = Asset("PETR4", feedBovespa);
3 var
4   compare : Boolean;
5 begin
6   compare := AssetHasValue(PETR4);
7   if compare then
8     Plottext(BoolToString(compare), clRed);
9 end
```

Exemplo Gráfico:



25.3 Função CompareFloat

A função `CompareFloat()` compara dois valores numéricos de ponto flutuante, considerando o número de casas decimais especificadas para precisão. Retorna **1** se `Valor_1` for maior, **0** se forem iguais e **-1** se `Valor_1` for menor. Também é possível usar constantes de comparação para facilitar as verificações.

Sintaxe:

```
Função CompareFloat(Valor_1: Float; Valor_2: Float; DígitosPrecisao: Integer = 9): Integer;
```

Parâmetros:

`Valor_1`: **Float**; Primeiro valor a ser comparado.

`Valor_2`: **Float**; Segundo valor a ser comparado.

`DígitosPrecisao`: **Integer** [Opcional]; Número de casas decimais a considerar (padrão = 9).

Exemplo de uso:

No exemplo abaixo, com precisão de 3 dígitos decimais, o resultado da comparação entre **3.141592** e **3.14** será atribuído à variável `resultado`, e então exibido com a função `Plot()`.

```
1 var
2     resultado : Integer;
3 begin
4     resultado := CompareFloat(3.141592, 3.14, 3);
5     Plot(resultado);
6 end;
```

Exemplo Gráfico:



25.4 Função FloatToString

A função `FloatToString()` converte um valor numérico de ponto flutuante em uma String, exibindo o número de casas decimais especificado. Essa função é útil para formatar valores numéricos antes de exibi-los em textos, legendas ou mensagens na tela.

Sintaxe:

Função `FloatToString(Valor: Float; Dígitos: Integer): String;`

Parâmetros:

Valor: `Float`; Valor numérico a ser convertido em texto.

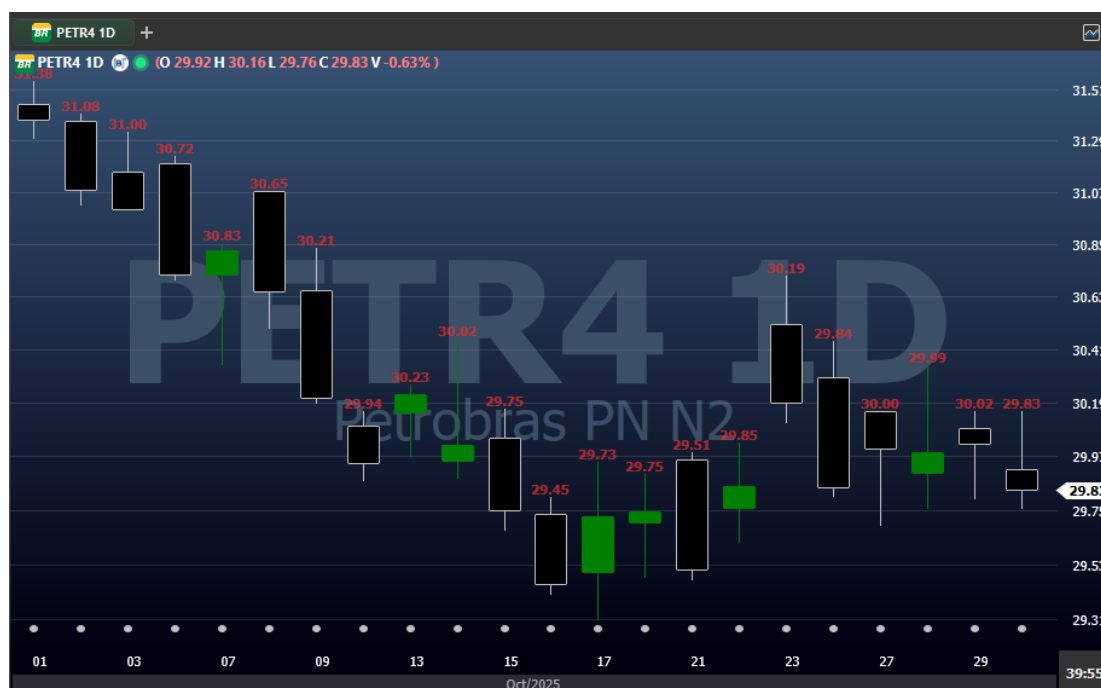
Dígitos: `Integer` [Opcional]; Número de casas decimais exibidas (padrão = 2).

Exemplo de uso:

No exemplo abaixo, o valor de fechamento do candle atual é convertido para texto com duas casas decimais e exibido no gráfico em vermelho, utilizando a função `PlotText()`.

```
1 begin
2   PlotText(FloatToString(Close, 2), clRed);
3 end;
```

Exemplo Gráfico:





26 Anexos

Este anexo apresenta exemplos de comportamentos incorretos que podem ocorrer ao desenvolver scripts em NTSL, seguidos das respectivas boas práticas para evitá-los. O objetivo é auxiliar desenvolvedores a escrever códigos mais seguros, previsíveis e compatíveis com o motor de execução da plataforma.

Esses casos foram observados com frequência no uso prático da linguagem e, quando corrigidos, contribuem diretamente para a estabilidade e a assertividade das estratégias criadas.

26.1 Exemplos de funcionamento de estratégias

1 - Nenhuma ordem será criada, já que existe um cancelamento de ordens.

```
1 begin
2   BuyAtMarket;
3   CancelPendingOrders;
4 end; //Fim do fluxo de execução
```

2 - Nenhuma ordem será criada, pois o ClosePosition vai cancelar a ordem BuyAtMarket.

```
1 begin
2   BuyAtMarket;
3   ClosePosition;
4 end; //Fim do fluxo de execução
```

3 - Nenhuma ordem será criada, pois o ReversePosition vai cancelar a ordem SellShortAtMarket.

```
1 begin
2   SellShortAtMarket;
3   ReversePosition;
4 end; //Fim do fluxo de execução
```

4 - Nenhuma ordem será criada, pois é realizada o agrupamento de ordens AtMarket que estão dentro do mesmo fluxo de execução.

```
1 begin
2   BuyAtMarket;
3   SellShortAtMarket;
4 end; //Fim do fluxo de execução
```

5 - O código abaixo não abre ordens covers. Pois ao passar pela linha BuyAtMarket não é gerada uma posição, pois não há execução da ordem. Desta maneira, a posição não é alterada e o SellToCover não será criado já que não há posição.

```
1 begin
2   if not HasPosition then
3     begin
4       BuyAtMarket;
5       SellToCoverLimit(Close + 10 * MinPriceIncrement);
6       SellToCoverStop(Close - 10 * MinPriceIncrement);
7     end;
8 end //Fim do fluxo de execução
```

Como alternativa, é aconselhável usar o código abaixo. Desta forma quando existir posição as ordens covers serão enviadas.



```
1 begin
2   if not HasPosition then
3     begin
4       BuyAtMarket;
5     end
6   else
7     begin
8       SellToCoverLimit(Close + 10 * MinPriceIncrement);
9       SellToCoverStop(Close - 10 * MinPriceIncrement);
10    end;
11 end //Fim do fluxo de execução
```

6 - Comparações diretas entre valores do tipo float podem resultar em comportamentos inesperados devido à natureza da representação de ponto flutuante em computadores. Pequenas diferenças de precisão podem fazer com que comparações como $a < b$, $a = b$ ou $a > b$ produzam resultados incorretos.

```
1 begin
2   if not HasPosition then
3     begin
4       BuyAtMarket;
5     end
6   else
7     begin
8       if(BuyPrice = Close + 10)then
9         ClosePosition;
10      end;
11 end //Fim do fluxo de execução
```

Para evitar esse tipo de problema, utilize a função `compareFloat`, que compara dois números float com uma precisão definida em casas decimais.

```
1 begin
2   if not HasPosition then
3     begin
4       BuyAtMarket;
5     end
6   else
7     begin
8       if (compareFloat(BuyPrice, Close + 10, 2) = 0) then
9         ClosePosition;
10      end;
11 end //Fim do fluxo de execução
```